

Oracle Database 11g: SQL Fundamentals II

Student Guide – Volume 1

D49994GC10

Edition 1.0

August 2007

D52148

ORACLE®

Author

Chaitanya Koratamaddi

**Technical Contributors
and Reviewers**

Claire Bennett
Ken Cooper
Yanti Chang
Laszlo Czinkoczki
Burt Demchick
Gerlinde Frenzen
Joel Goodman
Laura Garza
Richard Green
Nancy Greenberg
Akira Kinutani
Wendy Lo
Isabelle Marchand
Timothy Mcglue
Alan Paulson
Srinivas Putrevu
Bryan Roberts
Clinton Shaffer
Abhishek Singh
Jenny Tsai Smith
James Spiller
Lori Tritz
Lex van der Werff
Marcie Young

Editors

Nita Pavitrana
Arijit Ghosh
Raj Kumar

Graphic Designer

Satish Bettengowda

Publisher

Michael Sebastian Almeida

Copyright © 2007, Oracle. All rights reserved.

Disclaimer

This course provides an overview of features and enhancements planned in release 11g. It is intended solely to help you assess the business benefits of upgrading to 11g and to plan your IT projects.

This course in any form, including its course labs and printed matter, contains proprietary information that is the exclusive property of Oracle. This course and the information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This course and its contents are not part of your license agreement nor can they be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

This course is for informational purposes only and is intended solely to assist you in planning for the implementation and upgrade of the product features described. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, and timing of any features or functionality described in this document remain at the sole discretion of Oracle.

This document contains proprietary information and is protected by copyright and other intellectual property laws. You may copy and print this document solely for your own use in an Oracle training course. The document may not be modified or altered in any way. Except where your use constitutes "fair use" under copyright law, you may not use, share, download, upload, copy, print, display, perform, reproduce, publish, license, post, transmit, or distribute this document in whole or in part without the express authorization of Oracle.

The information contained in this document is subject to change without notice. If you find any problems in the document, please report them in writing to: Oracle University, 500 Oracle Parkway, Redwood Shores, California 94065 USA. This document is not warranted to be error-free.

Restricted Rights Notice

If this documentation is delivered to the United States Government or anyone using the documentation on behalf of the United States Government, the following notice is applicable:

U.S. GOVERNMENT RIGHTS

The U.S. Government's rights to use, modify, reproduce, release, perform, display, or disclose these training materials are restricted by the terms of the applicable Oracle license agreement and/or the applicable U.S. Government contract.

Trademark Notice

Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners.

Contents

Preface

I Introduction

Lesson Objectives	I-2
Lesson Agenda	I-3
Course Objectives	I-4
Course Prerequisites	I-5
Course Agenda	I-6
Lesson Agenda	I-7
Human Resources (HR) Schema Used in This Course	I-8
Appendixes Used in This Course	I-9
Development Environments	I-10
Lesson Agenda	I-11
Review of Using Oracle SQL Developer	I-12
Creating a Database Connection	I-13
Creating a Schema Object	I-14
Using the SQL Worksheet	I-15
Executing SQL Statements	I-16
Saving SQL Statements	I-17
Executing SQL Scripts	I-18
Lesson Agenda	I-19
SQL Statements in SQL*Plus	I-20
Review of Restricting Data	I-21
Review of Sorting Data	I-22
Review of SQL Functions	I-23
Review of Single-Row Functions	I-24
Review of Types of Group Functions	I-25
Review of Using Subqueries	I-26
Review of Manipulating Data	I-27
Lesson Agenda	I-28
Oracle Database 11g SQL Documentation	I-29
Additional Resources	I-30
Summary	I-31
Practice I: Overview	I-32

1 Controlling User Access

Objectives 1-2
Lesson Agenda 1-3
Controlling User Access 1-4
Privileges 1-5
System Privileges 1-6
Creating Users 1-7
User System Privileges 1-8
Granting System Privileges 1-9
Lesson Agenda 1-10
What Is a Role? 1-11
Creating and Granting Privileges to a Role 1-12
Changing Your Password 1-13
Lesson Agenda 1-14
Object Privileges 1-15
Granting Object Privileges 1-17
Passing On Your Privileges 1-18
Confirming Granted Privileges 1-19
Lesson Agenda 1-20
Revoking Object Privileges 1-21
Summary 1-23
Practice 1: Overview 1-24

2 Managing Schema Objects

Objectives 2-2
Lesson Agenda 2-3
`ALTER TABLE` Statement 2-4
Adding a Column 2-6
Modifying a Column 2-7
Dropping a Column 2-8
`SET UNUSED` Option 2-9
Lesson Agenda 2-11
Adding a Constraint Syntax 2-12
Adding a Constraint 2-13
`ON DELETE CASCADE` 2-14
Deferring Constraints 2-15
Difference Between `INITIALLY DEFERRED` and `INITIALLY IMMEDIATE` 2-16
Deferring Constraints: Example 2-17
Dropping a Constraint 2-18
Disabling Constraints 2-19

Enabling Constraints 2-20
Cascading Constraints 2-22
Renaming Table Columns and Constraints 2-24
Lesson Agenda 2-25
Overview of Indexes 2-26
CREATE INDEX with the CREATE TABLE Statement 2-27
Function-Based Indexes 2-29
Removing an Index 2-30
DROP TABLE ... PURGE 2-31
Lesson Agenda 2-32
FLASHBACK TABLE Statement 2-33
Using the FLASHBACK TABLE Statement 2-35
Lesson Agenda 2-36
External Tables 2-37
Creating a Directory for the External Table 2-38
Creating an External Table 2-40
Creating an External Table by Using ORACLE_LOADER 2-42
Querying External Tables 2-44
Creating an External Table by Using ORACLE_DATAPUMP: Example 2-45
Summary 2-46
Practice 2: Overview 2-47

3 Managing Objects with Data Dictionary Views

Objectives 3-2
Lesson Agenda 3-3
Data Dictionary 3-4
Data Dictionary Structure 3-5
How to Use the Dictionary Views 3-7
USER_OBJECTS and ALL_OBJECTS Views 3-8
USER_OBJECTS View 3-9
Lesson Agenda 3-10
Table Information 3-11
Column Information 3-12
Constraint Information 3-14
USER_CONSTRAINTS: Example 3-15
Querying USER_CONS_COLUMNS 3-16
Lesson Agenda 3-17
View Information 3-18
Sequence Information 3-19
Confirming Sequences 3-20

Index Information 3-21
USER_INDEXES: Examples 3-22
Querying USER_IND_COLUMNS 3-23
Synonym Information 3-24
Lesson Agenda 3-25
Adding Comments to a Table 3-26
Summary 3-27
Practice 3: Overview 3-28

4 Manipulating Large Data Sets

Objectives 4-2
Lesson Agenda 4-3
Using Subqueries to Manipulate Data 4-4
Retrieving Data Using a Subquery as Source 4-5
Inserting Using a Subquery as a Target 4-7
Using the WITH CHECK OPTION Keyword on DML Statements 4-9
Lesson Agenda 4-11
Overview of the Explicit Default Feature 4-12
Using Explicit Default Values 4-13
Copying Rows from Another Table 4-14
Lesson Agenda 4-15
Overview of Multitable INSERT Statements 4-16
Types of Multitable INSERT Statements 4-18
Multitable INSERT Statements 4-19
Unconditional INSERT ALL 4-21
Conditional INSERT ALL: Example 4-23
Conditional INSERT ALL 4-24
Conditional INSERT FIRST: Example 4-25
Conditional INSERT FIRST 4-26
Pivoting INSERT 4-28
Lesson Agenda 4-31
MERGE Statement 4-32
MERGE Statement Syntax 4-33
Merging Rows: Example 4-34
Lesson Agenda 4-37
Tracking Changes in Data 4-38
Example of the Flashback Version Query 4-39
VERSIONS BETWEEN Clause 4-41
Summary 4-42
Practice 4: Overview 4-43

5 Managing Data in Different Time Zones

Objectives 5-2

Lesson Agenda 5-3

Time Zones 5-4

TIME_ZONE Session Parameter 5-5

CURRENT_DATE, CURRENT_TIMESTAMP, and LOCALTIMESTAMP 5-6

Comparing Date and Time in a Session's Time Zone 5-7

DBTIMEZONE and SESSIONTIMEZONE 5-9

TIMESTAMP Data Types 5-10

TIMESTAMP Fields 5-11

Difference Between DATE and TIMESTAMP 5-12

Comparing TIMESTAMP Data Types 5-13

Lesson Agenda 5-14

INTERVAL Data Types 5-15

INTERVAL Fields 5-17

INTERVAL YEAR TO MONTH: Example 5-18

INTERVAL DAY TO SECOND Data Type: Example 5-20

Lesson Agenda 5-21

EXTRACT 5-22

TZ_OFFSET 5-23

FROM_TZ 5-25

TO_TIMESTAMP 5-26

TO_YMINTERVAL 5-27

TO_DSINTERVAL 5-28

Daylight Saving Time 5-29

Summary 5-31

Practice 5: Overview 5-32

6 Retrieving Data Using Subqueries

Objectives 6-2

Lesson Agenda 6-3

Multiple-Column Subqueries 6-4

Column Comparisons 6-5

Pairwise Comparison Subquery 6-6

Nonpairwise Comparison Subquery 6-8

Lesson Agenda 6-10

Scalar Subquery Expressions 6-11

Scalar Subqueries: Examples 6-12

Lesson Agenda 6-14

Correlated Subqueries 6-15

Using Correlated Subqueries 6-17
Lesson Agenda 6-19
Using the EXISTS Operator 6-20
Find Employees Who Have at Least One Person Reporting to Them 6-21
Find All Departments That Do Not Have Any Employees 6-22
Correlated UPDATE 6-23
Using Correlated UPDATE 6-24
Correlated DELETE 6-26
Using Correlated DELETE 6-27
Lesson Agenda 6-28
WITH Clause 6-29
WITH Clause: Example 6-30
Summary 6-32
Practice 6: Overview 6-34

7 Regular Expression Support

Objectives 7-2
Lesson Agenda 7-3
What Are Regular Expressions? 7-4
Benefits of Using Regular Expressions 7-5
Using the Regular Expressions Functions and Conditions in SQL and PL/SQL 7-6
Lesson Agenda 7-7
What Are Metacharacters? 7-8
Using Metacharacters with Regular Expressions 7-9
Lesson Agenda 7-11
Regular Expressions Functions and Conditions: Syntax 7-12
Performing a Basic Search Using the REGEXP LIKE Condition 7-13
Replacing Patterns Using the REGEXP_REPLACE Function 7-14
Finding Patterns Using the REGEXP_INSTR Function 7-15
Extracting Substrings Using the REGEXP_SUBSTR Function 7-16
Lesson Agenda 7-17
Subexpressions 7-18
Using Subexpressions with Regular Expression Support 7-19
Why Access the *n*th Subexpression? 7-20
REGEXP_SUBSTR: Example 7-21
Lesson Agenda 7-22
Using the REGEXP_COUNT Function 7-23
Regular Expressions and Check Constraints: Examples 7-24
Summary 7-25
Practice 7: Overview 7-26

Appendix A: Practice Solutions

Appendix B: Table Descriptions

Appendix C: Using the SQL Developer GUI

Appendix D: Using SQL*Plus

Appendix E: Generating Reports by Grouping Related Data

Appendix F: Hierarchical Retrieval

Appendix G: Writing Advanced Scripts

Appendix H: Oracle Database Architectural Components

Index

Additional Practices

Additional Practice Solutions

Oracle Internal & Oracle Academy
Use Only

Preface

Oracle Internal & Oracle Academy
Use Only

Profile

Before You Begin This Course

Before you begin this course, you should have working experience with SQL.

How This Course Is Organized

Oracle Database 11g: SQL Fundamentals II is an instructor-led course featuring lectures and hands-on exercises. Online demonstrations and written practice sessions reinforce the concepts and skills that are introduced.

Oracle Internal & Oracle Academy
Use Only

Related Publications

Oracle Publications

Title	Part Number
<i>Oracle® Database Reference 11g Release 1 (11.1)</i>	B28320-01
<i>Oracle® Database SQL Language Reference 11g – Release 1 (11.1)</i>	B28286-01
<i>Oracle® Database Concepts 11g Release 1 (11.1)</i>	B28318-01
<i>Oracle® Database Advanced Application Developer's Guide – 11g Release 1 (11.1)</i>	B28424-01
<i>SQL*Plus® User's Guide and Reference Release 11.1</i>	B31189-01
<i>Oracle Database SQL Developer User's Guide Release 1.2</i>	E10406-01

Additional Publications

- System release bulletins
- Installation and user's guides
- *read.me* files
- International Oracle User's Group (IOUG) articles
- *Oracle Magazine*

Typographic Conventions

What follows are two lists of typographical conventions that are used specifically within text or within code.

Typographic Conventions Within Text

Convention	Object or Term	Example
Uppercase	Commands, functions, column names, table names, PL/SQL objects, schemas	Use the <code>SELECT</code> command to view information stored in the <code>LAST_NAME</code> column of the <code>EMPLOYEES</code> table.
Lowercase, italic	Filenames, syntax variables, usernames, passwords	where: <i>role</i> is the name of the role to be created.
Initial cap	Trigger and button names	Assign a When-Validate-Item trigger to the ORD block. Choose Cancel.
Italic	Books, names of courses and manuals, and emphasized words or phrases	For more information on the subject see <i>Oracle SQL Reference Manual</i> Do <i>not</i> save changes to the database.
Quotation marks	Lesson module titles referenced within a course	This subject is covered in Lesson 3, “Working with Objects.”

Typographic Conventions (continued)

Typographic Conventions Within Code

Convention	Object or Term	Example
Uppercase	Commands, functions	<code>SELECT employee_id FROM employees;</code>
Lowercase, italic	Syntax variables	<code>CREATE ROLE <i>role</i>;</code>
Initial cap	Forms triggers	<code>Form module: ORD Trigger level: S_ITEM.QUANTITY item Trigger name: When-Validate-Item . . .</code>
Lowercase	Column names, table names, filenames, PL/SQL objects	<code>. . . OG_ACTIVATE_LAYER (OG_GET_LAYER ('prod_pie_layer')) . . . SELECT last_name FROM employees;</code>
Bold	Text that must be entered by a user	<code>CREATE USER scott IDENTIFIED BY tiger;</code>

I

Introduction

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Oracle Internal & Oracle Academy
Use Only

Lesson Objectives

After completing this lesson, you should be able to do the following:

- Discuss the goals of the course
- Identify the available environments that can be used in the course
- Describe the database schema and tables that are used in the course
- Review the basic features of SQL Developer
- Review some of the basic concepts of SQL

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Lesson Agenda

- Course objectives and course agenda
- The database schema and appendixes used in the course and the available development environment in this course
- Overview of Oracle SQL Developer
- Review of some basic concepts of SQL
- Oracle Database 11g documentation and additional resources

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Course Objectives

After completing this course, you should be able to do the following:

- Control database access to specific objects
- Add new users with different levels of access privileges
- Manage schema objects
- Manage objects with data dictionary views
- Manipulate large data sets in the Oracle database by using subqueries
- Manage data in different time zones
- Write multiple-column subqueries
- Use scalar and correlated subqueries
- Use the regular expression support in SQL

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Course Prerequisites

The *Oracle Database 11g: SQL Fundamentals I* course is a prerequisite for this course.

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Course Prerequisites

Required preparation for this course is *Oracle Database 11g: SQL Fundamentals I*.

This course offers you an introduction to Oracle Database 11g database technology. In this course, you learn the basic concepts of relational databases and the powerful SQL programming language. This course provides the essential SQL skills that enable you to write queries against single and multiple tables, manipulate data in tables, create database objects, and query metadata.

Course Agenda

- Day 1:
 - Introduction
 - Controlling User Access
 - Managing Schema Objects
 - Managing Objects with Data Dictionary Views
- Day 2:
 - Manipulating Large Data Sets
 - Managing Data in Different Time Zones
 - Retrieving Data Using Subqueries
 - Regular Expression Support

ORACLE

Copyright © 2007, Oracle. All rights reserved.

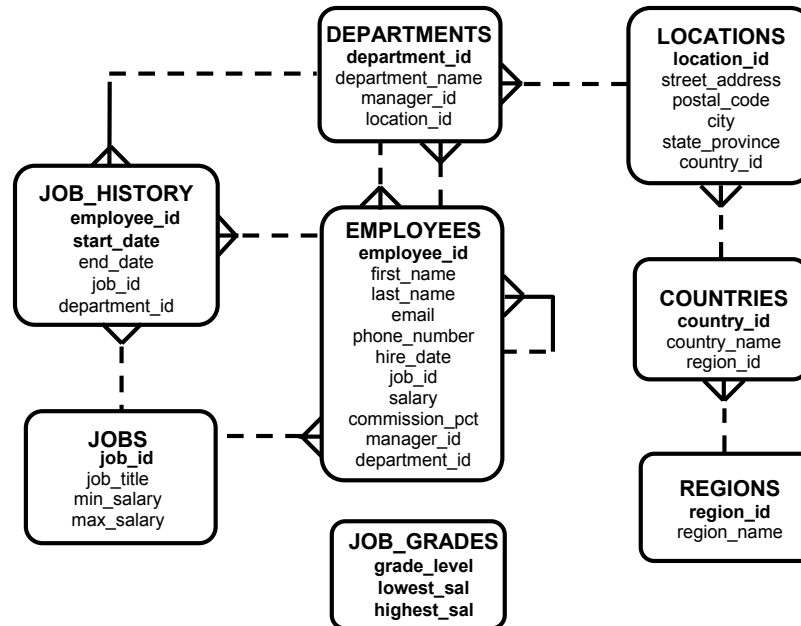
Lesson Agenda

- Course objectives and course agenda
- The database schema and appendixes used in the course and the available development environment in this course
- Overview of Oracle SQL Developer
- Review of some basic concepts of SQL
- Oracle Database 11g documentation and additional resources

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Human Resources (HR) Schema Used in This Course



ORACLE

Copyright © 2007, Oracle. All rights reserved.

Human Resources (HR) Schema Description

The Human Resources (HR) schema is part of the Oracle Sample Schemas that can be installed in an Oracle database. This course uses data from the HR schema.

Table descriptions

- The **EMPLOYEES** table contains information about all the employees such as their first and last names, job IDs, salaries, hire dates, department IDs, and manager IDs. This table is a child of the **DEPARTMENTS** table.
- The **DEPARTMENTS** table contains information such as the department ID, department name, manager ID, and location ID. This table is the primary key table to the **EMPLOYEES** table.
- The **LOCATIONS** table contains department location information. It contains location ID, street address, city, state province, postal code, and country ID information. It is the primary key table to the **DEPARTMENTS** table and is a child of the **COUNTRIES** table.
- The **COUNTRIES** table contains the country names, country IDs, and region IDs. It is a child of the **REGIONS** table. This table is the primary key table to the **LOCATIONS** table.
- The **REGIONS** table contains region IDs and region names of the various countries. It is a primary key table to the **COUNTRIES** table.
- The **JOB_GRADES** table identifies a salary range per job grade. The salary ranges do not overlap.
- The **JOB_HISTORY** table stores job history of the employees.
- The **JOBS** table contains job titles and salary ranges.

Appendixes Used in This Course

- Appendix A: Practice Solutions
- Appendix B: Table Descriptions
- Appendix C: Using the SQL Developer GUI
- Appendix D: Using SQL*Plus
- Appendix E: Generating Reports by Grouping Related Data
- Appendix F: Hierarchical Retrieval
- Appendix G: Writing Advanced Scripts
- Appendix H: Oracle Database Architectural Components

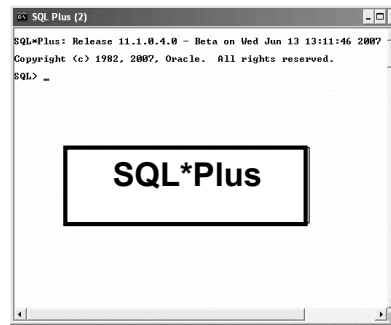
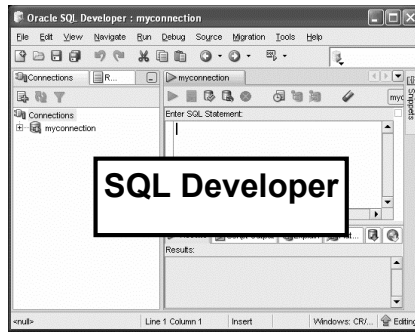
ORACLE

Copyright © 2007, Oracle. All rights reserved.

Development Environments

In this course:

- Primarily use SQL Developer 1.2
 - Included in the Oracle Database 11g Client installation for Windows
- Use SQL*Plus
 - In case you do not have access to SQL Developer



ORACLE

Copyright © 2007, Oracle. All rights reserved.

Development Environments

Oracle provides several tools for SQL development. Some of the development tools that are available for use in this course are:

- **Oracle SQL Developer:** A graphical tool (Oracle SQL Developer 1.2 is used in this course.)
- **Oracle SQL*Plus:** A window or command-line application

Note: The code and screen examples presented in the course notes were generated from output in the SQL Developer environment.

Lesson Agenda

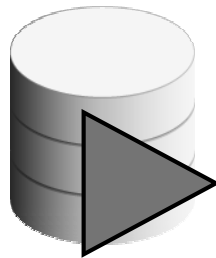
- Course objectives and course agenda
- The database schema and appendixes used in the course and the available development environment in this course
- Overview of Oracle SQL Developer
- Review of some basic concepts of SQL
- Oracle Database 11g documentation and additional resources

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Review of Using Oracle SQL Developer

- Oracle SQL Developer is a free graphical tool that enhances productivity and simplifies database development tasks.
- You can connect to any target Oracle database schema using standard Oracle database authentication.
- You use SQL Developer in this course.



SQL Developer

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Review of Using Oracle SQL Developer

Oracle SQL Developer is a free graphical tool designed to improve your productivity and simplify the development of everyday database tasks. With just a few clicks, you can easily create and debug stored procedures, test SQL statements, and view optimizer plans.

SQL Developer, the visual tool for database development, simplifies the following tasks:

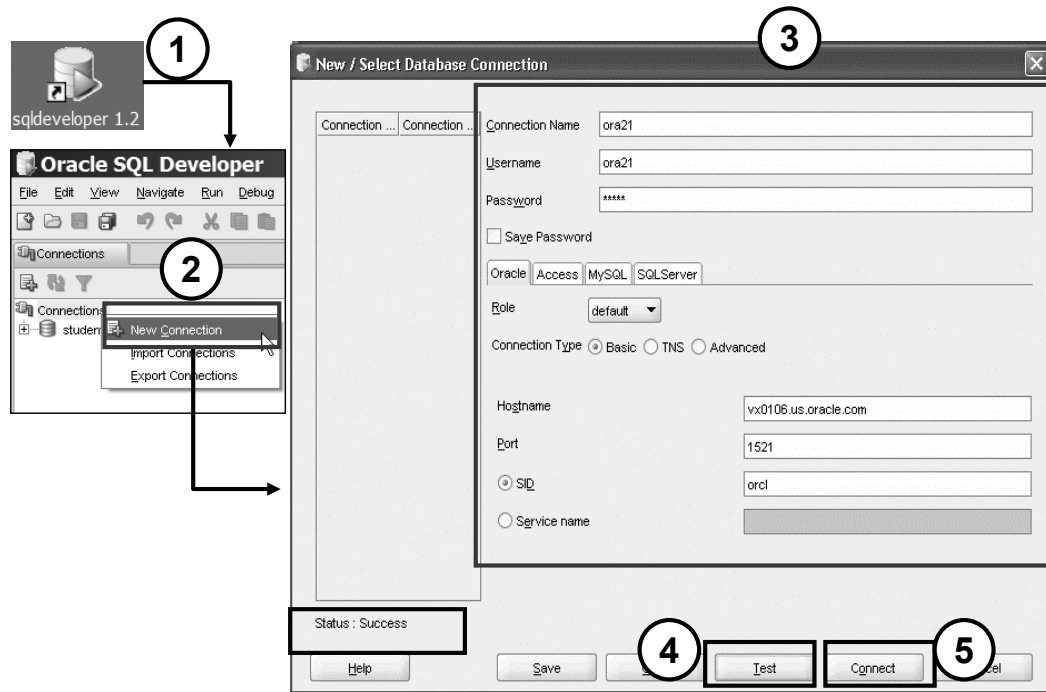
- Browsing and managing database objects
- Executing SQL statements and scripts
- Editing and debugging PL/SQL statements
- Creating reports

You can connect to any target Oracle database schema using standard Oracle database authentication. When connected, you can perform operations on objects in the database.

Note: The Oracle SQL Developer, release 1.2 is called the *Migration release* because it tightly integrates with *Developer Migration Workbench*. Therefore, it provides users with a single point to browse database objects and data in third party databases, and to migrate from these databases to Oracle. You can also connect to schemas for selected third-party (non-Oracle) databases, such as MySQL, Microsoft SQL Server, and Microsoft Access, and view metadata and data in these databases. Additionally Oracle SQL Developer, release 1.2 includes support for Oracle Application Express, release 3.0.1 (Oracle APEX).

For more information about using SQL Developer, see Appendix C.

Creating a Database Connection



Creating a Database Connection

To create a database connection, perform the following steps:

1. Double-click `<your_path>\sqldeveloper\sqldeveloper.exe`.
2. On the Connections tabbed page, right-click Connections and select New Connection.
3. Enter the connection name, username, password, host name, and SID for the database you want to connect.
4. Click Test to make sure that the connection has been set correctly.
5. Click Connect.

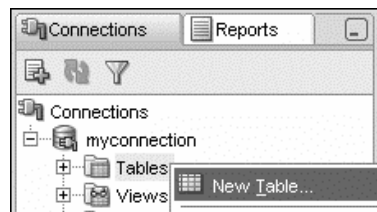
On the basic tabbed page, at the bottom, enter the following options:

- **Hostname:** Host system for the Oracle database
- **Port:** Listener port
- **System Identifier (SID):** Database name
- **Service Name:** Network service name for a remote database connection

If you select the Save Password check box, the password is saved to an XML file. So, after you close the SQL Developer connection and open it again, you will not be prompted for the password.

Creating a Schema Object

- You can create any schema object in SQL Developer using one of the following methods:
 - Executing a SQL statement in the SQL Worksheet
 - Using the context menu
- Edit the objects using an edit dialog box or one of the many context-sensitive menus.
- View the data definition language (DDL) for adjustments such as creating a new object or editing an existing schema object.



Expand **Connections**.
Right-click **Tables** and
select **New Table**.

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Creating a Schema Object

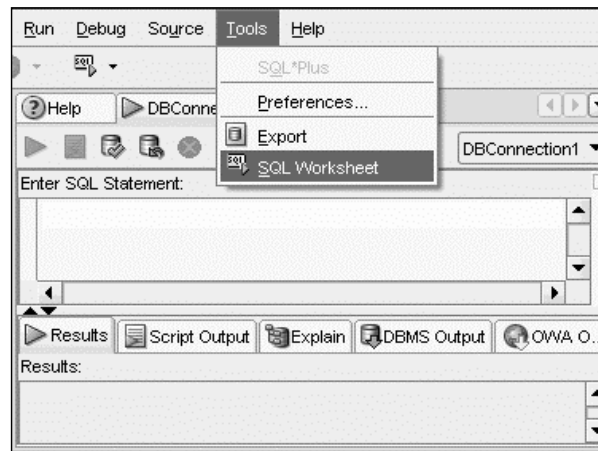
SQL Developer supports the creation of any schema object by executing a SQL statement in the SQL Worksheet. Alternatively, you can create objects using the shortcut menus. After they are created, you can edit the objects using an edit dialog box or one of many context-sensitive menus.

As new objects are created or existing objects are edited, the DDL for those adjustments is available for review. An Export DDL option is available if you want to create the full DDL for one or more objects in the schema.

The slide shows creating a table using the shortcut menu. To open a dialog box for creating a new table, right-click **Tables** and select **New Table**. The dialog boxes for creating and editing database objects have multiple tabs, each reflecting a logical grouping of properties for that type of object.

Using the SQL Worksheet

- Use the SQL Worksheet to enter and execute SQL and SQL*Plus statements.
- Specify any actions that can be processed by the database connection associated with the worksheet.



Copyright © 2007, Oracle. All rights reserved.

Using the SQL Worksheet

When you connect to a database, a SQL Worksheet window for that connection is automatically opened. You can use the SQL Worksheet to enter and execute SQL, PL/SQL, and SQL*Plus statements. The SQL Worksheet supports most of the SQL*Plus statements. SQL*Plus statements that are not supported by the SQL Worksheet are ignored and not passed to the database.

You can specify any actions that can be processed by the database connection associated with the worksheet, such as:

- Creating a table
- Inserting data
- Creating and editing a trigger
- Selecting data from a table
- Saving the selected data to a file

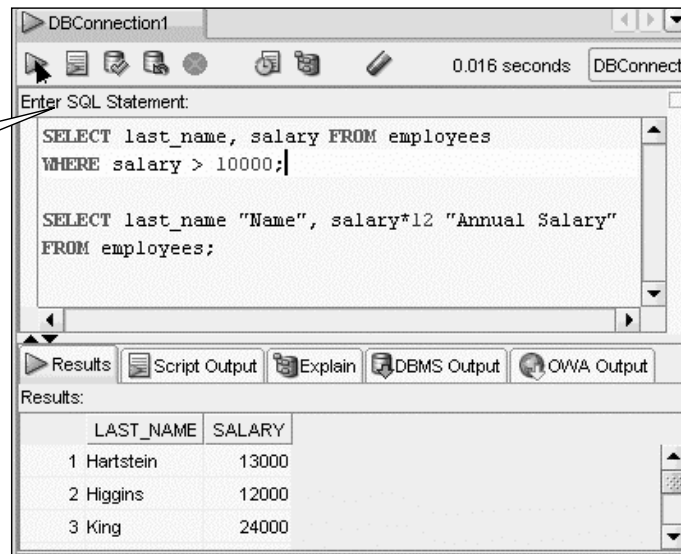
You can display a SQL Worksheet by using any of the following two options:

- Select **Tools > SQL Worksheet**.
- Click the **Open SQL Worksheet** icon.

Note: For the complete list of SQL*Plus statements that are supported, and not supported by the SQL Worksheet, refer to the “SQL*Plus Statements Supported and Not Supported in SQL Worksheet” topic in the SQL Developer online Help.

Executing SQL Statements

Use the Enter SQL Statement box to enter single or multiple SQL statements.



ORACLE

Copyright © 2007, Oracle. All rights reserved.

Executing SQL Statements

In the SQL Worksheet, you can use the Enter SQL Statement box to type a single or multiple SQL statements. For a single statement, the semicolon at the end is optional.

When you enter the statement, the SQL keywords are automatically highlighted. To execute a SQL statement, ensure that your cursor is within the statement and click the **Execute Statement** icon. Alternatively, you can press [F9].

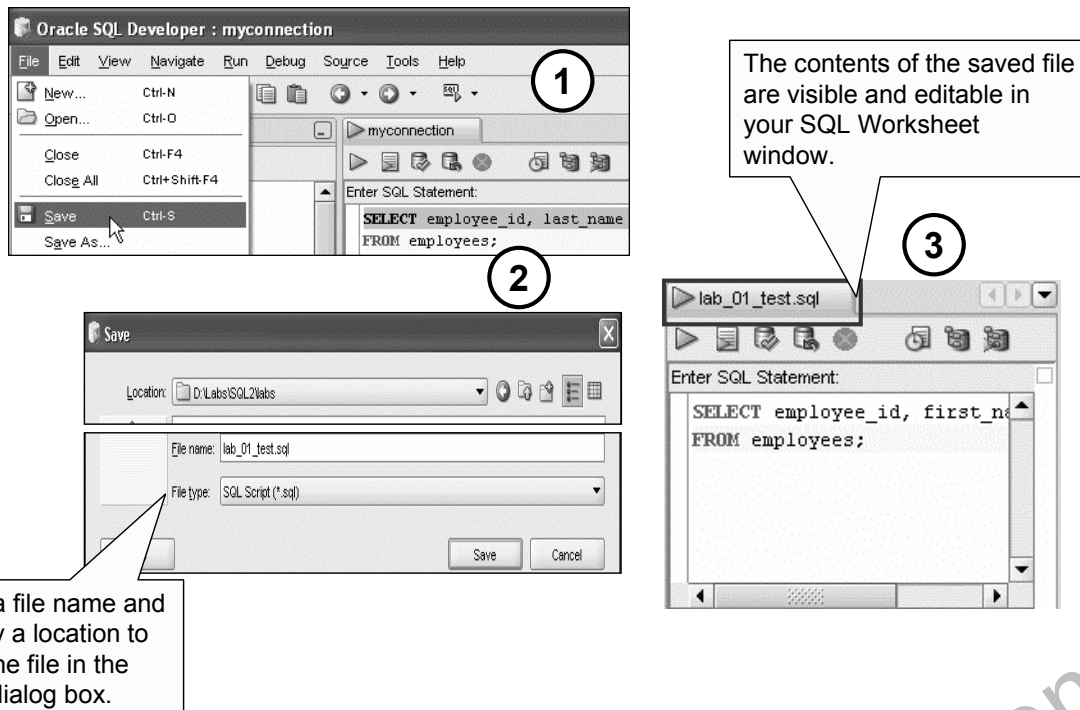
To execute multiple SQL statements and see the results, click the **Run Script** icon. Alternatively, you can press [F5].

In the example in the slide, because there are multiple SQL statements, the first statement is terminated with a semicolon. The cursor is in the first statement and so when the statement is executed, results corresponding to the first statement are displayed in the Results box.

Note: In this course, the following output options in SQL Developer are used for code examples and practices.

- The Results pane for simple SELECT statements
- The Script Output pane to display the output of DML and DDL operations

Saving SQL Statements



Copyright © 2007, Oracle. All rights reserved.

Saving SQL Statements

In most of the practices that you will perform, you will need to save a particular query in the SQL Worksheet as a .sql file. To do so, perform the following:

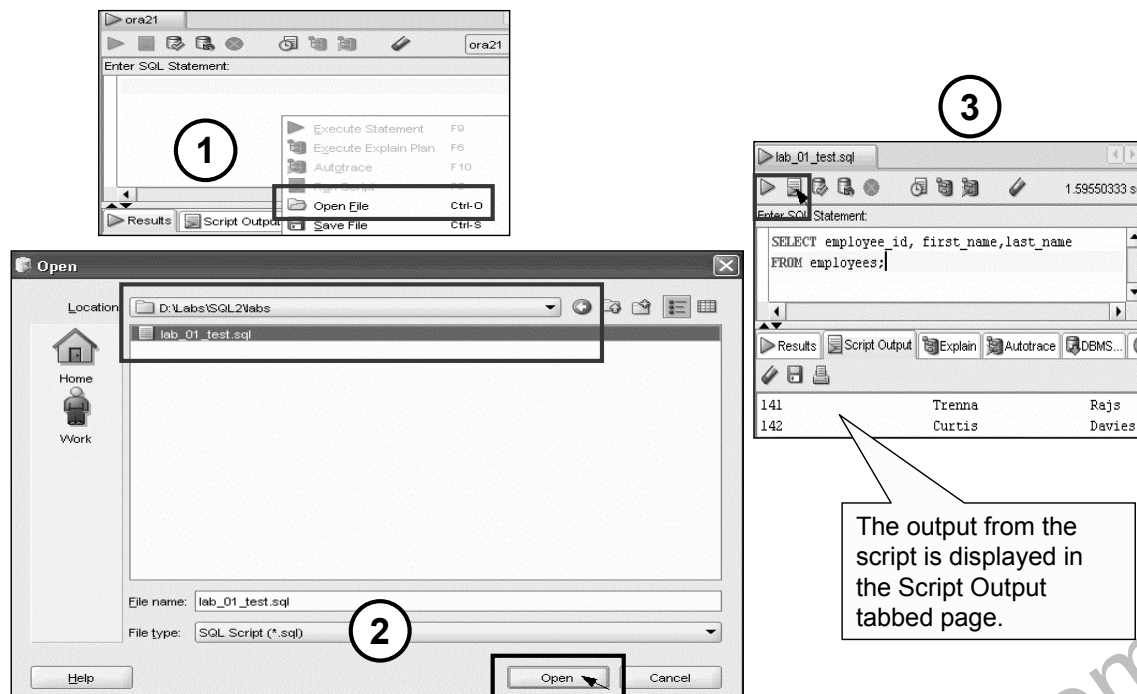
1. From the **File** menu, select **Save** or **Save As** (if you are renaming a current .sql script) or you can right-click in the SQL Worksheet and select **Save File**. Alternatively, you can use [CTRL] + [S].
2. In the Save dialog box, enter the appropriate file name. Make sure the extension is .sql or the file type is selected as SQL Script (*.sql). Click Save.
Note: For this course, you need to save your .sql scripts in the D:\labs\sql2\labs folder.
3. The SQL Worksheet is renamed to the file name you saved the script as. Make sure you do not enter any other SQL statements in the same worksheet. To continue with other SQL queries, open a new worksheet.

After you save the contents to a file, the Enter SQL Statement box displays a tabbed page of your file contents. You can have multiple files open at once. Each file appears as a tabbed page.

Script Pathing

You can select a default path to look for scripts and to save scripts. Under the **Tools > Preferences > Database > Worksheet Parameters**, enter a value in the **Select default path to look for scripts** field.

Executing SQL Scripts



The output from the script is displayed in the Script Output tabbed page.

Copyright © 2007, Oracle. All rights reserved.

Executing SQL Scripts

To run the saved SQL scripts, perform the following steps:

1. Right-click in the SQL Worksheet and select **Open File** or from the File menu, select **Open**. Alternatively, you can use [CTRL] + [O].
2. In the Open dialog box, move to the D: \labs\sql2\labs folder or to the location you saved the script file, select the file, and click **Open**.
3. The script file opens in a new worksheet. Now, you can run the script by either clicking the **Execute Statement** icon or the **Run Script** icon. Again, make sure you do not enter any other SQL statements in the same worksheet. To continue with other SQL queries, open a new worksheet.

Alternatively, to run a saved SQL script, perform the following steps:

1. In the Enter SQL Statement window, use the @ command, followed by the location and name of the file you want to run.
2. Click the **Run Script** icon.

When you execute a SQL script, the results are displayed on the Script Output tabbed page. You can also save the script output by clicking the Save icon on the Script Output tabbed page. The Windows File Save dialog box appears and you can identify a name and location for your file.

Lesson Agenda

- Course objectives and course agenda
- The database schema and appendixes used in the course and the available development environment in this course
- Overview of Oracle SQL Developer
- **Review of some basic concepts of SQL**
- Oracle Database 11g documentation and additional resources

ORACLE

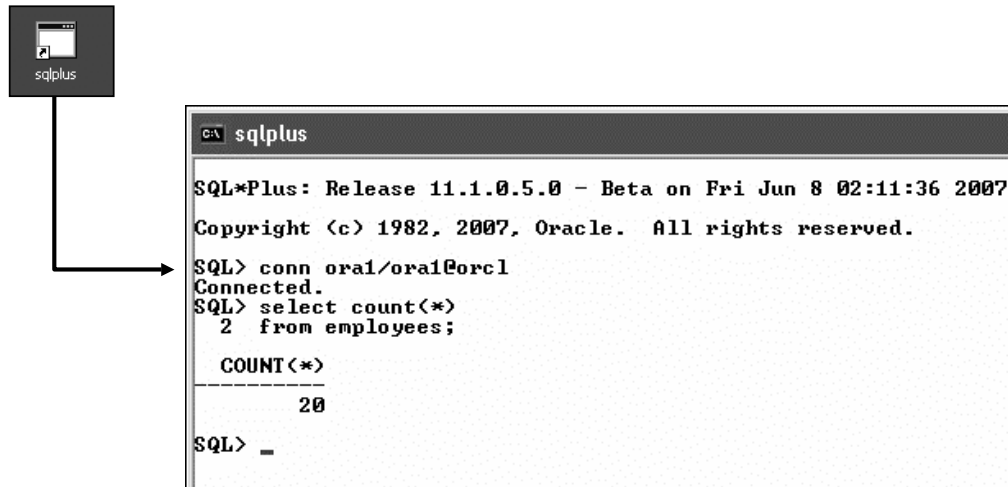
Copyright © 2007, Oracle. All rights reserved.

Lesson Agenda

The next few slides provide a brief overview of some of the basic concepts that you learned in the course titled *Oracle Database 11g: SQL Fundamentals I*.

SQL Statements in SQL*Plus

In Oracle Database 11g, SQL*Plus is a command-line interface.



ORACLE

Copyright © 2007, Oracle. All rights reserved.

SQL Statements in SQL*Plus

Oracle SQL*Plus is a command-line interface that enables you to submit SQL statements and PL/SQL blocks for execution and receive the results in an application or command window.

SQL*Plus is:

- Shipped with the database
- Installed on a client and on the database server system
- Accessed from an icon or the command line

Note: If you do not have access to SQL Developer and would prefer to use SQL* Plus, the classroom setup provides the SQL*Plus icon on your desktop. It may also be useful to use it in cases when SQL Developer does not support any SQL* Plus command.

Review of Restricting Data

- Restrict the rows that are returned by using the `WHERE` clause.
- Use comparison conditions to compare one expression with another value or expression.

Operator	Meaning
<code>BETWEEN</code> <code>...AND...</code>	Between two values (inclusive)
<code>IN (set)</code>	Match any of a list of values
<code>LIKE</code>	Match a character pattern

- Use logical conditions to combine the result of two component conditions and produce a single result based on those conditions.

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Review of Restricting and Sorting Data

You can restrict the rows that are returned from the query by using the `WHERE` clause. A `WHERE` clause contains a condition that must be met, and it directly follows the `FROM` clause.

The `WHERE` clause can compare values in columns, literal values, arithmetic expression, or functions.

It consists of three elements:

- Column name
- Comparison condition
- Column name, constant, or list of values

You use comparison conditions in the `WHERE` clause in the following format:

`... WHERE expr operator value`

Apart from those mentioned in the slide, you use other comparison conditions such as `=`, `<`, `>`, `<>`, `<=`, and `>=`.

Three logical operators are available in SQL:

- `AND`
- `OR`
- `NOT`

Review of Sorting Data

- Sort retrieved rows with the ORDER BY clause:
 - ASC: Ascending order, default
 - DESC: Descending order
- The ORDER BY clause comes last in the SELECT statement:

```
SELECT last_name, job_id, department_id, hire_date
FROM employees
ORDER BY hire_date ;
```

	LAST_NAME	JOB_ID	DEPARTMENT_ID	HIRE_DATE
1	King	AD_PRES		90 17-JUN-87
2	Whalen	AD_ASST		10 17-SEP-87
3	Kochhar	AD_VP		90 21-SEP-89
4	Hunold	IT_PROG		60 03-JAN-90
5	Ernst	IT_PROG		60 21-MAY-91
6	De Haan	AD_VP		90 13-JAN-93

...

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Review of Sorting Data

The order of rows that are returned in a query result is undefined. The ORDER BY clause can be used to sort the rows. If you use the ORDER BY clause, it must be the last clause of the SQL statement. You can specify an expression, an alias, or a column position as the sort condition.

Syntax

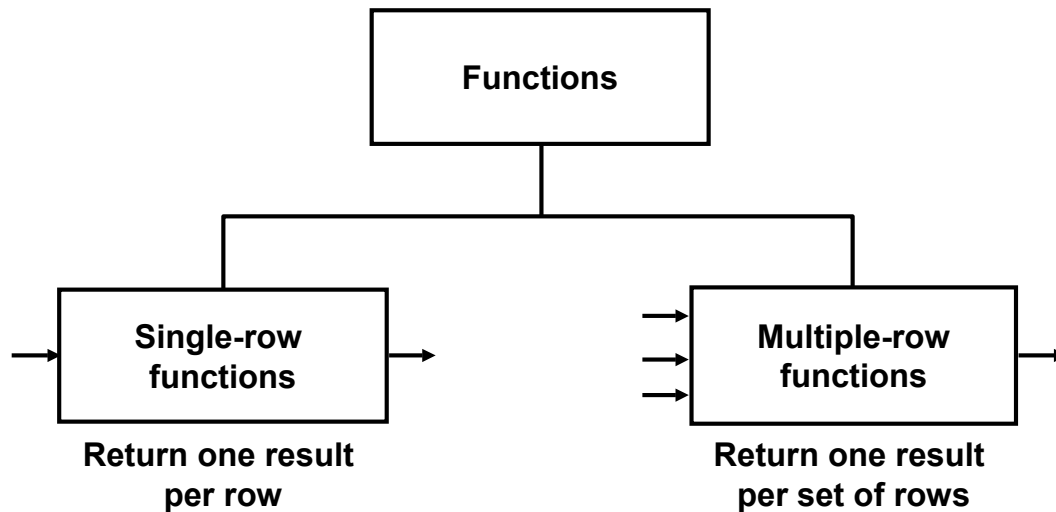
```
SELECT      expr
FROM        table
[WHERE      condition(s)]
[ORDER BY  {column, expr, numeric_position} [ASC|DESC]] ;
```

In the syntax:

ORDER BY	Specifies the order in which the retrieved rows are displayed
ASC	Orders the rows in ascending order (This is the default order.)
DESC	Orders the rows in descending order

If the ORDER BY clause is not used, the sort order is undefined, and the Oracle server may not fetch rows in the same order for the same query twice. Use the ORDER BY clause to display the rows in a specific order.

Review of SQL Functions



ORACLE

Copyright © 2007, Oracle. All rights reserved.

Review of SQL Functions

There are two types of functions:

- Single-row functions
- Multiple-row functions

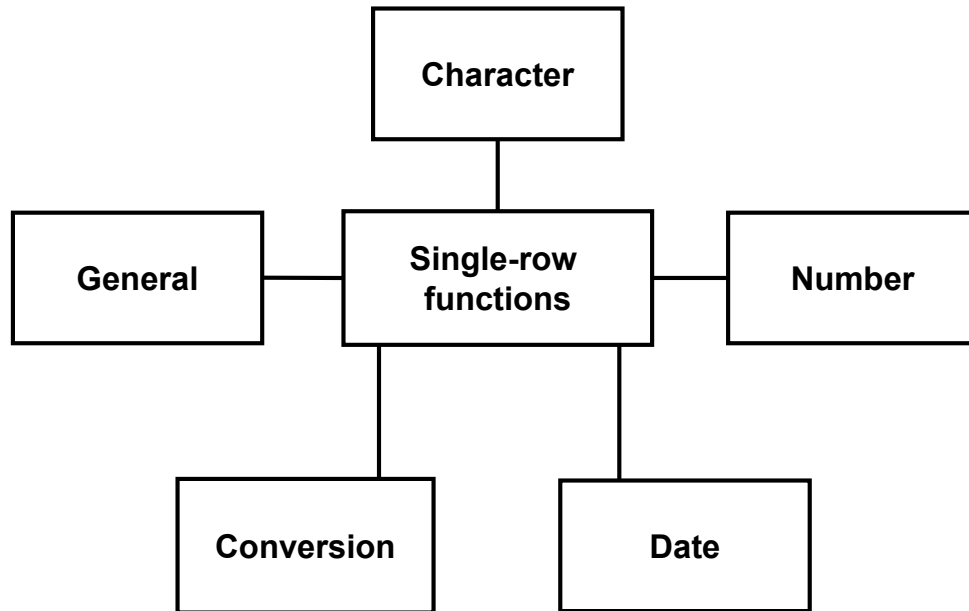
Single-Row Functions

These functions operate on single rows only and return one result per row. There are different types of single-row functions such as character, number, date, conversion, and general functions.

Multiple-Row Functions

Functions can manipulate groups of rows to give one result per group of rows. These functions are also known as *group functions*.

Review of Single-Row Functions



ORACLE

Copyright © 2007, Oracle. All rights reserved.

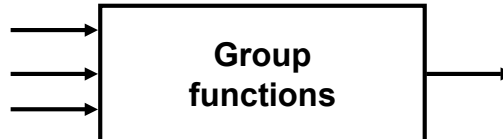
Review of Single-Row Functions

The following are different types of single-row functions:

- **Character functions:** Accept character input and can return both character and number values
- **Number functions:** Accept numeric input and return numeric values
- **Date functions:** Operate on values of the DATE data type (All date functions return a value of the DATE data type except the MONTHS_BETWEEN function, which returns a number.)
- **Conversion functions:** Convert a value from one data type to another
- **General functions:**
 - NVL
 - NVL2
 - NULLIF
 - COALESCE
 - CASE
 - DECODE

Review of Types of Group Functions

- AVG
- COUNT
- MAX
- MIN
- STDDEV
- SUM
- VARIANCE



ORACLE

Copyright © 2007, Oracle. All rights reserved.

Review of Types of Group Functions

Each of the functions accepts an argument. The following table identifies the options that you can use in the syntax:

Function	Description
AVG ([DISTINCT <u>ALL</u>] <i>n</i>)	Average value of <i>n</i> , ignoring null values
COUNT ({ * [DISTINCT <u>ALL</u>] <i>expr</i> })	Number of rows, where <i>expr</i> evaluates to something other than null (count all selected rows using *, including duplicates and rows with nulls)
MAX ([DISTINCT <u>ALL</u>] <i>expr</i>)	Maximum value of <i>expr</i> , ignoring null values
MIN ([DISTINCT <u>ALL</u>] <i>expr</i>)	Minimum value of <i>expr</i> , ignoring null values
STDDEV ([DISTINCT <u>ALL</u>] <i>x</i>)	Standard deviation of <i>n</i> , ignoring null values
SUM ([DISTINCT <u>ALL</u>] <i>n</i>)	Sum values of <i>n</i> , ignoring null values
VARIANCE ([DISTINCT <u>ALL</u>] <i>x</i>)	Variance of <i>n</i> , ignoring null values

Review of Using Subqueries

- A subquery is a `SELECT` statement nested in a clause of another `SELECT` statement.
- Syntax:

```
SELECT select_list
FROM   table
WHERE  expr operator
        (SELECT select_list
         FROM   table );
```

- Types of subqueries:

Single-row subquery	Multiple-row subquery
Returns only one row	Returns more than one row
Uses single-row comparison operators	Uses multiple-row comparison operators

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Review of Using Subqueries

You can build powerful statements out of simple ones by using subqueries. Subqueries are useful when a query is based on a search criterion with unknown intermediate values.

You can place the subquery in a number of SQL clauses, including the following:

- WHERE clause
- HAVING clause
- FROM clause

The subquery (inner query) executes once before the main query (outer query). The result of the subquery is used by the main query.

A single-row subquery uses a single-row operator such as `=`, `>`, `<`, `>=`, `<=`, and `<>`. With a multiple-row subquery, you use a multiple-row operator such as `IN`, `ANY`, and `ALL`.

Example: Display details of employees whose salary is equal to the minimum salary.

```
SELECT last_name, salary, job_id
FROM   employees
WHERE  salary = (SELECT MIN(salary)
                 FROM   employees );
```

In the example, the `MIN` group function returns a single value to the outer query.

Note: In this course, you learn how to use multiple-column subqueries. Multiple-column subqueries return more than one column from the inner `SELECT` statement.

Review of Manipulating Data

A data manipulation language (DML) statement is executed when you:

- Add new rows to a table
- Modify existing rows in a table
- Remove existing rows from a table

Function	Description
INSERT	Adds a new row to the table
UPDATE	Modifies existing rows in the table
DELETE	Removes existing rows from the table
MERGE	Updates, inserts, or deletes a row conditionally into/from a table

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Review of Manipulating Data

When you want to add, update, or delete data in the database, you execute a DML statement. A collection of DML statements that form a logical unit of work is called a transaction. You can add new rows to a table by using the INSERT statement. With the following syntax, only one row is inserted at a time.

```
INSERT INTO table [(column [, column...])]
VALUES (value[, value...]);
```

You can use the INSERT statement to add rows to a table where the values are derived from existing tables. In place of the VALUES clause, you use a subquery. The number of columns and their data types in the column list of the INSERT clause must match the number of values and their data types in the subquery.

The UPDATE statement modifies specific rows if you specify the WHERE clause.

```
UPDATE table
SET column = value [, column = value, ...]
[WHERE condition];
```

You can remove existing rows by using the DELETE statement. You can delete specific rows by specifying the WHERE clause in the DELETE statement.

```
DELETE [FROM] table
[WHERE condition];
```

You learn about the MERGE statement in the lesson titled “Manipulating Large Data Sets.”

Lesson Agenda

- Course objectives and course agenda
- The database schema and appendixes used in the course and the available development environment in this course
- Overview of Oracle SQL Developer
- Review of some basic concepts of SQL
- Oracle Database 11g documentation and additional resources

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Oracle Database 11g SQL Documentation

- *Oracle Database New Features Guide 11g Release 1 (11.1)*
- *Oracle Database Reference 11g Release 1 (11.1)*
- *Oracle Database SQL Language Reference 11g Release 1 (11.1)*
- *Oracle Database Concepts 11g Release 1 (11.1)*
- *Oracle Database SQL Developer User's Guide Release 1.2*

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Oracle Database 11g SQL Documentation

Navigate to <http://www.oracle.com/pls/db111/homepage> to access the Oracle Database 11g documentation library.

Additional Resources

For additional information about the new Oracle 11g SQL, refer to the following:

- *Oracle Database 11g: New Features eStudies*
- *Oracle by Example series (OBE): Oracle Database 11g*
 - http://www.oracle.com/technology/obe/11gr1_db/index.htm

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Summary

In this lesson, you should have learned the following:

- The course objectives
- The sample tables used in the course

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Oracle Internal & Oracle Academy
Use Only

Practice I: Overview

This practice covers the following topics:

- Running the SQL Developer online tutorial
- Starting SQL Developer and creating a new database connection and browsing the HR tables
- Executing SQL statements using the SQL Worksheet
- Reviewing the basic concepts of SQL

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Practice I: Overview

In this practice, you use SQL Developer to execute SQL statements to examine data in the HR schema.

Note: All written practices use SQL Developer as the development environment. Although it is recommended that you use SQL Developer, you can also use the SQL*Plus environment that is available in this course.

Practice 1

This is the first of many practices in this course. The solutions (if you require them) can be found in Appendix A. Practices are intended to cover most of the topics that are presented in the corresponding lesson.

Accessing SQL Developer resources

1. Access the SQL Developer home page.
 - a. Access the online SQL Developer Home Page available at:
http://www.oracle.com/technology/products/database/sql_developer/index.html
 - b. Bookmark the page for easier future access.
2. Access the SQL Developer tutorial available online at:
<http://st-curriculum.oracle.com/tutorial/SQLDeveloper/index.htm>
Then review the following sections and associated demos:
 - a. What to Do First
 - b. Working with Database Objects
 - c. Accessing Data

Starting SQL Developer, Creating and Testing a New SQL Developer Database Connection, and Connecting to your **oraxx** account

3. Start up SQL Developer using the Desktop icon.
4. Create a database connection using the following information:
 - a. Connection Name: `myconnection`
 - b. Username: `oraxx` where `xx` is the number of your PC (Ask your instructor to assign you one `ora` account out of the `ora21-ora40` range of accounts.)
 - c. Password: `oraxx`
 - d. Hostname: Enter the host name for your PC.
 - e. Port: `1521`
 - f. SID: `ORCL` (or the value provided to you by the instructor)
5. Test the new connection. If the status is Success, connect to the database using this new connection.
 - a. Click the Test button in the New/Select Database Connection window.
 - b. If the status is Success, click the Connect button.

Browsing the Tables

6. Browse the structure of the `EMPLOYEES` table and display its data.
 - a. Expand the `myconnection` connection by clicking the plus sign next to it.
 - b. Expand the Tables icon by clicking the plus sign next to it.
 - c. Display the structure of the `EMPLOYEES` table.
7. View the data of the `DEPARTMENTS` table.

Practice 1 (continued)

Using the SQL Worksheet

8. Execute some basic `SELECT` statements to query the data in the `EMPLOYEES` table in the SQL Worksheet area. Use both the Execute Statement (or press [F9]) and the Run Script (or press [F5]) icons to execute the `SELECT` statements. Review the results of both methods of executing the `SELECT` statements on the appropriate tabbed pages.
 - a. Write a query to select the last name and salary for any employee whose salary is less than or equal to \$3,000.
 - b. Write a query to display last name, job ID, and commission for all employees who are not entitled to receive a commission.
9. Set your script pathing preference to `D:\labs\SQL2`.
 - a. Select **Tools > Preferences > Database > Worksheet Parameters**.
 - b. Enter the value in the **Select default path to look for scripts** field.
10. Enter the following in the Enter SQL Statement box.

```
SELECT employee_id, first_name, last_name,  
       FROM employees;
```
11. Save the SQL statement to a script file using the **File > Save As** menu item.
 - a. Select **File > Save As**.
 - b. Name the file **intro_test.sql**.
 - c. Place the file under your `D:\labs\labs` folder.
12. Open and run `confidence.sql` from your `D:\Labs\labs` folder, and observe the output.

Quiz

1. What condition do you use to display rows based on a range of values?

2. The character pattern-matching operation is also referred to as:

3. Group functions return one result per row. True/False.

4. Which function converts mixed case or lowercase character strings to uppercase?

5. Which among the following are group functions?
 - a. MAX
 - b. TRUNC
 - c. STDDEV
 - d. MOD
 - e. CONCAT
 - f. SUM_____
6. What is the default sorting order of rows?

7. You can sort by a column that is not in the `SELECT` list. True/False.

8. You cannot specify a column position as the sort expression. True/False.

9. Which statement do you use to modify existing rows in a table?

Practice 1 (continued)

Quiz (continued)

10. What happens if you do not specify the WHERE clause in a DELETE statement?
-
11. Consider a SELECT statement that consists of three query blocks: the outer query and two inner queries. If both the inner queries return single values, what do you call such a SQL statement?
-
12. You can use subqueries only in the WHERE clause but not in the HAVING clause. True/False.
-
13. Observe the following SELECT statement. What happens when there is no employee named Kiran?
- ```
SELECT last_name, job_id
FROM employees
WHERE job_id =
 (SELECT job_id
 FROM employees
 WHERE last_name = 'Kiran');
```
- 
14. The subqueries are processed first by the Oracle server, after which the WHERE or HAVING clause uses the results. True/False.
-



# 1

## Controlling User Access

ORACLE®

Copyright © 2007, Oracle. All rights reserved.

Oracle Internal & Oracle Academy  
Use Only

## Objectives

After completing this lesson, you should be able to do the following:

- Differentiate system privileges from object privileges
- Grant privileges on tables
- Grant roles
- Distinguish between privileges and roles

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Objectives

In this lesson, you learn how to control database access to specific objects and add new users with different levels of access privileges.

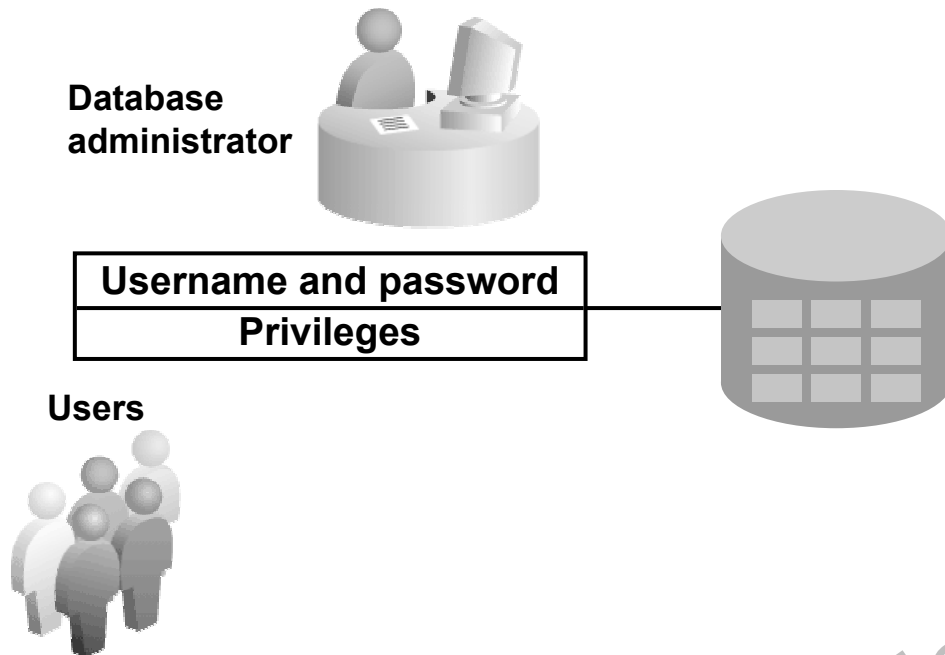
## Lesson Agenda

- System privileges
- Creating a role
- Object privileges
- Revoking object privileges

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Controlling User Access



ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Controlling User Access

In a multiple-user environment, you want to maintain security of the database access and use. With Oracle server database security, you can do the following:

- Control database access.
- Give access to specific objects in the database.
- Confirm given and received privileges with the Oracle data dictionary.

**Database security can be classified into two categories: system security and data security.**

System security covers access and use of the database at the system level, such as the username and password, the disk space allocated to users, and the system operations that users can perform. Database security covers access and use of the database objects and the actions that those users can perform on the objects.



# Privileges

- Database security:
  - System security
  - Data security
- System privileges: Performing a particular action within the database
- Object privileges: Manipulating the content of the database objects
- Schemas: Collection of objects such as tables, views, and sequences

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Privileges

A privilege is the right to execute particular SQL statements. The database administrator (DBA) is a high-level user with the ability to create users and grant users access to the database and its objects. Users require *system privileges* to gain access to the database and *object privileges* to manipulate the content of the objects in the database. Users can also be given the privilege to grant additional privileges to other users or to *roles*, which are named groups of related privileges.

### Schemas

A *schema* is a collection of objects such as tables, views, and sequences. The schema is owned by a database user and has the same name as that user.

A system privilege is the right to perform a particular action, or to perform an action on any schema objects of a particular type. An object privilege provides the user the ability to perform a particular action on a specific schema object.

For more information, see the *Oracle Database 2 Day DBA 11g Release 1 (11.1)* reference manual.

## System Privileges

- More than 100 privileges are available.
- The database administrator has high-level system privileges for tasks such as:
  - Creating new users
  - Removing users
  - Removing tables
  - Backing up tables

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### System Privileges

More than 100 distinct system privileges are available for users and roles. Typically, system privileges are provided by the DBA.

#### Typical DBA Privileges

| System Privilege | Operations Authorized                                                 |
|------------------|-----------------------------------------------------------------------|
| CREATE USER      | Grantee can create other Oracle users.                                |
| DROP USER        | Grantee can drop another user.                                        |
| DROP ANY TABLE   | Grantee can drop a table in any schema.                               |
| BACKUP ANY TABLE | Grantee can back up any table in any schema with the export utility.  |
| SELECT ANY TABLE | Grantee can query tables, views, or materialized views in any schema. |
| CREATE ANY TABLE | Grantee can create tables in any schema.                              |

## Creating Users

The database administrator (DBA) creates users with the `CREATE USER` statement.

```
CREATE USER user
IDENTIFIED BY password;
```

```
CREATE USER demo
IDENTIFIED BY demo;
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Creating Users

The DBA creates the user by executing the `CREATE USER` statement. The user does not have any privileges at this point. The DBA can then grant privileges to that user. These privileges determine what the user can do at the database level.

The slide gives the abridged syntax for creating a user.

In the syntax:

*user*                      Is the name of the user to be created  
*Password*                Specifies that the user must log in with this password

For more information, see the *Oracle Database 11g SQL Reference*.

**Note:** Starting Oracle Database 11g, passwords are case sensitive. Uppercase and lowercase characters are different characters when used in a password.

## User System Privileges

- After a user is created, the DBA can grant specific system privileges to that user.

```
GRANT privilege [, privilege...]
TO user [, user | role, PUBLIC...];
```

- An application developer, for example, may have the following system privileges:
  - CREATE SESSION
  - CREATE TABLE
  - CREATE SEQUENCE
  - CREATE VIEW
  - CREATE PROCEDURE

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Typical User Privileges

After the DBA creates a user, the DBA can assign privileges to that user.

| System Privilege | Operations Authorized                                                 |
|------------------|-----------------------------------------------------------------------|
| CREATE SESSION   | Connect to the database.                                              |
| CREATE TABLE     | Create tables in the user's schema.                                   |
| CREATE SEQUENCE  | Create a sequence in the user's schema.                               |
| CREATE VIEW      | Create a view in the user's schema.                                   |
| CREATE PROCEDURE | Create a stored procedure, function, or package in the user's schema. |

In the syntax:

*privilege*

Is the system privilege to be granted

*user* | *role* | *PUBLIC*

Is the name of the user, the name of the role, or *PUBLIC*  
(which designates that every user is granted the privilege)

**Note:** Current system privileges can be found in the *SESSION\_PRIVS* dictionary view. Data dictionary is a collection of tables and views created and maintained by the Oracle server. They contain information about the database.

## Granting System Privileges

The DBA can grant specific system privileges to a user.

```
GRANT create session, create table,
 create sequence, create view
TO demo;
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Granting System Privileges

The DBA uses the GRANT statement to allocate system privileges to the user. After the user has been granted the privileges, the user can immediately use those privileges.

In the example in the slide, the demo user has been assigned the privileges to create sessions, tables, sequences, and views.

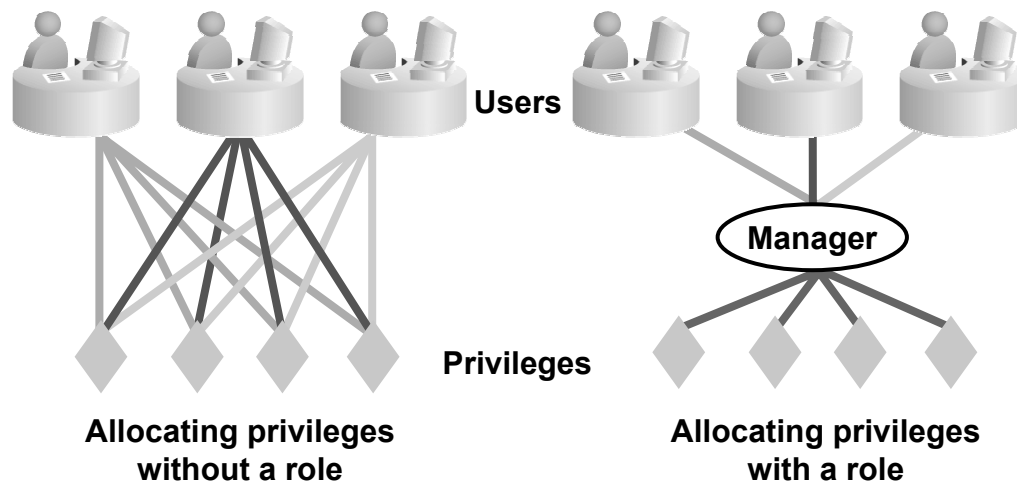
## Lesson Agenda

- System privileges
- Creating a role
- Object privileges
- Revoking object privileges

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## What Is a Role?



ORACLE

Copyright © 2007, Oracle. All rights reserved.

### What Is a Role?

A role is a named group of related privileges that can be granted to the user. This method makes it easier to revoke and maintain privileges.

A user can have access to several roles, and several users can be assigned the same role. Roles are typically created for a database application.

#### Creating and Assigning a Role

First, the DBA must create the role. Then the DBA can assign privileges to the role and assign the role to users.

#### Syntax

```
CREATE ROLE role;
```

In the syntax:

*role* Is the name of the role to be created

After the role is created, the DBA can use the GRANT statement to assign the role to users as well as assign privileges to the role.

## Creating and Granting Privileges to a Role

- Create a role:

```
CREATE ROLE manager;
```

- Grant privileges to a role:

```
GRANT create table, create view
TO manager;
```

- Grant a role to users:

```
GRANT manager TO BELL, KOCHHAR;
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Creating a Role

The example in the slide creates a `manager` role and then enables `manager` to create tables and views. It then grants Bell and Kochhar the role of managers. Now Bell and Kochhar can create tables and views.

If users have multiple roles granted to them, they receive all the privileges associated with all the roles.



## Changing Your Password

- The DBA creates your user account and initializes your password.
- You can change your password by using the ALTER USER statement.

```
ALTER USER demo
IDENTIFIED BY employ;
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Changing Your Password

The DBA creates an account and initializes a password for every user. You can change your password by using the ALTER USER statement.

The slide example shows that the demo user changes his password by using the ALTER USER statement.

#### Syntax

```
ALTER USER user IDENTIFIED BY password;
```

In the syntax:

|                 |                            |
|-----------------|----------------------------|
| <i>user</i>     | Is the name of the user    |
| <i>password</i> | Specifies the new password |

Although this statement can be used to change your password, there are many other options. You must have the ALTER USER privilege to change any other option.

For more information, see the *Oracle Database 11g SQL Reference* manual.

**Note:** SQL\*Plus has a PASSWORD command (PASSW) that can be used to change the password of a user when the user is logged in. This command is not available in SQL Developer.

## Lesson Agenda

- System privileges
- Creating a role
- Object privileges
- Revoking object privileges

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Object Privileges

| Object privilege | Table | View | Sequence |
|------------------|-------|------|----------|
| ALTER            | ✓     |      | ✓        |
| DELETE           | ✓     | ✓    |          |
| INDEX            | ✓     |      |          |
| INSERT           | ✓     | ✓    |          |
| REFERENCES       | ✓     |      |          |
| SELECT           | ✓     | ✓    | ✓        |
| UPDATE           | ✓     | ✓    |          |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Object Privileges

An *object privilege* is a privilege or right to perform a particular action on a specific table, view, sequence, or procedure. Each object has a particular set of grantable privileges. The table in the slide lists the privileges for various objects. Note that the only privileges that apply to a sequence are SELECT and ALTER. UPDATE, REFERENCES, and INSERT can be restricted by specifying a subset of updatable columns.

A SELECT privilege can be restricted by creating a view with a subset of columns and granting the SELECT privilege only on the view. A privilege granted on a synonym is converted to a privilege on the base table referenced by the synonym.

**Note:** With the REFERENCES privilege, you can ensure that other users can create FOREIGN KEY constraints that reference your table.

## Object Privileges

- Object privileges vary from object to object.
- An owner has all the privileges on the object.
- An owner can give specific privileges on that owner's object.

```
GRANT object_priv [(columns)]
ON object
TO {user|role|PUBLIC}
[WITH GRANT OPTION];
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Granting Object Privileges

Different object privileges are available for different types of schema objects. A user automatically has all object privileges for schema objects contained in the user's schema. A user can grant any object privilege on any schema object that the user owns to any other user or role. If the grant includes WITH GRANT OPTION, then the grantee can further grant the object privilege to other users; otherwise, the grantee can use the privilege but cannot grant it to other users.

In the syntax:

|                    |                                                                             |
|--------------------|-----------------------------------------------------------------------------|
| <i>object_priv</i> | Is an object privilege to be granted                                        |
| ALL                | Specifies all object privileges                                             |
| <i>columns</i>     | Specifies the column from a table or view on which privileges are granted   |
| ON <i>object</i>   | Is the object on which the privileges are granted                           |
| TO                 | Identifies to whom the privilege is granted                                 |
| PUBLIC             | Grants object privileges to all users                                       |
| WITH GRANT OPTION  | Enables the grantee to grant the object privileges to other users and roles |

**Note:** In the syntax, *schema* is the same as the owner's name.

## Granting Object Privileges

- Grant query privileges on the EMPLOYEES table:

```
GRANT select
ON employees
TO demo;
```

- Grant privileges to update specific columns to users and roles:

```
GRANT update (department_name, location_id)
ON departments
TO demo, manager;
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Guidelines

- To grant privileges on an object, the object must be in your own schema, or you must have been granted the object privileges WITH GRANT OPTION.
- An object owner can grant any object privilege on the object to any other user or role of the database.
- The owner of an object automatically acquires all object privileges on that object.

The first example in the slide grants the demo user the privilege to query your EMPLOYEES table. The second example grants UPDATE privileges on specific columns in the DEPARTMENTS table to demo and to the manager role.

For example, if your schema is oraxx, and the demo user now wants to use a SELECT statement to obtain data from your EMPLOYEES table, then the syntax he or she must use is:

```
SELECT * FROM oraxx.employees;
```

Alternatively, the demo user can create a synonym for the table and issue a SELECT statement from the synonym:

```
CREATE SYNONYM emp FOR oraxx.employees;
SELECT * FROM emp;
```

**Note:** DBAs generally allocate system privileges; any user who owns an object can grant object privileges.

## Passing On Your Privileges

- Give a user authority to pass along privileges:

```
GRANT select, insert
ON departments
TO demo
WITH GRANT OPTION;
```

- Allow all users on the system to query data from Alice's DEPARTMENTS table:

```
GRANT select
ON alice.departments
TO PUBLIC;
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Passing On Your Privileges

#### WITH GRANT OPTION Keyword

A privilege that is granted with the WITH GRANT OPTION clause can be passed on to other users and roles by the grantee. Object privileges granted with the WITH GRANT OPTION clause are revoked when the grantor's privilege is revoked.

The example in the slide gives the demo user access to your DEPARTMENTS table with the privileges to query the table and add rows to the table. The example also shows that user1 can give others these privileges.

#### PUBLIC Keyword

An owner of a table can grant access to all users by using the PUBLIC keyword.

The second example allows all users on the system to query data from Alice's DEPARTMENTS table.

## Confirming Granted Privileges

| Data Dictionary View | Description                                                    |
|----------------------|----------------------------------------------------------------|
| ROLE_SYS_PRIVS       | System privileges granted to roles                             |
| ROLE_TAB_PRIVS       | Table privileges granted to roles                              |
| USER_ROLE_PRIVS      | Roles accessible by the user                                   |
| USER_SYS_PRIVS       | System privileges granted to the user                          |
| USER_TAB_PRIVS_MADE  | Object privileges granted on the user's objects                |
| USER_TAB_PRIVS_RECD  | Object privileges granted to the user                          |
| USER_COL_PRIVS_MADE  | Object privileges granted on the columns of the user's objects |
| USER_COL_PRIVS_RECD  | Object privileges granted to the user on specific columns      |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Confirming Granted Privileges

If you attempt to perform an unauthorized operation, such as deleting a row from a table for which you do not have the `DELETE` privilege, the Oracle server does not permit the operation to take place.

If you receive the Oracle server error message “Table or view does not exist,” then you have done either of the following:

- Named a table or view that does not exist
- Attempted to perform an operation on a table or view for which you do not have the appropriate privilege

The data dictionary is organized in tables and views and contains information about the database. You can access the data dictionary to view the privileges that you have. The table in the slide describes various data dictionary views.

You learn more about data dictionary views in the lesson titled “Managing Objects with Data Dictionary Views.”

**Note:** The `ALL_TAB_PRIVS_MADE` dictionary view describes all the object grants made by the user or made on the objects owned by the user.

## Lesson Agenda

- System privileges
- Creating a role
- Object privileges
- Revoking object privileges

ORACLE

Copyright © 2007, Oracle. All rights reserved.



## Revoking Object Privileges

- You use the `REVOKE` statement to revoke privileges granted to other users.
- Privileges granted to others through the `WITH GRANT OPTION` clause are also revoked.

```
REVOKE {privilege [, privilege...] | ALL}
ON object
FROM {user[, user...] | role | PUBLIC}
[CASCADE CONSTRAINTS];
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Revoking Object Privileges

You can remove privileges granted to other users by using the `REVOKE` statement. When you use the `REVOKE` statement, the privileges that you specify are revoked from the users you name and from any other users to whom those privileges were granted by the revoked user.

In the syntax:

|                      |                                                                                                                                                           |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>CASCADE</code> | Is required to remove any referential integrity constraints made to the <code>CONSTRAINTS</code> object by means of the <code>REFERENCES</code> privilege |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|

For more information, see the *Oracle Database 11g SQL Reference*.

**Note:** If a user were to leave the company and you revoke his or her privileges, you must regrant any privileges that this user may have granted to other users. If you drop the user account without revoking privileges from it, then the system privileges granted by this user to other users are not affected by this action.

## Revoking Object Privileges

Revoke the SELECT and INSERT privileges given to the demo user on the DEPARTMENTS table.

```
REVOKE select, insert
ON departments
FROM demo;
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Revoking Object Privileges (continued)

The example in the slide revokes SELECT and INSERT privileges given to the demo user on the DEPARTMENTS table.

**Note:** If a user is granted a privilege with the WITH GRANT OPTION clause, that user can also grant the privilege with the WITH GRANT OPTION clause, so that a long chain of grantees is possible, but no circular grants (granting to a grant ancestor) are permitted. If the owner revokes a privilege from a user who granted the privilege to other users, then the revoking cascades to all the privileges granted.

For example, if user A grants a SELECT privilege on a table to user B including the WITH GRANT OPTION clause, user B can grant to user C the SELECT privilege with the WITH GRANT OPTION clause as well, and user C can then grant to user D the SELECT privilege. If user A revokes privileges from user B, then the privileges granted to users C and D are also revoked.

## Summary

In this lesson, you should have learned about statements that control access to the database and database objects.

| Statement   | Action                                                          |
|-------------|-----------------------------------------------------------------|
| CREATE USER | Creates a user (usually performed by a DBA)                     |
| GRANT       | Gives other users privileges to access the objects              |
| CREATE ROLE | Creates a collection of privileges (usually performed by a DBA) |
| ALTER USER  | Changes a user's password                                       |
| REVOKE      | Removes privileges on an object from users                      |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Summary

DBAs establish initial database security for users by assigning privileges to the users.

- The DBA creates users who must have a password. The DBA is also responsible for establishing the initial system privileges for a user.
- After the user has created an object, the user can pass along any of the available object privileges to other users or to all users by using the GRANT statement.
- A DBA can create roles by using the CREATE ROLE statement to pass along a collection of system or object privileges to multiple users. Roles make granting and revoking privileges easier to maintain.
- Users can change their passwords by using the ALTER USER statement.
- You can remove privileges from users by using the REVOKE statement.
- With data dictionary views, users can view the privileges granted to them and those that are granted on their objects.

## Practice 1: Overview

This practice covers the following topics:

- Granting other users privileges to your table
- Modifying another user's table through the privileges granted to you
- Creating a synonym

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Practice 1: Overview

Team up with other students for this exercise about controlling access to database objects.

## Practice 1

To complete question 8 and the subsequent ones, you need to connect to the database using SQL Developer. If you are already not connected, perform the following:

- A. Click the SQL Developer desktop icon.
  - B. In the Connections Navigator, use the *oraxx* account and the corresponding password provided by your instructor to log on to the database.
- 
1. What privilege should a user be given to log on to the Oracle server? Is this a system privilege or an object privilege?  
\_\_\_\_\_
  2. What privilege should a user be given to create tables?  
\_\_\_\_\_
  3. If you create a table, who can pass along privileges to other users on your table?  
\_\_\_\_\_
  4. You are the DBA. You are creating many users who require the same system privileges. What should you use to make your job easier?  
\_\_\_\_\_
  5. What command do you use to change your password?  
\_\_\_\_\_
  6. User1 is the owner of the EMP table and grants the DELETE privilege to User2 by using the WITH GRANT OPTION clause. User2 then grants the DELETE privilege on EMP to User3. User1 now finds that User3 has the privilege and revokes it from User2. Which user can now delete from the EMP table?  
\_\_\_\_\_
  7. You want to grant SCOTT the privilege to update data in the DEPARTMENTS table. You also want to enable SCOTT to grant this privilege to other users. What command do you use?  
\_\_\_\_\_
  8. Grant another user query privilege on your table. Then, verify whether that user can utilize the privilege.  
**Note:** For this exercise, team up with another group. For example, if you are user ora21, team up with another user ora22.
    - a. Grant another user privilege to view records in your REGIONS table. Include an option for this user to further grant this privilege to other users.
    - b. Have the user query your REGIONS table.
    - c. Have the user pass on the query privilege to a third user (for example, ora23).
    - d. Take back the privilege from the user who performs step b.**Note:** Each team can run exercises 9 and 10 independently.
  9. Grant another user query and data manipulation privileges on your COUNTRIES table. Make sure that the user cannot pass on these privileges to other users.
  10. Take back the privileges on the COUNTRIES table granted to another user.  
**Note:** For exercises 11 through 17, team up with another group
  11. Grant another user access to your DEPARTMENTS table. Have the user grant you query access to his or her DEPARTMENTS table.
  12. Query all the rows in your DEPARTMENTS table.

## Practice 1 (continued)

|   | DEPARTMENT_ID | DEPARTMENT_NAME | MANAGER_ID | LOCATION_ID |
|---|---------------|-----------------|------------|-------------|
| 1 | 10            | Administration  | 200        | 1700        |
| 2 | 20            | Marketing       | 201        | 1800        |
| 3 | 50            | Shipping        | 124        | 1500        |
| 4 | 60            | IT              | 103        | 1400        |
| 5 | 80            | Sales           | 149        | 2500        |
| 6 | 90            | Executive       | 100        | 1700        |
| 7 | 110           | Accounting      | 205        | 1700        |
| 8 | 190           | Contracting     | (null)     | 1700        |

13. Add a new row to your DEPARTMENTS table. Team 1 should add Education as department number 500. Team 2 should add Human Resources as department number 510. Query the other team's table.

14. Create a synonym for the other team's DEPARTMENTS table.

15. Query all the rows in the other team's DEPARTMENTS table by using your synonym.

*Team 1 SELECT statement results:*

|   | DEPARTMENT_ID | DEPARTMENT_NAME | MANAGER_ID | LOCATION_ID |
|---|---------------|-----------------|------------|-------------|
| 1 | 10            | Administration  | 200        | 1700        |
| 2 | 20            | Marketing       | 201        | 1800        |
| 3 | 50            | Shipping        | 124        | 1500        |
| 4 | 60            | IT              | 103        | 1400        |
| 5 | 80            | Sales           | 149        | 2500        |
| 6 | 90            | Executive       | 100        | 1700        |
| 7 | 110           | Accounting      | 205        | 1700        |
| 8 | 190           | Contracting     | (null)     | 1700        |
| 9 | 510           | Human Resources | (null)     | (null)      |

*Team 2 SELECT statement results:*

|   | DEPARTMENT_ID | DEPARTMENT_NAME | MANAGER_ID | LOCATION_ID |
|---|---------------|-----------------|------------|-------------|
| 1 | 10            | Administration  | 200        | 1700        |
| 2 | 20            | Marketing       | 201        | 1800        |
| 3 | 50            | Shipping        | 124        | 1500        |
| 4 | 60            | IT              | 103        | 1400        |
| 5 | 80            | Sales           | 149        | 2500        |
| 6 | 90            | Executive       | 100        | 1700        |
| 7 | 110           | Accounting      | 205        | 1700        |
| 8 | 190           | Contracting     | (null)     | 1700        |
| 9 | 500           | Education       | (null)     | (null)      |

**Practice 1 (continued)**

16. Revoke the `SELECT` privilege from the other team.
17. Remove the row you inserted into the `DEPARTMENTS` table in step 13 and save the changes.

Oracle Internal & Oracle Academy  
Use Only





# 2

## Managing Schema Objects

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Oracle Internal & Oracle Academy  
Use Only

## Objectives

After completing this lesson, you should be able to do the following:

- Add constraints
- Create indexes
- Create indexes using the `CREATE TABLE` statement
- Create function-based indexes
- Drop columns and set columns as `UNUSED`
- Perform `FLASHBACK` operations
- Create and use external tables

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Objectives

This lesson contains information about creating indexes and constraints, and altering existing objects. You also learn about external tables, and the provision to name the index at the time of creating a primary key constraint.

## Lesson Agenda

- Using the ALTER TABLE statement to add, modify, and drop a column
- Managing constraints
  - Adding and dropping a constraint
  - Deferring constraints
  - Enabling and disabling a constraint
- Creating indexes
  - Using the CREATE TABLE statement
  - Creating function-based indexes
  - Removing an index
- Performing flashback operations
- Creating and using external tables

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Oracle Internal & Oracle Academy  
Use Only

## ALTER TABLE Statement

Use the ALTER TABLE statement to:

- Add a new column
- Modify an existing column
- Define a default value for the new column
- Drop a column

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### ALTER TABLE Statement

After you create a table, you may need to change the table structure because you omitted a column, your column definition needs to be changed, or you need to remove columns. You can do this by using the ALTER TABLE statement.

## ALTER TABLE Statement

Use the ALTER TABLE statement to add, modify, or drop columns:

```
ALTER TABLE table
ADD (column datatype [DEFAULT expr]
 [, column datatype]...);
```

```
ALTER TABLE table
MODIFY (column datatype [DEFAULT expr]
 [, column datatype]...);
```

```
ALTER TABLE table
DROP (column);
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### ALTER TABLE Statement (continued)

You can add columns to a table, modify columns, and drop columns from a table by using the ALTER TABLE statement.

In the syntax:

|                     |                                           |
|---------------------|-------------------------------------------|
| <i>table</i>        | Is the name of the table                  |
| ADD   MODIFY   DROP | Is the type of modification               |
| <i>column</i>       | Is the name of the column                 |
| <i>datatype</i>     | Is the data type and length of the column |
| DEFAULT <i>expr</i> | Specifies the default value for a column  |

## Adding a Column

- You use the `ADD` clause to add columns:

```
ALTER TABLE dept80
ADD (job_id VARCHAR2(9));
```

```
ALTER TABLE dept80 succeeded.
```

- The new column becomes the last column:

|   | EMPLOYEE_ID | LAST_NAME | ANNSAL | HIRE_DATE | JOB_ID |
|---|-------------|-----------|--------|-----------|--------|
| 1 | 149         | Zlotkey   | 10500  | 29-JAN-00 | (null) |
| 2 | 174         | Abel      | 11000  | 11-MAY-96 | (null) |
| 3 | 176         | Taylor    | 8600   | 24-MAR-98 | (null) |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Guidelines for Adding a Column

- You can add or modify columns.
- You cannot specify where the column is to appear. The new column becomes the last column.

The example in the slide adds a column named `JOB_ID` to the `DEPT80` table. The `JOB_ID` column becomes the last column in the table.

**Note:** If a table already contains rows when a column is added, then the new column is initially null or takes the default value for all the rows. You can add a mandatory `NOT NULL` column to a table that contains data in the other columns only if you specify a default value. You can add a `NOT NULL` column to an empty table without the default value.

## Modifying a Column

- You can change a column's data type, size, and default value.

```
ALTER TABLE dept80
MODIFY (last_name VARCHAR2(30));
```

```
ALTER TABLE dept80 succeeded.
```

- A change to the default value affects only subsequent insertions to the table.

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Modifying a Column

You can modify a column definition by using the ALTER TABLE statement with the MODIFY clause. Column modification can include changes to a column's data type, size, and default value.

#### Guidelines

- You can increase the width or precision of a numeric column.
- You can increase the width of character columns.
- You can decrease the width of a column if:
  - The column contains only null values
  - The table has no rows
  - The decrease in column width is not less than the existing values in that column
- You can change the data type if the column contains only null values. The exception to this is CHAR-to-VARCHAR2 conversions, which can be done with data in the columns.
- You can convert a CHAR column to the VARCHAR2 data type or convert a VARCHAR2 column to the CHAR data type only if the column contains null values or if you do not change the size.
- A change to the default value of a column affects only subsequent insertions to the table.

## Dropping a Column

Use the `DROP COLUMN` clause to drop columns you no longer need from the table:

```
ALTER TABLE dept80
DROP COLUMN job_id;
```

```
ALTER TABLE dept80 succeeded.
```

|   | EMPLOYEE_ID | LAST_NAME | ANNSAL | HIRE_DATE |
|---|-------------|-----------|--------|-----------|
| 1 | 149         | Zlotkey   | 10500  | 29-JAN-00 |
| 2 | 174         | Abel      | 11000  | 11-MAY-96 |
| 3 | 176         | Taylor    | 8600   | 24-MAR-98 |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Dropping a Column

You can drop a column from a table by using the `ALTER TABLE` statement with the `DROP COLUMN` clause.

#### Guidelines

- The column may or may not contain data.
- Using the `ALTER TABLE DROP COLUMN` statement, only one column can be dropped at a time.
- The table must have at least one column remaining in it after it is altered.
- After a column is dropped, it cannot be recovered.
- A column cannot be dropped if it is part of a constraint or part of an index key unless the cascade option is added.
- Dropping a column can take a while if the column has a large number of values. In this case, it may be better to set it to be unused and drop it when there are fewer users on the system to avoid extended locks.

**Note:** Certain columns can never be dropped, such as columns that form part of the partitioning key of a partitioned table or columns that form part of the primary key of an index-organized table.



## SET UNUSED Option

- You use the SET UNUSED option to mark one or more columns as unused.
- You use the DROP UNUSED COLUMNS option to remove the columns that are marked as unused.

```
ALTER TABLE <table_name>
SET UNUSED(<column_name>);
```

OR

```
ALTER TABLE <table_name>
SET UNUSED COLUMN <column_name>;
```

```
ALTER TABLE <table_name>
DROP UNUSED COLUMNS;
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### SET UNUSED Option

The SET UNUSED option marks one or more columns as unused so that they can be dropped when the demand on system resources is lower. Specifying this clause does not actually remove the target columns from each row in the table (that is, it does not restore the disk space used by these columns). Therefore, the response time is faster than if you executed the DROP clause. Unused columns are treated as if they were dropped, even though their column data remains in the table's rows. After a column has been marked as unused, you have no access to that column. A SELECT \* query will not retrieve data from unused columns. In addition, the names and types of columns marked unused will not be displayed during a DESCRIBE statement, and you can add to the table a new column with the same name as an unused column. The SET UNUSED information is stored in the USER\_UNUSED\_COL\_TABS dictionary view.

**Note:** The guidelines for setting a column to be UNUSED are similar to those for dropping a column.

### **DROP UNUSED COLUMNS Option**

DROP UNUSED COLUMNS removes from the table all columns currently marked as unused. You can use this statement when you want to reclaim the extra disk space from unused columns in the table. If the table contains no unused columns, the statement returns with no errors.

```
ALTER TABLE dept80
SET UNUSED (last_name);
```

```
ALTER TABLE succeeded.
```

```
ALTER TABLE dept80
DROP UNUSED COLUMNS;
```

```
ALTER TABLE succeeded.
```

Oracle Internal & Oracle Academy  
Use Only

## Lesson Agenda

- Using the `ALTER TABLE` statement to add, modify, and drop a column
- Managing constraints
  - Adding and dropping a constraint
  - Deferring constraints
  - Enabling and disabling a constraint
- Creating indexes
  - Using the `CREATE TABLE` statement
  - Creating function-based indexes
  - Removing an index
- Performing flashback operations
- Creating and using external tables

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Oracle Internal & Oracle Academy  
Use Only

## Adding a Constraint Syntax

Use the ALTER TABLE statement to:

- Add or drop a constraint, but not modify its structure
- Enable or disable constraints
- Add a NOT NULL constraint by using the MODIFY clause

```
ALTER TABLE <table_name>
ADD [CONSTRAINT <constraint_name>]
type (<column_name>);
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Adding a Constraint

You can add a constraint for existing tables by using the ALTER TABLE statement with the ADD clause.

In the syntax:

|                   |                                                      |
|-------------------|------------------------------------------------------|
| <i>table</i>      | Is the name of the table                             |
| <i>constraint</i> | Is the name of the constraint                        |
| <i>type</i>       | Is the constraint type                               |
| <i>column</i>     | Is the name of the column affected by the constraint |

The constraint name syntax is optional, although recommended. If you do not name your constraints, the system generates constraint names.

#### Guidelines

- You can add, drop, enable, or disable a constraint, but you cannot modify its structure.
- You can add a NOT NULL constraint to an existing column by using the MODIFY clause of the ALTER TABLE statement.

**Note:** You can define a NOT NULL column only if the table is empty or if the column has a value for every row.

## Adding a Constraint

Add a FOREIGN KEY constraint to the EMP2 table indicating that a manager must already exist as a valid employee in the EMP2 table.

```
ALTER TABLE emp2
MODIFY employee_id Primary Key;
```

```
ALTER TABLE emp2 succeeded.
```

```
ALTER TABLE emp2
ADD CONSTRAINT emp_mgr_fk
FOREIGN KEY(manager_id)
REFERENCES emp2(employee_id);
```

```
ALTER TABLE succeeded.
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Adding a Constraint (continued)

The first example in the slide modifies the EMP2 table to add a PRIMARY KEY constraint on the EMPLOYEE\_ID column. Note that because no constraint name is provided, the constraint is automatically named by the Oracle server. The second example in the slide creates a FOREIGN KEY constraint on the EMP2 table. The constraint ensures that a manager exists as a valid employee in the EMP2 table.

## ON DELETE CASCADE

Delete child rows when a parent key is deleted:

```
ALTER TABLE Emp2 ADD CONSTRAINT emp_dt_fk
FOREIGN KEY (Department_id)
REFERENCES departments(department_id) ON DELETE CASCADE;
```

```
ALTER TABLE Emp2 succeeded.
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### ON DELETE CASCADE

The ON DELETE CASCADE action allows parent key data that is referenced from the child table to be deleted, but not updated. When data in the parent key is deleted, all rows in the child table that depend on the deleted parent key values are also deleted. To specify this referential action, include the ON DELETE CASCADE option in the definition of the FOREIGN KEY constraint.

# Deferring Constraints

Constraints can have the following attributes:

- DEFERRABLE or NOT DEFERRABLE
- INITIALLY DEFERRED or INITIALLY IMMEDIATE

```
ALTER TABLE dept2
ADD CONSTRAINT dept2_id_pk
PRIMARY KEY (department_id)
DEFERRABLE INITIALLY DEFERRED
```

Deferring constraint on creation

```
SET CONSTRAINTS dept2_id_pk IMMEDIATE
```

Changing a specific constraint attribute

```
ALTER SESSION
SET CONSTRAINTS= IMMEDIATE
```

Changing all constraints for a session

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Deferring Constraints

You can defer checking constraints for validity until the end of the transaction. A constraint is deferred if the system checks that it is satisfied only on commit. If a deferred constraint is violated, then commit causes the transaction to roll back. If a constraint is immediate (not deferred), then it is checked at the end of each statement. If it is violated, the statement is rolled back immediately. If a constraint causes an action (for example, DELETE CASCADE), that action is always taken as part of the statement that caused it, whether the constraint is deferred or immediate. Use the SET CONSTRAINTS statement to specify, for a particular transaction, whether a deferrable constraint is checked following each DML statement or when the transaction is committed. To create deferrable constraints, you must create a nonunique index for that constraint.

You can define constraints as either deferrable or not deferrable, and either initially deferred or initially immediate. These attributes can be different for each constraint.

**Usage scenario:** Company policy dictates that department number 40 should be changed to 45. Changing the DEPARTMENT\_ID column affects employees assigned to this department. Therefore, you make the primary key and foreign keys deferrable and initially deferred. You update both department and employee information, and at the time of commit, all rows are validated.

## Difference Between INITIALLY DEFERRED and INITIALLY IMMEDIATE

|                     |                                                             |
|---------------------|-------------------------------------------------------------|
| INITIALLY DEFERRED  | Waits to check the constraint until the transaction ends    |
| INITIALLY IMMEDIATE | Checks the constraint at the end of the statement execution |

```
CREATE TABLE emp_new_sal (salary NUMBER
CONSTRAINT sal_ck
CHECK (salary > 100)
DEFERRABLE INITIALLY IMMEDIATE,
bonus NUMBER
CONSTRAINT bonus_ck
CHECK (bonus > 0)
DEFERRABLE INITIALLY DEFERRED);
```

create table succeeded.

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Difference Between INITIALLY DEFERRED and INITIALLY IMMEDIATE

A constraint that is defined as deferrable can be specified as either INITIALLY DEFERRED or INITIALLY IMMEDIATE. The INITIALLY IMMEDIATE clause is the default.

In the slide example:

- The sal\_ck constraint is created as DEFERRABLE INITIALLY IMMEDIATE
- The bonus\_ck constraint is created as DEFERRABLE INITIALLY DEFERRED

After creating the emp\_new\_sal table as shown in the slide, you attempt to insert values into the table and observe the results. When both the sal\_ck and bonus\_ck constraints are satisfied, the rows are inserted without an error.

**Example 1:** Insert a row that violates sal\_ck. In the CREATE TABLE statement, sal\_ck is specified as an initially immediate constraint. This means that the constraint is verified immediately after the INSERT statement and you observe an error.

```
INSERT INTO emp_new_sal VALUES (90,5);
```

```
SQL Error: ORA-02290: check constraint (ORA21.SAL_CK) violated
02290. 00000 - "check constraint (%s.%s) violated"
```

**Example 2:** Insert a row that violates bonus\_ck. In the CREATE TABLE statement, bonus\_ck is specified as deferrable and also initially deferred. Therefore, the constraint is not verified until you COMMIT or set the constraint state back to immediate.



## Difference Between INITIALLY DEFERRED and INITIALLY IMMEDIATE (continued)

```
INSERT INTO emp_new_sal VALUES(110, -1);
```

```
1 rows inserted
```

The row insertion is successful. But, you observe an error when you commit the transaction.

```
COMMIT;
```

```
SQL Error: ORA-02091: transaction rolled back
ORA-02290: check constraint (ORA21.BONUS_CK) violated
02091. 00000 - "transaction rolled back"
```

The commit failed due to constraint violation. Therefore, at this point, the transaction is rolled back by the database.

**Example 3:** Set the DEFERRED status to all constraints that can be deferred. Note that you can also set the DEFERRED status to a single constraint if required.

```
SET CONSTRAINTS ALL DEFERRED;
```

```
SET CONSTRAINTS succeeded.
```

Now, if you attempt to insert a row that violates the sal\_ck constraint, the statement is executed successfully.

```
INSERT INTO emp_new_sal VALUES(90,5);
```

```
1 rows inserted
```

But, you observe an error when you commit the transaction. The transaction fails and is rolled back. This is because both the constraints are checked upon COMMIT.

```
COMMIT;
```

```
SQL Error: ORA-02091: transaction rolled back
ORA-02290: check constraint (ORA21.SAL_CK) violated
02091. 00000 - "transaction rolled back"
```

**Example 4:** Set the IMMEDIATE status to both the constraints that were set as DEFERRED in the previous example.

```
SET CONSTRAINTS ALL IMMEDIATE;
```

```
SET CONSTRAINTS succeeded.
```

You observe an error if you attempt to insert a row that violates either sal\_ck or bonus\_ck.

```
INSERT INTO emp_new_sal VALUES(110, -1);
```

```
SQL Error: ORA-02290: check constraint (ORA21.BONUS_CK) violated
02290. 00000 - "check constraint (%s.%s) violated"
```

**Note:** If you create a table without specifying constraint deferability, then the constraint is checked immediately at the end of each statement. For example, with the CREATE TABLE statement of the newemp\_details table, if you do not specify the newemp\_det\_pk constraint deferability, then the constraint is checked immediately.

```
CREATE TABLE newemp_details(emp_id NUMBER, emp_name
VARCHAR2(20),
CONSTRAINT newemp_det_pk PRIMARY KEY(emp_id));
```

When you attempt to defer the newemp\_det\_pk constraint that is not deferrable, you observe the following error:

```
SET CONSTRAINT newemp_det_pk DEFERRED;
```

```
SQL Error: ORA-02447: cannot defer a constraint that is not deferrable
```

## Dropping a Constraint

- Remove the manager constraint from the EMP2 table:

```
ALTER TABLE emp2
DROP CONSTRAINT emp_mgr_fk;
```

```
ALTER TABLE Emp2 succeeded.
```

- Remove the PRIMARY KEY constraint on the DEPT2 table and drop the associated FOREIGN KEY constraint on the EMP2.DEPARTMENT\_ID column:

```
ALTER TABLE dept2
DROP PRIMARY KEY CASCADE;
```

```
ALTER TABLE dept2 succeeded.
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Dropping a Constraint

To drop a constraint, you can identify the constraint name from the USER\_CONSTRAINTS and USER\_CONS\_COLUMNS data dictionary views. Then use the ALTER TABLE statement with the DROP clause. The CASCADE option of the DROP clause causes any dependent constraints also to be dropped.

#### Syntax

```
ALTER TABLE table
DROP PRIMARY KEY | UNIQUE (column) |
CONSTRAINT constraint [CASCADE];
```

In the syntax:

|                   |                                                      |
|-------------------|------------------------------------------------------|
| <i>table</i>      | Is the name of the table                             |
| <i>column</i>     | Is the name of the column affected by the constraint |
| <i>constraint</i> | Is the name of the constraint                        |

When you drop an integrity constraint, that constraint is no longer enforced by the Oracle server and is no longer available in the data dictionary.

## Disabling Constraints

- Execute the `DISABLE` clause of the `ALTER TABLE` statement to deactivate an integrity constraint.
- Apply the `CASCADE` option to disable dependent integrity constraints.

```
ALTER TABLE emp2
DISABLE CONSTRAINT emp_dt_fk;
```

```
ALTER TABLE Emp2 succeeded.
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Disabling a Constraint

You can disable a constraint without dropping it or re-creating it by using the `ALTER TABLE` statement with the `DISABLE` clause.

#### Syntax

```
ALTER TABLE table
DISABLE CONSTRAINT constraint [CASCADE];
```

In the syntax:

*table*                    Is the name of the table  
*constraint*            Is the name of the constraint

#### Guidelines

- You can use the `DISABLE` clause in both the `CREATE TABLE` statement and the `ALTER TABLE` statement.
- The `CASCADE` clause disables dependent integrity constraints.
- Disabling a unique or primary key constraint removes the unique index.

## Enabling Constraints

- Activate an integrity constraint currently disabled in the table definition by using the `ENABLE` clause.

```
ALTER TABLE emp2
ENABLE CONSTRAINT emp_dt_fk;
```

```
ALTER TABLE Emp2 succeeded.
```

- A `UNIQUE` index is automatically created if you enable a `UNIQUE` key or a `PRIMARY KEY` constraint.

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Enabling a Constraint

You can enable a constraint without dropping it or re-creating it by using the `ALTER TABLE` statement with the `ENABLE` clause.

#### Syntax

```
ALTER TABLE table
ENABLE CONSTRAINT constraint;
```

In the syntax:

*table*                    Is the name of the table  
*constraint*            Is the name of the constraint

#### Guidelines

- If you enable a constraint, that constraint applies to all the data in the table. All the data in the table must comply with the constraint.
- If you enable a `UNIQUE` key or a `PRIMARY KEY` constraint, a `UNIQUE` or `PRIMARY KEY` index is created automatically. If an index already exists, then it can be used by these keys.
- You can use the `ENABLE` clause in both the `CREATE TABLE` statement and the `ALTER TABLE` statement.

## Enabling a Constraint (continued)

### Guidelines (continued)

- Enabling a primary key constraint that was disabled with the CASCADE option does not enable any foreign keys that are dependent on the primary key.
- To enable a UNIQUE or PRIMARY KEY constraint, you must have the privileges necessary to create an index on the table.

Oracle Internal & Oracle Academy  
Use Only

## Cascading Constraints

- The CASCADE CONSTRAINTS clause is used along with the DROP COLUMN clause.
- The CASCADE CONSTRAINTS clause drops all referential integrity constraints that refer to the primary and unique keys defined on the dropped columns.
- The CASCADE CONSTRAINTS clause also drops all multicolumn constraints defined on the dropped columns.

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Cascading Constraints

This statement illustrates the usage of the CASCADE CONSTRAINTS clause. Assume that the TEST1 table is created as follows:

```
CREATE TABLE test1 (
 col1_pk NUMBER PRIMARY KEY,
 col2_fk NUMBER,
 col1 NUMBER,
 col2 NUMBER,
 CONSTRAINT fk_constraint FOREIGN KEY (col2_fk) REFERENCES
 test1,
 CONSTRAINT ck1 CHECK (col1_pk > 0 and col1 > 0),
 CONSTRAINT ck2 CHECK (col2_fk > 0));
```

An error is returned for the following statements:

```
ALTER TABLE test1 DROP (col1_pk); —col1_pk is a parent key.
ALTER TABLE test1 DROP (col1); —col1 is referenced by the multicolumn
constraint, ck1.
```

## Cascading Constraints

Example:

```
ALTER TABLE emp2
DROP COLUMN employee_id CASCADE CONSTRAINTS;
```

```
ALTER TABLE Emp2 succeeded.
```

```
ALTER TABLE test1
DROP (col1_pk, col2_fk, col1) CASCADE CONSTRAINTS;
```

```
ALTER TABLE test1 succeeded.
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Cascading Constraints (continued)

Submitting the following statement drops the EMPLOYEE\_ID column, the primary key constraint, and any foreign key constraints referencing the primary key constraint for the EMP2 table:

```
ALTER TABLE emp2 DROP COLUMN employee_id CASCADE CONSTRAINTS;
```

If all columns referenced by the constraints defined on the dropped columns are also dropped, then CASCADE CONSTRAINTS is not required. For example, assuming that no other referential constraints from other tables refer to the COL1\_PK column, it is valid to submit the following statement without the CASCADE CONSTRAINTS clause for the TEST1 table created on the previous page:

```
ALTER TABLE test1 DROP (col1_pk, col2_fk, col1);
```

## Renaming Table Columns and Constraints

Use the RENAME COLUMN clause of the ALTER TABLE statement to rename table columns.

**a**

```
ALTER TABLE marketing RENAME COLUMN team_id
TO id;
```

```
ALTER TABLE marketing succeeded.
```

Use the RENAME CONSTRAINT clause of the ALTER TABLE statement to rename any existing constraint for a table.

**b**

```
ALTER TABLE marketing RENAME CONSTRAINT mktg_pk
TO new_mktg_pk;
```

```
ALTER TABLE marketing succeeded.
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Renaming Table Columns and Constraints

When you rename a table column, the new name must not conflict with the name of any existing column in the table. You cannot use any other clauses in conjunction with the RENAME COLUMN clause.

The slide examples use the marketing table with the primary key mktg\_pk defined on the id column.

```
CREATE TABLE marketing (team_id NUMBER(10),
 target VARCHAR2(50),
 CONSTRAINT mktg_pk PRIMARY KEY(team_id));
```

```
CREATE TABLE succeeded.
```

Example **a** shows that the id column of the marketing table is renamed mktg\_id. Example **b** shows that mktg\_pk is renamed new\_mktg\_pk.

When you rename any existing constraint for a table, the new name must not conflict with any of your existing constraint names. You can use the RENAME CONSTRAINT clause to rename system-generated constraint names.



## Lesson Agenda

- Using the ALTER TABLE statement to add, modify, and drop a column
- Managing constraints
  - Adding and dropping a constraint
  - Deferring constraints
  - Enabling and disabling a constraint
- Creating indexes
  - Using the CREATE TABLE statement
  - Creating function-based indexes
  - Removing an index
- Performing flashback operations
- Creating and using external tables

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Oracle Internal & Oracle Academy  
Use Only

## Overview of Indexes

Indexes are created:

- Automatically
  - PRIMARY KEY creation
  - UNIQUE KEY creation
- Manually
  - The CREATE INDEX statement
  - The CREATE TABLE statement

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Overview of Indexes

Two types of indexes can be created. One type is a unique index. The Oracle server automatically creates a unique index when you define a column or group of columns in a table to have a PRIMARY KEY or a UNIQUE key constraint. The name of the index is the name given to the constraint.

The other type of index is a nonunique index, which a user can create. For example, you can create an index for a FOREIGN KEY column to be used in joins to improve retrieval speed.

You can create an index on one or more columns by issuing the CREATE INDEX statement.

For more information, see *Oracle Database 11g SQL Reference*.

**Note:** You can manually create a unique index, but it is recommended that you create a unique constraint, which implicitly creates a unique index.

## CREATE INDEX with the CREATE TABLE Statement

```
CREATE TABLE NEW_EMP
(employee_id NUMBER(6)
 PRIMARY KEY USING INDEX
 (CREATE INDEX emp_id_idx ON
 NEW_EMP(employee_id)),
first_name VARCHAR2(20),
last_name VARCHAR2(25));
```

CREATE TABLE succeeded.

```
SELECT INDEX_NAME, TABLE_NAME
FROM USER_INDEXES
WHERE TABLE_NAME = 'NEW_EMP';
```

|   | INDEX_NAME | TABLE_NAME |
|---|------------|------------|
| 1 | EMP_ID_IDX | NEW_EMP    |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## CREATE INDEX with the CREATE TABLE Statement

In the example in the slide, the CREATE INDEX clause is used with the CREATE TABLE statement to create a primary key index explicitly. You can name your indexes at the time of primary key creation to be different from the name of the PRIMARY KEY constraint.

You can query the USER\_INDEXES data dictionary view for information about your indexes.

**Note:** You learn more about USER\_INDEXES in the lesson titled “Managing Objects with Data Dictionary Views.”

The following example illustrates the database behavior if the index is not explicitly named:

```
CREATE TABLE EMP_UNNAMED INDEX
(employee_id NUMBER(6) PRIMARY KEY ,
first_name VARCHAR2(20),
last_name VARCHAR2(25));
```

CREATE TABLE succeeded.

```
SELECT INDEX_NAME, TABLE_NAME
FROM USER_INDEXES
WHERE TABLE_NAME = 'EMP_UNNAMED_INDEX';
```

|   | INDEX_NAME   | TABLE_NAME        |
|---|--------------|-------------------|
| 1 | SYS_C0017294 | EMP_UNNAMED_INDEX |

## CREATE INDEX with the CREATE TABLE Statement (continued)

Observe that the Oracle server gives a generic name to the index that is created for the PRIMARY KEY column.

You can also use an existing index for your PRIMARY KEY column—for example, when you are expecting a large data load and want to speed up the operation. You may want to disable the constraints while performing the load and then enable them, in which case having a unique index on the primary key will still cause the data to be verified during the load. So you can first create a nonunique index on the column designated as PRIMARY KEY, and then create the PRIMARY KEY column and specify that it should use the existing index. The following examples illustrate this process:

### Step 1: Create the table:

```
CREATE TABLE NEW_EMP2
 (employee_id NUMBER(6),
 first_name VARCHAR2(20),
 last_name VARCHAR2(25)
);
```

### Step 2: Create the index:

```
CREATE INDEX emp_id_idx2 ON
 new_emp2(employee_id);
```

### Step 3: Create the primary key:

```
ALTER TABLE new_emp2 ADD PRIMARY KEY (employee_id) USING INDEX
 emp_id_idx2;
```

## Function-Based Indexes

- A function-based index is based on expressions.
- The index expression is built from table columns, constants, SQL functions, and user-defined functions.

```
CREATE INDEX upper_dept_name_idx
ON dept2 (UPPER(department_name));
```

```
CREATE INDEX succeeded.
```

```
SELECT *
FROM dept2
WHERE UPPER(department_name) = 'SALES';
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Function-Based Indexes

Function-based indexes defined with the `UPPER(column_name)` or `LOWER(column_name)` keywords allow non-case-sensitive searches. For example, consider the following index:

```
CREATE INDEX upper_last_name_idx ON emp2 (UPPER(last_name));
```

This facilitates processing queries such as:

```
SELECT * FROM emp2 WHERE UPPER(last_name) = 'KING';
```

The Oracle server uses the index only when that particular function is used in a query. For example, the following statement may use the index, but without the `WHERE` clause, the Oracle server may perform a full table scan:

```
SELECT *
FROM employees
WHERE UPPER(last_name) IS NOT NULL
ORDER BY UPPER(last_name);
```

**Note:** The `QUERY_REWRITE_ENABLED` initialization parameter must be set to `TRUE` for a function-based index to be used.

The Oracle server treats indexes with columns marked `DESC` as function-based indexes. The columns marked `DESC` are sorted in descending order.

## Removing an Index

- Remove an index from the data dictionary by using the `DROP INDEX` command:

```
DROP INDEX index;
```

- Remove the `UPPER_DEPT_NAME_IDX` index from the data dictionary:

```
DROP INDEX upper_dept_name_idx;
```

```
DROP INDEX upper_dept_name_idx succeeded.
```

- To drop an index, you must be the owner of the index or have the `DROP ANY INDEX` privilege.

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Removing an Index

You cannot modify indexes. To change an index, you must drop it and then re-create it. Remove an index definition from the data dictionary by issuing the `DROP INDEX` statement. To drop an index, you must be the owner of the index or have the `DROP ANY INDEX` privilege.

In the syntax:

*index*                      Is the name of the index

**Note:** If you drop a table, then indexes, constraints, and triggers are automatically dropped, but views and sequences remain.

## DROP TABLE ... PURGE

```
DROP TABLE dept80 PURGE;
```

```
DROP TABLE dept80 succeeded.
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### DROP TABLE ... PURGE

Oracle Database provides a feature for dropping tables. When you drop a table, the database does not immediately release the space associated with the table. Rather, the database renames the table and places it in a recycle bin, where it can later be recovered with the `FLASHBACK TABLE` statement if you find that you dropped the table in error. If you want to immediately release the space associated with the table at the time you issue the `DROP TABLE` statement, then include the `PURGE` clause as shown in the statement in the slide.

Specify `PURGE` only if you want to drop the table and release the space associated with it in a single step. If you specify `PURGE`, then the database does not place the table and its dependent objects into the recycle bin.

Using this clause is equivalent to first dropping the table and then purging it from the recycle bin. This clause saves you one step in the process. It also provides enhanced security if you want to prevent sensitive material from appearing in the recycle bin.

**Note:** You cannot roll back a `DROP TABLE` statement with the `PURGE` clause, and you cannot recover the table if you drop it with the `PURGE` clause. This feature was not available in earlier releases.

## Lesson Agenda

- Using the `ALTER TABLE` statement to add, modify, and drop a column
- Managing constraints
  - Adding and dropping a constraint
  - Deferring constraints
  - Enabling and disabling a constraint
- Creating indexes
  - Using the `CREATE TABLE` statement
  - Creating function-based indexes
  - Removing an index
- Performing flashback operations
- Creating and using external tables

ORACLE

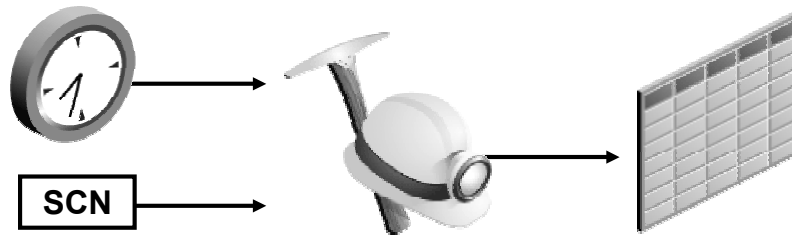
Copyright © 2007, Oracle. All rights reserved.

Oracle Internal & Oracle Academy  
Use Only



## FLASHBACK TABLE Statement

- Enables you to recover tables to a specified point in time with a single statement
- Restores table data along with associated indexes, and constraints
- Enables you to revert the table and its contents to a certain point in time or SCN



ORACLE

Copyright © 2007, Oracle. All rights reserved.

### FLASHBACK TABLE Statement

Oracle Flashback Table enables you to recover tables to a specified point in time with a single statement. You can restore table data along with associated indexes, and constraints, while the database is online, undoing changes to only the specified tables.

The Flashback Table feature is similar to a self-service repair tool. For example, if a user accidentally deleted important rows from a table and then wanted to recover the deleted rows, you can use the FLASHBACK TABLE statement to restore the table to the time before the deletion and see the missing rows in the table.

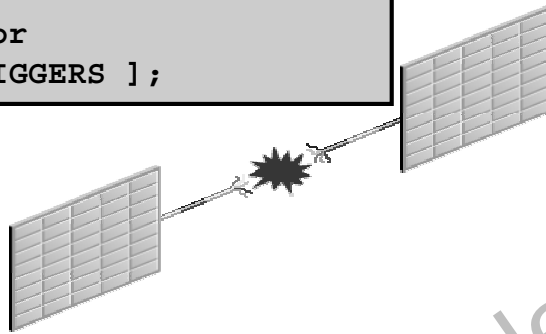
When using the FLASHBACK TABLE statement, you can revert the table and its contents to a certain time or to an SCN.

**Note:** The system change number (SCN) is an integer value associated with each change to the database. It is a unique incremental number in the database. Every time you commit a transaction, a new SCN is recorded.

## FLASHBACK TABLE Statement

- Repair tool for accidental table modifications
  - Restores a table to an earlier point in time
  - Benefits: Ease of use, availability, and fast execution
  - Is performed in place
- Syntax:

```
FLASHBACK TABLE [schema.] table [,
[schema.] table] ...
TO { TIMESTAMP | SCN } expr
[{ ENABLE | DISABLE } TRIGGERS] ;
```



ORACLE

Copyright © 2007, Oracle. All rights reserved.

### FLASHBACK TABLE Statement (continued)

#### Self-Service Repair Facility

Oracle Database provides a SQL data definition language (DDL) command, FLASHBACK TABLE, to restore the state of a table to an earlier point in time in case it is inadvertently deleted or modified. The FLASHBACK TABLE command is a self-service repair tool to restore data in a table along with associated attributes such as indexes or views. This is done while the database is online by rolling back only the subsequent changes to the given table. Compared to traditional recovery mechanisms, this feature offers significant benefits such as ease of use, availability, and faster restoration. It also takes the burden off the DBA to find and restore application-specific properties. The flashback table feature does not address physical corruption caused because of a bad disk.

#### Syntax

You can invoke a FLASHBACK TABLE operation on one or more tables, even on tables in different schemas. You specify the point in time to which you want to revert by providing a valid time stamp. By default, database triggers are disabled during the flashback operation for all tables involved. You can override this default behavior by specifying the ENABLE TRIGGERS clause.

**Note:** For more information about recycle bin and flashback semantics, refer to *Oracle Database Administrator's Guide 11g Release 1 (11.1)*.

## Using the FLASHBACK TABLE Statement

```
DROP TABLE emp2;
```

```
DROP TABLE emp2 succeeded.
```

```
SELECT original_name, operation, droptime FROM
recyclebin;
```

| ORIGINAL_NAME | OPERATION | DROPTIME            |
|---------------|-----------|---------------------|
| EMP2          | DROP      | 2007-07-02:06:07:41 |

...

```
FLASHBACK TABLE emp2 TO BEFORE DROP;
```

```
FLASHBACK TABLE succeeded.
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Using the FLASHBACK TABLE Statement

### Syntax and Examples

The example restores the EMP2 table to a state before a DROP statement.

The recycle bin is actually a data dictionary table containing information about dropped objects. Dropped tables and any associated objects—such as, indexes, constraints, nested tables, and so on—are not removed and still occupy space. They continue to count against user space quotas until specifically purged from the recycle bin or the situation where they must be purged by the database because of tablespace space constraints.

Each user can be thought of as an owner of a recycle bin because, unless a user has the SYSDBA privilege, the only objects that the user has access to in the recycle bin are those that the user owns. A user can view his or her objects in the recycle bin by using the following statement:

```
SELECT * FROM RECYCLEBIN;
```

When you drop a user, any objects belonging to that user are not placed in the recycle bin and any objects in the recycle bin are purged.

You can purge the recycle bin with the following statement:

```
PURGE RECYCLEBIN;
```

## Lesson Agenda

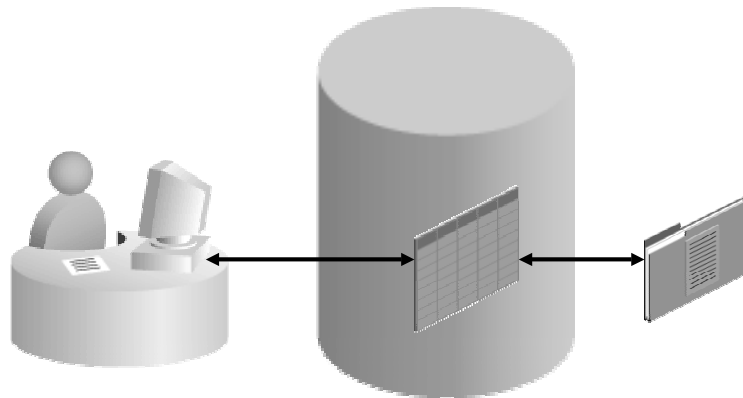
- Using the `ALTER TABLE` statement to add, modify, and drop a column
- Managing constraints
  - Adding and dropping a constraint
  - Deferring constraints
  - Enabling and disabling a constraint
- Creating indexes
  - Using the `CREATE TABLE` statement
  - Creating function-based indexes
  - Removing an index
- Performing flashback operations
- Creating and using external tables

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Oracle Internal & Oracle Academy  
Use Only

# External Tables



ORACLE

Copyright © 2007, Oracle. All rights reserved.

## External Tables

An external table is a read-only table whose metadata is stored in the database but whose data is stored outside the database. This external table definition can be thought of as a view that is used for running any SQL query against external data without requiring that the external data first be loaded into the database. The external table data can be queried and joined directly and in parallel without requiring that the external data first be loaded in the database. You can use SQL, PL/SQL and Java to query the data in an external table.

The main difference between external tables and regular tables is that externally organized tables are read-only. No data manipulation language (DML) operations are possible, and no indexes can be created on them. However, you can create an external table, and thus unload data, by using the `CREATE TABLE AS SELECT` command.

The Oracle server provides two major access drivers for external tables. One, the loader access driver (or `ORACLE_LOADER`) is used for reading data from external files whose format can be interpreted by the `SQL*Loader` utility. Note that not all `SQL*Loader` functionality is supported with external tables. The `ORACLE_DATAPUMP` access driver can be used to both import and export data using a platform-independent format. The `ORACLE_DATAPUMP` access driver writes rows from a `SELECT` statement to be loaded into an external table as part of a `CREATE TABLE . . . ORGANIZATION EXTERNAL . . . AS SELECT` statement. You can then use `SELECT` to read data out of that data file. You can also create an external table definition on another system and use that data file. This allows data to be moved between Oracle databases.

## Creating a Directory for the External Table

Create a `DIRECTORY` object that corresponds to the directory on the file system where the external data source resides.

```
CREATE OR REPLACE DIRECTORY emp_dir
AS '/.../emp_dir';

GRANT READ ON DIRECTORY emp_dir TO hr;
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Example of Creating an External Table

Use the `CREATE DIRECTORY` statement to create a directory object. A directory object specifies an alias for a directory on the server's file system where an external data source resides. You can use directory names when referring to an external data source, rather than hard code the operating system path name, for greater file management flexibility.

You must have `CREATE ANY DIRECTORY` system privileges to create directories. When you create a directory, you are automatically granted the `READ` and `WRITE` object privileges and can grant `READ` and `WRITE` privileges to other users and roles. The DBA can also grant these privileges to other users and roles.

A user needs `READ` privileges for all directories used in external tables to be accessed and `WRITE` privileges for the log, bad, and discard file locations being used.

In addition, a `WRITE` privilege is necessary when the external table framework is being used to unload data.

Oracle also provides the `ORACLE_DATAPUMP` type, with which you can unload data (that is, read data from a table in the database and insert it into an external table) and then reload it into an Oracle database. This is a one-time operation that can be done when the table is created. After the creation and initial population is done, you cannot update, insert, or delete any rows.

## Example of Creating an External Table (continued)

### Syntax

```
CREATE [OR REPLACE] DIRECTORY AS 'path_name' ;
```

In the syntax:

|             |                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OR REPLACE  | Specify OR REPLACE to re-create the directory database object if it already exists. You can use this clause to change the definition of an existing directory without dropping, re-creating, and regranting database object privileges previously granted on the directory. Users who were previously granted privileges on a redefined directory can continue to access the directory without requiring that the privileges be regranted. |
| directory   | Specify the name of the directory object to be created. The maximum length of the directory name is 30 bytes. You cannot qualify a directory object with a schema name.                                                                                                                                                                                                                                                                    |
| 'path_name' | Specify the full path name of the operating system directory to be accessed. The path name is case-sensitive.                                                                                                                                                                                                                                                                                                                              |

Oracle Internal & Oracle Academy  
Use Only

## Creating an External Table

```
CREATE TABLE <table_name>
 (<col_name> <datatype>, ...)
 ORGANIZATION EXTERNAL
 (TYPE <access_driver_type>
 DEFAULT DIRECTORY <directory_name>
 ACCESS PARAMETERS
 (...))
 LOCATION ('<location_specifier>'))
 REJECT LIMIT [0 | <number> | UNLIMITED];
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Creating an External Table

You create external tables using the `ORGANIZATION EXTERNAL` clause of the `CREATE TABLE` statement. You are not, in fact, creating a table. Rather, you are creating metadata in the data dictionary that you can use to access external data. You use the `ORGANIZATION` clause to specify the order in which the data rows of the table are stored. By specifying `EXTERNAL` in the `ORGANIZATION` clause, you indicate that the table is a read-only table located outside the database. Note that the external files must already exist outside the database.

`TYPE <access_driver_type>` indicates the access driver of the external table. The access driver is the application programming interface (API) that interprets the external data for the database. If you do not specify `TYPE`, Oracle uses the default access driver, `ORACLE_LOADER`. The other option is `ORACLE_DATAPUMP`.

You use the `DEFAULT DIRECTORY` clause to specify one or more Oracle database directory objects that correspond to directories on the file system where the external data sources may reside.

The optional `ACCESS PARAMETERS` clause enables you to assign values to the parameters of the specific access driver for this external table.



## Creating an External Table (continued)

Use the LOCATION clause to specify one external locator for each external data source. Usually, *<location\_specifier>* is a file, but it need not be.

The REJECT LIMIT clause enables you to specify how many conversion errors can occur during a query of the external data before an Oracle error is returned and the query is aborted. The default value is 0.

The syntax for using the ORACLE\_DATAPUMP access driver is as follows:

```
CREATE TABLE extract_emps
 ORGANIZATION EXTERNAL (TYPE ORACLE_DATAPUMP
 DEFAULT DIRECTORY ...
 ACCESS PARAMETERS (...)
 LOCATION (...)
 PARALLEL 4
 REJECT LIMIT UNLIMITED
 AS
 SELECT * FROM ...;
```

Oracle Internal & Oracle Academy  
Use Only

## Creating an External Table by Using ORACLE\_LOADER

```
CREATE TABLE oldemp (
 fname char(25), lname CHAR(25))
 ORGANIZATION EXTERNAL
 (TYPE ORACLE_LOADER
 DEFAULT DIRECTORY emp_dir
 ACCESS PARAMETERS
 (RECORDS DELIMITED BY NEWLINE
 NOBADFILE
 NOLOGFILE
 FIELDS TERMINATED BY ','
 (fname POSITION (1:20) CHAR,
 lname POSITION (22:41) CHAR))
 LOCATION ('emp.dat'))
 PARALLEL 5
 REJECT LIMIT 200;
```

CREATE TABLE succeeded.

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Example of Creating an External Table by Using the ORACLE\_LOADER Access Driver

Assume that there is a flat file that has records in the following format:

```
10,jones,11-Dec-1934
20,smith,12-Jun-1972
```

Records are delimited by new lines, and the fields are all terminated by a comma ( , ). The name of the file is /emp\_dir/emp.dat.

To convert this file as the data source for an external table, whose metadata will reside in the database, you must perform the following steps:

1. Create a directory object, emp\_dir, as follows:  
CREATE DIRECTORY emp\_dir AS '/emp\_dir' ;
2. Run the CREATE TABLE command shown in the slide.

The example in the slide illustrates the table specification to create an external table for the file:  
/emp\_dir/emp.dat

## Example of Creating an External Table by Using the ORACLE\_LOADER Access Driver (continued)

In the example, the TYPE specification is given only to illustrate its use. ORACLE\_LOADER is the default access driver if not specified. The ACCESS PARAMETERS option provides values to parameters of the specific access driver, which are interpreted by the access driver, not by the Oracle server.

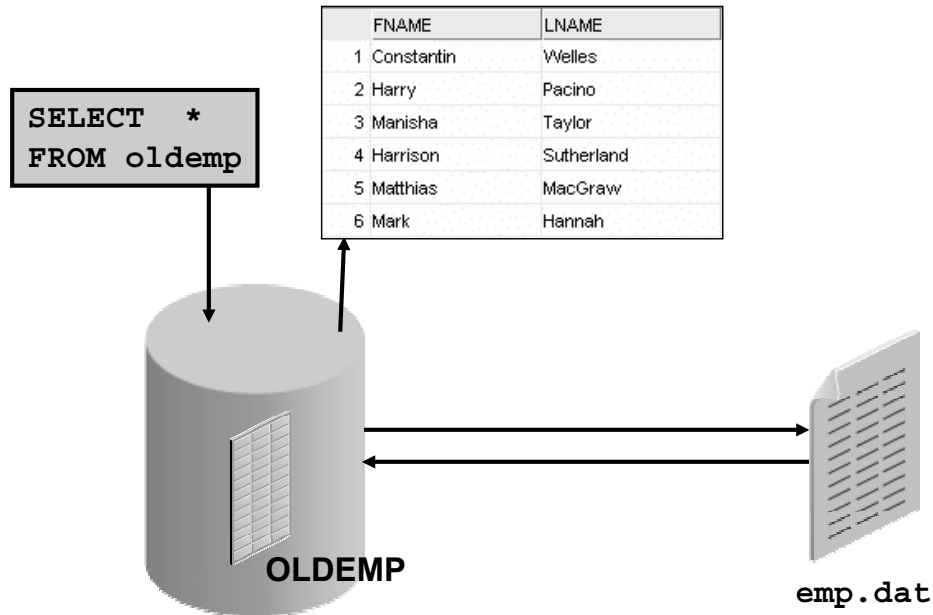
The PARALLEL clause enables five parallel execution servers to simultaneously scan the external data sources (files) when executing the INSERT INTO TABLE statement. For example, if PARALLEL=5 were specified, then more than one parallel execution server can be working on a data source. Because external tables can be very large, for performance reasons, it is advisable to specify the PARALLEL clause, or a parallel hint for the query.

The REJECT LIMIT clause specifies that if more than 200 conversion errors occur during a query of the external data, then the query be aborted and an error be returned. These conversion errors can arise when the access driver tries to transform the data in the data file to match the external table definition.

After the CREATE TABLE command executes successfully, the OLDEMP external table can be described and queried like a relational table.

Oracle Internal & Oracle Academy  
Use Only

## Querying External Tables



ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Querying External Tables

An external table does not describe any data that is stored in the database. It does not describe how data is stored in the external source. Instead, it describes how the external table layer must present the data to the server. It is the responsibility of the access driver and the external table layer to do the necessary transformations required on the data in the data file so that it matches the external table definition.

When the database server accesses data in an external source, it calls the appropriate access driver to get the data from an external source in a form that the database server expects.

It is important to remember that the description of the data in the data source is separate from the definition of the external table. The source file can contain more or fewer fields than there are columns in the table. Also, the data types for fields in the data source can be different from the columns in the table. The access driver takes care of ensuring that the data from the data source is processed so that it matches the definition of the external table.

## Creating an External Table by Using ORACLE\_DATAPUMP: Example

```
CREATE TABLE emp_ext
(employee_id, first_name, last_name)
ORGANIZATION EXTERNAL
(
 TYPE ORACLE_DATAPUMP
 DEFAULT DIRECTORY emp_dir
 LOCATION
 ('emp1.exp', 'emp2.exp')
)
PARALLEL
AS
SELECT employee_id, first_name, last_name
FROM employees;
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Creating an External Table by Using ORACLE\_DATAPUMP: Example

You can perform the unload and reload operations with external tables by using the ORACLE\_DATAPUMP access driver.

**Note:** In the context of external tables, loading data refers to the act of data being read from an external table and loaded into a table in the database. Unloading data refers to the act of reading data from a table and inserting it into an external table.

The example in the slide illustrates the table specification to create an external table by using the ORACLE\_DATAPUMP access driver. Data is then populated into the two files: emp1.exp and emp2.exp.

To populate data read from the EMPLOYEES table into an external table, you must perform the following steps:

1. Create a directory object, emp\_dir, as follows:  
CREATE DIRECTORY emp\_dir AS '/emp\_dir' ;
2. Run the CREATE TABLE command shown in the slide.

**Note:** The emp\_dir directory is the same as created in the previous example of using ORACLE\_LOADER.

You can query the external table by executing the following code:

```
SELECT * FROM emp_ext;
```

## Summary

In this lesson, you should have learned how to:

- Add constraints
- Create indexes
- Create indexes using the `CREATE TABLE` statement
- Create function-based indexes
- Drop columns and set columns as `UNUSED`
- Perform `FLASHBACK` operations
- Create and use external tables

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Summary

Alter tables to add or modify columns or constraints. Create indexes and function-based indexes using the `CREATE INDEX` statement. Drop unused columns. Use `FLASHBACK` mechanics to restore tables. Use the `external_table` clause to create an external table, which is a read-only table whose metadata is stored in the database but whose data is stored outside the database. Use external tables to query data without first loading it into the database. Name your `PRIMARY KEY` column indexes as you create the table with the `CREATE TABLE` statement.

## Practice 2: Overview

This practice covers the following topics:

- Altering tables
- Adding columns
- Dropping columns
- Creating indexes
- Creating external tables

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Practice 2: Overview

In this practice, you use the `ALTER TABLE` command to modify columns and add constraints. You use the `CREATE INDEX` command to create indexes when creating a table, along with the `CREATE TABLE` command. You create external tables.

## Practice 2

1. Create the DEPT2 table based on the following table instance chart. Type the syntax in the SQL Worksheet. Then, execute the statement to create the table. Confirm that the table is created.

| Column Name  | ID     | NAME     |
|--------------|--------|----------|
| Key Type     |        |          |
| Nulls/Unique |        |          |
| FK Table     |        |          |
| FK Column    |        |          |
| Data type    | NUMBER | VARCHAR2 |
| Length       | 7      | 25       |

| Name            | Null  | Type          |
|-----------------|-------|---------------|
| -----           | ----- | -----         |
| ID              |       | NUMBER (7)    |
| NAME            |       | VARCHAR2 (25) |
| 2 rows selected |       |               |

2. Populate the DEPT2 table with data from the DEPARTMENTS table. Include only the columns that you need.
3. Create the EMP2 table based on the following table instance chart. Type the syntax in the SQL Worksheet. Then execute the statement to create the table. Confirm that the table is created.

| Column Name  | ID     | LAST_NAME | FIRST_NAME | DEPT_ID |
|--------------|--------|-----------|------------|---------|
| Key Type     |        |           |            |         |
| Nulls/Unique |        |           |            |         |
| FK Table     |        |           |            |         |
| FK Column    |        |           |            |         |
| Data type    | NUMBER | VARCHAR2  | VARCHAR2   | NUMBER  |
| Length       | 7      | 25        | 25         | 7       |

| Name            | Null  | Type          |
|-----------------|-------|---------------|
| -----           | ----- | -----         |
| ID              |       | NUMBER (7)    |
| LAST_NAME       |       | VARCHAR2 (25) |
| FIRST_NAME      |       | VARCHAR2 (25) |
| DEPT_ID         |       | NUMBER (7)    |
| 4 rows selected |       |               |



## Practice 2 (continued)

4. Modify the EMP2 table to allow for longer employee last names. Confirm your modification.

| Name            | Null | Type          |
|-----------------|------|---------------|
| -----           |      |               |
| ID              |      | NUMBER (7)    |
| LAST_NAME       |      | VARCHAR2 (50) |
| FIRST_NAME      |      | VARCHAR2 (25) |
| DEPT_ID         |      | NUMBER (7)    |
| 4 rows selected |      |               |

5. Create the EMPLOYEES2 table based on the structure of the EMPLOYEES table. Include only the EMPLOYEE\_ID, FIRST\_NAME, LAST\_NAME, SALARY, and DEPARTMENT\_ID columns. Name the columns in your new table ID, FIRST\_NAME, LAST\_NAME, SALARY, and DEPT\_ID, respectively.
6. Drop the EMP2 table.
7. Query the recycle bin to see whether the table is present.

|   | ORIGINAL_NAME | OPERATION | DROPTIME            |
|---|---------------|-----------|---------------------|
| 1 | EMP2          | DROP      | 2007-07-02:04:52:43 |

8. Restore the EMP2 table to a state before the DROP statement.

| Name            | Null | Type          |
|-----------------|------|---------------|
| -----           |      |               |
| ID              |      | NUMBER (7)    |
| LAST_NAME       |      | VARCHAR2 (50) |
| FIRST_NAME      |      | VARCHAR2 (25) |
| DEPT_ID         |      | NUMBER (7)    |
| 4 rows selected |      |               |

9. Drop the FIRST\_NAME column from the EMPLOYEES2 table. Confirm your modification by checking the description of the table.

| Name            | Null     | Type          |
|-----------------|----------|---------------|
| -----           |          |               |
| ID              |          | NUMBER (6)    |
| LAST_NAME       | NOT NULL | VARCHAR2 (25) |
| SALARY          |          | NUMBER (8,2)  |
| DEPT_ID         |          | NUMBER (4)    |
| 4 rows selected |          |               |

10. In the EMPLOYEES2 table, mark the DEPT\_ID column as UNUSED. Confirm your modification by checking the description of the table.

## Practice 2 (continued)

| Name            | Null     | Type         |
|-----------------|----------|--------------|
| -----           | -----    | -----        |
| ID              |          | NUMBER(6)    |
| LAST_NAME       | NOT NULL | VARCHAR2(25) |
| SALARY          |          | NUMBER(8,2)  |
| 3 rows selected |          |              |

- Drop all the UNUSED columns from the EMPLOYEES2 table. Confirm your modification by checking the description of the table.
- Add a table-level PRIMARY KEY constraint to the EMP2 table on the ID column. The constraint should be named at creation. Name the constraint my\_emp\_id\_pk.
- Create a PRIMARY KEY constraint to the DEPT2 table using the ID column. The constraint should be named at creation. Name the constraint my\_dept\_id\_pk.
- Add a foreign key reference on the EMP2 table that ensures that the employee is not assigned to a nonexistent department. Name the constraint my\_emp\_dept\_id\_fk.
- Modify the EMP2 table. Add a COMMISSION column of the NUMBER data type, precision 2, scale 2. Add a constraint to the COMMISSION column that ensures that a commission value is greater than zero.
- Drop the EMP2 and DEPT2 tables so that they cannot be restored. Verify the recycle bin.
- Create the DEPT\_NAMED\_INDEX table based on the following table instance chart. Name the index for the PRIMARY KEY column as DEPT\_PK\_IDX.

| Column Name | Deptno | Dname    |
|-------------|--------|----------|
| Primary Key | Yes    |          |
| Data Type   | Number | VARCHAR2 |
| Length      | 4      | 30       |

- Create an external table library\_items\_ext. Use the ORACLE\_LOADER access driver. **Note:** The emp\_dir directory and library\_items.dat are already created for this exercise. library\_items.dat has records in the following format:

```
2354, 2264, 13.21, 150,
2355, 2289, 46.23, 200,
2355, 2264, 50.00, 100,
```

- Open the lab\_02\_18.sql file. Observe the code snippet to create the library\_items\_ext external table. Then replace <TODO1>, <TODO2>, <TODO3> and <TODO4> as appropriate and save the file as lab\_02\_18\_soln.sql. Run the script to create the external table.
- Query the library\_items\_ext table.

|   | 1           | 2       | 3          | 4        |
|---|-------------|---------|------------|----------|
|   | CATEGORY_ID | BOOK_ID | BOOK_PRICE | QUANTITY |
| 1 | 2354        | 2264    | 13.21      | 150      |
| 2 | 2355        | 2289    | 46.23      | 200      |
| 3 | 2355        | 2264    | 50         | 100      |

## Practice 2 (continued)

19. The HR department needs a report of addresses of all departments. Create an external table as `dept_add_ext` using the `ORACLE_LOADER` access driver. The report should show the location ID, street address, city, state or province, and country in the output. Use a `NATURAL JOIN` to produce the results.

**Note:** The `emp_dir` directory is already created for this exercise.

- Open the `lab_02_19.sql` file. Observe the code snippet to create the `dept_add_ext` external table. Then, replace `<TODO1>`, `<TODO2>` and `<TODO3>` with the appropriate code. Replace `<oraxx_emp4.exp>` and `<oraxx_emp5.exp>` with the appropriate file names. For example, if you are the `ora21` user, your file names are `ora21_emp4.exp` and `ora21_emp5.exp`. Save the script as `lab_02_19_soln.sql`.
- Run the `lab_02_19_soln.sql` script to create the external table.
- Query the `dept_add_ext` table.

| LOCATION_ID | STREET_ADDRESS                           | CITY                | STATE_PROVINCE | COUNTRY_NAME             |
|-------------|------------------------------------------|---------------------|----------------|--------------------------|
| 1800        | 460 Bloor St. W.                         | Toronto             | Ontario        | Canada                   |
| 1400        | 2014 Jabbawocky Rd                       | Southlake           | Texas          | United States of America |
| 1700        | 2004 Charade Rd                          | Seattle             | Washington     | United States of America |
| 2500        | Magdalen Centre, The Oxford Science Park | Oxford              | Oxford         | United Kingdom           |
| 1500        | 2011 Interiors Blvd                      | South San Francisco | California     | United States of America |

**Note:** When you perform the preceding step, two files `oraxx_emp4.exp` and `oraxx_emp5.exp` are created under the default directory `emp_dir`.

20. Create the `emp_books` table and populate it with data. Set the primary key as deferred and observe what happens at the end of the transaction.

- Run the `lab_02_20_a.sql` file to create the `emp_books` table. Observe that the `emp_books_pk` primary key is not created as deferrable.

```
create table succeeded.
```

- Run the `lab_02_20_b.sql` file to populate data into the `emp_books` table. What do you observe?

```
1 rows inserted

Error starting at line 2 in command:
insert into emp_books values(300,'Change Management')
Error report:
SQL Error: ORA-00001: unique constraint (ORA21.EMP_BOOKS_PK) violated
00001. 00000 - "unique constraint (%s.%s) violated"
*Cause: An UPDATE or INSERT statement attempted to insert a duplicate key.
 For Trusted Oracle configured in DBMS MAC mode, you may see
 this message if a duplicate entry exists at a different level.
*Action: Either remove the unique restriction or do not insert the key.
```

## Practice 2 (continued)

- c. Set the emp\_books\_pk constraint as deferred. What do you observe?

```
Error starting at line 1 in command:
set constraint emp_books_pk deferred
Error report:
SQL Error: ORA-02447: cannot defer a constraint that is not deferrable
02447. 00000 - "cannot defer a constraint that is not deferrable"
*Cause: An attempt was made to defer a nondeferrable constraint
*Action: Drop the constraint and create a new one that is deferrable
```

- d. Drop the emp\_books\_pk constraint.

```
alter table emp_books succeeded.
```

- e. Modify the emp\_books table definition to add the emp\_books\_pk constraint as deferrable this time.

```
alter table emp_books succeeded.
```

- f. Set the emp\_books\_pk constraint as deferred.

```
set constraint succeeded.
```

- g. Run the lab\_02\_20\_g.sql file to populate data into the emp\_books table. What do you observe?

```
1 rows inserted
1 rows inserted
1 rows inserted
```

- h. Commit the transaction. What do you observe?

```
Error report:
SQL Error: ORA-02091: transaction rolled back
ORA-00001: unique constraint (ORA21.EMP_BOOKS_PK) violated
02091. 00000 - "transaction rolled back"
*Cause: Also see error 2092. If the transaction is aborted at a remote
 site then you will only see 2091; if aborted at host then you will
 see 2092 and 2091.
*Action: Add rollback segment and retry the transaction.
```

# 3

## **Managing Objects with Data Dictionary Views**

ORACLE®

Copyright © 2007, Oracle. All rights reserved.

Oracle Internal & Oracle Academy  
Use Only

## Objectives

After completing this lesson, you should be able to do the following:

- Use the data dictionary views to research data on your objects
- Query various data dictionary views

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Objectives

In this lesson, you are introduced to the data dictionary views. You learn that the dictionary views can be used to retrieve metadata and create reports about your schema objects.

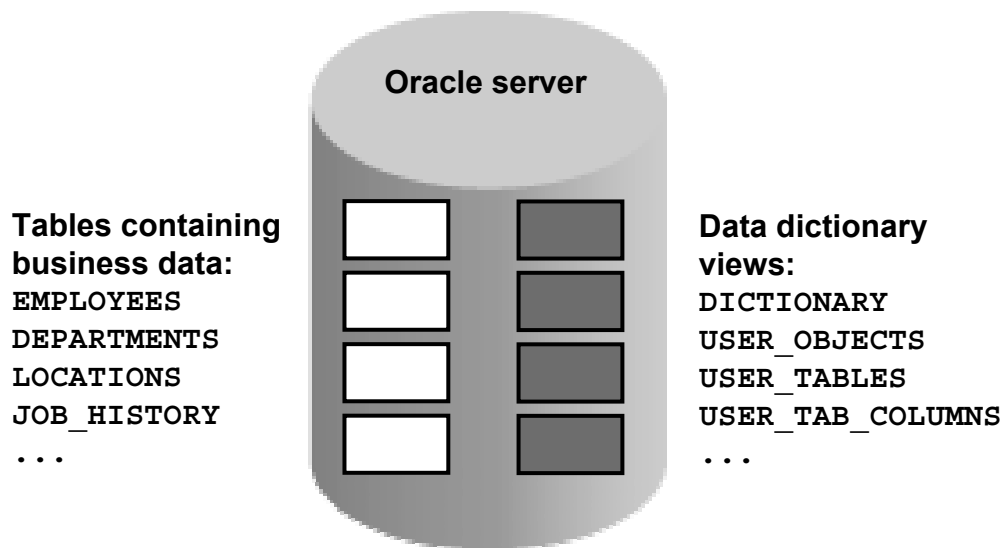
## Lesson Agenda

- Introduction to data dictionary
- Querying the dictionary views for the following:
  - Table information
  - Column information
  - Constraint information
- Querying the dictionary views for the following:
  - View information
  - Sequence information
  - Synonym information
  - Index information
- Adding a comment to a table and querying the dictionary views for comment information

ORACLE

Copyright © 2007, Oracle. All rights reserved.

# Data Dictionary



ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Data Dictionary

User tables are tables created by the user and contain business data, such as EMPLOYEES. There is another collection of tables and views in the Oracle database known as the data dictionary.

This collection is created and maintained by the Oracle server and contains information about the database. The *data dictionary* is structured in tables and views, just like other database data.

Not only is the data dictionary central to every Oracle database, but it is an important tool for all users, from end users to application designers and database administrators.

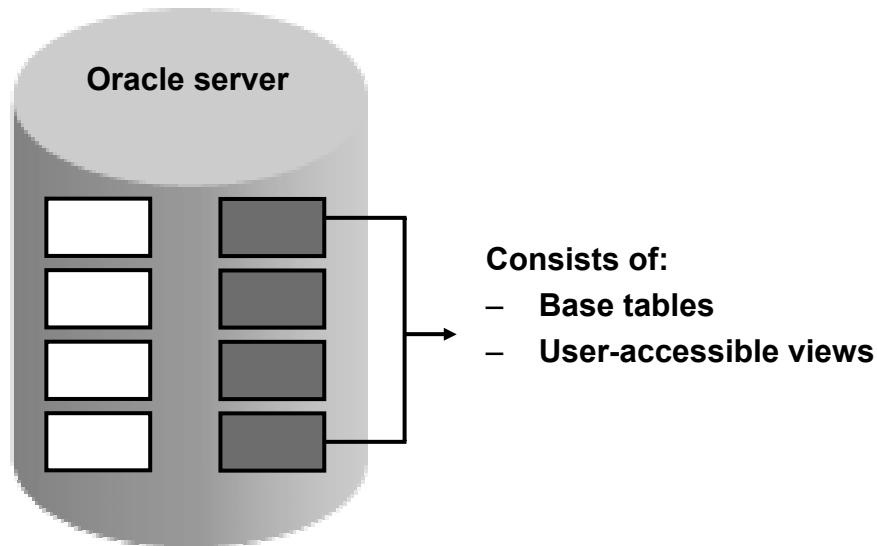
You use SQL statements to access the data dictionary. Because the data dictionary is read-only, you can issue only queries against its tables and views.

You can query the dictionary views that are based on the dictionary tables to find information such as:

- Definitions of all schema objects in the database (tables, views, indexes, synonyms, sequences, procedures, functions, packages, triggers, and so on)
- Default values for columns
- Integrity constraint information
- Names of Oracle users
- Privileges and roles that each user has been granted
- Other general database information



## Data Dictionary Structure



ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Data Dictionary Structure

Underlying base tables store information about the associated database. Only the Oracle server should write to and read from these tables. You rarely access them directly.

There are several views that summarize and display the information stored in the base tables of the data dictionary. These views decode the base table data into useful information (such as user or table names) using joins and WHERE clauses to simplify the information. Most users are given access to the views rather than the base tables.

The Oracle user SYS owns all base tables and user-accessible views of the data dictionary. No Oracle user should *ever* alter (UPDATE, DELETE, or INSERT) any rows or schema objects contained in the SYS schema because such activity can compromise data integrity.

## Data Dictionary Structure

View naming convention:

| View Prefix | Purpose                                                       |
|-------------|---------------------------------------------------------------|
| USER        | User's view (what is in your schema; what you own)            |
| ALL         | Expanded user's view (what you can access)                    |
| DBA         | Database administrator's view (what is in everyone's schemas) |
| V\$         | Performance-related data                                      |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Data Dictionary Structure (continued)

The data dictionary consists of sets of views. In many cases, a set consists of three views containing similar information and distinguished from each other by their prefixes. For example, there is a view named `USER_OBJECTS`, another named `ALL_OBJECTS`, and a third named `DBA_OBJECTS`.

These three views contain similar information about objects in the database, except that the scope is different. `USER_OBJECTS` contains information about objects that you own or created. `ALL_OBJECTS` contains information about all objects to which you have access. `DBA_OBJECTS` contains information about all objects that are owned by all users. For views that are prefixed with `ALL` or `DBA`, there is usually an additional column in the view named `OWNER` to identify who owns the object.

There is also a set of views that is prefixed with `V$`. These views are dynamic in nature and hold information about performance. Dynamic performance tables are not true tables, and they should not be accessed by most users. However, database administrators can query and create views on the tables and grant access to those views to other users. This course does not go into details about these views.

## How to Use the Dictionary Views

Start with `DICTIONARY`. It contains the names and descriptions of the dictionary tables and views.

### DESCRIBE DICTIONARY

| Name            | Null | Type           |
|-----------------|------|----------------|
| TABLE_NAME      |      | VARCHAR2(30)   |
| COMMENTS        |      | VARCHAR2(4000) |
| 2 rows selected |      |                |

```
SELECT *
FROM dictionary
WHERE table_name = 'USER_OBJECTS';
```

| TABLE_NAME     | COMMENTS                  |
|----------------|---------------------------|
| 1 USER_OBJECTS | Objects owned by the user |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## How to Use the Dictionary Views

To familiarize yourself with the dictionary views, you can use the dictionary view named `DICTIONARY`. It contains the name and short description of each dictionary view to which you have access.

You can write queries to search for information about a particular view name, or you can search the `COMMENTS` column for a word or phrase. In the example shown, the `DICTIONARY` view is described. It has two columns. The `SELECT` statement retrieves information about the dictionary view named `USER_OBJECTS`. The `USER_OBJECTS` view contains information about all the objects that you own.

You can write queries to search the `COMMENTS` column for a word or phrase. For example, the following query returns the names of all views that you are permitted to access in which the `COMMENTS` column contains the word *columns*:

```
SELECT table_name
FROM dictionary
WHERE LOWER(comments) LIKE '%columns%';
```

**Note:** The names in the data dictionary are uppercase.

## USER\_OBJECTS and ALL\_OBJECTS Views

### USER\_OBJECTS:

- Query USER\_OBJECTS to see all the objects that you own
- Is a useful way to obtain a listing of all object names and types in your schema, plus the following information:
  - Date created
  - Date of last modification
  - Status (valid or invalid)

### ALL\_OBJECTS:

- Query ALL\_OBJECTS to see all objects to which you have access

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## USER\_OBJECTS and ALL\_OBJECTS Views

You can query the USER\_OBJECTS view to see the names and types of all the objects in your schema. There are several columns in this view:

- **OBJECT\_NAME:** Name of the object
- **OBJECT\_ID:** Dictionary object number of the object
- **OBJECT\_TYPE:** Type of object (such as TABLE, VIEW, INDEX, SEQUENCE)
- **CREATED:** Time stamp for the creation of the object
- **LAST\_DDL\_TIME:** Time stamp for the last modification of the object resulting from a data definition language (DDL) command
- **STATUS:** Status of the object (VALID, INVALID, or N/A)
- **GENERATED:** Was the name of this object system generated? (Y | N)

**Note:** This is not a complete listing of the columns. For a complete listing, see “USER\_OBJECTS” in the *Oracle Database Reference*.

You can also query the ALL\_OBJECTS view to see a listing of all objects to which you have access.

## USER\_OBJECTS View

```
SELECT object_name, object_type, created, status
FROM user_objects
ORDER BY object_type;
```

|   | OBJECT_NAME | OBJECT_TYPE | CREATED   | STATUS |
|---|-------------|-------------|-----------|--------|
| 1 | REG_ID_PK   | INDEX       | 30-MAY-07 | VALID  |

...

|    |                  |          |           |       |
|----|------------------|----------|-----------|-------|
| 22 | DEPARTMENTS_SEQ  | SEQUENCE | 30-MAY-07 | VALID |
| 23 | LOCATIONS        | TABLE    | 30-MAY-07 | VALID |
| 24 | DEPARTMENTS      | TABLE    | 30-MAY-07 | VALID |
| 25 | REGIONS          | TABLE    | 30-MAY-07 | VALID |
| 26 | JOB_HISTORY      | TABLE    | 30-MAY-07 | VALID |
| 27 | JOBS             | TABLE    | 30-MAY-07 | VALID |
| 28 | EMPLOYEES        | TABLE    | 30-MAY-07 | VALID |
| 29 | COUNTRIES        | TABLE    | 30-MAY-07 | VALID |
| 30 | JOB_GRADES       | TABLE    | 30-MAY-07 | VALID |
| 31 | EMP_DETAILS_VIEW | VIEW     | 30-MAY-07 | VALID |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### USER\_OBJECTS View

The example shows the names, types, dates of creation, and status of all objects that are owned by this user.

The OBJECT\_TYPE column holds the values of either TABLE, VIEW, SEQUENCE, INDEX, PROCEDURE, FUNCTION, PACKAGE, or TRIGGER.

The STATUS column holds a value of VALID, INVALID, or N/A. Although tables are always valid, the views, procedures, functions, packages, and triggers may be invalid.

#### The CAT View

For a simplified query and output, you can query the CAT view. This view contains only two columns: TABLE\_NAME and TABLE\_TYPE. It provides the names of all your INDEX, TABLE, CLUSTER, VIEW, SYNONYM, SEQUENCE, or UNDEFINED objects.

**Note:** CAT is a synonym for USER\_CATALOG—a view that lists tables, views, synonyms and sequences owned by the user.

## Lesson Agenda

- Introduction to data dictionary
- Querying the dictionary views for the following:
  - Table information
  - Column information
  - Constraint information
- Querying the dictionary views for the following:
  - View information
  - Sequence information
  - Synonym information
  - Index information
- Adding a comment to a table and querying the dictionary views for comment information

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Table Information

USER\_TABLES:

```
DESCRIBE user_tables
```

| Name            | Null     | Type         |
|-----------------|----------|--------------|
| TABLE_NAME      | NOT NULL | VARCHAR2(30) |
| TABLESPACE_NAME |          | VARCHAR2(30) |
| CLUSTER_NAME    |          | VARCHAR2(30) |
| IOT_NAME        |          | VARCHAR2(30) |

...

```
SELECT table_name
FROM user_tables;
```

| TABLE_NAME    |
|---------------|
| 1 REGIONS     |
| 2 LOCATIONS   |
| 3 DEPARTMENTS |
| 4 JOBS        |
| 5 EMPLOYEES   |
| 6 JOB_HISTORY |

...

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Table Information

You can use the USER\_TABLES view to obtain the names of all your tables. The USER\_TABLES view contains information about your tables. In addition to providing the table name, it contains detailed information about the storage.

The TABS view is a synonym of the USER\_TABLES view. You can query it to see a listing of tables that you own:

```
SELECT table_name
FROM tabs;
```

**Note:** For a complete listing of the columns in the USER\_TABLES view, see “USER\_TABLES” in the *Oracle Database Reference*.

You can also query the ALL\_TABLES view to see a listing of all tables to which you have access.

## Column Information

USER\_TAB\_COLUMNS:

```
DESCRIBE user_tab_columns
```

| Name            | Null     | Type          |
|-----------------|----------|---------------|
| TABLE_NAME      | NOT NULL | VARCHAR2(30)  |
| COLUMN_NAME     | NOT NULL | VARCHAR2(30)  |
| DATA_TYPE       |          | VARCHAR2(106) |
| DATA_TYPE_MOD   |          | VARCHAR2(3)   |
| DATA_TYPE_OWNER |          | VARCHAR2(30)  |
| DATA_LENGTH     | NOT NULL | NUMBER        |
| DATA_PRECISION  |          | NUMBER        |
| DATA_SCALE      |          | NUMBER        |
| NULLABLE        |          | VARCHAR2(1)   |
| COLUMN_ID       |          | NUMBER        |
| DEFAULT_LENGTH  |          | NUMBER        |
| DATA_DEFAULT    |          | LONG()        |

...

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Column Information

You can query the USER\_TAB\_COLUMNS view to find detailed information about the columns in your tables. Although the USER\_TABLES view provides information about your table names and storage, detailed column information is found in the USER\_TAB\_COLUMNS view.

This view contains information such as:

- Column names
- Column data types
- Length of data types
- Precision and scale for NUMBER columns
- Whether nulls are allowed (Is there a NOT NULL constraint on the column?)
- Default value

**Note:** For a complete listing and description of the columns in the USER\_TAB\_COLUMNS view, see “USER\_TAB\_COLUMNS” in the *Oracle Database Reference*.



## Column Information

```
SELECT column_name, data_type, data_length,
 data_precision, data_scale, nullable
FROM user_tab_columns
WHERE table_name = 'EMPLOYEES';
```

|    | COLUMN_N...    | DATA_T... | DATA_LENGTH | DATA_PRECI... | DATA_SCALE | NULL |
|----|----------------|-----------|-------------|---------------|------------|------|
| 1  | EMPLOYEE_ID    | NUMBER    | 22          | 6             | 0          | N    |
| 2  | FIRST_NAME     | VARCHAR2  | 20          | (null)        | (null)     | Y    |
| 3  | LAST_NAME      | VARCHAR2  | 25          | (null)        | (null)     | N    |
| 4  | EMAIL          | VARCHAR2  | 25          | (null)        | (null)     | N    |
| 5  | PHONE_NUMBER   | VARCHAR2  | 20          | (null)        | (null)     | Y    |
| 6  | HIRE_DATE      | DATE      | 7           | (null)        | (null)     | N    |
| 7  | JOB_ID         | VARCHAR2  | 10          | (null)        | (null)     | N    |
| 8  | SALARY         | NUMBER    | 22          | 8             | 2          | Y    |
| 9  | COMMISSION_PCT | NUMBER    | 22          | 2             | 2          | Y    |
| 10 | MANAGER_ID     | NUMBER    | 22          | 6             | 0          | Y    |
| 11 | DEPARTMENT_ID  | NUMBER    | 22          | 4             | 0          | Y    |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Column Information (continued)

By querying the USER\_TAB\_COLUMNS table, you can find details about your columns such as the names, data types, data type lengths, null constraints, and default value for a column.

The example shown displays the columns, data types, data lengths, and null constraints for the EMPLOYEES table. Note that this information is similar to the output from the DESCRIBE command.

To view information about columns set as unused, you use the USER\_UNUSED\_COL\_TABS dictionary view.

## Constraint Information

- `USER_CONSTRAINTS` describes the constraint definitions on your tables.
- `USER_CONS_COLUMNS` describes columns that are owned by you and that are specified in constraints.

```
DESCRIBE user_constraints
```

| Name              | Null     | Type         |
|-------------------|----------|--------------|
| OWNER             | NOT NULL | VARCHAR2(30) |
| CONSTRAINT_NAME   | NOT NULL | VARCHAR2(30) |
| CONSTRAINT_TYPE   |          | VARCHAR2(1)  |
| TABLE_NAME        | NOT NULL | VARCHAR2(30) |
| SEARCH_CONDITION  |          | LONG()       |
| R_OWNER           |          | VARCHAR2(30) |
| R_CONSTRAINT_NAME |          | VARCHAR2(30) |
| DELETE_RULE       |          | VARCHAR2(9)  |
| STATUS            |          | VARCHAR2(8)  |

...

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Constraint Information

You can find out the names of your constraints, the type of constraint, the table name to which the constraint applies, the condition for check constraints, foreign key constraint information, deletion rule for foreign key constraints, the status, and many other types of information about your constraints.

**Note:** For a complete listing and description of the columns in the `USER_CONSTRAINTS` view, see “`USER_CONSTRAINTS`” in the *Oracle Database Reference*.

## USER\_CONSTRAINTS: Example

```
SELECT constraint_name, constraint_type,
 search_condition, r_constraint_name,
 delete_rule, status
FROM user_constraints
WHERE table_name = 'EMPLOYEES';
```

|    | CONSTRAINT...     | CONSTRAINT_T... | SEARCH_CONDITION        | R_CONSTRAINT... | DELETE_... | STATUS  |
|----|-------------------|-----------------|-------------------------|-----------------|------------|---------|
| 1  | EMP_LAST_NAME...  | C               | "LAST_NAME" IS NOT NULL | (null)          | (null)     | ENABLED |
| 2  | EMP_EMAIL_NN      | C               | "EMAIL" IS NOT NULL     | (null)          | (null)     | ENABLED |
| 3  | EMP_HIRE_DATE_... | C               | "HIRE_DATE" IS NOT NULL | (null)          | (null)     | ENABLED |
| 4  | EMP_JOB_NN        | C               | "JOB_ID" IS NOT NULL    | (null)          | (null)     | ENABLED |
| 5  | EMP_SALARY_MIN    | C               | salary > 0              | (null)          | (null)     | ENABLED |
| 6  | EMP_EMAIL_UK      | U               | (null)                  | (null)          | (null)     | ENABLED |
| 7  | EMP_EMP_ID_PK     | P               | (null)                  | (null)          | (null)     | ENABLED |
| 8  | EMP_DEPT_FK       | R               | (null)                  | DEPT_ID_PK      | NO ACTION  | ENABLED |
| 9  | EMP_JOB_FK        | R               | (null)                  | JOB_ID_PK       | NO ACTION  | ENABLED |
| 10 | EMP_MANAGER_FK    | R               | (null)                  | EMP_EMP_ID_PK   | NO ACTION  | ENABLED |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### USER\_CONSTRAINTS: Example

In the example shown, the USER\_CONSTRAINTS view is queried to find the names, types, check conditions, name of the unique constraint that the foreign key references, deletion rule for a foreign key, and status for constraints on the EMPLOYEES table.

The CONSTRAINT\_TYPE can be:

- C (check constraint on a table)
- P (primary key)
- U (unique key)
- R (referential integrity)
- V (with check option, on a view)
- O (with read-only, on a view)

The DELETE\_RULE can be:

- **CASCADE:** If the parent record is deleted, the child records are deleted too.
- **NO ACTION:** A parent record can be deleted only if no child records exist.

The STATUS can be:

- **ENABLED:** Constraint is active.
- **DISABLED:** Constraint is made not active.

## Querying USER\_CONS\_COLUMNS

```
DESCRIBE user_cons_columns
```

| Name            | Null     | Type           |
|-----------------|----------|----------------|
| OWNER           | NOT NULL | VARCHAR2(30)   |
| CONSTRAINT_NAME | NOT NULL | VARCHAR2(30)   |
| TABLE_NAME      | NOT NULL | VARCHAR2(30)   |
| COLUMN_NAME     |          | VARCHAR2(4000) |
| POSITION        |          | NUMBER         |

```
SELECT constraint_name, column_name
FROM user_cons_columns
WHERE table_name = 'EMPLOYEES';
```

|   | CONSTRAINT_NAME  | COLUMN_NAME |
|---|------------------|-------------|
| 1 | EMP_LAST_NAME_NN | LAST_NAME   |
| 2 | EMP_EMAIL_NN     | EMAIL       |
| 3 | EMP_HIRE_DATE_NN | HIRE_DATE   |
| 4 | EMP_JOB_NN       | JOB_ID      |

...

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Querying USER\_CONS\_COLUMNS

To find the names of the columns to which a constraint applies, query the USER\_CONS\_COLUMNS dictionary view. This view tells you the name of the owner of a constraint, the name of the constraint, the table that the constraint is on, the names of the columns with the constraint, and the original position of column or attribute in the definition of the object.

**Note:** A constraint may apply to more than one column.

You can also write a join between USER\_CONSTRAINTS and USER\_CONS\_COLUMNS to create customized output from both tables.

## Lesson Agenda

- Introduction to data dictionary
- Querying the dictionary views for the following:
  - Table information
  - Column information
  - Constraint information
- Querying the dictionary views for the following:
  - View information
  - Sequence information
  - Synonym information
  - Index information
- Adding a comment to a table and querying the dictionary views for comment information

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## View Information

1

```
DESCRIBE user_views
```

| Name        | Null     | Type         |
|-------------|----------|--------------|
| VIEW_NAME   | NOT NULL | VARCHAR2(30) |
| TEXT_LENGTH |          | NUMBER       |
| TEXT        |          | LONG()       |

2

```
SELECT DISTINCT view_name FROM user_views;
```

| VIEW_NAME          |
|--------------------|
| 1 EMP_DETAILS_VIEW |

3

```
SELECT text FROM user_views
WHERE view_name = 'EMP_DETAILS_VIEW';
```

| TEXT                                                                                                                                                                                                |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 SELECT e.employee_id, e.job_id, e.manager_id, e.department_id, d.location_id, l.country_id, e.first_name, e.last_name,<br>...<br>c.region_id = r.region_id AND j.job_id = e.job_id WITH READ ONLY |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### View Information

After your view is created, you can query the data dictionary view called `USER_VIEWS` to see the name of the view and the view definition. The text of the `SELECT` statement that constitutes your view is stored in a `LONG` column. The `LENGTH` column is the number of characters in the `SELECT` statement. By default, when you select from a `LONG` column, only the first 80 characters of the column's value are displayed. To see more than 80 characters in SQL\*Plus, use the `SET LONG` command:

```
SET LONG 1000
```

In the examples in the slide:

1. The `USER_VIEWS` columns are displayed. Note that this is a partial listing.
2. The names of your views are retrieved
3. The `SELECT` statement for the `EMP_DETAILS_VIEW` is displayed from the dictionary

### Data Access Using Views

When you access data using a view, the Oracle server performs the following operations:

- It retrieves the view definition from the data dictionary table `USER_VIEWS`.
- It checks access privileges for the view base table.
- It converts the view query into an equivalent operation on the underlying base table or tables. That is, data is retrieved from, or an update is made to, the base tables.

## Sequence Information

**DESCRIBE user\_sequences**

| Name          | Null     | Type         |
|---------------|----------|--------------|
| -----         | -----    | -----        |
| SEQUENCE_NAME | NOT NULL | VARCHAR2(30) |
| MIN_VALUE     |          | NUMBER       |
| MAX_VALUE     |          | NUMBER       |
| INCREMENT_BY  | NOT NULL | NUMBER       |
| CYCLE_FLAG    |          | VARCHAR2(1)  |
| ORDER_FLAG    |          | VARCHAR2(1)  |
| CACHE_SIZE    | NOT NULL | NUMBER       |
| LAST_NUMBER   | NOT NULL | NUMBER       |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Sequence Information

The `USER_SEQUENCES` view describes all sequences that you own. When you create the sequence, you specify criteria that are stored in the `USER_SEQUENCES` view. The columns in this view are:

- **SEQUENCE\_NAME:** Name of the sequence
- **MIN\_VALUE:** Minimum value of the sequence
- **MAX\_VALUE:** Maximum value of the sequence
- **INCREMENT\_BY:** Value by which the sequence is incremented
- **CYCLE\_FLAG:** Does sequence wrap around on reaching the limit?
- **ORDER\_FLAG:** Are sequence numbers generated in order?
- **CACHE\_SIZE:** Number of sequence numbers to cache
- **LAST\_NUMBER:** Last sequence number written to disk. If a sequence uses caching, the number written to disk is the last number placed in the sequence cache. This number is likely to be greater than the last sequence number that was used.





## Index Information

- `USER_INDEXES` provides information about your indexes.
- `USER_IND_COLUMNS` describes columns comprising your indexes and columns of indexes on your tables.

```
DESCRIBE user_indexes
```

| Name        | Null     | Type         |
|-------------|----------|--------------|
| INDEX_NAME  | NOT NULL | VARCHAR2(30) |
| INDEX_TYPE  |          | VARCHAR2(27) |
| TABLE_OWNER | NOT NULL | VARCHAR2(30) |
| TABLE_NAME  | NOT NULL | VARCHAR2(30) |
| TABLE_TYPE  |          | VARCHAR2(11) |
| UNIQUENESS  |          | VARCHAR2(9)  |

...

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Index Information

You query the `USER_INDEXES` view to find out the names of your indexes, the table name on which the index is created, and whether the index is unique.

**Note:** For a complete listing and description of the columns in the `USER_INDEXES` view, see “`USER_INDEXES`” in the *Oracle Database Reference 11g Release 1 (11.1)*.

## USER\_INDEXES: Examples

**a** `SELECT index_name, table_name, uniqueness  
FROM user_indexes  
WHERE table_name = 'EMPLOYEES';`

|   | INDEX_NAME        | TABLE_NAME | UNIQUENESS |
|---|-------------------|------------|------------|
| 1 | EMP_JOB_IX        | EMPLOYEES  | NONUNIQUE  |
| 2 | EMP_MANAGER_IX    | EMPLOYEES  | NONUNIQUE  |
| 3 | EMP_NAME_IX       | EMPLOYEES  | NONUNIQUE  |
| 4 | EMP_EMP_ID_PK     | EMPLOYEES  | UNIQUE     |
| 5 | EMP_EMAIL_UK      | EMPLOYEES  | UNIQUE     |
| 6 | EMP_DEPARTMENT_IX | EMPLOYEES  | NONUNIQUE  |

**b** `SELECT INDEX_NAME, TABLE_NAME  
FROM USER_INDEXES  
WHERE TABLE_NAME = 'EMP_LIB';`

|   | INDEX_NAME   | TABLE_NAME |
|---|--------------|------------|
| 1 | SYS_C0017560 | EMP_LIB    |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### USER\_INDEXES: Example

In the slide example **a**, the USER\_INDEXES view is queried to find the name of the index, name of the table on which the index is created, and whether the index is unique.

In the slide example **b**, observe that the Oracle server gives a generic name to the index that is created for the PRIMARY KEY column. The EMP\_LIB table is created by using the following code:

```
CREATE TABLE EMP_LIB
(book_id NUMBER(6) PRIMARY KEY ,
title VARCHAR2(25),
category VARCHAR2(20));
```

CREATE TABLE succeeded.

## Querying USER\_IND\_COLUMNS

```
DESCRIBE user_ind_columns
```

| Name            | Null | Type           |
|-----------------|------|----------------|
| INDEX_NAME      |      | VARCHAR2(30)   |
| TABLE_NAME      |      | VARCHAR2(30)   |
| COLUMN_NAME     |      | VARCHAR2(4000) |
| COLUMN_POSITION |      | NUMBER         |
| COLUMN_LENGTH   |      | NUMBER         |
| CHAR_LENGTH     |      | NUMBER         |
| DESCEND         |      | VARCHAR2(4)    |

```
SELECT INDEX_NAME, COLUMN_NAME, TABLE_NAME
FROM user_ind_columns
WHERE INDEX_NAME = 'LNAME_IDX';
```

|   | INDEX_NAME | COLUMN_NAME | TABLE_NAME |
|---|------------|-------------|------------|
| 1 | LNAME_IDX  | LAST_NAME   | EMP_TEST   |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Querying USER\_IND\_COLUMNS

The USER\_IND\_COLUMNS dictionary view provides information such as the name of the index, name of the indexed table, name of a column within the index, and the column's position within the index.

For the slide example, the emp\_test table and LNAME\_IDX index are created by using the following code:

```
CREATE TABLE emp_test AS SELECT * FROM employees;
CREATE INDEX LNAME_IDX ON emp_test (Last_Name);
```

## Synonym Information

```
DESCRIBE user_synonyms
```

| Name         | Null     | Type          |
|--------------|----------|---------------|
| SYNONYM_NAME | NOT NULL | VARCHAR2(30)  |
| TABLE_OWNER  |          | VARCHAR2(30)  |
| TABLE_NAME   | NOT NULL | VARCHAR2(30)  |
| DB_LINK      |          | VARCHAR2(128) |

```
SELECT *
FROM user_synonyms;
```

|   | SYNONYM_NAME | TABLE_OWNER | TABLE_NAME  | DB_LINK |
|---|--------------|-------------|-------------|---------|
| 1 | TEAM2        | ORA22       | DEPARTMENTS | (null)  |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Synonym Information

The USER\_SYNONYMS dictionary view describes private synonyms (synonyms that you own). You can query this view to find your synonyms. You can query ALL\_SYNONYMS to find out the name of all the synonyms that are available to you and the objects on which these synonyms apply.

The columns in this view are:

- **SYNONYM\_NAME:** Name of the synonym
- **TABLE\_OWNER:** Owner of the object that is referenced by the synonym
- **TABLE\_NAME:** Name of the table or view that is referenced by the synonym
- **DB\_LINK:** Name of the database link reference (if any)

## Lesson Agenda

- Introduction to data dictionary
- Querying the dictionary views for the following:
  - Table information
  - Column information
  - Constraint information
- Querying the dictionary views for the following:
  - View information
  - Sequence information
  - Synonym information
  - Index information
- Adding a comment to a table and querying the dictionary views for comment information

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Adding Comments to a Table

- You can add comments to a table or column by using the **COMMENT** statement:

```
COMMENT ON TABLE employees
IS 'Employee Information';
```

```
COMMENT ON COLUMN employees.first_name
IS 'First name of the employee';
```

- Comments can be viewed through the data dictionary views:
  - ALL\_COL\_COMMENTS
  - USER\_COL\_COMMENTS
  - ALL\_TAB\_COMMENTS
  - USER\_TAB\_COMMENTS

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Adding Comments to a Table

You can add a comment of up to 4,000 bytes about a column, table, view, or snapshot by using the **COMMENT** statement. The comment is stored in the data dictionary and can be viewed in one of the following data dictionary views in the **COMMENTS** column:

- ALL\_COL\_COMMENTS
- USER\_COL\_COMMENTS
- ALL\_TAB\_COMMENTS
- USER\_TAB\_COMMENTS

### Syntax

```
COMMENT ON {TABLE table | COLUMN table.column}
IS 'text';
```

In the syntax:

*table* Is the name of the table  
*column* Is the name of the column in a table  
*text* Is the text of the comment

You can drop a comment from the database by setting it to empty string (' '):

```
COMMENT ON TABLE employees IS ' ';
```

## Summary

In this lesson, you should have learned how to find information about your objects through the following dictionary views:

- `DICTIONARY`
- `USER_OBJECTS`
- `USER_TABLES`
- `USER_TAB_COLUMNS`
- `USER_CONSTRAINTS`
- `USER_CONS_COLUMNS`
- `USER_VIEWS`
- `USER_SEQUENCES`
- `USER_INDEXES`
- `USER_SYNONYMS`

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Summary

In this lesson, you learned about some of the dictionary views that are available to you. You can use these dictionary views to find information about your tables, constraints, views, sequences, and synonyms.

## Practice 3: Overview

This practice covers the following topics:

- Querying the dictionary views for table and column information
- Querying the dictionary views for constraint information
- Querying the dictionary views for view information
- Querying the dictionary views for sequence information
- Querying the dictionary views for synonym information
- Querying the dictionary views for index information
- Adding a comment to a table and querying the dictionary views for comment information

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Practice 3: Overview

In this practice, you query the dictionary views to find information about objects in your schema.



### Practice 3

1. Query the USER\_TABLES data dictionary view to see information about the tables that you own.

| TABLE_NAME    |
|---------------|
| 1 REGIONS     |
| 2 LOCATIONS   |
| 3 DEPARTMENTS |
| 4 JOBS        |
| 5 EMPLOYEES   |
| 6 JOB_HISTORY |
| 7 JOB_GRADES  |

...

2. Query the ALL\_TABLES data dictionary view to see information about all the tables that you can access. Exclude tables that you own.

**Note:** Your list may not exactly match the following list:

| TABLE_NAME             | OWNER |
|------------------------|-------|
| 1 DUAL                 | SYS   |
| 2 SYSTEM_PRIVILEGE_MAP | SYS   |
| 3 TABLE_PRIVILEGE_MAP  | SYS   |

...

|                         |              |
|-------------------------|--------------|
| 114 SDO_GR_STAT         | MDSYS        |
| 115 WWW_FLOW_LOV_TEMP   | FLows_030000 |
| 116 WWW_FLOW_TEMP_TABLE | FLows_030000 |

3. For a specified table, create a script that reports the column names, data types, and data types' lengths, as well as whether nulls are allowed. Prompt the user to enter the table name. Give appropriate aliases to the DATA\_PRECISION and DATA\_SCALE columns. Save this script in a file named lab\_03\_01.sql.

For example, if the user enters DEPARTMENTS, the following output results:

| COLUMN_NAME       | DATA_TYPE | DATA_LENGTH | PRECISION | SCALE    | NULLABLE |
|-------------------|-----------|-------------|-----------|----------|----------|
| 1 DEPARTMENT_ID   | NUMBER    | 22          | 4         | 0 N      |          |
| 2 DEPARTMENT_NAME | VARCHAR2  | 30          | (null)    | (null) N |          |
| 3 MANAGER_ID      | NUMBER    | 22          | 6         | 0 Y      |          |
| 4 LOCATION_ID     | NUMBER    | 22          | 4         | 0 Y      |          |

### Practice 3 (continued)

4. Create a script that reports the column name, constraint name, constraint type, search condition, and status for a specified table. You must join the `USER_CONSTRAINTS` and `USER_CONS_COLUMNS` tables to obtain all this information. Prompt the user to enter the table name. Save the script in a file named `lab_03_04.sql`.

For example, if the user enters `DEPARTMENTS`, the following output results:

|   | COLUMN_NAME     | CONSTRAINT_NAME | CONSTRAINT_TYPE | SEARCH_CONDITION              | STATUS  |
|---|-----------------|-----------------|-----------------|-------------------------------|---------|
| 1 | DEPARTMENT_NAME | DEPT_NAME_NN    | C               | "DEPARTMENT_NAME" IS NOT NULL | ENABLED |
| 2 | DEPARTMENT_ID   | DEPT_ID_PK      | P               | (null)                        | ENABLED |
| 3 | LOCATION_ID     | DEPT_LOC_FK     | R               | (null)                        | ENABLED |
| 4 | MANAGER_ID      | DEPT_MGR_FK     | R               | (null)                        | ENABLED |

5. Add a comment to the `DEPARTMENTS` table. Then query the `USER_TAB_COMMENTS` view to verify that the comment is present.

|   | COMMENTS                                                           |
|---|--------------------------------------------------------------------|
| 1 | Company department information including name, code, and location. |

6. Create a synonym for your `EMPLOYEES` table. Call it `EMP`. Then find the names of all synonyms that are in your schema.

|   | SYNONYM_NAME | TABLE_OWNER | TABLE_NAME  | DB_LINK |
|---|--------------|-------------|-------------|---------|
| 1 | TEAM2        | ORA22       | DEPARTMENTS | (null)  |
| 2 | EMP          | ORA21       | EMPLOYEES   | (null)  |

7. Run `lab_03_07.sql` to create the `dept50` view for this exercise. You need to determine the names and definitions of all the views in your schema. Create a report that retrieves view information: the view name and text from the `USER_VIEWS` data dictionary view.

**Note:** The `EMP_DETAILS_VIEW` was created as part of your schema.

**Note:** You can see the complete definition of the view if you use the Run Script (or press [F5]) in SQL Developer. If you use Execute Statement (or press [F9]) in SQL Developer, scroll horizontally in the result pane. If you use SQL\*Plus, to see more contents of a `LONG` column, use the command `SET LONG n`, where `n` is the value of the number of characters of the `LONG` column that you want to see.

|   | VIEW_NAME        | TEXT                                                                          |
|---|------------------|-------------------------------------------------------------------------------|
| 1 | DEPT50           | SELECT employee_id empno, last_name employee, department_id deptno            |
| 2 | EMP_DETAILS_VIEW | SELECT e.employee_id, e.job_id, e.manager_id, e.department_id, d.location_id, |

8. Find the names of your sequences. Write a query in a script to display the following information about your sequences: sequence name, maximum value, increment size, and last number. Name the script `lab_03_08.sql`. Run the statement in your script.





# 4

## Manipulating Large Data Sets

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Oracle Internal & Oracle Academy  
Use Only

## Objectives

After completing this lesson, you should be able to do the following:

- Manipulate data using subqueries
- Specify explicit default values in the `INSERT` and `UPDATE` statements
- Describe the features of multitable `INSERTs`
- Use the following types of multitable `INSERTs`:
  - Unconditional `INSERT`
  - Pivoting `INSERT`
  - Conditional `INSERT ALL`
  - Conditional `INSERT FIRST`
- Merge rows in a table
- Track the changes to data over a period of time

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Objectives

In this lesson, you learn how to manipulate data in the Oracle database by using subqueries. You learn how to use the `DEFAULT` keyword in `INSERT` and `UPDATE` statements to identify a default column value. You also learn about multitable `INSERT` statements, the `MERGE` statement, and tracking changes in the database.

## Lesson Agenda

- Manipulating data using subqueries
- Specifying explicit default values in the `INSERT` and `UPDATE` statements
- Using the following types of multitable `INSERT`s:
  - Unconditional `INSERT`
  - Pivoting `INSERT`
  - Conditional `INSERT ALL`
  - Conditional `INSERT FIRST`
- Merging rows in a table
- Tracking the changes to data over a period of time

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Oracle Internal & Oracle Academy  
Use Only

## Using Subqueries to Manipulate Data

You can use subqueries in data manipulation language (DML) statements to:

- Retrieve data using an inline view
- Copy data from one table to another
- Update data in one table based on the values of another table
- Delete rows from one table based on rows in another table

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Using Subqueries to Manipulate Data

Subqueries can be used to retrieve data from a table that you can use as input to an INSERT into a different table. In this way, you can easily copy large volumes of data from one table to another with one single SELECT statement. Similarly, you can use subqueries to do mass updates and deletes by using them in the WHERE clause of the UPDATE and DELETE statements. You can also use subqueries in the FROM clause of a SELECT statement. This is called an inline view.

**Note:** You learned how to update and delete rows based on another table in the course titled *Oracle Database 11g: SQL Fundamentals I*.



## Retrieving Data Using a Subquery as Source

```
SELECT department_name, city
FROM departments
NATURAL JOIN (SELECT l.location_id, l.city, l.country_id
 FROM loc l
 JOIN countries c
 ON(l.country_id = c.country_id)
 JOIN regions USING(region_id)
 WHERE region_name = 'Europe');
```

| DEPARTMENT_NAME | CITY   |
|-----------------|--------|
| 1 Sales         | Oxford |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Retrieving Data Using a Subquery as Source

You can use a subquery in the FROM clause of a SELECT statement, which is very similar to how views are used. A subquery in the FROM clause of a SELECT statement is also called an *inline* view. A subquery in the FROM clause of a SELECT statement defines a data source for that particular SELECT statement, and only that SELECT statement. As with a database view, the SELECT statement in the subquery can be as simple or as complex as you like.

When a database view is created, the associated SELECT statement is stored in the data dictionary. In situations where you do not have the necessary privileges to create database views, or when you would like to test the suitability of a SELECT statement to become a view, you can use an inline view.

With inline views, you can have all the code needed to support the query in one place. This means that you can avoid the complexity of creating a separate database view. The slide example shows how to use an inline view to display the department name and the city in Europe. The subquery in the FROM clause fetches the location ID, city name, and the country by joining three different tables. The output of the inner query is considered as a table for the outer query. The inner query is similar to that of a database view but does not have any physical name.

For the example in the slide, the loc table is created by running the following statement:

```
CREATE TABLE loc AS SELECT * FROM locations;
```

## Retrieving Data Using a Subquery as Source (continued)

You can display the same output as in the slide example by performing the following two steps:

1. Create a database view:

```
CREATE OR REPLACE VIEW european_cities
AS
SELECT l.location_id, l.city, l.country_id
FROM loc l
JOIN countries c
ON(l.country_id = c.country_id)
JOIN regions USING(region_id)
WHERE region_name = 'Europe';
```

2. Join the EUROPEAN\_CITIES view with the DEPARTMENTS table:

```
SELECT department_name, city
FROM departments
NATURAL JOIN european_cities;
```

**Note:** You learned how to create database views in the course titled *Oracle Database 11g: SQL Fundamentals I*.

Oracle Internal & Oracle Academy  
Use Only

## Inserting Using a Subquery as a Target

```
INSERT INTO (SELECT l.location_id, l.city, l.country_id
 FROM loc l
 JOIN countries c
 ON(l.country_id = c.country_id)
 JOIN regions USING(region_id)
 WHERE region_name = 'Europe')
VALUES (3300, 'Cardiff', 'UK');
```

1 rows inserted

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Inserting Using a Subquery as a Target

You can use a subquery in place of the table name in the INTO clause of the INSERT statement. The SELECT list of this subquery must have the same number of columns as the column list of the VALUES clause. Any rules on the columns of the base table must be followed in order for the INSERT statement to work successfully. For example, you cannot put in a duplicate location ID or leave out a value for a mandatory NOT NULL column.

This application of subqueries helps you avoid having to create a view just for performing an INSERT.

The slide example uses a subquery in the place of LOC to create a record for a new European city.

**Note:** You can also perform the INSERT operation on the EUROPEAN\_CITIES view by using the following code:

```
INSERT INTO european_cities
VALUES (3300, 'Cardiff', 'UK');
```

## Inserting Using a Subquery as a Target

Verify the results.

```
SELECT location_id, city, country_id
FROM loc
```

|   | LOCATION_ID | CITY                     | COUNTRY_ID |
|---|-------------|--------------------------|------------|
| 1 |             | 1400 Southlake           | US         |
| 2 |             | 1500 South San Francisco | US         |
| 3 |             | 1700 Seattle             | US         |
| 4 |             | 1800 Toronto             | CA         |
| 5 |             | 2500 Oxford              | UK         |
| 6 |             | 3300 Cardiff             | UK         |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Inserting Using a Subquery as a Target (continued)

The slide example shows that the insert via the inline view created a new record in the base table LOC.

The following example shows the results of the subquery that was used to identify the table for the INSERT statement.

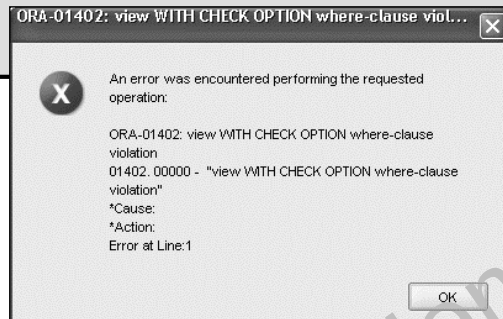
```
SELECT l.location_id, l.city, l.country_id
FROM loc l
JOIN countries c
ON(l.country_id = c.country_id)
JOIN regions USING(region_id)
WHERE region_name = 'Europe'
```

|   | LOCATION_ID | CITY         | COUNTRY_ID |
|---|-------------|--------------|------------|
| 1 |             | 2500 Oxford  | UK         |
| 2 |             | 3300 Cardiff | UK         |

## Using the WITH CHECK OPTION Keyword on DML Statements

The WITH CHECK OPTION keyword prohibits you from changing rows that are not in the subquery.

```
INSERT INTO (SELECT location_id, city, country_id
 FROM loc
 WHERE country_id IN
 (SELECT country_id
 FROM countries
 NATURAL JOIN regions
 WHERE region_name = 'Europe')
 WITH CHECK OPTION)
VALUES (3600, 'Washington', 'US');
```



ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Using the WITH CHECK OPTION Keyword on DML Statements

Specify the WITH CHECK OPTION keyword to indicate that if the subquery is used in place of a table in an INSERT, UPDATE, or DELETE statement, no changes that produce rows that are not included in the subquery are permitted to that table.

The slide example shows how to use an inline view with the WITH CHECK OPTION. The INSERT statement prevents the creation of records in the LOC table for a city that is not in Europe.

The following example executes successfully because of the changes in the VALUES list.

```
INSERT INTO (SELECT location_id, city, country_id
 FROM loc
 WHERE country_id IN
 (SELECT country_id
 FROM countries
 NATURAL JOIN regions
 WHERE region_name = 'Europe')
 WITH CHECK OPTION)
VALUES (3500, 'Berlin', 'DE');
```

## Using the WITH CHECK OPTION Keyword on DML Statements (continued)

The application of an inline view with the WITH CHECK OPTION provides an easy method to prevent changes to the table.

To prevent the creation of a non-European city, you can also use a database view by performing the following steps:

1. Create a database view:

```
CREATE OR REPLACE VIEW european_cities
AS
SELECT location_id, city, country_id
FROM locations
WHERE country_id in
 (SELECT country_id
 FROM countries
 NATURAL JOIN regions
 WHERE region_name = 'Europe')
WITH CHECK OPTION;
```

2. Verify the results by inserting data:

```
INSERT INTO european_cities
VALUES (3400, 'New York', 'US');
```

The second step produces the same error as shown in the slide.

Oracle Internal & Oracle Academy  
Use Only

## Lesson Agenda

- Manipulating data using subqueries
- Specifying explicit default values in the INSERT and UPDATE statements
- Using the following types of multitable INSERTs:
  - Unconditional INSERT
  - Pivoting INSERT
  - Conditional INSERT ALL
  - Conditional INSERT FIRST
- Merging rows in a table
- Tracking the changes to data over a period of time

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Oracle Internal & Oracle Academy  
Use Only

## Overview of the Explicit Default Feature

- Use the `DEFAULT` keyword as a column value where the default column value is desired.
- This allows the user to control where and when the default value should be applied to data.
- Explicit defaults can be used in `INSERT` and `UPDATE` statements.

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Explicit Defaults

The `DEFAULT` keyword can be used in `INSERT` and `UPDATE` statements to identify a default column value. If no default value exists, a null value is used.

The `DEFAULT` option saves you from hard coding the default value in your programs or querying the dictionary to find it, as was done before this feature was introduced. Hard coding the default is a problem if the default changes because the code consequently needs changing. Accessing the dictionary is not usually done in an application program, so this is a very important feature.



## Using Explicit Default Values

- DEFAULT with INSERT:

```
INSERT INTO deptm3
 (department_id, department_name, manager_id)
VALUES (300, 'Engineering', DEFAULT);
```

- DEFAULT with UPDATE:

```
UPDATE deptm3
SET manager_id = DEFAULT
WHERE department_id = 10;
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Using Explicit Default Values

Specify DEFAULT to set the column to the value previously specified as the default value for the column. If no default value for the corresponding column has been specified, the Oracle server sets the column to null.

In the first example in the slide, the INSERT statement uses a default value for the MANAGER\_ID column. If there is no default value defined for the column, a null value is inserted instead.

The second example uses the UPDATE statement to set the MANAGER\_ID column to a default value for department 10. If no default value is defined for the column, it changes the value to null.

**Note:** When creating a table, you can specify a default value for a column. This is discussed in *SQL Fundamentals I*.

## Copying Rows from Another Table

- Write your INSERT statement with a subquery.

```
INSERT INTO sales_reps(id, name, salary, commission_pct)
SELECT employee_id, last_name, salary, commission_pct
FROM employees
WHERE job_id LIKE '%REP%';
```

4 rows inserted

- Do not use the VALUES clause.
- Match the number of columns in the INSERT clause with that in the subquery.

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Copying Rows from Another Table

You can use the INSERT statement to add rows to a table where the values are derived from existing tables. In place of the VALUES clause, you use a subquery.

#### Syntax

```
INSERT INTO table [column (, column)] subquery;
```

In the syntax:

|                 |                                                    |
|-----------------|----------------------------------------------------|
| <i>table</i>    | Is the table name                                  |
| <i>column</i>   | Is the name of the column in the table to populate |
| <i>subquery</i> | Is the subquery that returns rows into the table   |

The number of columns and their data types in the column list of the INSERT clause must match the number of values and their data types in the subquery. To create a copy of the rows of a table, use SELECT \* in the subquery.

```
INSERT INTO EMPL3
SELECT *
FROM employees;
```

**Note:** You use the LOG ERRORS clause in your DML statement to enable the DML operation to complete regardless of errors. Oracle writes the details of the error message to an error-logging table that you have created. For more information, see *Oracle Database 11g SQL Reference*.

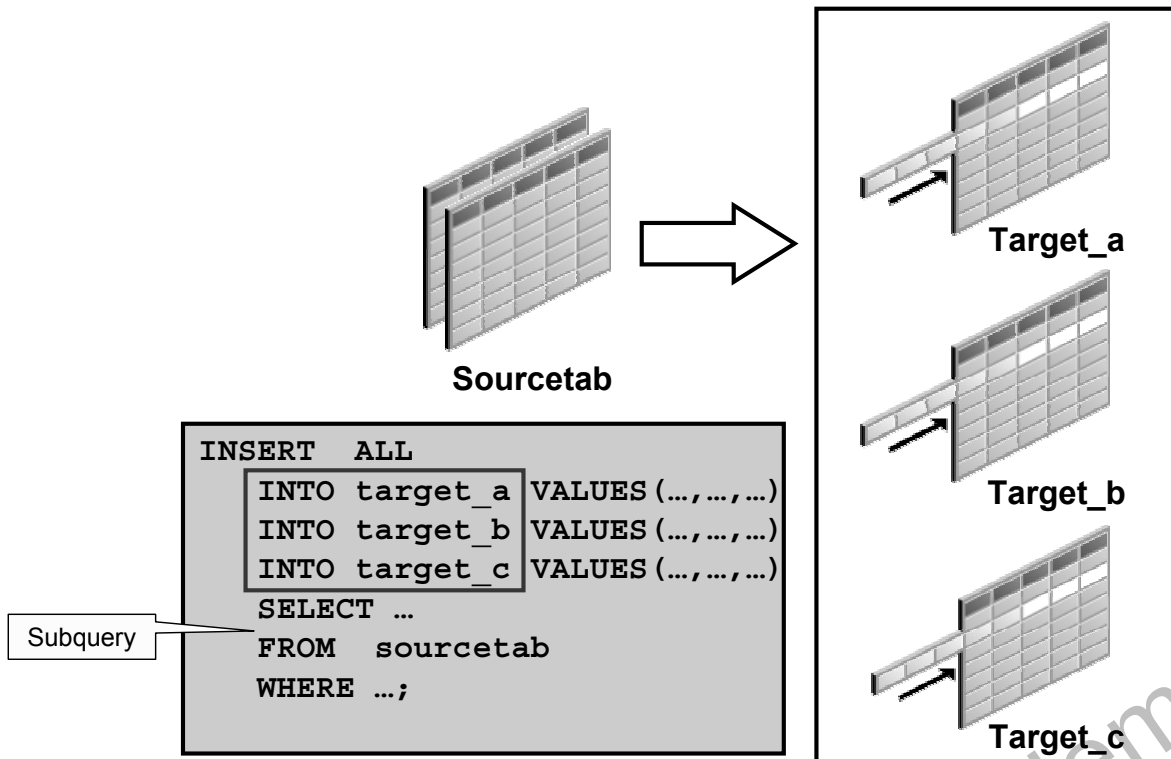
## Lesson Agenda

- Manipulating data using subqueries
- Specifying explicit default values in the `INSERT` and `UPDATE` statements
- Using the following types of multitable `INSERT`s:
  - Unconditional `INSERT`
  - Pivoting `INSERT`
  - Conditional `INSERT ALL`
  - Conditional `INSERT FIRST`
- Merging rows in a table
- Tracking the changes to data over a period of time

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Overview of Multitable INSERT Statements



ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Overview of Multitable INSERT Statements

In a multitable INSERT statement, you insert computed rows derived from the rows returned from the evaluation of a subquery into one or more tables.

Multitable INSERT statements are useful in a data warehouse scenario. You need to load your data warehouse regularly so that it can serve its purpose of facilitating business analysis. To do this, data from one or more operational systems must be extracted and copied into the warehouse. The process of extracting data from the source system and bringing it into the data warehouse is commonly called ETL, which stands for extraction, transformation, and loading.

During extraction, the desired data must be identified and extracted from many different sources, such as database systems and applications. After extraction, the data must be physically transported to the target system or an intermediate system for further processing. Depending on the chosen means of transportation, some transformations can be done during this process. For example, a SQL statement that directly accesses a remote target through a gateway can concatenate two columns as part of the SELECT statement.

After data is loaded into the Oracle database, data transformations can be executed using SQL operations. A multitable INSERT statement is one of the techniques for implementing SQL data transformations.

## Overview of Multitable INSERT Statements

- Use the `INSERT...SELECT` statement to insert rows into multiple tables as part of a single DML statement.
- Multitable `INSERT` statements are used in data warehousing systems to transfer data from one or more operational sources to a set of target tables.
- They provide significant performance improvement over:
  - Single DML versus multiple `INSERT...SELECT` statements
  - Single DML versus a procedure to perform multiple inserts by using the `IF . . . THEN` syntax

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Overview of Multitable INSERT Statements (continued)

Multitable `INSERT` statements offer the benefits of the `INSERT . . . SELECT` statement when multiple tables are involved as targets. Without multitable `INSERT`, you had to deal with  $n$  independent `INSERT . . . SELECT` statements, thus processing the same source data  $n$  times and increasing the transformation workload  $n$  times.

As with the existing `INSERT . . . SELECT` statement, the new statement can be parallelized and used with the direct-load mechanism for faster performance.

Each record from any input stream, such as a nonrelational database table, can now be converted into multiple records for a more relational database table environment. To alternatively implement this functionality, you were required to write multiple `INSERT` statements.

## Types of Multitable INSERT Statements

The different types of multitable INSERT statements are:

- Unconditional INSERT
- Conditional INSERT ALL
- Pivoting INSERT
- Conditional INSERT FIRST

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Types of Multitable INSERT Statements

You use different clauses to indicate the type of INSERT to be executed. The types of multitable INSERT statements are:

- **Unconditional INSERT:** For each row returned by the subquery, a row is inserted into each of the target tables.
- **Conditional INSERT ALL:** For each row returned by the subquery, a row is inserted into each target table if the specified condition is met.
- **Pivoting INSERT:** This is a special case of the unconditional INSERT ALL.
- **Conditional INSERT FIRST:** For each row returned by the subquery, a row is inserted into the very first target table in which the condition is met.

## Multitable INSERT Statements

- Syntax for multitable INSERT:

```
INSERT [conditional_insert_clause]
[insert_into_clause values_clause] (subquery)
```

- conditional\_insert\_clause:

```
[ALL] [FIRST]
[WHEN condition THEN] [insert_into_clause values_clause]
[ELSE] [insert_into_clause values_clause]
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Multitable INSERT Statements

The slide displays the generic format for multitable INSERT statements.

#### Unconditional INSERT: ALL into\_clause

Specify ALL followed by multiple insert\_into\_clauses to perform an unconditional multitable INSERT. The Oracle server executes each insert\_into\_clause once for each row returned by the subquery.

#### Conditional INSERT: conditional\_insert\_clause

Specify the conditional\_insert\_clause to perform a conditional multitable INSERT. The Oracle server filters each insert\_into\_clause through the corresponding WHEN condition, which determines whether that insert\_into\_clause is executed. A single multitable INSERT statement can contain up to 127 WHEN clauses.

#### Conditional INSERT: ALL

If you specify ALL, the Oracle server evaluates each WHEN clause regardless of the results of the evaluation of any other WHEN clause. For each WHEN clause whose condition evaluates to true, the Oracle server executes the corresponding INTO clause list.

## **Multitable INSERT Statements (continued)**

### **Conditional INSERT: FIRST**

If you specify **FIRST**, the Oracle server evaluates each **WHEN** clause in the order in which it appears in the statement. If the first **WHEN** clause evaluates to true, the Oracle server executes the corresponding **INTO** clause and skips subsequent **WHEN** clauses for the given row.

### **Conditional INSERT: ELSE Clause**

For a given row, if no **WHEN** clause evaluates to true:

- If you have specified an **ELSE** clause, the Oracle server executes the **INTO** clause list associated with the **ELSE** clause
- If you did not specify an **ELSE** clause, the Oracle server takes no action for that row

### **Restrictions on Multitable INSERT Statements**

- You can perform multitable **INSERT** statements only on tables, and not on views or materialized views.
- You cannot perform a multitable **INSERT** on a remote table.
- You cannot specify a table collection expression when performing a multitable **INSERT**.
- In a multitable **INSERT**, all `insert_into_clauses` cannot combine to specify more than 999 target columns.

Oracle Internal & Oracle Academy  
Use Only



## Unconditional INSERT ALL

- Select the EMPLOYEE\_ID, HIRE\_DATE, SALARY, and MANAGER\_ID values from the EMPLOYEES table for those employees whose EMPLOYEE\_ID is greater than 200.
- Insert these values into the SAL\_HISTORY and MGR\_HISTORY tables by using a multitable INSERT.

```
INSERT ALL
 INTO sal_history VALUES (EMPID, HIREDATE, SAL)
 INTO mgr_history VALUES (EMPID, MGR, SAL)
 SELECT employee_id EMPID, hire_date HIREDATE,
 salary SAL, manager_id MGR
 FROM employees
 WHERE employee_id > 200;
```

8 rows inserted

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Unconditional INSERT ALL

The example in the slide inserts rows into both the SAL\_HISTORY and the MGR\_HISTORY tables. The SELECT statement retrieves the details of employee ID, hire date, salary, and manager ID of those employees whose employee ID is greater than 200 from the EMPLOYEES table. The details of the employee ID, hire date, and salary are inserted into the SAL\_HISTORY table. The details of employee ID, manager ID, and salary are inserted into the MGR\_HISTORY table.

This INSERT statement is referred to as an unconditional INSERT because no further restriction is applied to the rows that are retrieved by the SELECT statement. All the rows retrieved by the SELECT statement are inserted into the two tables: SAL\_HISTORY and MGR\_HISTORY. The VALUES clause in the INSERT statements specifies the columns from the SELECT statement that must be inserted into each of the tables. Each row returned by the SELECT statement results in two insertions: one for the SAL\_HISTORY table and one for the MGR\_HISTORY table.

## Unconditional INSERT ALL (continued)

A total of eight rows were inserted:

```
SELECT COUNT(*) total_in_sal FROM sal_history;
```

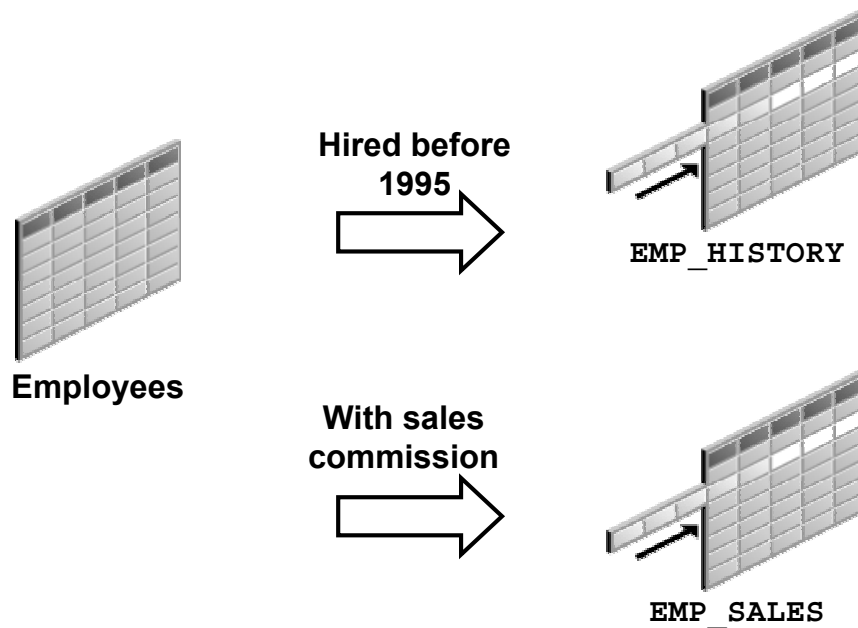
|   |  TOTAL_IN_SAL |
|---|------------------------------------------------------------------------------------------------|
| 1 | 4                                                                                              |

```
SELECT COUNT(*) total_in_mgr FROM mgr_history;
```

|   |  TOTAL_IN_MGR |
|---|------------------------------------------------------------------------------------------------|
| 1 | 4                                                                                              |

Oracle Internal & Oracle Academy  
Use Only

## Conditional INSERT ALL: Example



ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Conditional INSERT ALL: Example

For all employees in the employees tables, if the employee was hired before 1995, then insert that employee record into the employee history. If the employee earns a sales commission, then insert the record information into the EMP\_SALES table. The SQL statement is shown on the next page.

## Conditional INSERT ALL

```
INSERT ALL
 WHEN HIREDATE < '01-JAN-95' THEN
 INTO emp_history VALUES (EMPID, HIREDATE, SAL)
 WHEN COMM IS NOT NULL THEN
 INTO emp_sales VALUES (EMPID, COMM, SAL)
SELECT employee_id EMPID, hire_date HIREDATE,
 salary SAL, commission_pct COMM
FROM employees
```

12 rows inserted

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Conditional INSERT ALL

The example in the slide is similar to the example in the previous slide because it inserts rows into both the EMP\_HISTORY and the EMP\_SALES tables. The SELECT statement retrieves details such as employee ID, hire date, salary, and commission percentage for all employees from the EMPLOYEES table. Details such as employee ID, hire date, and salary are inserted into the EMP\_HISTORY table. Details such as employee ID, commission percentage, and salary are inserted into the EMP\_SALES table.

This INSERT statement is referred to as a conditional INSERT ALL because a further restriction is applied to the rows that are retrieved by the SELECT statement. From the rows that are retrieved by the SELECT statement, only those rows in which the hire date was prior to 1995 are inserted in the EMP\_HISTORY table. Similarly, only those rows where the value of commission percentage is not null are inserted in the EMP\_SALES table.

```
SELECT count(*) FROM emp_history;
```

|   | AZ | COUNT(*) |
|---|----|----------|
| 1 |    | 8        |

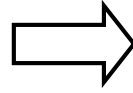
```
SELECT count(*) FROM emp_sales;
```

|   | AZ | COUNT(*) |
|---|----|----------|
| 1 |    | 4        |

## Conditional INSERT FIRST: Example

**Scenario:** If an employee salary is 2,000, the record is inserted into the SAL\_LOW table only.

**Salary < 5,000**



**SAL\_LOW**

SAL\_LOW

## EMPLOYEES

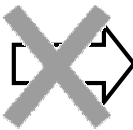
**5000 <= Salary  
<= 10,000**



**SAL 1**

SAL MID

**Otherwise**



**SAL H**

SAL HIGH

ORACLE®

Copyright © 2007, Oracle. All rights reserved.

### Conditional INSERT FIRST: Example

For all employees in the EMPLOYEES table, insert the employee information into the first target table that meets the condition. In the example, if an employee has a salary of 2,000, the record is inserted into the SAL LOW table only. The SQL statement is shown on the next page.

## Conditional INSERT FIRST

```
INSERT FIRST
WHEN salary < 5000 THEN
 INTO sal_low VALUES (employee_id, last_name, salary)
WHEN salary between 5000 and 10000 THEN
 INTO sal_mid VALUES (employee_id, last_name, salary)
ELSE
 INTO sal_high VALUES (employee_id, last_name, salary)
SELECT employee_id, last_name, salary
FROM employees
```

20 rows inserted

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Conditional INSERT FIRST

The SELECT statement retrieves details such as employee ID, last name, and salary for every employee in the EMPLOYEES table. For each employee record, it is inserted into the very first target table that meets the condition.

This INSERT statement is referred to as a conditional INSERT FIRST. The WHEN salary < 5000 condition is evaluated first. If this first WHEN clause evaluates to true, the Oracle server executes the corresponding INTO clause and inserts the record into the SAL\_LOW table. It skips subsequent WHEN clauses for this row.

If the row that does not satisfy the first WHEN condition (WHEN salary < 5000), the next condition (WHEN salary between 5000 and 10000) is evaluated. If this condition evaluates to true, the record is inserted into the SAL\_MID table, and the last condition is skipped.

If neither the first condition (WHEN salary < 5000) nor the second condition (WHEN salary between 5000 and 10000) is evaluated to true, the Oracle server executes the corresponding INTO clause for the ELSE clause.

## Conditional INSERT FIRST (continued)

A total of 20 rows were inserted:

```
SELECT count(*) low FROM sal_low;
```

| A<br>Z |  | LOW |
|--------|--|-----|
| 1      |  | 6   |

```
SELECT count(*) mid FROM sal_mid;
```

| A<br>Z |  | MID |
|--------|--|-----|
| 1      |  | 7   |

```
SELECT count(*) high FROM sal_high;
```

| A<br>Z |  | HIGH |
|--------|--|------|
| 1      |  | 7    |

Oracle Internal & Oracle Academy  
Use Only

## Pivoting INSERT

Convert the set of sales records from the nonrelational database table to relational format.

| Emp_ID | Week_ID | MON  | TUES | WED  | THUR | FRI  |
|--------|---------|------|------|------|------|------|
| 176    | 6       | 2000 | 3000 | 4000 | 5000 | 6000 |



| Employee_ID | WEEK | SALES |
|-------------|------|-------|
| 176         | 6    | 2000  |
| 176         | 6    | 3000  |
| 176         | 6    | 4000  |
| 176         | 6    | 5000  |
| 176         | 6    | 6000  |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Pivoting INSERT

Pivoting is an operation in which you must build a transformation such that each record from any input stream, such as a nonrelational database table, must be converted into multiple records for a more relational database table environment.

Suppose you receive a set of sales records from a nonrelational database table:

SALES\_SOURCE\_DATA, in the following format:

EMPLOYEE\_ID, WEEK\_ID, SALES\_MON, SALES\_TUE, SALES\_WED,  
SALES\_THUR, SALES\_FRI

You want to store these records in the SALES\_INFO table in a more typical relational format:

EMPLOYEE\_ID, WEEK, SALES

To solve this problem, you must build a transformation such that each record from the original nonrelational database table, SALES\_SOURCE\_DATA, is converted into five records for the data warehouse's SALES\_INFO table. This operation is commonly referred to as *pivoting*.

The solution to this problem is shown on the next page.



## Pivoting INSERT

```
INSERT ALL
 INTO sales_info VALUES (employee_id,week_id,sales_MON)
 INTO sales_info VALUES (employee_id,week_id,sales_TUE)
 INTO sales_info VALUES (employee_id,week_id,sales_WED)
 INTO sales_info VALUES (employee_id,week_id,sales_THUR)
 INTO sales_info VALUES (employee_id,week_id,sales_FRI)
SELECT EMPLOYEE_ID, week_id, sales_MON, sales_TUE,
 sales_WED, sales_THUR,sales_FRI
FROM sales_source_data;
```

5 rows inserted

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Pivoting INSERT (continued)

In the example in the slide, the sales data is received from the nonrelational database table SALES\_SOURCE\_DATA, which is the details of the sales performed by a sales representative on each day of a week, for a week with a particular week ID.

DESC SALES\_SOURCE\_DATA

| Name        | Null | Type        |
|-------------|------|-------------|
| EMPLOYEE_ID |      | NUMBER(6)   |
| WEEK_ID     |      | NUMBER(2)   |
| SALES_MON   |      | NUMBER(8,2) |
| SALES_TUE   |      | NUMBER(8,2) |
| SALES_WED   |      | NUMBER(8,2) |
| SALES_THUR  |      | NUMBER(8,2) |
| SALES_FRI   |      | NUMBER(8,2) |

## Pivoting INSERT (continued)

```
SELECT * FROM SALES_SOURCE_DATA;
```

|   | EMPLOYEE_ID | WEEK_ID | SALES_MON | SALES_TUE | SALES_WED | SALES_THUR | SALES_FRI |
|---|-------------|---------|-----------|-----------|-----------|------------|-----------|
| 1 | 178         | 6       | 1750      | 2200      | 1500      | 1500       | 3000      |

```
DESC SALES_INFO
```

| Name        | Null  | Type        |
|-------------|-------|-------------|
| -----       | ----- | -----       |
| EMPLOYEE_ID |       | NUMBER(6)   |
| WEEK        |       | NUMBER(2)   |
| SALES       |       | NUMBER(8,2) |

```
SELECT * FROM sales_info;
```

|   | EMPLOYEE_ID | WEEK | SALES |
|---|-------------|------|-------|
| 1 | 178         | 6    | 1750  |
| 2 | 178         | 6    | 2200  |
| 3 | 178         | 6    | 1500  |
| 4 | 178         | 6    | 1500  |
| 5 | 178         | 6    | 3000  |

Observe in the preceding example that by using a pivoting INSERT, one row from the SALES\_SOURCE\_DATA table is converted into five records for the relational table, SALES\_INFO.

## Lesson Agenda

- Manipulating data using subqueries
- Specifying explicit default values in the `INSERT` and `UPDATE` statements
- Using the following types of multitable `INSERT`s:
  - Unconditional `INSERT`
  - Pivoting `INSERT`
  - Conditional `INSERT ALL`
  - Conditional `INSERT FIRST`
- Merging rows in a table
- Tracking the changes to data over a period of time

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Oracle Internal & Oracle Academy  
Use Only

## MERGE Statement

- Provides the ability to conditionally update, insert, or delete data into a database table
- Performs an `UPDATE` if the row exists, and an `INSERT` if it is a new row:
  - Avoids separate updates
  - Increases performance and ease of use
  - Is useful in data warehousing applications

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### MERGE Statement

The Oracle server supports the `MERGE` statement for `INSERT`, `UPDATE`, and `DELETE` operations. Using this statement, you can update, insert, or delete a row conditionally into a table, thus avoiding multiple DML statements. The decision whether to update, insert, or delete into the target table is based on a condition in the `ON` clause.

You must have the `INSERT` and `UPDATE` object privileges on the target table and the `SELECT` object privilege on the source table. To specify the `DELETE` clause of `merge_update_clause`, you must also have the `DELETE` object privilege on the target table.

The `MERGE` statement is deterministic. You cannot update the same row of the target table multiple times in the same `MERGE` statement.

An alternative approach is to use PL/SQL loops and multiple DML statements. The `MERGE` statement, however, is easy to use and more simply expressed as a single SQL statement.

The `MERGE` statement is suitable in a number of data warehousing applications. For example, in a data warehousing application, you may need to work with data coming from multiple sources, some of which may be duplicates. With the `MERGE` statement, you can conditionally add or modify rows.

## MERGE Statement Syntax

You can conditionally insert, update, or delete rows in a table by using the MERGE statement.

```
MERGE INTO table_name table_alias
 USING (table/view/sub_query) alias
 ON (join condition)
 WHEN MATCHED THEN
 UPDATE SET
 col1 = col1_val,
 col2 = col2_val
 WHEN NOT MATCHED THEN
 INSERT (column_list)
 VALUES (column_values);
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Merging Rows

You can update existing rows, and insert new rows conditionally by using the MERGE statement. Using the MERGE statement, you can delete obsolete rows at the same time as you update rows in a table. To do this, you include a DELETE clause with its own WHERE clause in the syntax of the MERGE statement.

In the syntax:

|                  |                                                                                                |
|------------------|------------------------------------------------------------------------------------------------|
| INTO clause      | Specifies the target table you are updating or inserting into                                  |
| USING clause     | Identifies the source of the data to be updated or inserted; can be a table, view, or subquery |
| ON clause        | The condition on which the MERGE operation either updates or inserts                           |
| WHEN MATCHED     | Instructs the server how to respond to the results of the join condition                       |
| WHEN NOT MATCHED |                                                                                                |

For more information, see *Oracle Database 11g SQL Reference*.

## Merging Rows: Example

Insert or update rows in the COPY\_EMP3 table to match the EMPLOYEES table.

```
MERGE INTO copy_emp3 c
USING (SELECT * FROM EMPLOYEES) e
ON (c.employee_id = e.employee_id)
WHEN MATCHED THEN
UPDATE SET
c.first_name = e.first_name,
c.last_name = e.last_name,
...
DELETE WHERE (E.COMMISSION_PCT IS NOT NULL)
WHEN NOT MATCHED THEN
INSERT VALUES(e.employee_id, e.first_name, e.last_name,
e.email, e.phone_number, e.hire_date, e.job_id,
e.salary, e.commission_pct, e.manager_id,
e.department_id);
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Merging Rows: Example

```
MERGE INTO copy_emp3 c
USING (SELECT * FROM EMPLOYEES) e
ON (c.employee_id = e.employee_id)
WHEN MATCHED THEN
UPDATE SET
c.first_name = e.first_name,
c.last_name = e.last_name,
c.email = e.email,
c.phone_number = e.phone_number,
c.hire_date = e.hire_date,
c.job_id = e.job_id,
c.salary = e.salary*2,
c.commission_pct = e.commission_pct,
c.manager_id = e.manager_id,
c.department_id = e.department_id
DELETE WHERE (E.COMMISSION_PCT IS NOT NULL)
WHEN NOT MATCHED THEN
INSERT VALUES(e.employee_id, e.first_name, e.last_name,
e.email, e.phone_number, e.hire_date, e.job_id,
e.salary, e.commission_pct, e.manager_id,
e.department_id);
```

## Merging Rows: Example (continued)

The COPY\_EMP3 table is created by using the following code:

```
CREATE TABLE COPY_EMP3 AS SELECT * FROM EMPLOYEES
WHERE SALARY<10000;
```

Then query the COPY\_EMP3 table.

```
SELECT employee_id, salary, commission_pct FROM COPY_EMP3;
```

|    | EMPLOYEE_ID | SALARY | COMMISSION_PCT |
|----|-------------|--------|----------------|
| 1  | 103         | 9000   | (null)         |
| 2  | 104         | 6000   | (null)         |
| 3  | 107         | 4200   | (null)         |
| 4  | 124         | 5800   | (null)         |
| 5  | 141         | 3500   | (null)         |
| 6  | 142         | 3100   | (null)         |
| 7  | 143         | 2600   | (null)         |
| 8  | 144         | 2500   | (null)         |
| 9  | 176         | 8600   | 0.2            |
| 10 | 178         | 7000   | 0.15           |
| 11 | 200         | 4400   | (null)         |
| 12 | 202         | 6000   | (null)         |
| 13 | 206         | 8300   | (null)         |

Observe that there are thirteen employees with SALARY < 10000 and there are two employees with COMMISSION\_PCT.

The example in the slide matches the EMPLOYEE\_ID in the COPY\_EMP3 table to the EMPLOYEE\_ID in the EMPLOYEES table. If a match is found, the row in the COPY\_EMP3 table is updated to match the row in the EMPLOYEES table and the salary of the employee is doubled. The records of the two employees with values in the COMMISSION\_PCT column are deleted. If the match is not found, then rows are inserted into the COPY\_EMP3 table.

## Merging Rows: Example

```
TRUNCATE TABLE copy_emp3;
SELECT * FROM copy_emp3;
0 rows selected
```

```
MERGE INTO copy_emp3 c
USING (SELECT * FROM EMPLOYEES) e
ON (c.employee_id = e.employee_id)
WHEN MATCHED THEN
UPDATE SET
c.first_name = e.first_name,
c.last_name = e.last_name,
...
DELETE WHERE (E.COMMISSION_PCT IS NOT NULL)
WHEN NOT MATCHED THEN
INSERT VALUES(e.employee_id, e.first_name, ...
```

```
SELECT * FROM copy_emp3;
20 rows selected.
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Merging Rows: Example (continued)

The slide examples show that the COPY\_EMP3 table is empty. The `c.employee_id = e.employee_id` condition is evaluated. The condition returns false—there are no matches. The logic falls into the `WHEN NOT MATCHED` clause, and the `MERGE` command inserts the rows of the `EMPLOYEES` table into the `COPY_EMP3` table. This means that the `COPY_EMP3` table now has exactly the same data as in the `EMPLOYEES` table.

```
SELECT employee_id, salary, commission_pct from copy_emp3;
```

|     | EMPLOYEE_ID | SALARY | COMMISSION_PCT |
|-----|-------------|--------|----------------|
| 1   | 144         | 2500   | (null)         |
| 2   | 143         | 2600   | (null)         |
| 3   | 202         | 6000   | (null)         |
| 4   | 141         | 3500   | (null)         |
| 5   | 174         | 11000  | 0.3            |
| ... |             |        |                |
| 15  | 149         | 10500  | 0.2            |
| 16  | 206         | 8300   | (null)         |
| 17  | 176         | 8600   | 0.2            |
| 18  | 124         | 5800   | (null)         |
| 19  | 205         | 12000  | (null)         |
| 20  | 178         | 7000   | 0.15           |



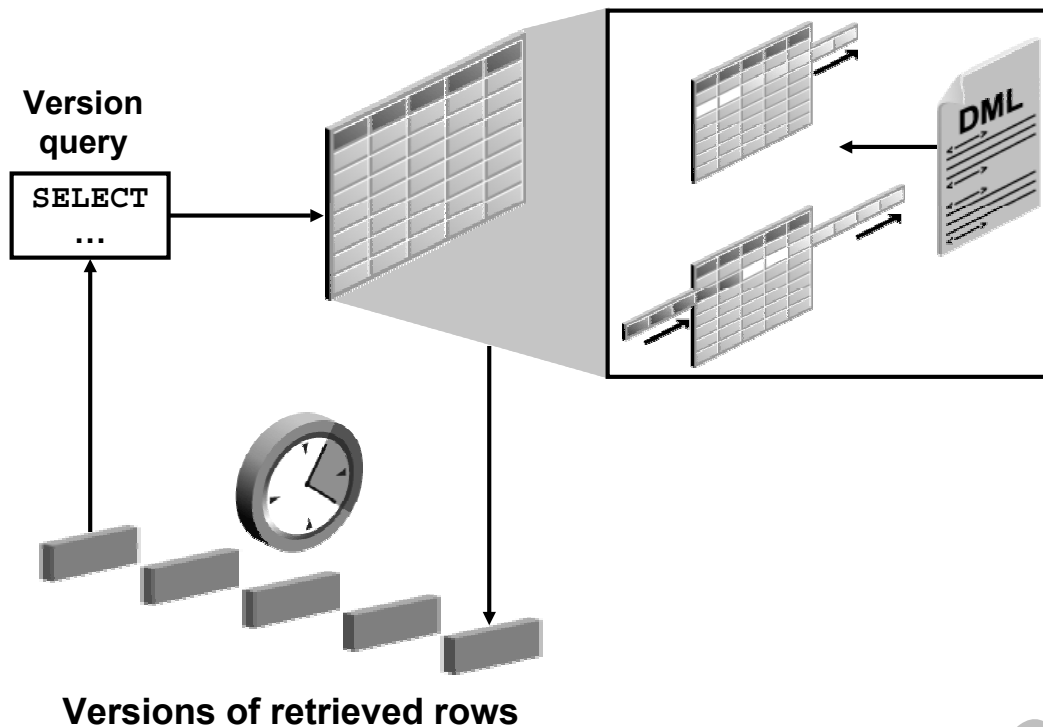
## Lesson Agenda

- Manipulating data using subqueries
- Specifying explicit default values in the `INSERT` and `UPDATE` statements
- Using the following types of multitable `INSERT`s:
  - Unconditional `INSERT`
  - Pivoting `INSERT`
  - Conditional `INSERT ALL`
  - Conditional `INSERT FIRST`
- Merging rows in a table
- Tracking the changes to data over a period of time

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Tracking Changes in Data



ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Tracking Changes in Data

You may discover that somehow data in a table has been inappropriately changed. To research this, you can use multiple flashback queries to view row data at specific points in time. More efficiently, you can use the Flashback Version Query feature to view all changes to a row over a period of time. This feature enables you to append a `VERSIONS` clause to a `SELECT` statement that specifies a system change number (SCN) or the time stamp range within which you want to view changes to row values. The query also can return associated metadata, such as the transaction responsible for the change.

Further, after you identify an erroneous transaction, you can use the Flashback Transaction Query feature to identify other changes that were done by the transaction. You then have the option of using the Flashback Table feature to restore the table to a state before the changes were made.

You can use a query on a table with a `VERSIONS` clause to produce all the versions of all the rows that exist or ever existed between the time the query was issued and the `undo_retention` seconds before the current time. `undo_retention` is an initialization parameter, which is an autotuned parameter. A query that includes a `VERSIONS` clause is referred to as a version query. The results of a version query behaves as though the `WHERE` clause were applied to the versions of the rows. The version query returns versions of the rows only across transactions.

**System change number (SCN):** The Oracle server assigns an System change number to identify the redo records for each committed transaction.

## Example of the Flashback Version Query

```
SELECT salary FROM employees3
WHERE employee_id = 107;
```

1

```
UPDATE employees3 SET salary = salary * 1.30
WHERE employee_id = 107;
```

2

```
COMMIT;
```

```
SELECT salary FROM employees3
 VERSIONS BETWEEN SCN MINVALUE AND MAXVALUE
WHERE employee_id = 107;
```

3

1

|   | R2 | SALARY |
|---|----|--------|
| 1 |    | 4200   |

3

|   | R2 | SALARY |
|---|----|--------|
| 1 |    | 5460   |
| 2 |    | 4200   |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Example of the Flashback Version Query

In the example in the slide, the salary for employee 107 is retrieved (1). The salary for employee 107 is increased by 30 percent and this change is committed (2). The different versions of salary are displayed (3).

The VERSIONS clause does not change the plan of the query. For example, if you run a query on a table that uses the index access method, then the same query on the same table with a VERSIONS clause continues to use the index access method. The versions of the rows returned by the version query are versions of the rows across transactions. The VERSIONS clause has no effect on the transactional behavior of a query. This means that a query on a table with a VERSIONS clause still inherits the query environment of the ongoing transaction.

The default VERSIONS clause can be specified as VERSIONS BETWEEN {SCN|TIMESTAMP} MINVALUE AND MAXVALUE.

The VERSIONS clause is a SQL extension only for queries. You can have DML and DDL operations that use a VERSIONS clause within subqueries. The row version query retrieves all the committed versions of the selected rows. Changes made by the current active transaction are not returned. The version query retrieves all incarnations of the rows. This essentially means that versions returned include deleted and subsequent reinserted versions of the rows.

### Example of the Flashback Version Query (continued)

The row access for a version query can be defined in one of the following two categories:

- **ROWID-based row access:** In case of ROWID-based access, all versions of the specified ROWID are returned irrespective of the row content. This essentially means that all versions of the slot in the block indicated by the ROWID are returned.
- **All other row access:** For all other row access, all versions of the rows are returned.

Oracle Internal & Oracle Academy  
Use Only

## VERSIONS BETWEEN Clause

```
SELECT versions_starttime "START_DATE",
 versions_endtime "END_DATE",
 salary
FROM employees
 VERSIONS BETWEEN SCN MINVALUE
 AND MAXVALUE
WHERE last_name = 'Lorentz';
```

|   | START_DATE                    | END_DATE                      | R | SALARY |
|---|-------------------------------|-------------------------------|---|--------|
| 1 | 10-JUL-07 10.55.49.0000000000 | (null)                        |   | 5460   |
| 2 | (null)                        | 10-JUL-07 10.55.49.0000000000 |   | 4200   |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### VERSIONS BETWEEN Clause

You can use the VERSIONS BETWEEN clause to retrieve all the versions of the rows that exist or have ever existed between the time the query was issued and a point back in time.

If the undo retention time is less than the lower bound time or the SCN of the BETWEEN clause, the query retrieves versions up to the undo retention time only. The time interval of the BETWEEN clause can be specified as an SCN interval or a wall-clock interval. This time interval is closed at both the lower and the upper bounds.

In the example, Lorentz's salary changes are retrieved. The NULL value for END\_DATE for the first version indicates that this was the existing version at the time of the query. The NULL value for START\_DATE for the last version indicates that this version was created at a time before the undo retention time.

## Summary

In this lesson, you should have learned how to:

- Use DML statements and control transactions
- Describe the features of multitable `INSERTs`
- Use the following types of multitable `INSERTs`:
  - Unconditional `INSERT`
  - Pivoting `INSERT`
  - Conditional `INSERT ALL`
  - Conditional `INSERT FIRST`
- Merge rows in a table
- Manipulate data by using subqueries
- Track the changes to data over a period of time

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Summary

In this lesson, you should have learned how to manipulate data in the Oracle database by using subqueries. You also should have learned about multitable `INSERT` statements, the `MERGE` statement, and tracking changes in the database.

## Practice 4: Overview

This practice covers the following topics:

- Performing multitable `INSERTs`
- Performing `MERGE` operations
- Tracking row versions

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Oracle Internal & Oracle Academy  
Use Only

## Practice 4

1. Run the lab\_04\_01.sql script in the lab folder to create the SAL\_HISTORY table.
2. Display the structure of the SAL\_HISTORY table.

| Column Name | Data Type   | Nullable | Data Default | COLUMN ID | Primary Key |
|-------------|-------------|----------|--------------|-----------|-------------|
| EMPLOYEE_ID | NUMBER(6,0) | Yes      | (null)       | 1         | (null)      |
| HIRE_DATE   | DATE        | Yes      | (null)       | 2         | (null)      |
| SALARY      | NUMBER(8,2) | Yes      | (null)       | 3         | (null)      |

3. Run the lab\_04\_03.sql script in the lab folder to create the MGR\_HISTORY table.
4. Display the structure of the MGR\_HISTORY table.

| Column Name | Data Type   | Nullable | Data Default | COLUMN ID | Primary Key |
|-------------|-------------|----------|--------------|-----------|-------------|
| EMPLOYEE_ID | NUMBER(6,0) | Yes      | (null)       | 1         | (null)      |
| MANAGER_ID  | NUMBER(6,0) | Yes      | (null)       | 2         | (null)      |
| SALARY      | NUMBER(8,2) | Yes      | (null)       | 3         | (null)      |

5. Run the lab\_04\_05.sql script in the lab folder to create the SPECIAL\_SAL table.
6. Display the structure of the SPECIAL\_SAL table.

| Column Name | Data Type   | Nullable | Data Default | COLUMN ID | Primary Key |
|-------------|-------------|----------|--------------|-----------|-------------|
| EMPLOYEE_ID | NUMBER(6,0) | Yes      | (null)       | 1         | (null)      |
| SALARY      | NUMBER(8,2) | Yes      | (null)       | 2         | (null)      |

7. a. Write a query to do the following:
  - Retrieve details such as the employee ID, hire date, salary, and manager ID of those employees whose employee ID is less than 125 from the EMPLOYEES table.
  - If the salary is more than \$20,000, insert details such as the employee ID and salary into the SPECIAL\_SAL table.
  - Insert details such as the employee ID, hire date, and salary into the SAL\_HISTORY table.
  - Insert details such as the employee ID, manager ID, and salary into the MGR\_HISTORY table.



### Practice 4 (continued)

- b. Display the records from the SPECIAL\_SAL table.

|   | EMPLOYEE_ID | SALARY |
|---|-------------|--------|
| 1 | 100         | 24000  |

- c. Display the records from the SAL\_HISTORY table.

|   | EMPLOYEE_ID | HIRE_DATE | SALARY |
|---|-------------|-----------|--------|
| 1 | 101         | 21-SEP-89 | 17000  |
| 2 | 102         | 13-JAN-93 | 17000  |
| 3 | 103         | 03-JAN-90 | 9000   |
| 4 | 104         | 21-MAY-91 | 6000   |
| 5 | 107         | 07-FEB-99 | 4200   |
| 6 | 124         | 16-NOV-99 | 5800   |

- d. Display the records from the MGR\_HISTORY table.

|   | EMPLOYEE_ID | MANAGER_ID | SALARY |
|---|-------------|------------|--------|
| 1 | 101         | 100        | 17000  |
| 2 | 102         | 100        | 17000  |
| 3 | 103         | 102        | 9000   |
| 4 | 104         | 103        | 6000   |
| 5 | 107         | 103        | 4200   |
| 6 | 124         | 100        | 5800   |

Oracle Internal & Oracle Academy  
Use Only

## Practice 4 (continued)

8. a. Run the lab\_04\_08a.sql script in the lab folder to create the SALES\_WEEK\_DATA table.
- b. Run the lab\_04\_08b.sql script in the lab folder to insert records into the SALES\_WEEK\_DATA table.
- c. Display the structure of the SALES\_WEEK\_DATA table.

```
DESCRIBE sales_week_data
```

| Name     | Null | Type        |
|----------|------|-------------|
| ID       |      | NUMBER(6)   |
| WEEK_ID  |      | NUMBER(2)   |
| QTY_MON  |      | NUMBER(8,2) |
| QTY_TUE  |      | NUMBER(8,2) |
| QTY_WED  |      | NUMBER(8,2) |
| QTY_THUR |      | NUMBER(8,2) |
| QTY_FRI  |      | NUMBER(8,2) |

7 rows selected

- d. Display the records from the SALES\_WEEK\_DATA table.

| ID  | WEEK_ID | QTY_MON | QTY_TUE | QTY_WED | QTY_THUR | QTY_FRI |
|-----|---------|---------|---------|---------|----------|---------|
| 200 | 6       | 2050    | 2200    | 1700    | 1200     | 3000    |

- e. Run the lab\_04\_08c.sql script in the lab folder to create the EMP\_SALES\_INFO table.
- f. Display the structure of the EMP\_SALES\_INFO table.

```
describe emp_sales_info
```

| Name      | Null | Type        |
|-----------|------|-------------|
| ID        |      | NUMBER(6)   |
| WEEK      |      | NUMBER(2)   |
| QTY_SALES |      | NUMBER(8,2) |

3 rows selected

## Practice 4 (continued)

- g. Write a query to do the following:
- Retrieve details such as ID, week ID, sales quantity on Monday, sales quantity on Tuesday, sales quantity on Wednesday, sales quantity on Thursday, and sales quantity on Friday from the `SALES_WEEK_DATA` table.
  - Build a transformation such that each record retrieved from the `SALES_WEEK_DATA` table is converted into multiple records for the `EMP_SALES_INFO` table.
- Hint:** Use a pivoting `INSERT` statement.
- h. Display the records from the `EMP_SALES_INFO` table.

|   | ID  | WEEK | QTY_SALES |
|---|-----|------|-----------|
| 1 | 200 | 6    | 2050      |
| 2 | 200 | 6    | 2200      |
| 3 | 200 | 6    | 1700      |
| 4 | 200 | 6    | 1200      |
| 5 | 200 | 6    | 3000      |

9. You have the data of past employees stored in a flat file called `emp.data`. You want to store the names and email IDs of all employees, past and present, in a table. To do this, first create an external table called `EMP_DATA` using the `emp.dat` source file in the `emp_dir` directory. Use the `lab_04_09.sql` script to do this.
10. Next, run the `lab_04_10.sql` script to create the `EMP_HIST` table.
- Increase the size of the email column to 45.
  - Merge the data in the `EMP_DATA` table created in the last lab into the data in the `EMP_HIST` table. Assume that the data in the external `EMP_DATA` table is the most up-to-date. If a row in the `EMP_DATA` table matches the `EMP_HIST` table, update the email column of the `EMP_HIST` table to match the `EMP_DATA` table row. If a row in the `EMP_DATA` table does not match, insert it into the `EMP_HIST` table. Rows are considered matching when the employee's first and last names are identical.
  - Retrieve the rows from `EMP_HIST` after the merge.

## Practice 4 (continued)

Partial results (48 rows retrieved)

|    | 1 2       | FIRST_NAME | 1 2              | LAST_NAME | 1 2 | EMAIL |
|----|-----------|------------|------------------|-----------|-----|-------|
| 1  | Ellen     | Abel       | EABEL            |           |     |       |
| 2  | Curtis    | Davies     | CDAVIES          |           |     |       |
| 3  | Lex       | De Haan    | LDEHAAN          |           |     |       |
| 4  | Bruce     | Ernst      | BERNST           |           |     |       |
| 5  | Pat       | Fay        | PFAY             |           |     |       |
| 6  | William   | Gietz      | WGIEZT           |           |     |       |
| 7  | Kimberely | Grant      | KGRANT           |           |     |       |
| 8  | Michael   | Hartstein  | MHARTSTE         |           |     |       |
| 9  | Shelley   | Higgins    | SHIGGINS         |           |     |       |
| 10 | Alexander | Hunold     | AHun@MOORHEN.COM |           |     |       |
| 11 | Steven    | King       | SKING            |           |     |       |
| 12 | Neena     | Kochhar    | nkochh@pipit.com |           |     |       |

11. Create the EMP3 table by using the lab\_04\_11.sql script. In the EMP3 table, change the department for Kochhar to 60 and commit your change. Next, change the department for Kochhar to 50 and commit your change. Track the changes to Kochhar by using the Row Versions feature.

|   | START_DATE                      | END_DATE                        | 1 2 | DEPARTMENT_ID |
|---|---------------------------------|---------------------------------|-----|---------------|
| 1 | 19-JUN-07 10.29.57.000000000 PM | (null)                          |     | 50            |
| 2 | 19-JUN-07 10.29.54.000000000 PM | 19-JUN-07 10.29.57.000000000 PM |     | 60            |
| 3 | 19-JUN-07 10.29.50.000000000 PM | 19-JUN-07 10.29.54.000000000 PM |     | 60            |
| 4 | (null)                          | 19-JUN-07 10.29.50.000000000 PM |     | 90            |

# 5

## Managing Data in Different Time Zones

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Oracle Internal & Oracle Academy  
Use Only

## Objectives

After completing this lesson, you should be able to do the following:

- Use data types similar to `DATE` that store fractional seconds and track time zones
- Use data types that store the difference between two datetime values
- Use the following datetime functions:
  - `CURRENT_DATE`
  - `CURRENT_TIMESTAMP`
  - `LOCALTIMESTAMP`
  - `DBTIMEZONE`
  - `SESSIONTIMEZONE`
  - `EXTRACT`
  - `TZ_OFFSET`
  - `FROM_TZ`
  - `TO_TIMESTAMP`
  - `TO_YMINTERVAL`
  - `TO_DSINTERVAL`

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Objectives

In this lesson, you learn how to use data types similar to `DATE` that store fractional seconds and track time zones. This lesson addresses some of the datetime functions available in the Oracle database.

## Lesson Agenda

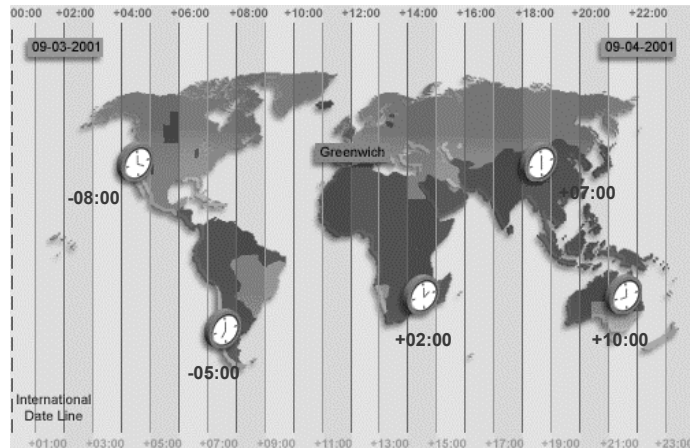
- `CURRENT_DATE`, `CURRENT_TIMESTAMP`,  
and `LOCALTIMESTAMP`
- `INTERVAL` data types
- Using the following functions:
  - `EXTRACT`
  - `TZ_OFFSET`
  - `FROM_TZ`
  - `TO_TIMESTAMP`
  - `TO_YMINTERVAL`
  - `TO_DSINTERVAL`

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Oracle Internal & Oracle Academy  
Use Only

# Time Zones



**The image represents the time for each time zone when Greenwich time is 12:00.**

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Time Zones

The hours of the day are measured by the turning of the earth. The time of day at any particular moment depends on where you are. When it is noon in Greenwich, England, it is midnight along the International Date Line. The earth is divided into 24 time zones, one for each hour of the day. The time along the prime meridian in Greenwich, England, is known as Greenwich Mean Time, or GMT. GMT is now known as Coordinated Universal Time or UTC. UTC is the time standard against which all other time zones in the world are referenced. It is the same all year round and is not affected by summer time or daylight saving time. The meridian line is an imaginary line that runs from the North Pole to the South Pole. It is known as zero longitude and it is the line from which all other lines of longitude are measured. All time is measured relative to UTC and all places have a latitude (their distance north or south of the equator) and a longitude (their distance east or west of the Greenwich meridian).



## TIME\_ZONE Session Parameter

TIME\_ZONE may be set to:

- An absolute offset
- Database time zone
- OS local time zone
- A named region

```
ALTER SESSION SET TIME_ZONE = '-05:00';
ALTER SESSION SET TIME_ZONE = dbtimezone;
ALTER SESSION SET TIME_ZONE = local;
ALTER SESSION SET TIME_ZONE = 'America/New_York';
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### TIME\_ZONE Session Parameter

The Oracle database supports storing the time zone in your date and time data, as well as fractional seconds. The ALTER SESSION command can be used to change time zone values in a user's session. The time zone values can be set to an absolute offset, a named time zone, a database time zone, or the local time zone.

## CURRENT\_DATE, CURRENT\_TIMESTAMP, and LOCALTIMESTAMP

- CURRENT\_DATE:
  - Returns the current date from the user session
  - Has a data type of DATE
- CURRENT\_TIMESTAMP:
  - Returns the current date and time from the user session
  - Has a data type of TIMESTAMP WITH TIME ZONE
- LOCALTIMESTAMP:
  - Returns the current date and time from the user session
  - Has a data type of TIMESTAMP

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### CURRENT\_DATE, CURRENT\_TIMESTAMP, and LOCALTIMESTAMP

The CURRENT\_DATE and CURRENT\_TIMESTAMP functions return the current date and current time stamp, respectively. The data type of CURRENT\_DATE is DATE. The data type of CURRENT\_TIMESTAMP is TIMESTAMP WITH TIME ZONE. The values returned display the time zone displacement of the SQL session executing the functions. The time zone displacement is the difference (in hours and minutes) between local time and Coordinated Universal Time (UTC). The TIMESTAMP WITH TIME ZONE data type has the format:

TIMESTAMP [ (fractional\_seconds\_precision) ] WITH TIME ZONE

where fractional\_seconds\_precision optionally specifies the number of digits in the fractional part of the SECOND datetime field and can be a number in the range 0 through 9. The default is 6.

The LOCALTIMESTAMP function returns the current date and time in the session time zone. The difference between LOCALTIMESTAMP and CURRENT\_TIMESTAMP is that LOCALTIMESTAMP returns a TIMESTAMP value, whereas CURRENT\_TIMESTAMP returns a TIMESTAMP WITH TIME ZONE value.

These functions are national language support (NLS)–sensitive—that is, the results will be in the current NLS calendar and datetime formats.

**Note:** The SYSDATE function returns the current date and time as a DATE data type. You learned how to use the SYSDATE function in the course titled *Oracle Database 11g: SQL Fundamentals I*.

## Comparing Date and Time in a Session's Time Zone

The `TIME_ZONE` parameter is set to `-5:00` and then `SELECT` statements for each date and time is executed to compare differences.

```
ALTER SESSION
SET NLS_DATE_FORMAT = 'DD-MON-YYYY HH24:MI:SS';
ALTER SESSION SET TIME_ZONE = '-5:00';
SELECT SESSIONTIMEZONE, CURRENT_DATE FROM DUAL;
SELECT SESSIONTIMEZONE, CURRENT_TIMESTAMP FROM DUAL;
SELECT SESSIONTIMEZONE, LOCALTIMESTAMP FROM DUAL;
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Comparing Date and Time in a Session's Time Zone

The `ALTER SESSION` command sets the date format of the session to `'DD-MON-YYYY HH24:MI:SS'`—that is, day of month (1–31)-abbreviated name of month-4-digit year hour of day (0–23):minute (0–59):second (0–59).

The example in the slide illustrates that the session is altered to set the `TIME_ZONE` parameter to `-5:00`. Then the `SELECT` statement for `CURRENT_DATE`, `CURRENT_TIMESTAMP`, and `LOCALTIMESTAMP` is executed to observe the differences in format.

**Note:** The `TIME_ZONE` parameter specifies the default local time zone displacement for the current SQL session. `TIME_ZONE` is a session parameter only, not an initialization parameter. The `TIME_ZONE` parameter is set as follows:

`TIME_ZONE = '[+ | -] hh:mm'`

The format mask `([+ | -] hh:mm)` indicates the hours and minutes before or after UTC (Coordinated Universal Time, formerly known as Greenwich Mean Time).

## Comparing Date and Time in a Session's Time Zone

Results of queries:

|                              |                                        |
|------------------------------|----------------------------------------|
| alter session set succeeded. |                                        |
| SESSIONTIMEZONE              | CURRENT_DATE                           |
| -05:00                       | 11-JUN-2007 14:12:06                   |
| 1 rows selected              |                                        |
| SESSIONTIMEZONE              | CURRENT_TIMESTAMP                      |
| -05:00                       | 11-JUN-07 02.12.06.076200000 PM -05:00 |
| 1 rows selected              |                                        |
| SESSIONTIMEZONE              | LOCALTIMESTAMP                         |
| -05:00                       | 11-JUN-07 02.12.06.164828000 PM        |
| 1 rows selected              |                                        |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Comparing Date and Time in a Session's Time Zone (continued)

In this case, the `CURRENT_DATE` function returns the current date in the session's time zone, the `CURRENT_TIMESTAMP` function returns the current date and time in the session's time zone as a value of the data type `TIMESTAMP WITH TIME ZONE`, and the `LOCALTIMESTAMP` function returns the current date and time in the session's time zone.

## DBTIMEZONE and SESSIONTIMEZONE

- Display the value of the database time zone:

```
SELECT DTIMEZONE FROM DUAL;
```

|   | DTIMEZONE |
|---|-----------|
| 1 | +00:00    |

- Display the value of the session's time zone:

```
SELECT SESSIONTIMEZONE FROM DUAL;
```

|   | SESSIONTIMEZONE |
|---|-----------------|
| 1 | -05:00          |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### DBTIMEZONE and SESSIONTIMEZONE

The DBA sets the database's default time zone by specifying the SET TIME\_ZONE clause of the CREATE DATABASE statement. If omitted, the default database time zone is the operating system time zone. The database time zone cannot be changed for a session with an ALTER SESSION statement.

The DBTIMEZONE function returns the value of the database time zone. The return type is a time zone offset (a character type in the format: '[+ | -] TZH:TZM') or a time zone region name, depending on how the user specified the database time zone value in the most recent CREATE DATABASE or ALTER DATABASE statement. The example in the slide shows that the database time zone is set to "-05:00," as the TIME\_ZONE parameter is in the format:

```
TIME_ZONE = '[+ | -] hh:mm'
```

The SESSIONTIMEZONE function returns the value of the current session's time zone. The return type is a time zone offset (a character type in the format '[+ | -] TZH:TZM') or a time zone region name, depending on how the user specified the session time zone value in the most recent ALTER SESSION statement. The example in the slide shows that the session time zone is offset to UTC by -8 hours. Observe that the database time zone is different from the current session's time zone.

## TIMESTAMP Data Types

| Data Type                      | Fields                                                                                                         |
|--------------------------------|----------------------------------------------------------------------------------------------------------------|
| TIMESTAMP                      | Year, Month, Day, Hour, Minute, Second with fractional seconds                                                 |
| TIMESTAMP WITH TIME ZONE       | Same as the TIMESTAMP data type; also includes:<br>TIMEZONE_HOUR, and<br>TIMEZONE_MINUTE or<br>TIMEZONE_REGION |
| TIMESTAMP WITH LOCAL TIME ZONE | Same as the TIMESTAMP data type; also includes a time zone offset in its value                                 |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### TIMESTAMP Data Types

The TIMESTAMP data type is an extension of the DATE data type.

**TIMESTAMP (fractional\_seconds\_precision)**

This data type contains the year, month, and day values of date, as well as hour, minute, and second values of time, where significant fractional seconds precision is the number of digits in the fractional part of the SECOND datetime field. The accepted values of significant fractional\_seconds\_precision are 0 through 9. The default is 6.

**TIMESTAMP (fractional\_seconds\_precision) WITH TIME ZONE**

This data type contains all values of TIMESTAMP as well as time zone displacement value.

**TIMESTAMP (fractional\_seconds\_precision) WITH LOCAL TIME ZONE**

This data type contains all values of TIMESTAMP, with the following exceptions:

- Data is normalized to the database time zone when it is stored in the database.
- When the data is retrieved, users see the data in the session time zone.

## TIMESTAMP Fields

| Datetime Field  | Valid Values                          |
|-----------------|---------------------------------------|
| YEAR            | –4712 to 9999 (excluding year 0)      |
| MONTH           | 01 to 12                              |
| DAY             | 01 to 31                              |
| HOURL           | 00 to 23                              |
| MINUTE          | 00 to 59                              |
| SECOND          | 00 to 59.9(N) where 9(N) is precision |
| TIMEZONE_HOUR   | –12 to 14                             |
| TIMEZONE_MINUTE | 00 to 59                              |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### TIMESTAMP Fields

Each datetime data type is composed of several of these fields. Datetimes are mutually comparable and assignable only if they have the same datetime fields.

## Difference Between DATE and TIMESTAMP

A

```
-- when hire_date is
of type DATE
```

```
SELECT hire_date
FROM employees;
```

|    | HIRE_DATE |
|----|-----------|
| 1  | 17-JUN-87 |
| 2  | 21-SEP-89 |
| 3  | 13-JAN-93 |
| 4  | 03-JAN-90 |
| 5  | 21-MAY-91 |
| 6  | 25-JUN-97 |
| 7  | 05-FEB-98 |
| 8  | 07-FEB-99 |
| 9  | 17-AUG-94 |
| 10 | 16-AUG-94 |

B

```
ALTER TABLE employees
MODIFY hire_date TIMESTAMP;
```

```
SELECT hire_date
FROM employees;
```

|    | HIRE_DATE                       |
|----|---------------------------------|
| 1  | 17-JUN-87 12.00.00.000000000 AM |
| 2  | 21-SEP-89 12.00.00.000000000 AM |
| 3  | 13-JAN-93 12.00.00.000000000 AM |
| 4  | 03-JAN-90 12.00.00.000000000 AM |
| 5  | 21-MAY-91 12.00.00.000000000 AM |
| 6  | 25-JUN-97 12.00.00.000000000 AM |
| 7  | 05-FEB-98 12.00.00.000000000 AM |
| 8  | 07-FEB-99 12.00.00.000000000 AM |
| 9  | 17-AUG-94 12.00.00.000000000 AM |
| 10 | 16-AUG-94 12.00.00.000000000 AM |

...

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### TIMESTAMP Data Type: Example

In the slide, example A shows the data from the `hire_date` column of the `EMPLOYEES` table when the data type of the column is `DATE`. In example B, the table is altered and the data type of the `hire_date` column is made into `TIMESTAMP`. The output shows the differences in display. You can convert from `DATE` to `TIMESTAMP` when the column has data, but you cannot convert from `DATE` or `TIMESTAMP` to `TIMESTAMP WITH TIME ZONE` unless the column is empty.

You can specify the fractional seconds precision for time stamp. If none is specified, as in this example, it defaults to 6.

For example, the following statement sets the fractional seconds precision as 7:

```
ALTER TABLE employees
MODIFY hire_date TIMESTAMP(7);
```

**Note:** The Oracle date data type by default appears as shown in this example. However, the date data type also contains additional information such as hours, minutes, seconds, AM, and PM. To obtain the date in this format, you can apply a format mask or a function to the date value.



## Comparing TIMESTAMP Data Types

```
CREATE TABLE web_orders
(order_date TIMESTAMP WITH TIME ZONE,
 delivery_time TIMESTAMP WITH LOCAL TIME ZONE);
```

```
INSERT INTO web_orders values
(current_date, current_timestamp + 2);
```

```
SELECT * FROM web_orders;
```

| ORDER_DATE                               | DELIVERY_TIME                   |
|------------------------------------------|---------------------------------|
| 1 11-JUN-07 02.44.14.000000000 PM -05:00 | 13-JUN-07 02.44.14.000000000 PM |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Comparing TIMESTAMP Data Types

In the example in the slide, a new table `web_orders` is created with a column of data type `TIMESTAMP WITH TIME ZONE` and a column of data type `TIMESTAMP WITH LOCAL TIME ZONE`. This table is populated whenever a `web_order` is placed. The time stamp and time zone for the user placing the order is inserted based on the `CURRENT_DATE` value. The local time stamp and time zone is populated by inserting two days from the `CURRENT_TIMESTAMP` value into it every time an order is placed. When a Web-based company guarantees shipping, they can estimate their delivery time based on the time zone of the person placing the order.

## Lesson Agenda

- `CURRENT_DATE`, `CURRENT_TIMESTAMP`, and `LOCALTIMESTAMP`
- **INTERVAL data types**
- Using the following functions:
  - `EXTRACT`
  - `TZ_OFFSET`
  - `FROM_TZ`
  - `TO_TIMESTAMP`
  - `TO_YMINTERVAL`
  - `TO_DSINTERVAL`

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Oracle Internal & Oracle Academy  
Use Only

## INTERVAL Data Types

- INTERVAL data types are used to store the difference between two datetime values.
- There are two classes of intervals:
  - Year-month
  - Day-time
- The precision of the interval is:
  - The actual subset of fields that constitutes an interval
  - Specified in the interval qualifier

| Data Type              | Fields                                             |
|------------------------|----------------------------------------------------|
| INTERVAL YEAR TO MONTH | Year, Month                                        |
| INTERVAL DAY TO SECOND | Days, Hour, Minute, Second with fractional seconds |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### INTERVAL Data Types

INTERVAL data types are used to store the difference between two datetime values. There are two classes of intervals: year-month intervals and day-time intervals. A year-month interval is made up of a contiguous subset of fields of YEAR and MONTH, whereas a day-time interval is made up of a contiguous subset of fields consisting of DAY, HOUR, MINUTE, and SECOND. The actual subset of fields that constitute an interval is called the precision of the interval and is specified in the interval qualifier. Because the number of days in a year is calendar dependent, the year-month interval is NLS dependent, whereas day-time interval is NLS independent.

The interval qualifier may also specify the leading field precision, which is the number of digits in the leading or only field, and in case the trailing field is SECOND, it may also specify the fractional seconds precision, which is the number of digits in the fractional part of the SECOND value. If not specified, the default value for leading field precision is 2 digits, and the default value for fractional seconds precision is 6 digits.

## **INTERVAL Data Types (continued)**

### **INTERVAL YEAR (year\_precision) TO MONTH**

This data type stores a period of time in years and months, where `year_precision` is the number of digits in the YEAR datetime field. The accepted values are 0 through 9. The default is 6.

### **INTERVAL DAY (day\_precision) TO SECOND (fractional\_seconds\_precision)**

This data type stores a period of time in days, hours, minutes, and seconds, where `day_precision` is the maximum number of digits in the DAY datetime field (accepted values are 0 through 9; the default is 2), and `fractional_seconds_precision` is the number of digits in the fractional part of the SECOND field. The accepted values are 0 through 9. The default is 6.

Oracle Internal & Oracle Academy  
Use Only

## INTERVAL Fields

| INTERVAL Field | Valid Values for Interval             |
|----------------|---------------------------------------|
| YEAR           | Any positive or negative integer      |
| MONTH          | 00 to 11                              |
| DAY            | Any positive or negative integer      |
| HOURL          | 00 to 23                              |
| MINUTE         | 00 to 59                              |
| SECOND         | 00 to 59.9(N) where 9(N) is precision |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### INTERVAL Fields

INTERVAL YEAR TO MONTH can have fields of YEAR and MONTH.

INTERVAL DAY TO SECOND can have fields of DAY, HOUR, MINUTE, and SECOND.

The actual subset of fields that constitute an item of either type of interval is defined by an interval qualifier, and this subset is known as the precision of the item.

Year-month intervals are mutually comparable and assignable only with other year-month intervals, and day-time intervals are mutually comparable and assignable only with other day-time intervals.

## INTERVAL YEAR TO MONTH: Example

```
CREATE TABLE warranty
(prod_id number, warranty_time INTERVAL YEAR(3) TO
MONTH);
INSERT INTO warranty VALUES (123, INTERVAL '8' MONTH);
INSERT INTO warranty VALUES (155, INTERVAL '200'
YEAR(3));
INSERT INTO warranty VALUES (678, '200-11');
SELECT * FROM warranty;
```

|   | PROD_ID | WARRANTY_TIME |
|---|---------|---------------|
| 1 | 123     | 0-8           |
| 2 | 155     | 200-0         |
| 3 | 678     | 200-11        |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### INTERVAL YEAR TO MONTH Data Type

INTERVAL YEAR TO MONTH stores a period of time using the YEAR and MONTH datetime fields. Specify INTERVAL YEAR TO MONTH as follows:

INTERVAL YEAR [(year\_precision)] TO MONTH

where year\_precision is the number of digits in the YEAR datetime field. The default value of year\_precision is 2.

**Restriction:** The leading field must be more significant than the trailing field. For example, INTERVAL '0-1' MONTH TO YEAR is not valid.

#### Examples

- INTERVAL '123-2' YEAR(3) TO MONTH  
Indicates an interval of 123 years, 2 months
- INTERVAL '123' YEAR(3)  
Indicates an interval of 123 years, 0 months
- INTERVAL '300' MONTH(3)  
Indicates an interval of 300 months
- INTERVAL '123' YEAR  
Returns an error because the default precision is 2, and 123 has 3 digits

### **INTERVAL YEAR TO MONTH Data Type (continued)**

The Oracle database supports two interval data types: `INTERVAL YEAR TO MONTH` and `INTERVAL DAY TO SECOND`; the column type, PL/SQL argument, variable, and return type must be one of the two. However, for interval literals, the system recognizes other American National Standards Institute (ANSI) interval types such as `INTERVAL '2' YEAR` or `INTERVAL '10' HOUR`. In these cases, each interval is converted to one of the two supported types.

In the example in the slide, a `WARRANTY` table is created, which contains a `warranty_time` column that takes the `INTERVAL YEAR (3) TO MONTH` data type. Different values are inserted into it to indicate years and months for various products. When these rows are retrieved from the table, you see a year value separated from the month value by a (-).

Oracle Internal & Oracle Academy  
Use Only

## INTERVAL DAY TO SECOND

### Data Type: Example

```
CREATE TABLE lab
(exp_id number, test_time INTERVAL DAY(2) TO SECOND);

INSERT INTO lab VALUES (100012, '90 00:00:00');
INSERT INTO lab VALUES (56098,
 INTERVAL '6 03:30:16' DAY TO SECOND);
```

```
SELECT * FROM lab;
```

|   | EXP_ID | TEST_TIME   |
|---|--------|-------------|
| 1 | 10012  | 90 0:0:0.0  |
| 2 | 56098  | 6 3:30:16.0 |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### INTERVAL DAY TO SECOND Data Type: Example

In the example in the slide, you are creating the lab table with a test\_time column of the INTERVAL DAY TO SECOND data type. You then insert into it the value “90 00:00:00” to indicate 90 days and 0 hours, 0 minutes, and 0 seconds, and INTERVAL '6 03:30:16' DAY TO SECOND to indicate 6 days, 3 hours, 30 minutes, and 16 seconds. The SELECT statement shows how this data is displayed in the database.



## Lesson Agenda

- `CURRENT_DATE`, `CURRENT_TIMESTAMP`, and `LOCALTIMESTAMP`
- `INTERVAL` data types
- Using the following functions:
  - `EXTRACT`
  - `TZ_OFFSET`
  - `FROM_TZ`
  - `TO_TIMESTAMP`
  - `TO_YMINTERVAL`
  - `TO_DSINTERVAL`

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Oracle Internal & Oracle Academy  
Use Only

## EXTRACT

- Display the YEAR component from the SYSDATE.

```
SELECT EXTRACT (YEAR FROM SYSDATE) FROM DUAL;
```

| EXTRACT(YEARFROMSYSDATE) |
|--------------------------|
| 1 2007                   |

- Display the MONTH component from the HIRE\_DATE for those employees whose MANAGER\_ID is 100.

```
SELECT last name, hire date,
 EXTRACT (MONTH FROM HIRE_DATE)
FROM employees
WHERE manager_id = 100;
```

|   | LAST_NAME | HIRE_DATE                       | EXTRACT(MONTHFROMHIRE_DATE) |
|---|-----------|---------------------------------|-----------------------------|
| 1 | Kochhar   | 21-SEP-89 12.00.00.000000000 AM | 9                           |
| 2 | De Haan   | 13-JAN-93 12.00.00.000000000 AM | 1                           |
| 3 | Raphaely  | 07-DEC-94 12.00.00.000000000 AM | 12                          |
| 4 | Weiss     | 18-JUL-96 12.00.00.000000000 AM | 7                           |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## EXTRACT

The EXTRACT expression extracts and returns the value of a specified datetime field from a datetime or interval value expression. You can extract any of the components mentioned in the following syntax using the EXTRACT function. The syntax of the EXTRACT function is:

```
SELECT EXTRACT ([YEAR] [MONTH] [DAY] [HOUR] [MINUTE] [SECOND]
 [TIMEZONE_HOUR] [TIMEZONE_MINUTE]
 [TIMEZONE_REGION] [TIMEZONE_ABBR]
FROM [datetime_value_expression] [interval_value_expression]);
```

When you extract a TIMEZONE\_REGION or TIMEZONE\_ABBR (abbreviation), the value returned is a string containing the appropriate time zone name or abbreviation. When you extract any of the other values, the value returned is a date in the Gregorian calendar. When extracting from a datetime with a time zone value, the value returned is in UTC.

In the first example in the slide, the EXTRACT function is used to extract the YEAR from SYSDATE. In the second example in the slide, the EXTRACT function is used to extract the MONTH from the HIRE\_DATE column of the EMPLOYEES table for those employees who report to the manager whose EMPLOYEE\_ID is 100.

## TZ\_OFFSET

Display the time zone offset for the 'US/Eastern',  
'Canada/Yukon' and 'Europe/London' time zones:

```
SELECT TZ_OFFSET('US/Eastern'),
 TZ_OFFSET('Canada/Yukon'),
 TZ_OFFSET('Europe/London')
FROM DUAL;
```

|   |  TZ_OFFSET('US/EASTERN') |  TZ_OFFSET('CANADA/YUKON') |  TZ_OFFSET('EUROPE/LONDON') |
|---|-----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| 1 | -04:00                                                                                                    | -07:00                                                                                                      | +01:00                                                                                                       |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### TZ\_OFFSET

The TZ\_OFFSET function returns the time zone offset corresponding to the value entered. The return value is dependent on the date when the statement is executed. For example, if the TZ\_OFFSET function returns a value -08:00, this value indicates that the time zone where the command was executed is eight hours behind UTC. You can enter a valid time zone name, a time zone offset from UTC (which simply returns itself), or the keyword SESSIONTIMEZONE or DBTIMEZONE. The syntax of the TZ\_OFFSET function is:

```
TZ_OFFSET (['time_zone_name'] '[+ | -] hh:mm']
 [SESSIONTIMEZONE] [DBTIMEZONE]
```

The Fold Motor Company has its headquarters in Michigan, USA, which is in the US/Eastern time zone. The company president, Mr. Fold, wants to conduct a conference call with the vice president of the Canadian operations and the vice president of European operations, who are in the Canada/Yukon and Europe/London time zones, respectively. Mr. Fold wants to find out the time in each of these places to make sure that his senior management will be available to attend the meeting. His secretary, Mr. Scott, helps by issuing the queries shown in the example and gets the following results:

- The 'US/Eastern' time zone is four hours behind UTC.
- The 'Canada/Yukon' time zone is seven hours behind UTC.
- The 'Europe/London' time zone is one hour ahead of UTC.

## TZ\_OFFSET (continued)

For a listing of valid time zone name values, you can query the V\$TIMEZONE\_NAMES dynamic performance view.

```
SELECT * FROM V$TIMEZONE_NAMES;
```

|    |  TZNAME |  TZABBREV |
|----|------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| 1  | Africa/Abidjan                                                                           | LMT                                                                                        |
| 2  | Africa/Abidjan                                                                           | GMT                                                                                        |
| 3  | Africa/Accra                                                                             | LMT                                                                                        |
| 4  | Africa/Accra                                                                             | GMT                                                                                        |
| 5  | Africa/Accra                                                                             | GHST                                                                                       |
| 6  | Africa/Addis_Ababa                                                                       | LMT                                                                                        |
| 7  | Africa/Addis_Ababa                                                                       | ADMT                                                                                       |
| 8  | Africa/Addis_Ababa                                                                       | EAT                                                                                        |
| 9  | Africa/Algiers                                                                           | LMT                                                                                        |
| 10 | Africa/Algiers                                                                           | PMT                                                                                        |
| 11 | Africa/Algiers                                                                           | WET                                                                                        |
| 12 | Africa/Algiers                                                                           | WEST                                                                                       |
| 13 | Africa/Algiers                                                                           | CET                                                                                        |

...

Oracle Internal & Oracle Academy  
Use Only

## FROM\_TZ

Display the **TIMESTAMP** value '2000-03-28 08:00:00' as a **TIMESTAMP WITH TIME ZONE** value for the 'Australia/North' time zone region.

```
SELECT FROM_TZ(TIMESTAMP
 '2000-07-12 08:00:00', 'Australia/North')
FROM DUAL;
```

|                                                           |
|-----------------------------------------------------------|
| FROM_TZ(TIMESTAMP'2000-07-12 08:00:00','AUSTRALIA/NORTH') |
| 1 12-JUL-00 08.00.00.000000000 AM AUSTRALIA/NORTH         |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### FROM\_TZ

The **FROM\_TZ** function converts a **TIMESTAMP** value to a **TIMESTAMP WITH TIME ZONE** value. The syntax of the **FROM\_TZ** function is as follows:

**FROM\_TZ**(**TIMESTAMP** timestamp\_value, time\_zone\_value)

where **time\_zone\_value** is a character string in the format 'TZH:TZM' or a character expression that returns a string in TZR (time zone region) with an optional TZD format. TZD is an abbreviated time zone string with daylight saving information. TZR represents the time zone region in datetime input strings. Examples are 'Australia/North', 'PST' for US/Pacific standard time, 'PDT' for US/Pacific daylight time, and so on.

The example in the slide converts a **TIMESTAMP** value to **TIMESTAMP WITH TIME ZONE**.

**Note:** To see a listing of valid values for the TZR and TZD format elements, query the **V\$TIMEZONE\_NAMES** dynamic performance view.

## TO\_TIMESTAMP

Display the character string '2007-03-06 11:00:00'  
as a TIMESTAMP value:

```
SELECT TO_TIMESTAMP ('2007-03-06 11:00:00',
 'YYYY-MM-DD HH:MI:SS')
FROM DUAL;
```

```
TO_TIMESTAMP('2007-03-06 11:00:00','YYYY-MM-DDHH:MI:SS')
06-MAR-07 11.00.00.000000000
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### TO\_TIMESTAMP

The TO\_TIMESTAMP function converts a string of CHAR, VARCHAR2, NCHAR, or NVARCHAR2 data type to a value of the TIMESTAMP data type. The syntax of the TO\_TIMESTAMP function is:

```
TO_TIMESTAMP (char, [fmt], ['nlsparam'])
```

The optional *fmt* specifies the format of *char*. If you omit *fmt*, then the string must be in the default format of the TIMESTAMP data type. The optional *nlsparam* specifies the language in which month and day names, and abbreviations are returned. This argument can have this form:

```
'NLS_DATE_LANGUAGE = language'
```

If you omit *nlsparams*, this function uses the default date language for your session.

The example in the slide converts a character string to a value of TIMESTAMP.

**Note:** You use the TO\_TIMESTAMP\_TZ function to convert a string of CHAR, VARCHAR2, NCHAR, or NVARCHAR2 data type to a value of the TIMESTAMP WITH TIME ZONE data type. For more information about this function, see *Oracle Database SQL Reference 11g Release 1 (11.1)*.

## TO\_YMINTERVAL

Display a date that is one year and two months after the hire date for the employees working in the department with the DEPARTMENT\_ID 20.

```
SELECT hire_date,
 hire_date + TO_YMINTERVAL('01-02') AS
 HIRE_DATE_YMININTERVAL
FROM employees
WHERE department_id = 20;
```

|   | HIRE_DATE                    | HIRE_DATE_YMININTERVAL       |
|---|------------------------------|------------------------------|
| 1 | 17-FEB-96 00.00.00.000000000 | 17-APR-97 00.00.00.000000000 |
| 2 | 17-AUG-97 00.00.00.000000000 | 17-OCT-98 00.00.00.000000000 |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### TO\_YMINTERVAL

The TO\_YMINTERVAL function converts a character string of CHAR, VARCHAR2, NCHAR, or NVARCHAR2 data type to an INTERVAL YEAR TO MONTH data type. The INTERVAL YEAR TO MONTH data type stores a period of time using the YEAR and MONTH datetime fields. The format of INTERVAL YEAR TO MONTH is as follows:

INTERVAL YEAR [(year\_precision)] TO MONTH

where year\_precision is the number of digits in the YEAR datetime field. The default value of year\_precision is 2.

The syntax of the TO\_YMINTERVAL function is:

TO\_YMINTERVAL (char)

where char is the character string to be converted

The example in the slide calculates a date that is one year and two months after the hire date for the employees working in the department 20 of the EMPLOYEES table.

## TO\_DSINTERVAL

Display a date that is 100 days and 10 hours after the hire date for all the employees.

```
SELECT last_name,
 TO_CHAR(hire_date, 'mm-dd-yy:hh:mi:ss') hire_date,
 TO_CHAR(hire_date +
 TO_DSINTERVAL('100 10:00:00'),
 'mm-dd-yy:hh:mi:ss') hiredate2
FROM employees;
```

|   | LAST_NAME | HIRE_DATE         | HIREDATE2         |
|---|-----------|-------------------|-------------------|
| 1 | King      | 06-17-87:12:00:00 | 09-25-87:10:00:00 |
| 2 | Kochhar   | 09-21-89:12:00:00 | 12-30-89:10:00:00 |
| 3 | De Haan   | 01-13-93:12:00:00 | 04-23-93:10:00:00 |
| 4 | Hunold    | 01-03-90:12:00:00 | 04-13-90:10:00:00 |
| 5 | Ernst     | 05-21-91:12:00:00 | 08-29-91:10:00:00 |
| 6 | Lorentz   | 02-07-99:12:00:00 | 05-18-99:10:00:00 |

...

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### TO\_DSINTERVAL

TO\_DSINTERVAL converts a character string of the CHAR, VARCHAR2, NCHAR, or NVARCHAR2 data type to an INTERVAL DAY TO SECOND data type.

In the example in the slide, the date 100 days and 10 hours after the hire date is obtained.



## Daylight Saving Time

- First Sunday in April
  - Time jumps from 01:59:59 AM to 03:00:00 AM.
  - Values from 02:00:00 AM to 02:59:59 AM are not valid.
- Last Sunday in October
  - Time jumps from 02:00:00 AM to 01:00:01 AM
  - Values from 01:00:01 AM to 02:00:00 AM are ambiguous because they are visited twice.

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Daylight Saving Time (DST)

Most western nations advance the clock ahead one hour during the summer months. This period is called daylight saving time. Daylight saving time lasts from the first Sunday in April to the last Sunday in October in the most of the United States, Mexico, and Canada. The nations of the European Union observe daylight saving time, but they call it the summer time period. Europe's summer time period begins a week earlier than its North American counterpart, but ends at the same time.

The Oracle database automatically determines, for any given time zone region, whether daylight saving time is in effect and returns local time values accordingly. The datetime value is sufficient for the Oracle database to determine whether daylight saving time is in effect for a given region in all cases except boundary cases. A boundary case occurs during the period when daylight saving time goes into or out of effect. For example, in the US/Eastern region, when daylight saving time goes into effect, the time changes from 01:59:59 AM to 03:00:00 AM. The one-hour interval between 02:00:00 AM and 02:59:59 AM. does not exist. When daylight saving time goes out of effect, the time changes from 02:00:00 AM back to 01:00:01 AM, and the one-hour interval between 01:00:01 AM and 02:00:00 AM is repeated.

## Daylight Saving Time (DST) (continued)

### **ERROR\_ON\_OVERLAP\_TIME**

The `ERROR_ON_OVERLAP_TIME` is a session parameter to notify the system to issue an error when it encounters a datetime that occurs in the overlapped period and no time zone abbreviation was specified to distinguish the period.

For example, daylight saving time ends on October 31, at 02:00:01 AM. The overlapped periods are:

- 10/31/2004 01:00:01 AM to 10/31/2004 02:00:00 AM (EDT)
- 10/31/2004 01:00:01 AM to 10/31/2004 02:00:00 AM (EST)

If you input a datetime string that occurs in one of these two periods, you need to specify the time zone abbreviation (for example, EDT or EST) in the input string for the system to determine the period. Without this time zone abbreviation, the system does the following:

If the `ERROR_ON_OVERLAP_TIME` parameter is `FALSE`, it assumes that the input time is standard time (for example, EST). Otherwise, an error is raised.

Oracle Internal & Oracle Academy  
Use Only

## Summary

In this lesson, you should have learned how to use the following functions:

- CURRENT\_DATE
- CURRENT\_TIMESTAMP
- LOCALTIMESTAMP
- DBTIMEZONE
- SESSIONTIMEZONE
- EXTRACT
- TZ\_OFFSET
- FROM\_TZ
- TO\_TIMESTAMP
- TO\_YMINTERVAL
- TO\_DSINTERVAL

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Summary

This lesson addressed some of the datetime functions available in the Oracle database.

## Practice 5: Overview

This practice covers using the datetime functions.

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Practice 5: Overview

In this practice, you display time zone offsets, `CURRENT_DATE`, `CURRENT_TIMESTAMP`, and `LOCALTIMESTAMP`. You also set time zones and use the `EXTRACT` function.

## Practice 5

1. Alter the session to set NLS\_DATE\_FORMAT to DD-MON-YYYY HH24:MI:SS.
2. a. Write queries to display the time zone offsets (TZ\_OFFSET) for the following time zones.

- *US/Pacific-New*

|   |                                                                                                               |
|---|---------------------------------------------------------------------------------------------------------------|
|   |  TZ_OFFSET('US/PACIFIC-NEW') |
| 1 | -07:00                                                                                                        |

- *Singapore*

|   |                                                                                                          |
|---|----------------------------------------------------------------------------------------------------------|
|   |  TZ_OFFSET('SINGAPORE') |
| 1 | +08:00                                                                                                   |

- *Egypt*

|   |                                                                                                      |
|---|------------------------------------------------------------------------------------------------------|
|   |  TZ_OFFSET('EGYPT') |
| 1 | +03:00                                                                                               |

- b. Alter the session to set the TIME\_ZONE parameter value to the time zone offset of US/Pacific-New.
- c. Display CURRENT\_DATE, CURRENT\_TIMESTAMP, and LOCALTIMESTAMP for this session.

|   | CURRENT_DATE         | CURRENT_TIMESTAMP                      | LOCALTIMESTAMP                  |
|---|----------------------|----------------------------------------|---------------------------------|
| 1 | 12-jun-2007 09:57:27 | 12-JUN-07 09.57.27.318008000 AM -07:00 | 12-JUN-07 09.57.27.318008000 AM |

- d. Alter the session to set the TIME\_ZONE parameter value to the time zone offset of Singapore.
- e. Display CURRENT\_DATE, CURRENT\_TIMESTAMP, and LOCALTIMESTAMP for this session.

**Note:** The output might be different based on the date when the command is executed.

|   | CURRENT_DATE         | CURRENT_TIMESTAMP                      | LOCALTIMESTAMP                  |
|---|----------------------|----------------------------------------|---------------------------------|
| 1 | 13-jun-2007 00:58:53 | 13-JUN-07 12.58.53.066575000 AM +08:00 | 13-JUN-07 12.58.53.066575000 AM |

**Note:** Observe in the preceding practice that CURRENT\_DATE, CURRENT\_TIMESTAMP, and LOCALTIMESTAMP are all sensitive to the session time zone.

3. Write a query to display DBTIMEZONE and SESSIONTIMEZONE.

|   |                                                                                                |                                                                                                     |
|---|------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
|   |  DBTIMEZONE |  SESSIONTIMEZONE |
| 1 | +00:00                                                                                         | +08:00                                                                                              |

## Practice 5 (continued)

- Write a query to extract the YEAR from the HIRE\_DATE column of the EMPLOYEES table for those employees who work in department 80.

|   | LAST_NAME | EXTRACT(YEARFROMHIRE_DATE) |
|---|-----------|----------------------------|
| 1 | Zlotkey   | 2000                       |
| 2 | Abel      | 1996                       |
| 3 | Taylor    | 1998                       |

- Alter the session to set NLS\_DATE\_FORMAT to DD-MON-YYYY.
- Examine and run the lab\_05\_06.sql script to create the SAMPLE\_DATES table and populate it.
  - Select from the table and view the data.

|   | DATE_COL    |
|---|-------------|
| 1 | 12-jun-2007 |

- Modify the data type of the DATE\_COL column and change it to TIMESTAMP. Select from the table to view the data.

|   | DATE_COL                        |
|---|---------------------------------|
| 1 | 12-JUN-07 05.13.46.000000000 PM |

- Try to modify the data type of the DATE\_COL column and change it to TIMESTAMP WITH TIME ZONE. What happens?
- Create a query to retrieve last names from the EMPLOYEES table and calculate the review status. If the year hired was 1998, display Needs Review for the review status; otherwise, display not this year! Name the review status column Review. Sort the results by the HIRE\_DATE column.

**Hint:** Use a CASE expression with the EXTRACT function to calculate the review status.

## Practice 5 (continued)

|  | LAST_NAME |  | Review         |
|-----------------------------------------------------------------------------------|-----------|-----------------------------------------------------------------------------------|----------------|
| 1                                                                                 | King      |                                                                                   | not this year! |
| 2                                                                                 | Whalen    |                                                                                   | not this year! |
| 3                                                                                 | Kochhar   |                                                                                   | not this year! |
| 4                                                                                 | Hunold    |                                                                                   | not this year! |
| 5                                                                                 | Ernst     |                                                                                   | not this year! |
| 6                                                                                 | De Haan   |                                                                                   | not this year! |
| 7                                                                                 | Higgins   |                                                                                   | not this year! |
| 8                                                                                 | Gietz     |                                                                                   | not this year! |
| 9                                                                                 | Rajs      |                                                                                   | not this year! |
| 10                                                                                | Hartstein |                                                                                   | not this year! |

|    |         |  |                |
|----|---------|--|----------------|
| 11 | Abel    |  | not this year! |
| 12 | Davies  |  | not this year! |
| 13 | Fay     |  | not this year! |
| 14 | Matos   |  | Needs Review   |
| 15 | Taylor  |  | Needs Review   |
| 16 | Vargas  |  | Needs Review   |
| 17 | Lorentz |  | not this year! |
| 18 | Grant   |  | not this year! |
| 19 | Mourgos |  | not this year! |
| 20 | Zlotkey |  | not this year! |

Oracle Internal & Oracle Academy  
Use Only

### Practice 5 (continued)

8. Create a query to print the last names and the number of years of service for each employee. If the employee has been employed for five or more years, then print 5 years of service. If the employee has been employed for 10 or more years, then print 10 years of service. If the employee has been employed for 15 or more years, then print 15 years of service. If none of these conditions match, then print maybe next year! Sort the results by the HIRE\_DATE column. Use the EMPLOYEES table.

**Hint:** Use CASE expressions and TO\_YMINTERVAL.

| R  | LAST_NAME | HIRE_DATE                     | R           | SYSDATE | R                   | Awards |
|----|-----------|-------------------------------|-------------|---------|---------------------|--------|
| 1  | King      | 17-JUN-87 00.00.00.0000000000 | 18-JUL-2007 |         | 15 years of service |        |
| 2  | Kochhar   | 21-SEP-89 00.00.00.0000000000 | 18-JUL-2007 |         | 15 years of service |        |
| 3  | De Haan   | 13-JAN-93 00.00.00.0000000000 | 18-JUL-2007 |         | 10 years of service |        |
| 4  | Hunold    | 03-JAN-90 00.00.00.0000000000 | 18-JUL-2007 |         | 15 years of service |        |
| 5  | Ernst     | 21-MAY-91 00.00.00.0000000000 | 18-JUL-2007 |         | 15 years of service |        |
| 6  | Lorentz   | 07-FEB-99 00.00.00.0000000000 | 18-JUL-2007 |         | 5 years of service  |        |
| 7  | Mourgos   | 16-NOV-99 00.00.00.0000000000 | 18-JUL-2007 |         | 5 years of service  |        |
| 8  | Rajs      | 17-OCT-95 00.00.00.0000000000 | 18-JUL-2007 |         | 10 years of service |        |
| 9  | Davies    | 29-JAN-97 00.00.00.0000000000 | 18-JUL-2007 |         | 10 years of service |        |
| 10 | Matos     | 15-MAR-98 00.00.00.0000000000 | 18-JUL-2007 |         | 5 years of service  |        |
| 11 | Vargas    | 09-JUL-98 00.00.00.0000000000 | 18-JUL-2007 |         | 5 years of service  |        |
| 12 | Zlotkey   | 29-JAN-00 00.00.00.0000000000 | 18-JUL-2007 |         | 5 years of service  |        |
| 13 | Abel      | 11-MAY-96 00.00.00.0000000000 | 18-JUL-2007 |         | 10 years of service |        |
| 14 | Taylor    | 24-MAR-98 00.00.00.0000000000 | 18-JUL-2007 |         | 5 years of service  |        |
| 15 | Grant     | 24-MAY-99 00.00.00.0000000000 | 18-JUL-2007 |         | 5 years of service  |        |
| 16 | Whalen    | 17-SEP-87 00.00.00.0000000000 | 18-JUL-2007 |         | 15 years of service |        |
| 17 | Hartstein | 17-FEB-96 00.00.00.0000000000 | 18-JUL-2007 |         | 10 years of service |        |
| 18 | Fay       | 17-AUG-97 00.00.00.0000000000 | 18-JUL-2007 |         | 5 years of service  |        |
| 19 | Higgins   | 07-JUN-94 00.00.00.0000000000 | 18-JUL-2007 |         | 10 years of service |        |
| 20 | Gietz     | 07-JUN-94 00.00.00.0000000000 | 18-JUL-2007 |         | 10 years of service |        |



## Retrieving Data Using Subqueries

ORACLE<sup>®</sup>

Copyright © 2007, Oracle. All rights reserved.

## Objectives

After completing this lesson, you should be able to do the following:

- Write a multiple-column subquery
- Use scalar subqueries in SQL
- Solve problems with correlated subqueries
- Update and delete rows using correlated subqueries
- Use the `EXISTS` and `NOT EXISTS` operators
- Use the `WITH` clause

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Objectives

In this lesson, you learn how to write multiple-column subqueries and subqueries in the `FROM` clause of a `SELECT` statement. You also learn how to solve problems by using scalar, correlated subqueries and the `WITH` clause.

## Lesson Agenda

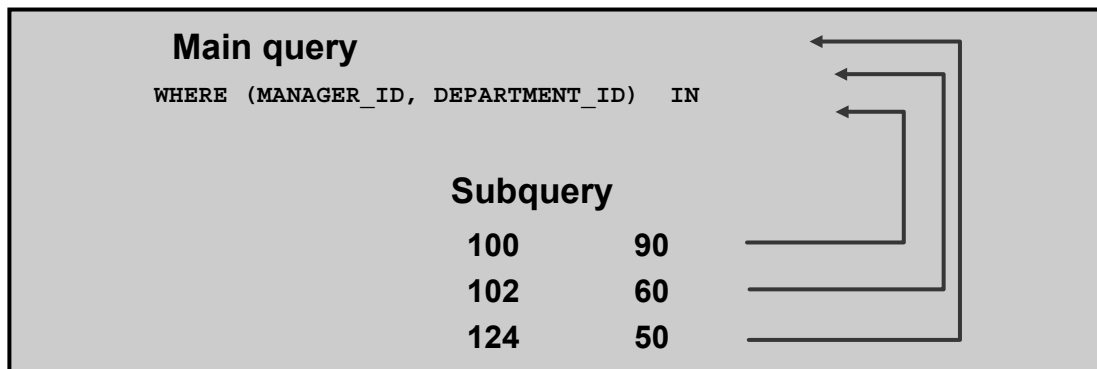
- Writing a multiple-column subquery
- Using scalar subqueries in SQL
- Solving problems with correlated subqueries
- Using the EXISTS and NOT EXISTS operators
- Using the WITH clause

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Oracle Internal & Oracle Academy  
Use Only

## Multiple-Column Subqueries



**Each row of the main query is compared to values from a multiple-row and multiple-column subquery.**

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Multiple-Column Subqueries

So far, you have written single-row subqueries and multiple-row subqueries where only one column is returned by the inner `SELECT` statement and this is used to evaluate the expression in the parent `SELECT` statement. If you want to compare two or more columns, you must write a compound `WHERE` clause using logical operators. Using multiple-column subqueries, you can combine duplicate `WHERE` conditions into a single `WHERE` clause.

#### Syntax

```
SELECT column, column, ...
FROM table
WHERE (column, column, ...) IN
 (SELECT column, column, ...
 FROM table
 WHERE condition);
```

The graphic in the slide illustrates that the values of `MANAGER_ID` and `DEPARTMENT_ID` from the main query are being compared with the `MANAGER_ID` and `DEPARTMENT_ID` values retrieved by the subquery. Because the number of columns that are being compared is more than one, the example qualifies as a multiple-column subquery.

**Note:** Before you run the examples in the next few slides, you need to create the `empl_demo` table and populate data into it by using the `lab_06_insert_empdata.sql` file.

## Column Comparisons

Multiple-column comparisons involving subqueries can be:

- Nonpairwise comparisons
- Pairwise comparisons

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Pairwise Versus Nonpairwise Comparisons

Multiple-column comparisons involving subqueries can be nonpairwise comparisons or pairwise comparisons. If you consider the example “Display the details of the employees who work in the same department, and have the same manager, as ‘Daniel’?,” you get the correct result with the following statement:

```
SELECT first_name, last_name, manager_id, department_id
FROM empl_demo
WHERE manager_id IN (SELECT manager_id
 FROM empl_demo
 WHERE first_name = 'Daniel')
AND department_id IN (SELECT department_id
 FROM empl_demo
 WHERE first_name = 'Daniel');
```

There is only one “Daniel” in the EMPL\_DEMO table (Daniel Faviet, who is managed by employee 108 and works in department 100). However, if the subqueries return more than one row, the result might not be correct. For example, if you run the same query but substitute “John” for “Daniel,” you get an incorrect result. This is because the combination of department\_id and manager\_id is important. To get the correct result for this query, you need a pairwise comparison.

## Pairwise Comparison Subquery

Display the details of the employees who are managed by the same manager and work in the same department as employees with the first name of “John.”

```
SELECT employee_id, manager_id, department_id
FROM empl_demo
WHERE (manager_id, department_id) IN
 (SELECT manager_id, department_id
 FROM empl_demo
 WHERE first_name = 'John')
AND first_name <> 'John';
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Pairwise Comparison Subquery

The example in the slide compares the combination of values in the `MANAGER_ID` column and the `DEPARTMENT_ID` column of each row in the `EMPL_DEMO` table with the values in the `MANAGER_ID` column and the `DEPARTMENT_ID` column for the employees with the `FIRST_NAME` of “John.” First, the subquery to retrieve the `MANAGER_ID` and `DEPARTMENT_ID` values for the employees with the `FIRST_NAME` of “John” is executed. This subquery returns the following:

|   | MANAGER_ID | DEPARTMENT_ID |
|---|------------|---------------|
| 1 | 108        | 100           |
| 2 | 123        | 50            |
| 3 | 100        | 80            |

### Pairwise Comparison Subquery (continued)

These values are compared with the `MANAGER_ID` column and the `DEPARTMENT_ID` column of each row in the `EMPL_DEMO` table. If the combination matches, the row is displayed. In the output, the records of the employees with the `FIRST_NAME` of “John” will not be displayed. The following is the output of the query in the slide:

|    | EMPLOYEE_ID | MANAGER_ID | DEPARTMENT_ID |
|----|-------------|------------|---------------|
| 1  | 113         | 108        | 100           |
| 2  | 112         | 108        | 100           |
| 3  | 111         | 108        | 100           |
| 4  | 109         | 108        | 100           |
| 5  | 195         | 123        | 50            |
| 6  | 194         | 123        | 50            |
| 7  | 193         | 123        | 50            |
| 8  | 192         | 123        | 50            |
| 9  | 140         | 123        | 50            |
| 10 | 138         | 123        | 50            |
| 11 | 137         | 123        | 50            |
| 12 | 149         | 100        | 80            |
| 13 | 148         | 100        | 80            |
| 14 | 147         | 100        | 80            |
| 15 | 146         | 100        | 80            |

## Nonpairwise Comparison Subquery

Display the details of the employees who are managed by the same manager as the employees with the first name of “John” and work in the same department as the employees with the first name of “John.”

```
SELECT employee_id, manager_id, department_id
FROM empl_demo
WHERE manager_id IN
 (SELECT manager_id
 FROM empl_demo
 WHERE first_name = 'John')
AND department_id IN
 (SELECT department_id
 FROM empl_demo
 WHERE first_name = 'John')
AND first_name <> 'John';
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Nonpairwise Comparison Subquery

The example shows a nonpairwise comparison of the columns. First, the subquery to retrieve the MANAGER\_ID values for the employees with the FIRST\_NAME of “John” is executed. Similarly, the second subquery to retrieve the DEPARTMENT\_ID values for the employees with the FIRST\_NAME of “John” is executed. The retrieved values of the MANAGER\_ID and DEPARTMENT\_ID columns are compared with the MANAGER\_ID and DEPARTMENT\_ID columns for each row in the EMPL\_DEMO table. If the MANAGER\_ID column of the row in the EMPL\_DEMO table matches with any of the values of MANAGER\_ID retrieved by the inner subquery and if the DEPARTMENT\_ID column of the row in the EMPL\_DEMO table matches with any of the values of DEPARTMENT\_ID retrieved by the second subquery, the record is displayed.



### Nonpairwise Comparison Subquery (continued)

The following is the output of the query in the previous slide:

|    | EMPLOYEE_ID | MANAGER_ID | DEPARTMENT_ID |
|----|-------------|------------|---------------|
| 1  | 109         | 108        | 100           |
| 2  | 111         | 108        | 100           |
| 3  | 112         | 108        | 100           |
| 4  | 113         | 108        | 100           |
| 5  | 120         | 100        | 50            |
| 6  | 121         | 100        | 50            |
| 7  | 122         | 100        | 50            |
| 8  | 123         | 100        | 50            |
| 9  | 124         | 100        | 50            |
| 10 | 137         | 123        | 50            |
| 11 | 138         | 123        | 50            |
| 12 | 140         | 123        | 50            |
| 13 | 192         | 123        | 50            |
| 14 | 193         | 123        | 50            |
| 15 | 194         | 123        | 50            |
| 16 | 195         | 123        | 50            |
| 17 | 146         | 100        | 80            |
| 18 | 147         | 100        | 80            |
| 19 | 148         | 100        | 80            |
| 20 | 149         | 100        | 80            |

This query retrieves additional rows than the pairwise comparison (those with the combination of manager\_id=100 and department\_id=50 or 80, although no employee named “John” has such a combination).

## Lesson Agenda

- Writing a multiple-column subquery
- Using scalar subqueries in SQL
- Solving problems with correlated subqueries
- Using the EXISTS and NOT EXISTS operators
- Using the WITH clause

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Oracle Internal & Oracle Academy  
Use Only

## Scalar Subquery Expressions

- A scalar subquery expression is a subquery that returns exactly one column value from one row.
- Scalar subqueries can be used in:
  - The condition and expression part of `DECODE` and `CASE`
  - All clauses of `SELECT` except `GROUP BY`
  - The `SET` clause and `WHERE` clause of an `UPDATE` statement

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Scalar Subqueries in SQL

A subquery that returns exactly one column value from one row is also referred to as a scalar subquery. Multiple-column subqueries that are written to compare two or more columns, using a compound `WHERE` clause and logical operators, do not qualify as scalar subqueries.

The value of the scalar subquery expression is the value of the select list item of the subquery. If the subquery returns 0 rows, the value of the scalar subquery expression is `NULL`. If the subquery returns more than one row, the Oracle server returns an error. The Oracle server has always supported the usage of a scalar subquery in a `SELECT` statement. You can use scalar subqueries in:

- The condition and expression part of `DECODE` and `CASE`
- All clauses of `SELECT` except `GROUP BY`
- The `SET` clause and `WHERE` clause of an `UPDATE` statement

However, scalar subqueries are not valid expressions in the following places:

- As default values for columns and hash expressions for clusters
- In the `RETURNING` clause of data manipulation language (DML) statements
- As the basis of a function-based index
- In `GROUP BY` clauses, `CHECK` constraints, and `WHEN` conditions
- In `CONNECT BY` clauses
- In statements that are unrelated to queries, such as `CREATE PROFILE`

## Scalar Subqueries: Examples

- Scalar subqueries in CASE expressions:

```
SELECT employee_id, last_name,
 (CASE
 WHEN department_id = 20
 (SELECT department_id
 FROM departments
 WHERE location_id = 1800)
 THEN 'Canada' ELSE 'USA' END) location
FROM employees;
```

- Scalar subqueries in the ORDER BY clause:

```
SELECT employee_id, last_name
FROM employees e
ORDER BY (SELECT department_name
 FROM departments d
 WHERE e.department_id = d.department_id);
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Scalar Subqueries: Examples

The first example in the slide demonstrates that scalar subqueries can be used in CASE expressions. The inner query returns the value 20, which is the department ID of the department whose location ID is 1800. The CASE expression in the outer query uses the result of the inner query to display the employee ID, last names, and a value of Canada or USA, depending on whether the department ID of the record retrieved by the outer query is 20 or not.

The following is the result of the first example in the slide:

...

|    | EMPLOYEE_ID | LAST_NAME | LOCATION |
|----|-------------|-----------|----------|
| 14 | 176         | Taylor    | USA      |
| 15 | 178         | Grant     | USA      |
| 16 | 200         | Whalen    | USA      |
| 17 | 201         | Hartstein | Canada   |
| 18 | 202         | Fay       | Canada   |
| 19 | 205         | Higgins   | USA      |
| 20 | 206         | Gietz     | USA      |

### Scalar Subqueries: Examples (continued)

The second example in the slide demonstrates that scalar subqueries can be used in the ORDER BY clause. The example orders the output based on the DEPARTMENT\_NAME by matching the DEPARTMENT\_ID from the EMPLOYEES table with the DEPARTMENT\_ID from the DEPARTMENTS table. This comparison is done in a scalar subquery in the ORDER BY clause.

The following is the result of the second example:

|    | EMPLOYEE_ID | LAST_NAME |
|----|-------------|-----------|
| 1  | 205         | Higgins   |
| 2  | 206         | Gietz     |
| 3  | 200         | Whalen    |
| 4  | 102         | De Haan   |
| 5  | 100         | King      |
| 6  | 101         | Kochhar   |
| 7  | 103         | Hunold    |
| 8  | 104         | Ernst     |
| 9  | 107         | Lorentz   |
| 10 | 202         | Fay       |
| 11 | 201         | Hartstein |
| 12 | 149         | Zlotkey   |
| 13 | 174         | Abel      |
| 14 | 176         | Taylor    |
| 15 | 143         | Matos     |
| 16 | 142         | Davies    |
| 17 | 141         | Rajs      |
| 18 | 124         | Mourgos   |
| 19 | 144         | Vargas    |
| 20 | 178         | Grant     |

The second example uses a correlated subquery. In a correlated subquery, the subquery references a column from a table referred to in the parent statement. Correlated subqueries are explained later in this lesson.

## Lesson Agenda

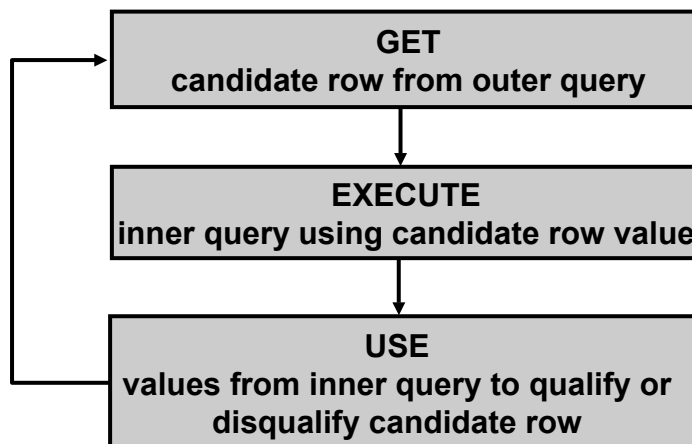
- Writing a multiple-column subquery
- Using scalar subqueries in SQL
- Solving problems with correlated subqueries
- Using the `EXISTS` and `NOT EXISTS` operators
- Using the `WITH` clause

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Correlated Subqueries

Correlated subqueries are used for row-by-row processing. Each subquery is executed once for every row of the outer query.



ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Correlated Subqueries

The Oracle server performs a correlated subquery when the subquery references a column from a table referred to in the parent statement. A correlated subquery is evaluated once for each row processed by the parent statement. The parent statement can be a `SELECT`, `UPDATE`, or `DELETE` statement.

#### Nested Subqueries Versus Correlated Subqueries

With a normal nested subquery, the inner `SELECT` query runs first and executes once, returning values to be used by the main query. A correlated subquery, however, executes once for each candidate row considered by the outer query. That is, the inner query is driven by the outer query.

#### Nested Subquery Execution

- The inner query executes first and finds a value.
- The outer query executes once, using the value from the inner query.

#### Correlated Subquery Execution

- Get a candidate row (fetched by the outer query).
- Execute the inner query using the value of the candidate row.
- Use the values resulting from the inner query to qualify or disqualify the candidate.
- Repeat until no candidate row remains.

## Correlated Subqueries

The subquery references a column from a table in the parent query.

```
SELECT column1, column2, ...
FROM table1 outer
WHERE column1 operator
 (SELECT column1, column2
 FROM table2
 WHERE expr1 =
 outer.expr2);
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Correlated Subqueries (continued)

A correlated subquery is one way of reading every row in a table and comparing values in each row against related data. It is used whenever a subquery must return a different result or set of results for each candidate row considered by the main query. That is, you use a correlated subquery to answer a multipart question whose answer depends on the value in each row processed by the parent statement.

The Oracle server performs a correlated subquery when the subquery references a column from a table in the parent query.

**Note:** You can use the ANY and ALL operators in a correlated subquery.



## Using Correlated Subqueries

Find all employees who earn more than the average salary in their department.

```
SELECT last_name, salary, department_id
FROM employees outer
WHERE salary > (SELECT AVG(salary)
 FROM employees
 WHERE department_id =
 outer.department_id);
```

Each time a row from the outer query is processed, the inner query is evaluated.

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Using Correlated Subqueries

The example in the slide determines which employees earn more than the average salary of their department. In this case, the correlated subquery specifically computes the average salary for each department.

Because both the outer query and inner query use the EMPLOYEES table in the FROM clause, an alias is given to EMPLOYEES in the outer SELECT statement for clarity. The alias makes the entire SELECT statement more readable. Without the alias, the query would not work properly because the inner statement would not be able to distinguish the inner table column from the outer table column.

## Using Correlated Subqueries

Display details of those employees who have changed jobs at least twice.

```
SELECT e.employee_id, last_name, e.job_id
FROM employees e
WHERE 2 <= (SELECT COUNT(*)
 FROM job_history
 WHERE employee_id = e.employee_id);
```

|   | EMPLOYEE_ID | LAST_NAME | JOB_ID  |
|---|-------------|-----------|---------|
| 1 | 101         | Kochhar   | AD_VP   |
| 2 | 176         | Taylor    | SA_REP  |
| 3 | 200         | Whalen    | AD_ASST |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Using Correlated Subqueries (continued)

The example in the slide displays the details of those employees who have changed jobs at least twice. The Oracle server evaluates a correlated subquery as follows:

1. Select a row from the table specified in the outer query. This will be the current candidate row.
2. Store the value of the column referenced in the subquery from this candidate row. (In the example in the slide, the column referenced in the subquery is E.EMPLOYEE\_ID.)
3. Perform the subquery with its condition referencing the value from the outer query's candidate row. (In the example in the slide, the COUNT(\*) group function is evaluated based on the value of the E.EMPLOYEE\_ID column obtained in step 2.)
4. Evaluate the WHERE clause of the outer query on the basis of results of the subquery performed in step 3. This determines whether the candidate row is selected for output. (In the example, the number of times an employee has changed jobs, evaluated by the subquery, is compared with 2 in the WHERE clause of the outer query. If the condition is satisfied, that employee record is displayed.)
5. Repeat the procedure for the next candidate row of the table, and so on, until all the rows in the table have been processed.

The correlation is established by using an element from the outer query in the subquery. In this example, you compare EMPLOYEE\_ID from the table in the subquery with EMPLOYEE\_ID from the table in the outer query.

## Lesson Agenda

- Writing a multiple-column subquery
- Using scalar subqueries in SQL
- Solving problems with correlated subqueries
- **Using the EXISTS and NOT EXISTS operators**
- Using the WITH clause

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Using the EXISTS Operator

- The EXISTS operator tests for existence of rows in the results set of the subquery.
- If a subquery row value is found:
  - The search does not continue in the inner query
  - The condition is flagged TRUE
- If a subquery row value is not found:
  - The condition is flagged FALSE
  - The search continues in the inner query

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### EXISTS Operator

With nesting SELECT statements, all logical operators are valid. In addition, you can use the EXISTS operator. This operator is frequently used with correlated subqueries to test whether a value retrieved by the outer query exists in the results set of the values retrieved by the inner query. If the subquery returns at least one row, the operator returns TRUE. If the value does not exist, it returns FALSE. Accordingly, NOT EXISTS tests whether a value retrieved by the outer query is not a part of the results set of the values retrieved by the inner query.

## Find Employees Who Have at Least One Person Reporting to Them

```
SELECT employee_id, last_name, job_id, department_id
FROM employees outer
WHERE EXISTS (SELECT 'X'
 FROM employees
 WHERE manager_id =
 outer.employee_id);
```

|   | EMPLOYEE_ID | LAST_NAME | JOB_ID  | DEPARTMENT_ID |
|---|-------------|-----------|---------|---------------|
| 1 | 100         | King      | AD_PRES | 90            |
| 2 | 101         | Kochhar   | AD_VP   | 90            |
| 3 | 102         | De Haan   | AD_VP   | 90            |
| 4 | 103         | Hunold    | IT_PROG | 60            |
| 5 | 124         | Mourgos   | ST_MAN  | 50            |
| 6 | 149         | Zlotkey   | SA_MAN  | 80            |
| 7 | 201         | Hartstein | MK_MAN  | 20            |
| 8 | 205         | Higgins   | AC_MGR  | 110           |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Using the EXISTS Operator

The EXISTS operator ensures that the search in the inner query does not continue when at least one match is found for the manager and employee number by the condition:

```
WHERE manager_id = outer.employee_id.
```

Note that the inner SELECT query does not need to return a specific value, so a constant can be selected.

## Find All Departments That Do Not Have Any Employees

```
SELECT department_id, department_name
FROM departments d
WHERE NOT EXISTS (SELECT 'X'
 FROM employees
 WHERE department_id = d.department_id);
```

|   | DEPARTMENT_ID | DEPARTMENT_NAME |
|---|---------------|-----------------|
| 1 |               | 190 Contracting |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Using the NOT EXISTS Operator

#### Alternative Solution

A NOT IN construct can be used as an alternative for a NOT EXISTS operator, as shown in the following example:

```
SELECT department_id, department_name
FROM departments
WHERE department_id NOT IN (SELECT department_id
 FROM employees);
```

No rows selected.

However, NOT IN evaluates to FALSE if any member of the set is a NULL value. Therefore, your query will not return any rows even if there are rows in the departments table that satisfy the WHERE condition.

## Correlated UPDATE

Use a correlated subquery to update rows in one table based on rows from another table.

```
UPDATE table1 alias1
SET column = (SELECT expression
 FROM table2 alias2
 WHERE alias1.column =
 alias2.column);
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Correlated UPDATE

In the case of the UPDATE statement, you can use a correlated subquery to update rows in one table based on rows from another table.

## Using Correlated UPDATE

- Denormalize the EMPL6 table by adding a column to store the department name.
- Populate the table by using a correlated update.

```
ALTER TABLE empl6
ADD(department_name VARCHAR2(25));
```

```
UPDATE empl6 e
SET department_name =
 (SELECT department_name
 FROM departments d
 WHERE e.department_id = d.department_id);
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Correlated UPDATE (continued)

The example in the slide denormalizes the EMPL6 table by adding a column to store the department name and then populates the table by using a correlated update.

Following is another example for a correlated update.

#### Problem Statement

The REWARDS table has a list of employees who have exceeded expectations in their performance. Use a correlated subquery to update rows in the EMPL6 table based on rows from the REWARDS table:

```
UPDATE empl6
SET salary = (SELECT empl6.salary + rewards.pay_raise
 FROM rewards
 WHERE employee_id =
 empl6.employee_id
 AND payraise_date =
 (SELECT MAX(payraise_date)
 FROM rewards
 WHERE employee_id = empl6.employee_id))
WHERE empl6.employee_id
IN (SELECT employee_id FROM rewards);
```



### **Correlated UPDATE (continued)**

This example uses the REWARDS table. The REWARDS table has the following columns: EMPLOYEE\_ID, PAY\_RAISE, and PAYRAISE\_DATE. Every time an employee gets a pay raise, a record with details such as the employee ID, the amount of the pay raise, and the date of receipt of the pay raise is inserted into the REWARDS table. The REWARDS table can contain more than one record for an employee. The PAYRAISE\_DATE column is used to identify the most recent pay raise received by an employee.

In the example, the SALARY column in the EMPL6 table is updated to reflect the latest pay raise received by the employee. This is done by adding the current salary of the employee with the corresponding pay raise from the REWARDS table.

Oracle Internal & Oracle Academy  
Use Only

## Correlated DELETE

Use a correlated subquery to delete rows in one table based on rows from another table.

```
DELETE FROM table1 alias1
WHERE column operator
 (SELECT expression
 FROM table2 alias2
 WHERE alias1.column = alias2.column);
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Correlated DELETE

In the case of a DELETE statement, you can use a correlated subquery to delete only those rows that also exist in another table. If you decide that you will maintain only the last four job history records in the JOB\_HISTORY table, then when an employee transfers to a fifth job, you delete the oldest JOB\_HISTORY row by looking up the JOB\_HISTORY table for the MIN(START\_DATE) for the employee. The following code illustrates how the preceding operation can be performed using a correlated DELETE:

```
DELETE FROM emp_history JH
WHERE employee_id =
 (SELECT employee_id
 FROM employees E
 WHERE JH.employee_id = E.employee_id
 AND START_DATE =
 (SELECT MIN(start_date)
 FROM job_history JH
 WHERE JH.employee_id = E.employee_id)
 AND 5 > (SELECT COUNT(*)
 FROM job_history JH
 WHERE JH.employee_id = E.employee_id
 GROUP BY EMPLOYEE_ID
 HAVING COUNT(*) >= 4));
```

## Using Correlated DELETE

Use a correlated subquery to delete only those rows from the EMPL6 table that also exist in the EMP\_HISTORY table.

```
DELETE FROM empl6 E
WHERE employee_id =
 (SELECT employee_id
 FROM emp_history
 WHERE employee_id = E.employee_id);
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Correlated DELETE (continued)

#### Example

Two tables are used in this example. They are:

- The EMPL6 table, which provides details of all the current employees
- The EMP\_HISTORY table, which provides details of previous employees

EMP\_HISTORY contains data regarding previous employees, so it would be erroneous if the same employee's record existed in both the EMPL6 and EMP\_HISTORY tables. You can delete such erroneous records by using the correlated subquery shown in the slide.

## Lesson Agenda

- Writing a multiple-column subquery
- Using scalar subqueries in SQL
- Solving problems with correlated subqueries
- Using the `EXISTS` and `NOT EXISTS` operators
- Using the `WITH` clause

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Oracle Internal & Oracle Academy  
Use Only

## WITH Clause

- Using the `WITH` clause, you can use the same query block in a `SELECT` statement when it occurs more than once within a complex query.
- The `WITH` clause retrieves the results of a query block and stores it in the user's temporary tablespace.
- The `WITH` clause may improve performance.

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### WITH Clause

Using the `WITH` clause, you can define a query block before using it in a query. The `WITH` clause (formally known as `subquery_factoring_clause`) enables you to reuse the same query block in a `SELECT` statement when it occurs more than once within a complex query. This is particularly useful when a query has many references to the same query block and there are joins and aggregations.

Using the `WITH` clause, you can reuse the same query when it is costly to evaluate the query block and it occurs more than once within a complex query. Using the `WITH` clause, the Oracle server retrieves the results of a query block and stores it in the user's temporary tablespace. This can improve performance.

#### WITH Clause Benefits

- Makes the query easy to read
- Evaluates a clause only once, even if it appears multiple times in the query
- In most cases, may improve performance for large queries

## WITH Clause: Example

Using the `WITH` clause, write a query to display the department name and total salaries for those departments whose total salary is greater than the average salary across departments.

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### WITH Clause: Example

The problem in the slide would require the following intermediate calculations:

1. Calculate the total salary for every department, and store the result using a `WITH` clause.
2. Calculate the average salary across departments, and store the result using a `WITH` clause.
3. Compare the total salary calculated in the first step with the average salary calculated in the second step. If the total salary for a particular department is greater than the average salary across departments, then display the department name and the total salary for that department.

The solution for this problem is provided on the next page.

## WITH Clause: Example

```
WITH
dept_costs AS (
 SELECT d.department_name, SUM(e.salary) AS dept_total
 FROM employees e JOIN departments d
 ON e.department_id = d.department_id
 GROUP BY d.department_name),
avg_cost AS (
 SELECT SUM(dept_total)/COUNT(*) AS dept_avg
 FROM dept_costs)
SELECT *
FROM dept_costs
WHERE dept_total >
 (SELECT dept_avg
 FROM avg_cost)
ORDER BY department_name;
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### WITH Clause: Example (continued)

The SQL code in the slide is an example of a situation in which you can improve performance and write SQL more simply by using the WITH clause. The query creates the query names DEPT\_COSTS and AVG\_COST and then uses them in the body of the main query. Internally, the WITH clause is resolved either as an in-line view or a temporary table. The optimizer chooses the appropriate resolution depending on the cost or benefit of temporarily storing the results of the WITH clause.

The output generated by the SQL code in the slide is as follows:

|   | DEPARTMENT_NAME | DEPT_TOTAL |
|---|-----------------|------------|
| 1 | Executive       | 58000      |
| 2 | Sales           | 30100      |

### WITH Clause Usage Notes

- It is used only with SELECT statements.
- A query name is visible to all WITH element query blocks (including their subquery blocks) defined after it and the main query block itself (including its subquery blocks).
- When the query name is the same as an existing table name, the parser searches from the inside out, and the query block name takes precedence over the table name.
- The WITH clause can hold more than one query. Each query is then separated by a comma.

## Summary

In this lesson, you should have learned that:

- A multiple-column subquery returns more than one column
- Multiple-column comparisons can be pairwise or nonpairwise
- A multiple-column subquery can also be used in the `FROM` clause of a `SELECT` statement

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Summary

You can use multiple-column subqueries to combine multiple `WHERE` conditions in a single `WHERE` clause. Column comparisons in a multiple-column subquery can be pairwise comparisons or nonpairwise comparisons.

You can use a subquery to define a table to be operated on by a containing query.

Scalar subqueries can be used in:

- The condition and expression part of `DECODE` and `CASE`
- All clauses of `SELECT` except `GROUP BY`
- A `SET` clause and `WHERE` clause of the `UPDATE` statement



## Summary

- Correlated subqueries are useful whenever a subquery must return a different result for each candidate row
- The `EXISTS` operator is a Boolean operator that tests the presence of a value
- Correlated subqueries can be used with `SELECT`, `UPDATE`, and `DELETE` statements
- You can use the `WITH` clause to use the same query block in a `SELECT` statement when it occurs more than once

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Summary (continued)

The Oracle server performs a correlated subquery when the subquery references a column from a table referred to in the parent statement. A correlated subquery is evaluated once for each row processed by the parent statement. The parent statement can be a `SELECT`, `UPDATE`, or `DELETE` statement. Using the `WITH` clause, you can reuse the same query when it is costly to reevaluate the query block and it occurs more than once within a complex query.

## Practice 6: Overview

This practice covers the following topics:

- Creating multiple-column subqueries
- Writing correlated subqueries
- Using the `EXISTS` operator
- Using scalar subqueries
- Using the `WITH` clause

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Practice 6: Overview

In this practice, you write multiple-column subqueries, and correlated and scalar subqueries. You also solve problems by writing the `WITH` clause.

## Practice 6

1. Write a query to display the last name, department number, and salary of any employee whose department number and salary both match the department number and salary of any employee who earns a commission.

|  | LAST_NAME |  | DEPARTMENT_ID |  | SALARY |
|-----------------------------------------------------------------------------------|-----------|-----------------------------------------------------------------------------------|---------------|-----------------------------------------------------------------------------------|--------|
| 1                                                                                 | Taylor    |                                                                                   | 80            |                                                                                   | 8600   |
| 2                                                                                 | Zlotkey   |                                                                                   | 80            |                                                                                   | 10500  |
| 3                                                                                 | Abel      |                                                                                   | 80            |                                                                                   | 11000  |

2. Display the last name, department name, and salary of any employee whose salary and commission match the salary and commission of any employee located in location ID 1700.

|  | LAST_NAME |  | DEPARTMENT_NAME |  | SALARY |
|-----------------------------------------------------------------------------------|-----------|-----------------------------------------------------------------------------------|-----------------|-----------------------------------------------------------------------------------|--------|
| 1                                                                                 | King      |                                                                                   | Executive       |                                                                                   | 24000  |
| 2                                                                                 | Higgins   |                                                                                   | Accounting      |                                                                                   | 12000  |
| 3                                                                                 | De Haan   |                                                                                   | Executive       |                                                                                   | 17000  |
| 4                                                                                 | Gietz     |                                                                                   | Accounting      |                                                                                   | 8300   |
| 5                                                                                 | Kochhar   |                                                                                   | Executive       |                                                                                   | 17000  |
| 6                                                                                 | Whalen    |                                                                                   | Administration  |                                                                                   | 4400   |

3. Create a query to display the last name, hire date, and salary for all employees who have the same salary and commission as Kochhar.

**Note:** Do not display Kochhar in the result set.

|  | LAST_NAME | HIRE_DATE |  | SALARY |
|-------------------------------------------------------------------------------------|-----------|-----------|-------------------------------------------------------------------------------------|--------|
| 1                                                                                   | De Haan   | 13-JAN-93 |                                                                                     | 17000  |

4. Create a query to display the employees who earn a salary that is higher than the salary of all the sales managers (JOB\_ID = 'SA\_MAN'). Sort the results on salary from the highest to the lowest.

|  | LAST_NAME |  | JOB_ID  |  | SALARY |
|-------------------------------------------------------------------------------------|-----------|-------------------------------------------------------------------------------------|---------|-------------------------------------------------------------------------------------|--------|
| 1                                                                                   | King      |                                                                                     | AD_PRES |                                                                                     | 24000  |
| 2                                                                                   | De Haan   |                                                                                     | AD_VP   |                                                                                     | 17000  |
| 3                                                                                   | Kochhar   |                                                                                     | AD_VP   |                                                                                     | 17000  |
| 4                                                                                   | Hartstein |                                                                                     | MK_MAN  |                                                                                     | 13000  |
| 5                                                                                   | Higgins   |                                                                                     | AC_MGR  |                                                                                     | 12000  |
| 6                                                                                   | Abel      |                                                                                     | SA_REP  |                                                                                     | 11000  |

## Practice 6 (continued)

5. Display details such as the employee ID, last name, and department ID of those employees who live in cities the names of which begin with *T*.

|   |  EMPLOYEE_ID |  LAST_NAME |  DEPARTMENT_ID |
|---|-----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| 1 | 201                                                                                           | Hartstein                                                                                   | 20                                                                                              |
| 2 | 202                                                                                           | Fay                                                                                         | 20                                                                                              |

6. Write a query to find all employees who earn more than the average salary in their departments. Display last name, salary, department ID, and the average salary for the department. Sort by average salary and round to 2 decimals. Use aliases for the columns retrieved by the query as shown in the sample output.

|   |  ENAME |  SALARY |  DEPTNO |  DEPT_AVG |
|---|-----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| 1 | Mourgos                                                                                 | 5800                                                                                     | 50                                                                                       | 3500                                                                                       |
| 2 | Hunold                                                                                  | 9000                                                                                     | 60                                                                                       | 6400                                                                                       |
| 3 | Hartstein                                                                               | 13000                                                                                    | 20                                                                                       | 9500                                                                                       |
| 4 | Zlotkey                                                                                 | 10500                                                                                    | 80                                                                                       | 10033.33                                                                                   |
| 5 | Abel                                                                                    | 11000                                                                                    | 80                                                                                       | 10033.33                                                                                   |
| 6 | Higgins                                                                                 | 12000                                                                                    | 110                                                                                      | 10150                                                                                      |
| 7 | King                                                                                    | 24000                                                                                    | 90                                                                                       | 19333.33                                                                                   |

7. Find all employees who are not supervisors.  
a. First do this using the NOT EXISTS operator

|    |  LAST_NAME |
|----|-----------------------------------------------------------------------------------------------|
| 1  | Abel                                                                                          |
| 2  | Davies                                                                                        |
| 3  | Ernst                                                                                         |
| 4  | Fay                                                                                           |
| 5  | Gietz                                                                                         |
| 6  | Grant                                                                                         |
| 7  | Lorentz                                                                                       |
| 8  | Matos                                                                                         |
| 9  | Rajs                                                                                          |
| 10 | Taylor                                                                                        |
| 11 | Vargas                                                                                        |
| 12 | Whalen                                                                                        |

- b. Can this be done by using the NOT IN operator? How, or why not?

### Practice 6 (continued)

8. Write a query to display the last names of the employees who earn less than the average salary in their departments.

|    | LAST_NAME |
|----|-----------|
| 1  | Kochhar   |
| 2  | De Haan   |
| 3  | Fay       |
| 4  | Gietz     |
| 5  | Davies    |
| 6  | Matos     |
| 7  | Vargas    |
| 8  | Taylor    |
| 9  | Ernst     |
| 10 | Lorentz   |

9. Write a query to display the last names of the employees who have one or more coworkers in their departments with later hire dates but higher salaries.

|   | LAST_NAME |
|---|-----------|
| 1 | Vargas    |
| 2 | Matos     |
| 3 | Davies    |
| 4 | Rajs      |
| 5 | Taylor    |

10. Write a query to display the employee ID, last names, and department names of all employees.

**Note:** Use a scalar subquery to retrieve the department name in the SELECT statement.

|     | EMPLOYEE_ID | LAST_NAME | DEPARTMENT     |
|-----|-------------|-----------|----------------|
| 1   | 205         | Higgins   | Accounting     |
| 2   | 206         | Gietz     | Accounting     |
| 3   | 200         | Whalen    | Administration |
| 4   | 102         | De Haan   | Executive      |
| 5   | 100         | King      | Executive      |
| ... |             |           |                |
| 18  | 124         | Mourgos   | Shipping       |
| 19  | 144         | Vargas    | Shipping       |
| 20  | 178         | Grant     | (null)         |

### Practice 6 (continued)

11. Write a query to display the department names of those departments whose total salary cost is above one-eighth ( $1/8$ ) of the total salary cost of the whole company. Use the WITH clause to write this query. Name the query SUMMARY.

|   | DEPARTMENT_NAME | DEPT_TOTAL |
|---|-----------------|------------|
| 1 | Executive       | 58000      |
| 2 | Sales           | 30100      |

Oracle Internal & Oracle Academy  
Use Only

# 7

## Regular Expression Support

ORACLE<sup>®</sup>

Copyright © 2007, Oracle. All rights reserved.

Oracle Internal & Oracle Academy  
Use Only

## Objectives

After completing this lesson, you should be able to do the following:

- List the benefits of using regular expressions
- Use regular expressions to search for, match, and replace strings

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Objectives

In this lesson, you learn to use the regular expression support feature. Regular expression support is available in both SQL and PL/SQL.



## Lesson Agenda

- Introduction to regular expressions
- Using metacharacters with regular expressions
- Using the regular expressions functions
  - REGEXP\_LIKE
  - REGEXP\_REPLACE
  - REGEXP\_INSTR
  - REGEXP\_SUBSTR
- Accessing subexpressions
- Using the REGEXP\_COUNT function
- Regular expressions and check constraints

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## What Are Regular Expressions?

- You use regular expressions to search for (and manipulate) simple and complex patterns in string data by using standard syntax conventions.
- You use a set of SQL functions and conditions to search for and manipulate strings in SQL and PL/SQL.
- You specify a regular expression by using:
  - Metacharacters, which are operators that specify the search algorithms
  - Literals, which are the characters for which you are searching

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### What Are Regular Expressions?

Oracle Database provides support for regular expressions. The implementation complies with the Portable Operating System for UNIX (POSIX) standard, controlled by the Institute of Electrical and Electronics Engineers (IEEE), for ASCII data-matching semantics and syntax. Oracle's multilingual capabilities extend the matching capabilities of the operators beyond the POSIX standard. Regular expressions are a method of describing both simple and complex patterns for searching and manipulating.

String manipulation and searching contribute to a large percentage of the logic within a Web-based application. Usage ranges from the simple, such as finding the word "San Francisco" in a specified text, to the complex task of extracting all URLs from the text and the more complex task of finding all words whose every second character is a vowel.

When coupled with native SQL, the use of regular expressions allows for very powerful search and manipulation operations on any data stored in an Oracle database. You can use this feature to easily solve problems that would otherwise involve complex programming.

## Benefits of Using Regular Expressions

Regular expressions enable you to implement complex match logic in the database with the following benefits:

- By centralizing match logic in Oracle Database, you avoid intensive string processing of SQL results sets by middle-tier applications.
- Using server-side regular expressions to enforce constraints, you eliminate the need to code data validation logic on the client.
- The built-in SQL and PL/SQL regular expression functions and conditions make string manipulations more powerful and easier than in previous releases of Oracle Database 10g.

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Benefits of Using Regular Expressions

Regular expressions are a powerful text-processing component of programming languages such as PERL and Java. For example, a PERL script can process each HTML file in a directory, read its contents into a scalar variable as a single string, and then use regular expressions to search for URLs in the string. One reason for many developers writing in PERL is that it has a robust pattern-matching functionality. Oracle's support of regular expressions enables developers to implement complex match logic in the database. This technique is useful for the following reasons:

- By centralizing match logic in Oracle Database, you avoid intensive string processing of SQL results sets by middle-tier applications. The SQL regular expression functions move the processing logic closer to the data, thereby providing a more efficient solution.
- Before Oracle Database 10g, developers often coded data validation logic on the client, requiring the same validation logic to be duplicated for multiple clients. Using server-side regular expressions to enforce constraints solves this problem.
- The built-in SQL and PL/SQL regular expression functions and conditions make string manipulations more powerful and less cumbersome than in previous releases of Oracle Database 10g.

## Using the Regular Expressions Functions and Conditions in SQL and PL/SQL

| Function or Condition Name | Description                                                                                                           |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------|
| REGEXP_LIKE                | Similar to the LIKE operator, but performs regular expression matching instead of simple pattern matching (condition) |
| REGEXP_REPLACE             | Searches for a regular expression pattern and replaces it with a replacement string                                   |
| REGEXP_INSTR               | Searches a string for a regular expression pattern and returns the position where the match is found                  |
| REGEXP_SUBSTR              | Searches for a regular expression pattern within a given string and extracts the matched substring                    |
| REGEXP_COUNT               | Returns the number of times a pattern match is found in an input string                                               |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Using the Regular Expressions Functions and Conditions in SQL and PL/SQL

Oracle Database provides a set of SQL functions that you use to search and manipulate strings using regular expressions. You use these functions on a text literal, bind variable, or any column that holds character data such as CHAR, NCHAR, CLOB, NCLOB, NVARCHAR2, and VARCHAR2 (but not LONG). A regular expression must be enclosed or wrapped between single quotation marks. This ensures that the entire expression is interpreted by the SQL function and can improve the readability of your code.

**REGEXP\_LIKE:** This condition searches a character column for a pattern. Use this condition in the WHERE clause of a query to return rows matching the regular expression you specify.

**REGEXP\_REPLACE:** This function searches for a pattern in a character column and replaces each occurrence of that pattern with the pattern you specify.

**REGEXP\_INSTR:** This function searches a string for a given occurrence of a regular expression pattern. You specify which occurrence you want to find and the start position to search from. This function returns an integer indicating the position in the string where the match is found.

**REGEXP\_SUBSTR:** This function returns the actual substring matching the regular expression pattern you specify.

**REGEXP\_COUNT:** This function returns the number of times a pattern match is found in the input string.

## Lesson Agenda

- Introduction to regular expressions
- Using metacharacters with regular expressions
- Using the regular expressions functions
  - REGEXP\_LIKE
  - REGEXP\_REPLACE
  - REGEXP\_INSTR
  - REGEXP\_SUBSTR
- Accessing subexpressions
- Using the REGEXP\_COUNT function

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## What Are Metacharacters?

- Metacharacters are special characters that have a special meaning such as a wildcard, a repeating character, a nonmatching character, or a range of characters.
- You can use several predefined metacharacter symbols in the pattern matching.
- For example, the `^(f|ht)tps?:$` regular expression searches for the following from the beginning of the string:
  - The literals `f` or `ht`
  - The `t` literal
  - The `p` literal, optionally followed by the `s` literal
  - The colon “:” literal at the end of the string

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### What Are Metacharacters?

The regular expression in the slide matches the `http:`, `https:`, `ftp:`, and `ftps:` strings.

**Note:** For a complete list of the regular expressions’ metacharacters, see the *Oracle Database Advanced Application Developer’s Guide 11g Release 1 (11.1)*

## Using Metacharacters with Regular Expressions

| Syntax    | Description                                                                                                                                                  |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| .         | Matches any character in the supported character set, except NULL                                                                                            |
| +         | Matches one or more occurrences                                                                                                                              |
| ?         | Matches zero or one occurrence                                                                                                                               |
| *         | Matches zero or more occurrences of the preceding subexpression                                                                                              |
| {m}       | Matches exactly <i>m</i> occurrences of the preceding expression                                                                                             |
| {m, }     | Matches at least <i>m</i> occurrences of the preceding subexpression                                                                                         |
| {m, n}    | Matches at least <i>m</i> , but not more than <i>n</i> , occurrences of the preceding subexpression                                                          |
| [...]     | Matches any single character in the list within the brackets                                                                                                 |
|           | Matches one of the alternatives                                                                                                                              |
| ( . . . ) | Treats the enclosed expression within the parentheses as a unit. The subexpression can be a string of literals or a complex expression containing operators. |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Using Metacharacters in Regular Expressions Functions

**Any character**, “.”: **a.b** matches the strings **abb**, **acb**, and **adb**, but not **acc**.

**One or more**, “+”: **a+** matches the strings **a**, **aa**, and **aaa**, but does not match **bbb**.

**Zero or one**, “?”: **ab?c** matches the strings **abc** and **ac**, but does not match **abbc**.

**Zero or more**, “\*”: **ab\*c** matches the strings **ac**, **abc**, and **abbc**, but does not match **abb**.

**Exact count**, “{m}”: **a{3}** matches the strings **aaa**, but does not match **aa**.

**At least count**, “{m,}”: **a{3,}** matches the strings **aaa** and **aaaa**, but not **aa**.

**Between count**, “{m,n}”: **a{3,5}** matches the strings **aaa**, **aaaa**, and **aaaaa**, but not **aa**.

**Matching character list**, “[...]”: **[abc]** matches the first character in the strings **a11**, **bill**, and **cold**, but does not match any characters in **doll**.

**Or**, “|”: **a|b** matches character **a** or character **b**.

**Subexpression**, “(...)”: **(abc)?def** matches the optional string **abc**, followed by **def**. The expression matches **abcdefghi** and **def**, but does not match **ghi**. The subexpression can be a string of literals or a complex expression containing operators.

## Using Metacharacters with Regular Expressions

| Syntax                   | Description                                                                                                                                                                          |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>^</code>           | Matches the beginning of a string                                                                                                                                                    |
| <code>\$</code>          | Matches the end of a string                                                                                                                                                          |
| <code>\</code>           | Treats the subsequent metacharacter in the expression as a literal                                                                                                                   |
| <code>\n</code>          | Matches the <i>n</i> th (1–9) preceding subexpression of whatever is grouped within parentheses. The parentheses cause an expression to be remembered; a backreference refers to it. |
| <code>\d</code>          | A digit character                                                                                                                                                                    |
| <code>[ :class:]</code>  | Matches any character belonging to the specified POSIX character class                                                                                                               |
| <code>[ ^:class:]</code> | Matches any single character <i>not</i> in the list within the brackets                                                                                                              |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Using Metacharacters in Regular Expressions Functions (continued)

**Beginning/end of line anchor, “`^`” and “`$`”:** `^def` matches `def` in the string `defghi` but does not match `def` in `abcdef`. `def$` matches `def` in the string `abcdef` but does not match `def` in the string `defghi`.

**Escape character “`\`”:** `\+` searches for a `+`. It matches the plus character in the string `abc+def`, but does not match `Abcdef`.

**Backreference, “`\n`”:** `(abc | def) xy \1` matches the strings `abcxyabc` and `defxydef`, but does not match `abcxydef` or `abcxy`. A backreference enables you to search for a repeated string without knowing the actual string ahead of time. For example, the expression `^(.*) \1$` matches a line consisting of two adjacent instances of the same string.

**Digit character, “`\d`”:** The expression `^\[d{3}\] \d{3}-\d{4}$` matches `[650] 555-1212` but does not match `650-555-1212`.

**Character class, “`[ :class:]`”:** `[[:upper:]]+` searches for one or more consecutive uppercase characters. This matches `DEF` in the string `abcDEFghi` but does not match the string `abcdefghi`.

**Nonmatching character list (or class), “`[ ^... ]`”:** `[^abc]` matches the character `d` in the string `abcdef`, but not the character `a`, `b`, or `c`.



## Lesson Agenda

- Introduction to regular expressions
- Using metacharacters with regular expressions
- **Using the regular expressions functions**
  - REGEXP\_LIKE
  - REGEXP\_REPLACE
  - REGEXP\_INSTR
  - REGEXP\_SUBSTR
- Accessing subexpressions
- Using the REGEXP\_COUNT function

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Regular Expressions Functions and Conditions: Syntax

```
REGEXP_LIKE (source_char, pattern [,match_option]
```

```
REGEXP_INSTR (source_char, pattern [, position
[, occurrence [, return_option
[, match_option [, subexpr]]]])
```

```
REGEXP_SUBSTR (source_char, pattern [, position
[, occurrence [, match_option
[, subexpr]]])
```

```
REGEXP_REPLACE(source_char, pattern [,replacestr
[, position [, occurrence
[, match_option]]])
```

```
REGEXP_COUNT (source_char, pattern [, position
[, occurrence [, match_option]])
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Regular Expressions Functions and Conditions: Syntax

The syntax for the regular expressions functions and conditions is as follows:

- **source\_char**: A character expression that serves as the search value
- **pattern**: A regular expression, a text literal
- **occurrence**: A positive integer indicating which occurrence of pattern in **source\_char** Oracle Server should search for. The default is 1.
- **position**: A positive integer indicating the character of **source\_char** where Oracle Server should begin the search. The default is 1.
- **return\_option**:
  - 0: Returns the position of the first character of the occurrence (default)
  - 1: Returns the position of the character following the occurrence
- **Replacestr**: Character string replacing pattern
- **match\_parameter**:
  - "c": Uses case-sensitive matching (default)
  - "i": Uses non-case-sensitive matching
  - "n": Allows match-any-character operator
  - "m": Treats source string as multiple lines
- **subexpr**: Fragment of pattern enclosed in parentheses. You learn more about subexpressions later in this lesson.

## Performing a Basic Search Using the REGEXP\_LIKE Condition

```
REGEXP_LIKE(source_char, pattern [, match_parameter])
```

```
SELECT first_name, last_name
FROM employees
WHERE REGEXP_LIKE (first_name, '^Ste(v|ph)en$');
```

|   | FIRST_NAME | LAST_NAME |
|---|------------|-----------|
| 1 | Steven     | King      |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Performing a Basic Search Using the REGEXP\_LIKE Condition

REGEXP\_LIKE is similar to the LIKE condition, except that REGEXP\_LIKE performs regular-expression matching instead of the simple pattern matching performed by LIKE. This condition evaluates strings using characters as defined by the input character set.

#### Example of REGEXP\_LIKE

In this query, against the EMPLOYEES table, all employees with first names containing either Steven or Stephen are displayed. In the expression used,

'^Ste(v|ph)en\$'

- ^ indicates the beginning of the expression
- \$ indicates the end of the expression
- | indicates either/or

## Replacing Patterns Using the REGEXP\_REPLACE Function

```
REGEXP_REPLACE(source_char, pattern [,replacestr
[, position [, occurrence [, match_option]]]])
```

```
SELECT REGEXP_REPLACE(phone_number, '\.', '-') AS phone
FROM employees;
```

Original

|   | LAST_NAME | PHONE_NUMBER |
|---|-----------|--------------|
| 1 | King      | 515.123.4567 |
| 2 | Kochhar   | 515.123.4568 |
| 3 | De Haan   | 515.123.4569 |
| 4 | Hunold    | 590.423.4567 |

Partial results

|   | LAST_NAME | PHONE        |
|---|-----------|--------------|
| 1 | King      | 515-123-4567 |
| 2 | Kochhar   | 515-123-4568 |
| 3 | De Haan   | 515-123-4569 |
| 4 | Hunold    | 590-423-4567 |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Replacing Patterns Using the REGEXP\_REPLACE Function

Using the REGEXP\_REPLACE function, you reformat the phone number to replace the period (.) delimiter with a dash (-) delimiter. Here is an explanation of each of the elements used in the regular expression example:

- phone\_number is the source column.
- '\.' is the search pattern.
  - Use single quotation marks ( ' ') to search for the literal character period (.).
  - Use a backslash (\) to search for a character that is normally treated as a metacharacter.
- '-' is the replace string.

## Finding Patterns Using the REGEXP\_INSTR Function

```
REGEXP_INSTR (source_char, pattern [, position [,
occurrence [, return_option [, match_option]]])
```

```
SELECT street_address,
REGEXP_INSTR(street_address,'[[:alpha:]]') AS
 First_Alpha_Position
FROM locations;
```

| STREET_ADDRESS                             | FIRST_ALPHA_POSITION |
|--------------------------------------------|----------------------|
| 1 2014 Jabberwocky Rd                      | 6                    |
| 2 2011 Interiors Blvd                      | 6                    |
| 3 2004 Charade Rd                          | 6                    |
| 4 460 Bloor St. W.                         | 5                    |
| 5 Magdalen Centre, The Oxford Science Park | 1                    |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Finding Patterns Using the REGEXP\_INSTR Function

In this example, the REGEXP\_INSTR function is used to search the street address to find the location of the first alphabetic character, regardless of whether it is in uppercase or lowercase. Note that `[<class>:]` implies a character class and matches any character from within that class; `[[:alpha:]]` matches with any alphabetic character. The partial results are displayed.

In the expression used in the query `'[[:alpha:]]'`:

- `[` starts the expression.
- `[[:alpha:]]` indicates alphabetic character class.
- `]` ends the expression.

**Note:** The POSIX character class operator enables you to search for an expression within a character list that is a member of a specific POSIX character class. You can use this operator to search for specific formatting, such as uppercase characters, or you can search for special characters such as digits or punctuation characters. The full set of POSIX character classes is supported. Use the syntax `[[:class:]]` where `class` is the name of the POSIX character class to search for. The following regular expression searches for one or more consecutive uppercase characters: `[[:upper:]]+`.

## Extracting Substrings Using the REGEXP\_SUBSTR Function

```
REGEXP_SUBSTR (source_char, pattern [, position
[, occurrence [, match_option]])
```

```
SELECT REGEXP_SUBSTR(street_address , ' [^]+ ') AS Road
FROM locations;
```

|   | ROAD        |
|---|-------------|
| 1 | Jabberwocky |
| 2 | Interiors   |
| 3 | Charade     |
| 4 | Bloor       |
| 5 | Centre,     |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Extracting Substrings Using the REGEXP\_SUBSTR Function

In this example, the road names are extracted from the LOCATIONS table. To do this, the contents in the STREET\_ADDRESS column that are after the first space are returned using the REGEXP\_SUBSTR function. In the expression used in the query ' [^ ]+ ':

- [ starts the expression.
- ^ indicates NOT.
- indicates space.
- ] ends the expression.
- + indicates 1 or more.
- indicates space.

## Lesson Agenda

- Introduction to regular expressions
- Using metacharacters with regular expressions
- Using the regular expressions functions
  - REGEXP\_LIKE
  - REGEXP\_REPLACE
  - REGEXP\_INSTR
  - REGEXP\_SUBSTR
- Accessing subexpressions
- Using the REGEXP\_COUNT function

ORACLE

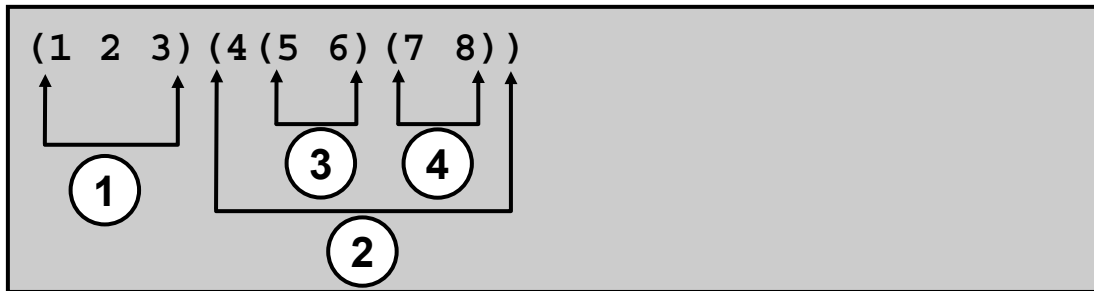
Copyright © 2007, Oracle. All rights reserved.

## Subexpressions

Examine this expression:

```
(1 2 3) (4 (5 6) (7 8))
```

The subexpressions are:



ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Subexpressions

Oracle Database 11g provides regular expression support parameter to access a subexpression. In the slide example, a string of digits is shown. The parentheses identify the subexpressions within the string of digits. Reading from left to right, and from inner parentheses to the outer parentheses, the subexpressions in the string of digits are:

1. 123
2. 45678
3. 56
4. 78

You can search for any of those subexpressions with the `REGEXP_INSTR` and `REGEXP_SUBSTR` functions.



## Using Subexpressions with Regular Expression Support

```
SELECT
 REGEXP_INSTR
① ('0123456789', -- source char or search value
② '(123)(4(56)(78))', -- regular expression patterns
③ 1, -- position to start searching
④ 1, -- occurrence
⑤ 0, -- return option
⑥ 'i', -- match option (case insensitive)
⑦ 1) -- sub-expression on which to search
 "Position"
FROM dual;
```

| Position |
|----------|
| 1        |
| 2        |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Using Subexpressions with Regular Expression Support

REGEXP\_INSTR and REGEXP\_SUBSTR have an optional SUBEXPR parameter that lets you target a particular substring of the regular expression being evaluated.

In the example shown in the slide, you may want to search for the first subexpression pattern in your list of subexpressions. The example shown identifies several parameters for the REGEXP\_INSTR function.

1. The string you are searching is identified.
2. The subexpressions are identified. The first subexpression is 123. The second subexpression is 45678, the third is 56, and the fourth is 78.
3. The third parameter identifies from which position to start searching.
4. The fourth parameter identifies the occurrence of the pattern you want to find. 1 means find the first occurrence.
5. The fifth parameter is the return option. This is the position of the first character of the occurrence. (If you specify 1, then the position of the character following the occurrence is returned.)
6. The sixth parameter identifies whether your search should be case-sensitive or not.
7. The last parameter is the parameter added in Oracle Database 11g. This parameter specifies which subexpression you want to find. In the example shown, you are searching for the first subexpression, which is 123.

## Why Access the *n*th Subexpression?

- A more realistic use: DNA sequencing
- You may need to find a specific subpattern that identifies a protein needed for immunity in mouse DNA.

```
SELECT
 REGEXP_INSTR('ccacctttccctccactcctcacgttctcacctgtaaagcgctccctc
cctcatcccatgcccccttaccctgcagggtagtaggctagaaccagagagctccaagc
tccatctgtggagaggtgccatccttgggctgcagagagaggagaatttgcccaaagctgcc
tgcagagcttcaccacccttagtctcacaaagccttgagttcatagcatttcttgagtttca
ccctgccagcaggacactgcagcaccctaaaggcttcccaggagtagggttgcctcaagag
gctcttgggtctgatggccacatcctggaattgttttcaagttgatggtcacagccctgaggc
atgtagggcggtgggatgcgtctgtctgtctctcctctcctgaaccctgaaccctctggc
taccagagcacttagagccag',
 '(gtc(tcac)(aaag))',
 1, 1, 0, 'i',
 1) "Position"
FROM dual;
```

| 1 | Position |
|---|----------|
| 1 | 195      |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Why Access the *n*th Subexpression?

In life sciences, you may need to extract the offsets of subexpression matches from a DNA sequence for further processing. For example, you may need to find a specific protein sequence, such as the begin offset for the DNA sequence preceded by `gtc` and followed by `tcac` followed by `aaag`. To accomplish this goal, you can use the `REGEXP_INSTR` function, which returns the position where a match is found.

In the slide example, the position of the first subexpression (`gtc`) is returned. `gtc` appears starting in position 195 of the DNA string.

If you modify the slide example to search for the second subexpression (`tcac`), the query results in the following output. `tcac` appears starting in position 198 of the DNA string.

| 1 | Position |
|---|----------|
| 1 | 198      |

If you modify the slide example to search for the third subexpression (`aaag`), the query results in the following output. `aaag` appears starting in position 202 of the DNA string.

| 1 | Position |
|---|----------|
| 1 | 202      |

## REGEXP\_SUBSTR: Example

```
SELECT
 REGEXP_SUBSTR
 ① ('acgctgcactgca', -- source char or search value
 ② 'acg(.*)gca', -- regular expression pattern
 ③ 1, -- position to start searching
 ④ 1, -- occurrence
 ⑤ 'i', -- match option (case insensitive)
 ⑥ 1) -- sub-expression
 "Value"
FROM dual;
```

| Value     |
|-----------|
| 1 ctgcact |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### REGEXP\_SUBSTR Example

In the example shown in the slide:

1. acgctgcactgca is the source to be searched.
2. acg ( . \* ) gca is the pattern to be searched. Find acg followed by gca with potential characters between the acg and the gca.
3. Start searching at the first character of the source.
4. Search for the first occurrence of the pattern.
5. Use non-case-sensitive matching on the source.
6. Use a nonnegative integer value that identifies the *n*th subexpression to be targeted. This is the subexpression parameter. In this example, 1 indicates the first subexpression. You can use a value from 0–9. A zero means that no subexpression is targeted. The default value for this parameter is 0.

## Lesson Agenda

- Introduction to regular expressions
- Using metacharacters with regular expressions
- Using the regular expressions functions
  - REGEXP\_LIKE
  - REGEXP\_REPLACE
  - REGEXP\_INSTR
  - REGEXP\_SUBSTR
- Accessing subexpressions
- Using the REGEXP\_COUNT function

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Using the REGEXP\_COUNT Function

```
REGEXP_COUNT (source_char, pattern [, position
[, occurrence [, match_option]]])
```

```
SELECT REGEXP_COUNT (
 'ccacctttccctccactcctcactgttctcacctgttaaagcgccctccctcatccccatgcccccttaccctgcag
 ggtagagtaggctagaaccagagagctccaagctccatctgtggagaggtgccatccttgggctgcagagagaggag
 aatttgcccaaagctgcctgcagagcttcaccacccttagtctcacaaagccttgagttcatagcatttcttgagtt
 ttcacctgcccagcaggacactgcagcacccaaagggctcccaggagtaggggtgcctcaagaggctcttgggtc
 tgatggccacatcctggaattgtttcaagttgatggtcacagccctgaggcatgtagggcggtgggatgcgctctg
 ctctgctctcctcctgaaccctgaaccctctggctacccagagcacttagagccag',
 'gtc') AS Count

FROM dual;
```

|   | A<br>Z | COUNT |
|---|--------|-------|
| 1 |        | 4     |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Using the REGEXP\_COUNT Function

The REGEXP\_COUNT function evaluates strings using characters as defined by the input character set. It returns an integer indicating the number of occurrences of pattern. If no match is found, then the function returns 0.

In the slide example, the number of occurrences for a DNA substring is determined using the REGEXP\_COUNT function.

The following example shows that the number of times the pattern 123 occurs in the string 123123123123 is 3 times. The search starts from the second position of the string.

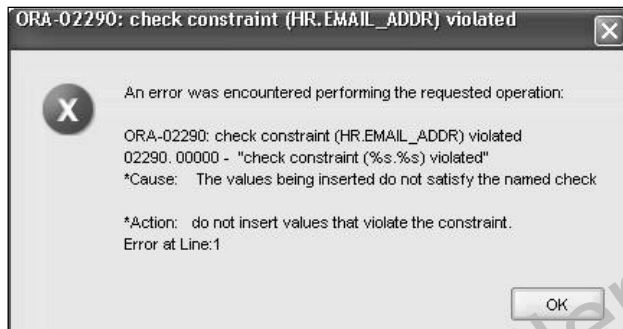
```
SELECT REGEXP_COUNT
 ('123123123123', -- source char or search value
 '123', -- regular expression pattern
 2, -- position where the search should start
 'i') -- match option (case insensitive)
 AS Count
FROM dual;
```

|   | A<br>Z | COUNT |
|---|--------|-------|
| 1 |        | 3     |

## Regular Expressions and Check Constraints: Examples

```
ALTER TABLE emp8
ADD CONSTRAINT email_addr
CHECK (REGEXP_LIKE(email, '@')) NOVALIDATE;
```

```
INSERT INTO emp8 VALUES
(500, 'Christian', 'Patel', 'ChrisP2creme.com',
1234567890, '12-Jan-2004', 'HR_REP', 2000, null, 102, 40);
```



ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Regular Expressions and Check Constraints: Examples

Regular expressions can also be used in CHECK constraints. In this example, a CHECK constraint is added on the EMAIL column of the EMPLOYEES table. This ensures that only strings containing an “@” symbol are accepted. The constraint is tested. The CHECK constraint is violated because the email address does not contain the required symbol. The NOVALIDATE clause ensures that existing data is not checked.

For the slide example, the emp8 table is created by using the following code:

```
CREATE TABLE emp8 AS SELECT * FROM employees;
```

**Note:** The example in the slide is executed by using the “Execute Statement” option in SQL Developer. The output format differs if you use the “Run Script” option.

## Summary

In this lesson, you should have learned how to use regular expressions to search for, match, and replace strings.

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Summary

In this lesson, you have learned to use the regular expression support features. Regular expression support is available in both SQL and PL/SQL.

## Practice 7: Overview

This practice covers using regular expressions functions to do the following:

- Searching for, replacing, and manipulating data
- Creating a new `CONTACTS` table and adding a `CHECK` constraint to the `p_number` column to ensure that phone numbers are entered into the database in a specific standard format
- Testing the adding of some phone numbers into the `p_number` column by using various formats

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Practice 7: Overview

In this practice, you use regular expressions functions to search for, replace, and manipulate data. You also create a new `CONTACTS` table and add a `CHECK` constraint to the `p_number` column to ensure that phone numbers are entered into the database in a specific standard format.



## Practice 7

In this practice, you use regular expressions functions to search for, replace, and manipulate data. You also create a new contacts table and add a check constraint to the `p_number` column to enforce that phone numbers are entered into the database in a specific standard format.

1. Write a query to search the `EMPLOYEES` table for all employees whose first names start with “Ki” or “Ko.”

|   | FIRST_NAME | LAST_NAME |
|---|------------|-----------|
| 1 | Steven     | King      |
| 2 | Neena      | Kochhar   |

2. Create a query that removes the spaces in the `STREET_ADDRESS` column of the `LOCATIONS` table in the display. Use “Street Address” as the column heading.

|   | Street Address                      |
|---|-------------------------------------|
| 1 | 2014JabberwockyRd                   |
| 2 | 2011InteriorsBlvd                   |
| 3 | 2004CharadeRd                       |
| 4 | 460BloorSt.W.                       |
| 5 | MagdalenCentre,TheOxfordSciencePark |

3. Create a query that displays “St” replaced by “Street” in the `STREET_ADDRESS` column of the `LOCATIONS` table. Be careful that you do not affect any rows that already have “Street” in them. Display only those rows that are affected.

|   | REGEXP_REPLACE(STREET_ADDRESS,'ST\$','STREET') |
|---|------------------------------------------------|
| 1 | 460 Bloor St. W.                               |

4. Create a contacts table and add a check constraint to the `p_number` column to enforce the following format mask to ensure that phone numbers are entered into the database in the following standard format: (XXX) XXX-XXXX. The table should have the following columns:

- `l_name` `varchar2(30)`
- `p_number` `varchar2(30)`

### Practice 7 (continued)

5. Run the SQL script `lab_07_05.sql` to insert the following seven phone numbers into the contacts table. Which numbers are added?

| <b>l_name</b> Column Value | <b>p_number</b> Column Value |
|----------------------------|------------------------------|
| NULL                       | '(650) 555-5555'             |
| NULL                       | '(215) 555-3427'             |
| NULL                       | '650 555-5555'               |
| NULL                       | '650 555 5555'               |
| NULL                       | '650-555-5555'               |
| NULL                       | '(650) 555-5555'             |
| NULL                       | ' (650) 555-5555'            |

6. Write a query to find the number of occurrences of the DNA pattern `ctc` in the string `gtctcgtctcgttctgtctgtcgttctg`. Ignore case sensitivity.

Oracle Internal & Oracle Academy  
Use Only

---

## **Appendix A: Practice Solutions**

---

Oracle Internal & Oracle Academy  
Use Only

Note the following points before you begin with the practices:

- Enter your SQL statements in a SQL Worksheet. To save a script in SQL Developer, from the File menu, select **Save As** or right-click in the SQL Worksheet and select **Save file** to save your SQL statement. To modify an existing script, use **File > Open** to open the script file, make changes and then make sure you use **Save As** to save it with a different file name.
- To run the query, click the **Execute Statement** icon (or press [F9]) in the SQL Worksheet. For DML and DDL statements, use the **Run Script** icon (or press [F5]).
- After you have executed a saved script, make sure that you do not enter your next query in the same worksheet. Open a new worksheet.

Oracle Internal & Oracle Academy  
Use Only

## Practice I: Solutions

In this practice, you review the available SQL Developer resources. You also learn about your user account that you will use in this course. You then start SQL Developer, create a new database connection, and browse your HR tables. You also set some SQL Developer preferences, execute SQL statements, and execute an anonymous PL/SQL block using SQL Worksheet. Finally, you access and bookmark the Oracle Database 11g documentation and other useful Web sites that you can use in this course.

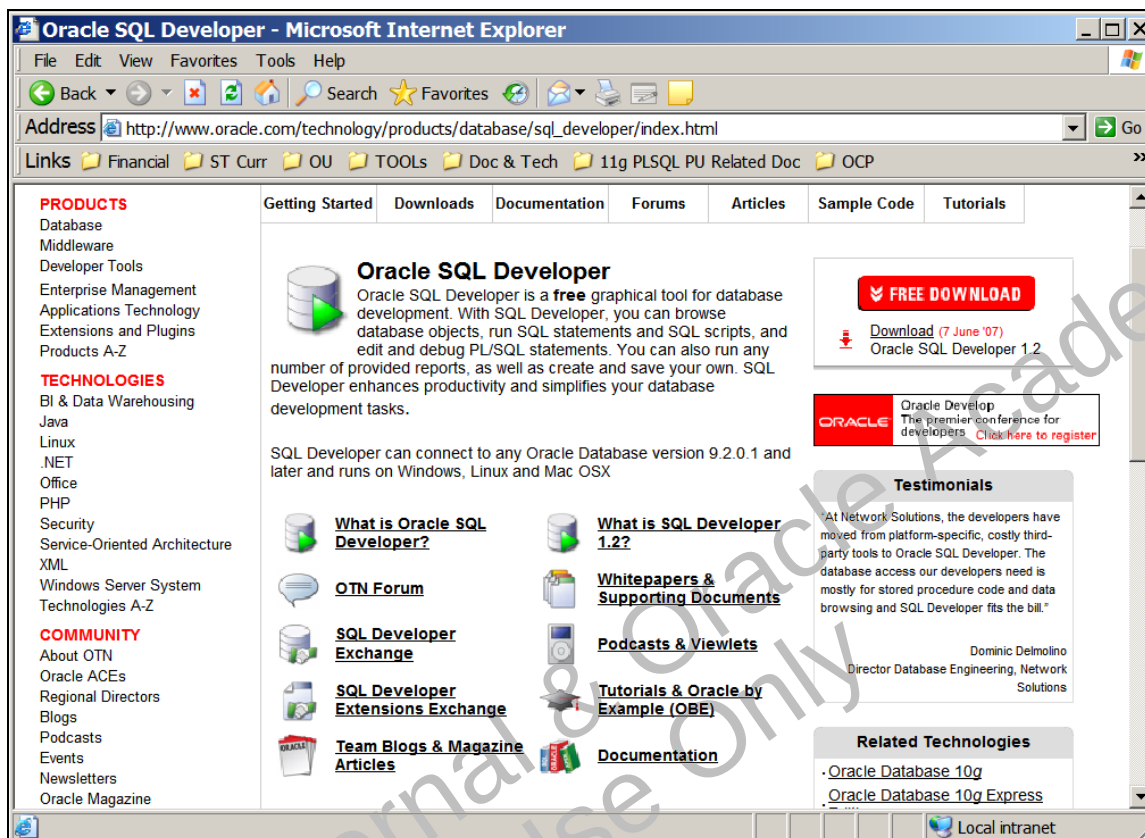
### Accessing SQL Developer Resources

1) Access the SQL Developer home page.

a) Access the online SQL Developer home page available online at:

[http://www.oracle.com/technology/products/database/sql\\_developer/index.html](http://www.oracle.com/technology/products/database/sql_developer/index.html)

The SQL Developer home page is displayed as follows:



b) Bookmark the page for easier future access.

## Practice I: Solutions (continued)

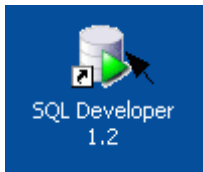
- 2) Access the SQL Developer tutorial available online at:  
<http://st-curriculum.oracle.com/tutorial/SQLDeveloper/index.htm>

Then, review the following sections and associated demos:

- a) What to Do First
- b) Working with Database Objects
- c) Accessing Data

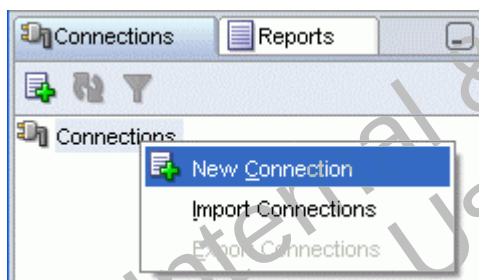
### Starting SQL Developer, Creating and Testing a New SQL Developer Database Connection, and Connecting to Your **oraxx** Account

- 3) Start up SQL Developer using the Desktop icon.



- 4) Create a database connection using the following information:

- a. Connection Name: myconnection.
- b. Username: oraxx (Ask your instructor to assign you one ora account out of the ora21-ora40 range of accounts.)
- c. Password: oraxx
- d. Hostname: Enter the host name for your PC.
- e. Port: 1521
- f. SID: ORCL (or the value provided to you by the instructor)



## Practice I: Solutions (continued)

Connect... Connect... Connection Name myconnection

Username ora21

Password \*\*\*\*\*

☐ Save Password

Oracle Access MySQL SQLServer

Role default

Connection Type ☒ Basic ☐ TNS ☐ Advanced

Hostname vx0106.us.oracle.com

Port 1521

☒ SID orcl

☐ Service name

Status :

Help Save Clear Test Conn... Cancel

5) Test the new connection. If the Status is Success, connect to the database using this new connection.

a) Click the **Test** button in the New/Select Database Connection window.

Status :

Help Save Clear Test Conn... Cancel

b) If the status is Success, click the **Connect** button.

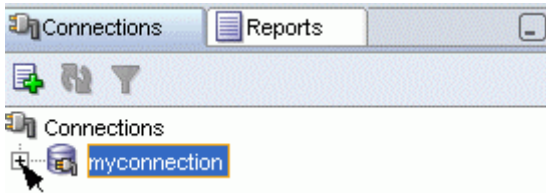
Status : Success

Help Save Clear Test Conn... Cancel

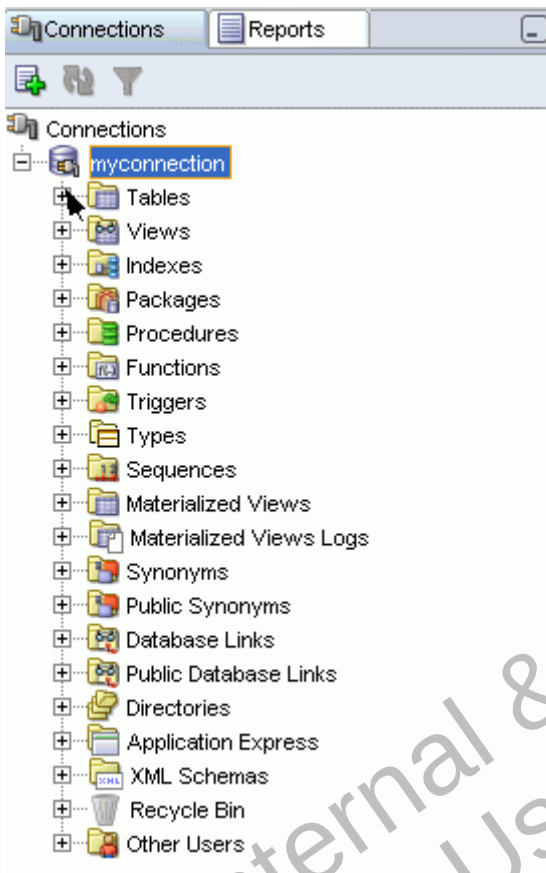
## Practice I: Solutions (continued)

### Browsing the Tables

- 6) Browse the structure of the EMPLOYEES table and display its data.
  - a) Expand the **myconnection** connection by clicking the plus sign next to it.

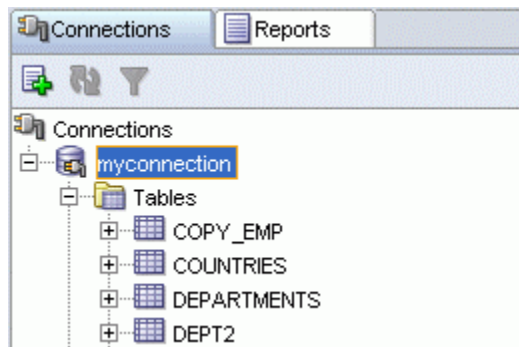


- b) Expand the **Tables** icon by clicking the plus sign next to it.





## Practice I: Solutions (continued)



c) Display the structure of the EMPLOYEES table.

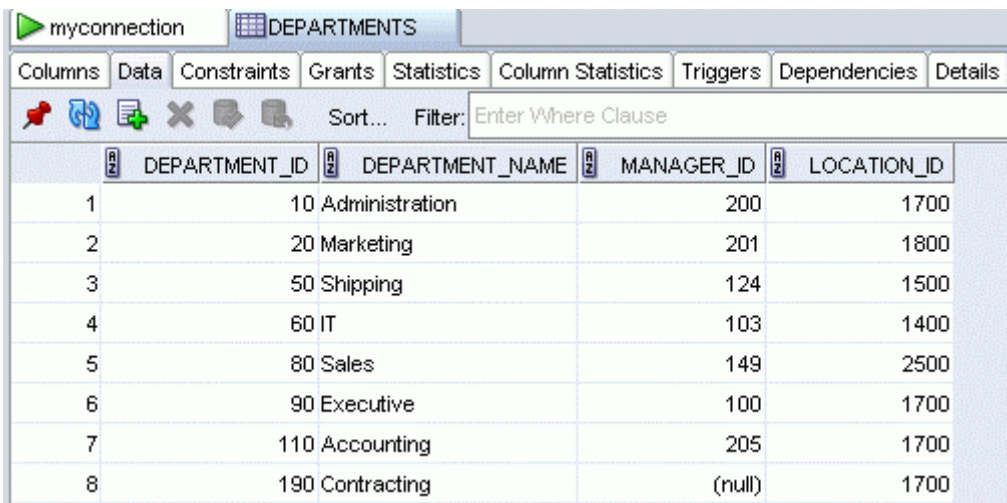
Click the **EMPLOYEES** table. The Columns tab displays the columns in the EMPLOYEES table as follows:

| Column Name    | Data Type         | Nullable | Data Default | COLUMN ID | Primary Key          | COMMENTS            |
|----------------|-------------------|----------|--------------|-----------|----------------------|---------------------|
| EMPLOYEE_ID    | NUMBER(6,0)       | No       | (null)       | 1         | 1 Primary key of ... |                     |
| FIRST_NAME     | VARCHAR2(20 BYTE) | Yes      | (null)       | 2         | (null)               | First name of t...  |
| LAST_NAME      | VARCHAR2(25 BYTE) | No       | (null)       | 3         | (null)               | Last name of t...   |
| EMAIL          | VARCHAR2(25 BYTE) | No       | (null)       | 4         | (null)               | Email id of the ... |
| PHONE_NUMBER   | VARCHAR2(20 BYTE) | Yes      | (null)       | 5         | (null)               | Phone number ...    |
| HIRE_DATE      | DATE              | No       | (null)       | 6         | (null)               | Date when the...    |
| JOB_ID         | VARCHAR2(10 BYTE) | No       | (null)       | 7         | (null)               | Current job of t... |
| SALARY         | NUMBER(8,2)       | Yes      | (null)       | 8         | (null)               | Monthly salary...   |
| COMMISSION_PCT | NUMBER(2,2)       | Yes      | (null)       | 9         | (null)               | Commission pe...    |
| MANAGER_ID     | NUMBER(6,0)       | Yes      | (null)       | 10        | (null)               | Manager id of t...  |
| DEPARTMENT_ID  | NUMBER(4,0)       | Yes      | (null)       | 11        | (null)               | Department id ...   |

7) View the data of the DEPARTMENTS table.

In the Connections navigator, click the **DEPARTMENTS** table. Then click the **Data** tab.

## Practice I: Solutions (continued)



The screenshot shows the Oracle SQL Developer interface with the 'DEPARTMENTS' table selected. The table has four columns: DEPARTMENT\_ID, DEPARTMENT\_NAME, MANAGER\_ID, and LOCATION\_ID. The data is as follows:

|   | DEPARTMENT_ID | DEPARTMENT_NAME | MANAGER_ID | LOCATION_ID |
|---|---------------|-----------------|------------|-------------|
| 1 | 10            | Administration  | 200        | 1700        |
| 2 | 20            | Marketing       | 201        | 1800        |
| 3 | 50            | Shipping        | 124        | 1500        |
| 4 | 60            | IT              | 103        | 1400        |
| 5 | 80            | Sales           | 149        | 2500        |
| 6 | 90            | Executive       | 100        | 1700        |
| 7 | 110           | Accounting      | 205        | 1700        |
| 8 | 190           | Contracting     | (null)     | 1700        |

- 8) Execute some basic SELECT statements to query the data in the EMPLOYEES table in the SQL Worksheet area. Use both the Execute Statement (or press [F9]) and the Run Script icons (or press [F5]) to execute the SELECT statements. Review the results of both methods of executing the SELECT statements on the appropriate tabbed pages.

- a) Write a query to select the last name and salary for any employee whose salary is less than or equal to \$3,000.

```
SELECT last_name, salary
FROM employees
WHERE salary <= 3000;
```

- b) Write a query to display last name, job ID, and commission for all employees who are not entitled to receive a commission.

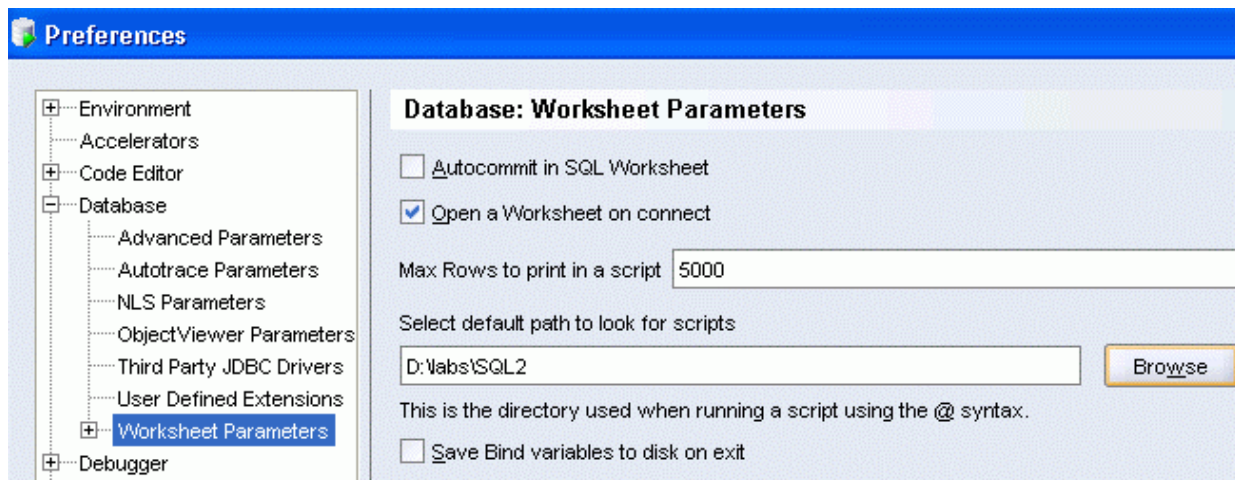
```
SELECT last_name, job_id, commission_pct
FROM employees
WHERE commission_pct IS NULL;
```

- 9) Set your script pathing preference to D:\labs\SQL2.

- a) Select **Tools > Preferences > Database > Worksheet Parameters**.

- b) Enter the value in the **Select default path to look for scripts** field. Then, click **OK**.

## Practice I: Solutions (continued)

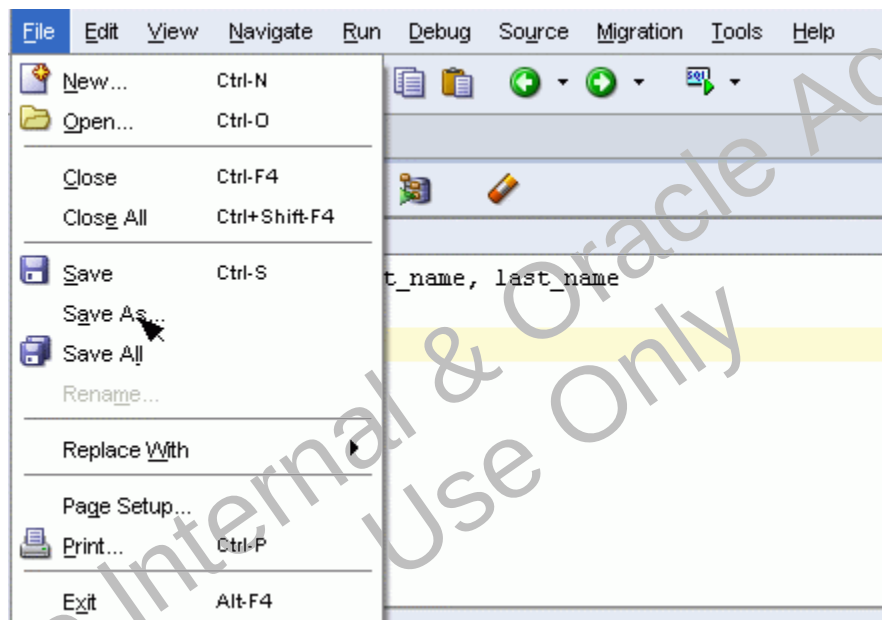


10) Enter the following in the Enter SQL Statement box.

```
SELECT employee_id, first_name, last_name
FROM employees;
```

11) Save the SQL statement to a script file using the **File > Save As** menu item.

a) Select **File > Save As**

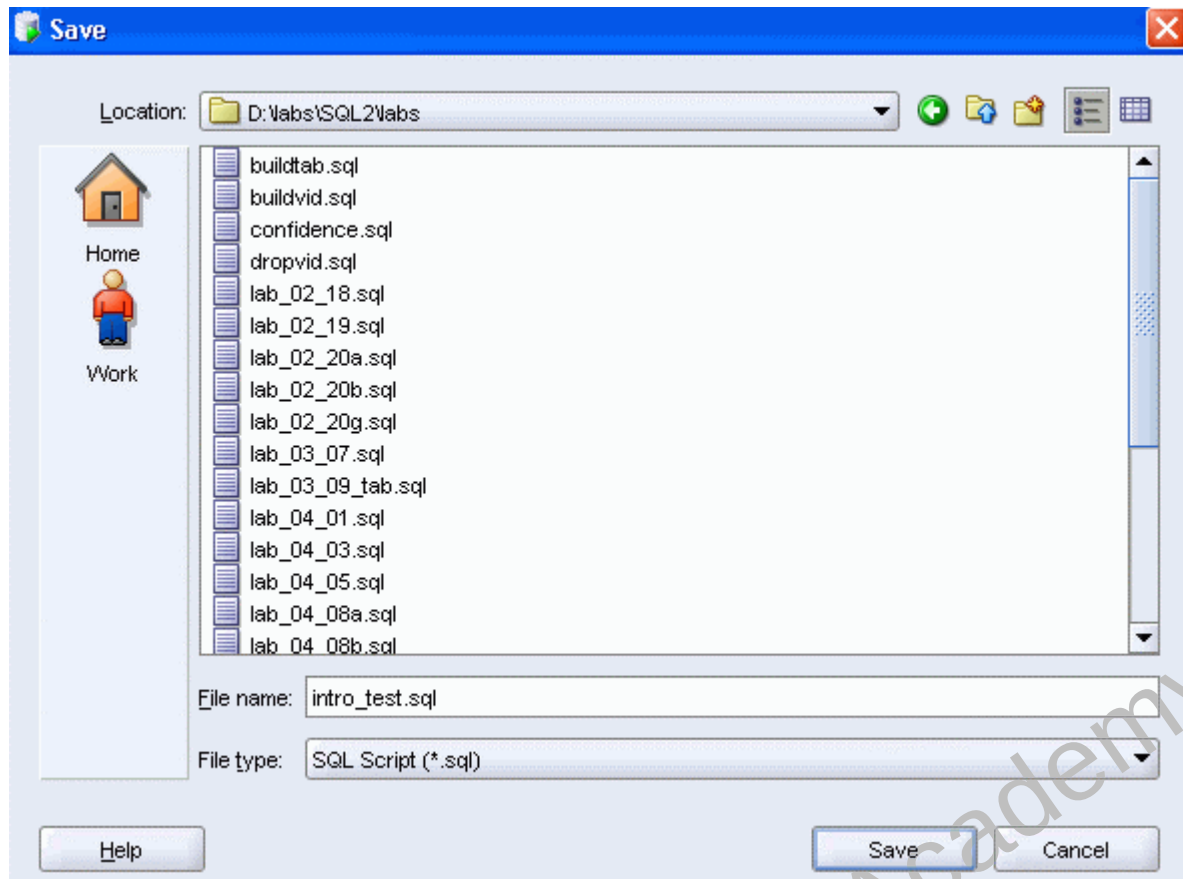


b) Name the file **intro\_test.sql**

Type **intro\_test.sql** in the **File\_name** text box.

## Practice I: Solutions (continued)

c) Place the file under **D:\labs\SQL2\labs** folder.



Then, Click **Save**.

12) Open and run `confidence.sql` from your `D:\Labs\labs` folder, and observe the output.

Right-click in the Enter SQL Statement box to open the file. Then, press [F5].

The following is the expected result.

## Practice I: Solutions (continued)

```
COUNT(*)

9

1 rows selected

COUNT(*)

20

1 rows selected

COUNT(*)

4

1 rows selected

COUNT(*)

4

1 rows selected
```

```
COUNT(*)

5

1 rows selected

COUNT(*)

8

1 rows selected

COUNT(*)

12

1 rows selected

COUNT(*)

10

1 rows selected
```

## Practice I: Solutions (continued)

### Quiz:

1. What condition do you use to display rows based on a range of values?

**BETWEEN**

2. The character pattern-matching operation is also referred to as:

**Wildcard search**

3. Group functions return one result per row. True/False.

**False. Group functions give one result per group of rows. Single-row functions operate on single rows only and return one result per row.**

4. Which function converts mixed case or lowercase character strings to uppercase?

**UPPER**

5. Which among the following are group functions?

a . MAX

b . TRUNC

c . STDDEV

d . MOD

e . CONCAT

f . SUM

**MAX, STDDEV, SUM**

6. What is the default sorting order of rows?

**Ascending**

7. You can sort by a column that is not in the SELECT list. True/False.

**True.**

8. You cannot specify a column position as the sort expression. True/False.

**False. You can specify an expression, an alias, or a column position as the sort condition.**

## Practice I: Solutions (continued)

9. Which statement do you use to modify existing rows in a table?

**UPDATE**

10. What happens if you do not specify the WHERE clause in a DELETE statement?

**If you omit the WHERE clause, all rows in the table are deleted.**

11. Consider a SELECT statement that consists of three query blocks: the outer query and two inner queries. If both the inner queries return single values, what do you call such a SQL statement?

**Single-row subquery**

12. You can use subqueries only in the WHERE clause but not in the HAVING clause. True/False.

**False. You can use subqueries not only in the WHERE clause but also in the HAVING clause.**

13. Observe the following SELECT statement. What happens when there is no employee named Kiran?

```
SELECT last_name, job_id
FROM employees
WHERE job_id =
 (SELECT job_id
 FROM employees
 WHERE last_name = 'Kiran');
```

**The subquery returns no rows. The outer query takes the results of the subquery (null) and uses these results in its WHERE clause. The outer query finds no employee with a job ID equal to null, and so returns no rows. If a job existed with a value of null, the row is not returned because comparison of two null values yields a null; therefore, the WHERE condition is not true.**

14. The subqueries are processed first by the Oracle server, after which the WHERE or HAVING clause uses the results. True/False.

**True.**

## Practice 1: Solutions

To complete question 8 and the subsequent ones, you need to connect to the database using SQL Developer.

1. What privilege should a user be given to log on to the Oracle server? Is this a system or an object privilege?  
**The CREATE SESSION system privilege**
2. What privilege should a user be given to create tables?  
**The CREATE TABLE privilege**
3. If you create a table, who can pass along privileges to other users on your table?  
**You can, or anyone you have given those privileges to, by using WITH GRANT OPTION**
4. You are the DBA. You are creating many users who require the same system privileges. What should you use to make your job easier?  
**Create a role containing the system privileges and grant the role to the users.**
5. What command do you use to change your password?  
**The ALTER USER statement**
6. User1 is the owner of the EMP table and grants DELETE privileges to User2 by using the WITH GRANT OPTION clause. User2 then grants DELETE privileges on EMP to User3. User1 now finds that User3 has the privilege and revokes it from User2. Which user can now delete data from the EMP table.  
**User1 only**
7. You want to grant SCOTT the privilege to update data in the DEPARTMENTS table. You also want to enable SCOTT to grant this privilege to other users. What command do you use?  
**GRANT UPDATE ON departments TO scott WITH GRANT OPTION;**



## Practice 1: Solutions (continued)

8. Grant another user query privilege on your table. Then, verify whether that user can utilize the privilege.

**Note:** For this exercise, team up with another group. For example, if you are user ora21, team up with another user ora22.

- a) Grant another user privilege to view records in your REGIONS table. Include an option for this user to further grant this privilege to other users.

*Team 1 executes this statement:*

```
GRANT select
ON regions
TO <team2_oraxx> WITH GRANT OPTION;
```

- b) Have the user query your REGIONS table.

*Team 2 executes this statement:*

```
SELECT * FROM <team1_oraxx>.regions;
```

- c) Have the user pass on the query privilege to a third user (for example ora23).

*Team 2 executes this statement.*

```
GRANT select
ON <team1_oraxx>.regions
TO <team3_oraxx>;
```

- d) Take back the privilege from the user who performs step b.

*Team 1 executes this statement.*

```
REVOKE select
ON regions
FROM <team2_oraxx>;
```

9. Grant another user query and data manipulation privileges on your COUNTRIES table. Make sure the user cannot pass on these privileges to other users.

*Team 1 executes this statement*

```
GRANT select, update, insert
ON COUNTRIES
TO <team2_oraxx>;
```

## Practice 1: Solutions (continued)

10. Take back the privileges on the COUNTRIES table granted to another user.

*Team 1 executes this statement*

```
REVOKE select, update, insert ON COUNTRIES FROM
<team2_oraxx>;
```

**Note:** For the exercises 11 through 17, team up with another group.

11. Grant another user access to your DEPARTMENTS table. Have the user grant you query access to his or her DEPARTMENTS table.

*Team 2 executes the GRANT statement.*

```
GRANT select
ON departments
TO <team1_oraxx>;
```

*Team 1 executes the GRANT statement.*

```
GRANT select
ON departments
TO <team2_oraxx>;
```

Here, <team1\_oraxx> is the username of Team 1 and <team2\_oraxx> is the username of Team 2.

12. Query all the rows in your DEPARTMENTS table.

```
SELECT *
FROM departments;
```

13. Add a new row to your DEPARTMENTS table. Team 1 should add Education as department number 500. Team 2 should add Human Resources as department number 510. Query the other team's table.

*Team 1 executes this INSERT statement.*

```
INSERT INTO departments(department_id, department_name)
VALUES (500, 'Education');
COMMIT;
```

*Team 2 executes this INSERT statement.*

```
INSERT INTO departments(department_id, department_name)
VALUES (510, 'Human Resources');
COMMIT;
```

## Practice 1: Solutions (continued)

14. Create a synonym for the other team's DEPARTMENTS table.

*Team 1 creates a synonym named team2.*

```
CREATE SYNONYM team2
FOR <team2_oraxx>.DEPARTMENTS;
```

*Team 2 creates a synonym named team1.*

```
CREATE SYNONYM team1
FOR <team1_oraxx>. DEPARTMENTS;
```

15. Query all the rows in the other team's DEPARTMENTS table by using your synonym.

*Team 1 executes this SELECT statement.*

```
SELECT *
FROM team2;
```

*Team 2 executes this SELECT statement.*

```
SELECT *
FROM team1;
```

16. Revoke the SELECT privilege from the other team.

*Team 1 revokes the privilege.*

```
REVOKE select
ON departments
FROM <team2_oraxx>;
```

*Team 2 revokes the privilege.*

```
REVOKE select
ON departments
FROM <team1_oraxx>;
```

## Practice 1: Solutions (continued)

17. Remove the row you inserted into the DEPARTMENTS table in step 8 and save the changes.

*Team 1 executes this DELETE statement.*

```
DELETE FROM departments
WHERE department_id = 500;
COMMIT;
```

*Team 2 executes this DELETE statement.*

```
DELETE FROM departments
WHERE department_id = 510;
COMMIT;
```

Oracle Internal & Oracle Academy  
Use Only

## Practice 2: Solutions

1. Create the DEPT2 table based on the following table instance chart. Type the syntax in the SQL Worksheet. Then, execute the statement to create the table. Confirm that the table is created.

| Column Name  | ID     | NAME     |
|--------------|--------|----------|
| Key Type     |        |          |
| Nulls/Unique |        |          |
| FK Table     |        |          |
| FK Column    |        |          |
| Data type    | NUMBER | VARCHAR2 |
| Length       | 7      | 25       |

```
CREATE TABLE dept2
(id NUMBER(7),
 name VARCHAR2(25));

DESCRIBE dept2
```

2. Populate the DEPT2 table with data from the DEPARTMENTS table. Include only the columns that you need.

```
INSERT INTO dept2
SELECT department_id, department_name
FROM departments;
```

3. Create the EMP2 table based on the following table instance chart. Type the syntax in the SQL Worksheet. Then execute the statement to create the table. Confirm that the table is created.

| Column Name  | ID     | LAST_NAME | FIRST_NAME | DEPT_ID |
|--------------|--------|-----------|------------|---------|
| Key Type     |        |           |            |         |
| Nulls/Unique |        |           |            |         |
| FK Table     |        |           |            |         |
| FK Column    |        |           |            |         |
| Data type    | NUMBER | VARCHAR2  | VARCHAR2   | NUMBER  |
| Length       | 7      | 25        | 25         | 7       |

## Practice 2: Solutions (continued)

```
CREATE TABLE emp2
(id NUMBER(7),
 last_name VARCHAR2(25),
 first_name VARCHAR2(25),
 dept_id NUMBER(7));

DESCRIBE emp2
```

4. Modify the EMP2 table to allow for longer employee last names. Confirm your modification.

```
ALTER TABLE emp2
MODIFY (last_name VARCHAR2(50));

DESCRIBE emp2
```

5. Create the EMPLOYEES2 table based on the structure of the EMPLOYEES table. Include only the EMPLOYEE\_ID, FIRST\_NAME, LAST\_NAME, SALARY, and DEPARTMENT\_ID columns. Name the columns in your new table ID, FIRST\_NAME, LAST\_NAME, SALARY, and DEPT\_ID, respectively.

```
CREATE TABLE employees2 AS
SELECT employee_id id, first_name, last_name, salary,
 department_id dept_id
FROM employees;
```

6. Drop the EMP2 table.

```
DROP TABLE emp2;
```

## Practice 2: Solutions (continued)

7. Query the recycle bin to see whether the table is present.

```
SELECT original_name, operation, droptime
FROM recyclebin;
```

8. Restore the EMP2 table to a state before the DROP statement.

```
FLASHBACK TABLE emp2 TO BEFORE DROP;
DESC emp2;
```

9. Drop the FIRST\_NAME column from the EMPLOYEES2 table. Confirm your modification by checking the description of the table.

```
ALTER TABLE employees2
DROP COLUMN first_name;

DESCRIBE employees2
```

10. In the EMPLOYEES2 table, mark the DEPT\_ID column as UNUSED. Confirm your modification by checking the description of the table.

```
ALTER TABLE employees2
SET UNUSED (dept_id);

DESCRIBE employees2
```

11. Drop all the UNUSED columns from the EMPLOYEES2 table. Confirm your modification by checking the description of the table.

```
ALTER TABLE employees2
DROP UNUSED COLUMNS;

DESCRIBE employees2
```

## Practice 2: Solutions (continued)

12. Add a table-level PRIMARY KEY constraint to the EMP2 table on the ID column. The constraint should be named at creation. Name the constraint my\_emp\_id\_pk.

```
ALTER TABLE emp2
ADD CONSTRAINT my_emp_id_pk PRIMARY KEY (id);
```

13. Create a PRIMARY KEY constraint to the DEPT2 table using the ID column. The constraint should be named at creation. Name the constraint my\_dept\_id\_pk.

```
ALTER TABLE dept2
ADD CONSTRAINT my_dept_id_pk PRIMARY KEY(id);
```

14. Add a foreign key reference on the EMP2 table that ensures that the employee is not assigned to a nonexistent department. Name the constraint my\_emp\_dept\_id\_fk.

```
ALTER TABLE emp2
ADD CONSTRAINT my_emp_dept_id_fk
FOREIGN KEY (dept_id) REFERENCES dept2(id);
```

15. Modify the EMP2 table. Add a COMMISSION column of the NUMBER data type, precision 2, scale 2. Add a constraint to the COMMISSION column that ensures that a commission value is greater than zero.

```
ALTER TABLE emp2
ADD commission NUMBER(2,2)
CONSTRAINT my_emp_comm_ck CHECK (commission > 0);
```

16. Drop the EMP2 and DEPT2 tables so that they cannot be restored. Verify the recycle bin.

```
DROP TABLE emp2 PURGE;
DROP TABLE dept2 PURGE;

SELECT original_name, operation, droptime
FROM recyclebin;
```

17. Create the DEPT\_NAMED\_INDEX table based on the following table instance chart. Name the index for the PRIMARY KEY column as DEPT\_PK\_IDX.

| Column Name | Deptno | Dname    |
|-------------|--------|----------|
| Primary Key | Yes    |          |
| Data Type   | Number | VARCHAR2 |
| Length      | 4      | 30       |



## Practice 2: Solutions (continued)

```
CREATE TABLE DEPT_NAMED_INDEX
(deptno NUMBER(4)
PRIMARY KEY USING INDEX
(CREATE INDEX dept_pk_idx ON
DEPT_NAMED_INDEX(deptno)),
dname VARCHAR2(30));
```

18. Create an external table `library_items_ext`. Use the `ORACLE_LOADER` access driver.

**Note:** The `emp_dir` directory and `library_items.dat` are already created for this exercise.

`library_items.dat` has records in the following format:

2354, 2264, 13.21, 150,

2355, 2289, 46.23, 200,

2355, 2264, 50.00, 100,

- a) Open the `lab_02_18.sql` file. Observe the code snippet to create the `library_items_ext` external table. Then, replace `<TODO1>`, `<TODO2>`, `<TODO3>`, and `<TODO4>` as appropriate and save the file as `lab_02_18_soln.sql`.  
Run the script to create the external table.

```
CREATE TABLE library_items_ext (category_id number(12)
 , book_id number(6)
 , book_price number(8,2)
 , quantity number(8)
)
ORGANIZATION EXTERNAL
(TYPE ORACLE_LOADER
DEFAULT DIRECTORY emp_dir
ACCESS PARAMETERS (RECORDS DELIMITED BY NEWLINE
 FIELDS TERMINATED BY ',')
LOCATION ('library_items.dat')
)
REJECT LIMIT UNLIMITED;
```

## Practice 2: Solutions (continued)

- b) Query the `library_items_ext` table.

```
SELECT * FROM library_items_ext;
```

19. The HR department needs a report of addresses of all departments. Create an external table as `dept_add_ext` using the `ORACLE_DATAPUMP` access driver. The report should show the location ID, street address, city, state or province, and country in the output. Use a `NATURAL JOIN` to produce the results.

**Note:** The `emp_dir` directory is already created for this exercise.

- a) Open the `lab_02_19.sql` file. Observe the code snippet to create the `dept_add_ext` external table. Then, replace `<TODO1>`, `<TODO2>` and `<TODO3>` with appropriate code. Replace `<oraxx_emp4.exp>` and `<oraxx_emp5.exp>` with appropriate file names. For example, if you are user `ora21`, your file names are `ora21_emp4.exp` and `ora21_emp5.exp`. Save the script as `lab_02_19_soln.sql`.

```
CREATE TABLE dept_add_ext (location_id,
 street_address, city,
 state_province,
 country_name)

ORGANIZATION EXTERNAL(
 TYPE ORACLE_DATAPUMP
 DEFAULT DIRECTORY emp_dir
 LOCATION ('oraxx_emp4.exp', 'oraxx_emp5.exp'))
PARALLEL
AS
SELECT location_id, street_address, city,
 state_province, country_name
FROM locations
NATURAL JOIN countries;
```

**Note:** When you perform the preceding step, two files `oraxx_emp4.exp` and `oraxx_emp5.exp` are created under the default directory `emp_dir`.

Run the `lab_02_19_soln.sql` script to create the external table.

## Practice 2: Solutions (continued)

- b) Query the dept\_add\_ext table.

```
SELECT * FROM dept_add_ext;
```

20. Create the emp\_books table and populate it with data. Set the primary key as deferred and observe what happens at the end of the transaction.

- a) Run the lab\_02\_20a.sql script to create the emp\_books table. Observe that the emp\_books\_pk primary key is not created as deferrable.

```
CREATE TABLE emp_books (book_id number,
 title varchar2(20),
CONSTRAINT emp_books_pk PRIMARY KEY (book_id));
```

- b) Run the lab\_02\_20b.sql script to populate data into the emp\_books table. What do you observe?

```
INSERT INTO emp_books VALUES(300,'Organizations');

INSERT INTO emp_books VALUES(300,'Change
Management');
```

The first row is inserted. But, you see ora-00001 error with the second row insertion.

- c) Set the emp\_books\_pk constraint as deferred. What do you observe?

```
SET CONSTRAINT emp_books_pk DEFERRED;
```

You see the error “ORA-02447: Cannot defer a constraint that is not deferrable”

- d) Drop the emp\_books\_pk constraint.

```
ALTER TABLE emp_books DROP CONSTRAINT emp_books_pk;
```

- e) Modify the emp\_books table definition to add the emp\_books\_pk constraint as deferrable this time.

```
ALTER TABLE emp_books ADD (CONSTRAINT emp_books_pk
PRIMARY KEY (book_id) DEFERRABLE);
```

- f) Set the emp\_books\_pk constraint as deferred.

```
SET CONSTRAINT emp_books_pk DEFERRED;
```

## Practice 2: Solutions (continued)

- g) Run the lab\_02\_20g.sql script to populate data into the emp\_books table.  
What do you observe?

```
INSERT INTO emp_books VALUES (300, 'Change
Management');

INSERT INTO emp_books VALUES (300, 'Personality');

INSERT INTO emp_books VALUES (350, 'Creativity');
```

You see that all of the rows are inserted.

- h) Commit the transaction. What do you observe?

```
COMMIT;
```

You see that the transaction is rolled back.

Oracle Internal & Oracle Academy  
Use Only

### Practice 3: Solutions

1. Query the data dictionary to see information about the tables you own.

```
SELECT table_name
FROM user_tables;
```

2. Query the dictionary view to see information about all the tables that you can access. Exclude tables that you own.

```
SELECT table_name, owner
FROM all_tables
WHERE owner <> 'oraxx';
```

3. For a specified table, create a script that reports the column names, data types, and data types' lengths, as well as whether nulls are allowed. Prompt the user to enter the table name. Give appropriate aliases to the DATA\_PRECISION and DATA\_SCALE columns. Save this script in a file named lab\_03\_01.sql.

```
SELECT column_name, data_type, data_length,
 data_precision PRECISION, data_scale SCALE, nullable
FROM user_tab_columns
WHERE table_name = UPPER('&tab_name');
```

To test, run the script and enter DEPARTMENTS as the table name.

4. Create a script that reports the column name, constraint name, constraint type, search condition, and status for a specified table. You must join the USER\_CONSTRAINTS and USER\_CONS\_COLUMNS tables to obtain all this information. Prompt the user to enter the table name. Save the script in a file named lab\_03\_04.sql.

```
SELECT ucc.column_name, uc.constraint_name, uc.constraint_type,
 uc.search_condition, uc.status
FROM user_constraints uc JOIN user_cons_columns ucc
ON uc.table_name = ucc.table_name
AND uc.constraint_name = ucc.constraint_name
AND uc.table_name = UPPER('&tab_name');
```

To test, run the script and enter DEPARTMENTS as the table name.

### Practice 3: Solutions (continued)

5. Add a comment to the DEPARTMENTS table. Then query the USER\_TAB\_COMMENTS view to verify that the comment is present.

```
COMMENT ON TABLE departments IS
 'Company department information including name, code, and
 location.';

SELECT COMMENTS
FROM user_tab_comments
WHERE table_name = 'DEPARTMENTS';
```

6. Create a synonym for your EMPLOYEES table. Call it EMP. Then, find the names of all synonyms that are in your schema.

```
CREATE SYNONYM emp FOR EMPLOYEES;
SELECT *
FROM user_synonyms;
```

7. Run lab\_03\_07.sql to create the dept50 view for this exercise. You need to determine the names and definitions of all the views in your schema. Create a report that retrieves view information: the view name and text from the USER\_VIEWS data dictionary view.

**Note:** The EMP\_DETAILS\_VIEW was created as part of your schema.

**Note:** You can see the complete definition of the view if you use the Run Script (or press [F5]) in SQL Developer. If you use Execute Statement (or press [F9]) in SQL Developer, scroll horizontally in the result pane. If you use SQL\*Plus to see more contents of a LONG column, use the SET LONG n command, where n is the value of the number of characters of the LONG column that you want to see.

```
SELECT view_name, text
FROM user_views;
```

### Practice 3: Solutions (continued)

8. Find the names of your sequences. Write a query in a script to display the following information about your sequences: sequence name, maximum value, increment size, and last number. Name the script `lab_11_06.sql`. Run the statement in your script.

```
SELECT sequence_name, max_value, increment_by, last_number
FROM user_sequences;
```

Run the `lab_03_09_tab.sql` script as a prerequisite for exercises 9 through 11. Alternatively, open the script file to copy the code and paste it into your SQL Worksheet. Then execute the script. This script does the following:

- Drops the DEPT2 and EMP2 tables
- Creates tables DEPT2 and EMP2

**Note:** In Practice 2, you should have already dropped DEPT2 and EMP2 tables so that they cannot be restored.

9. Confirm that both the DEPT2 and EMP2 tables are stored in the data dictionary.

```
SELECT table_name
FROM user_tables
WHERE table_name IN ('DEPT2', 'EMP2');
```

10. Query the data dictionary to find out the constraint names and types for both the tables.

```
SELECT constraint_name, constraint_type
FROM user_constraints
WHERE table_name IN ('EMP2', 'DEPT2');
```

11. Query the data dictionary to display the object names and types for both the tables.

```
SELECT object_name, object_type
FROM user_objects
WHERE object_name LIKE 'EMP%'
OR object_name LIKE 'DEPT%';
```

### Practice 3: Solutions (continued)

12. Create the SALES\_DEPT table based on the following table instance chart. Name the index for the PRIMARY KEY column as SALES\_PK\_IDX. Then query the data dictionary view to find the index name, table name, and whether the index is unique.

|             |         |          |
|-------------|---------|----------|
| Column Name | Team_Id | Location |
| Primary Key | Yes     |          |
| Data Type   | Number  | VARCHAR2 |
| Length      | 3       | 30       |

```
CREATE TABLE SALES_DEPT
(team_id NUMBER(3)
PRIMARY KEY USING INDEX
(CREATE INDEX sales_pk_idx ON
SALES_DEPT(team_id)),
location VARCHAR2(30));
SELECT INDEX_NAME, TABLE_NAME, UNIQUENESS
FROM USER_INDEXES
WHERE TABLE_NAME = 'SALES_DEPT';
```



## Practice 4: Solutions

1. Run the `lab_04_01.sql` script in the lab folder to create the `SAL_HISTORY` table.
2. Display the structure of the `SAL_HISTORY` table.

```
DESC sal_history
```

3. Run the `lab_04_03.sql` script in the lab folder to create the `MGR_HISTORY` table.
4. Display the structure of the `MGR_HISTORY` table.

```
DESC mgr_history
```

5. Run the `lab_04_05.sql` script in the lab folder to create the `SPECIAL_SAL` table.
6. Display the structure of the `SPECIAL_SAL` table.

```
DESC special_sal
```

7. a) Write a query to do the following:
  - Retrieve details such as the employee ID, hire date, salary, and manager ID of those employees whose employee ID is less than 125 from the `EMPLOYEES` table.
  - If the salary is more than \$20,000, insert details such as the employee ID and salary into the `SPECIAL_SAL` table.
  - Insert details such as the employee ID, hire date, and salary into the `SAL_HISTORY` table.
  - Insert details such as the employee ID, manager ID, and salary into the `MGR_HISTORY` table.

## Practice 4: Solutions (continued)

```
INSERT ALL
WHEN SAL > 20000 THEN
 INTO special_sal VALUES (EMPID, SAL)
ELSE
 INTO sal_history VALUES(EMPID,HIREDATE,SAL)
 INTO mgr_history VALUES(EMPID,MGR,SAL)
SELECT employee_id EMPID, hire_date HIREDATE,
 salary SAL, manager_id MGR
FROM employees
WHERE employee_id < 125;
```

b) Display the records from the SPECIAL\_SAL table.

```
SELECT * FROM special_sal;
```

c) Display the records from the SAL\_HISTORY table.

```
SELECT * FROM sal_history;
```

d) Display the records from the MGR\_HISTORY table.

```
SELECT * FROM mgr_history;
```

8. a) Run the lab\_04\_08a.sql script in the lab folder to create the SALES\_WEEK\_DATA table.
- b) Run the lab\_04\_08b.sql script in the lab folder to insert records into the SALES\_WEEK\_DATA table.
- c) Display the structure of the SALES\_WEEK\_DATA table.

```
DESC sales_week_data
```

## Practice 4: Solutions (continued)

d) Display the records from the SALES\_WEEK\_DATA table.

```
SELECT * FROM SALES_WEEK_DATA;
```

e) Run the lab\_04\_08c.sql script in the lab folder to create the EMP\_SALES\_INFO table.

f) Display the structure of the EMP\_SALES\_INFO table.

```
DESC emp_sales_info
```

g) Write a query to do the following:

- Retrieve details such as the employee ID, week ID, sales quantity on Monday, sales quantity on Tuesday, sales quantity on Wednesday, sales quantity on Thursday, and sales quantity on Friday from the SALES\_WEEK\_DATA table.
- Build a transformation such that each record retrieved from the SALES\_WEEK\_DATA table is converted into multiple records for the EMP\_SALES\_INFO table.

**Hint:** Use a pivoting INSERT statement.

```
INSERT ALL
 INTO emp_sales_info VALUES (id, week_id, QTY_MON)
 INTO emp_sales_info VALUES (id, week_id, QTY_TUE)
 INTO emp_sales_info VALUES (id, week_id, QTY_WED)
 INTO emp_sales_info VALUES (id, week_id, QTY_THUR)
 INTO emp_sales_info VALUES (id, week_id, QTY_FRI)
 SELECT ID, week_id, QTY_MON, QTY_TUE, QTY_WED,
 QTY_THUR, QTY_FRI FROM sales_week_data;
```

h) Display the records from the EMP\_SALES\_INFO table.

```
SELECT * FROM emp_sales_info;
```

## Practice 4: Solutions (continued)

9. You have the data of past employees stored in a flat file called `emp.dat`. You want to store the names and email IDs of all employees past and present in a table. To do this, first create an external table called `EMP_DATA` using the `emp.dat` source file in the `emp_dir` directory. You can use the script in `lab_04_09.sql` to do this.

```
CREATE TABLE emp_data
 (first_name VARCHAR2(20)
 ,last_name VARCHAR2(20)
 , email VARCHAR2(30)
)
ORGANIZATION EXTERNAL
(
 TYPE oracle_loader
 DEFAULT DIRECTORY emp_dir
 ACCESS PARAMETERS
 (
 RECORDS DELIMITED BY NEWLINE CHARACTERSET US7ASCII
 NOBADFILE
 NOLOGFILE
 FIELDS
 (first_name POSITION (1:20) CHAR
 , last_name POSITION (22:41) CHAR
 , email POSITION (43:72) CHAR)
)
 LOCATION ('emp.dat')) ;
```

10. Next, run the `lab_04_10.sql` script to create the `EMP_HIST` table.
- a) Increase the size of the email column to 45.

```
ALTER TABLE emp_hist MODIFY email varchar(45);
```

- b) Merge the data in the `EMP_DATA` table created in the last lab into the data in the `EMP_HIST` table. Assume that the data in the external `EMP_DATA` table is the most up-to-date. If a row in the `EMP_DATA` table matches the `EMP_HIST` table, update the email column of the `EMP_HIST` table to match the `EMP_DATA` table row. If a row in the `EMP_DATA` table does not match, insert it into the `EMP_HIST` table. Rows are considered matching when the employee's first and last names are identical.

## Practice 4: Solutions (continued)

```
MERGE INTO EMP_HIST f USING EMP_DATA h
 ON (f.first_name = h.first_name
 AND f.last_name = h.last_name)
 WHEN MATCHED THEN
 UPDATE SET f.email = h.email
 WHEN NOT MATCHED THEN
 INSERT (f.first_name
 , f.last_name
 , f.email)
 VALUES (h.first_name
 , h.last_name
 , h.email);
```

- c) Retrieve the rows from EMP\_HIST after the merge.

```
SELECT * FROM emp_hist;
```

11. Create the EMP3 table using the lab\_04\_11.sql script. In the EMP3 table, change the department for Kochhar to 60 and commit your change. Next, change the department for Kochhar to 50 and commit your change. Track the changes to Kochhar using the Row Versions feature.

```
UPDATE emp3 SET department_id = 60
WHERE last_name = 'Kochchar';
COMMIT;
UPDATE emp3 SET department_id = 50
WHERE last_name = 'Kochchar';
COMMIT;
```

```
SELECT VERSIONS STARTTIME "START_DATE",
 VERSIONS_ENDTIME "END_DATE", DEPARTMENT_ID
FROM EMP3
 VERSIONS BETWEEN SCN MINVALUE AND MAXVALUE
WHERE LAST_NAME = 'Kochhar';
```

## Practice 5: Solutions

1. Alter the session to set NLS\_DATE\_FORMAT to DD-MON-YYYY HH24:MI:SS.

```
ALTER SESSION SET NLS_DATE_FORMAT =
'DD-MON-YYYY HH24:MI:SS';
```

2. a. Write queries to display the time zone offsets (TZ\_OFFSET) for the following time zones: *US/Pacific-New*, *Singapore*, and *Egypt*.

*US/Pacific-New*

```
SELECT TZ_OFFSET ('US/Pacific-New') from dual;
```

*Singapore*

```
SELECT TZ_OFFSET ('Singapore') from dual;
```

*Egypt*

```
SELECT TZ_OFFSET ('Egypt') from dual;
```

- b. Alter the session to set the TIME\_ZONE parameter value to the time zone offset of *US/Pacific-New*.

```
ALTER SESSION SET TIME_ZONE = '-7:00';
```

- c. Display the CURRENT\_DATE, CURRENT\_TIMESTAMP, and LOCALTIMESTAMP for this session.

**Note:** The output may be different based on the date when the command is executed.

```
SELECT CURRENT_DATE, CURRENT_TIMESTAMP,
LOCALTIMESTAMP FROM DUAL;
```

- d. Alter the session to set the TIME\_ZONE parameter value to the time zone offset of *Singapore*.

```
ALTER SESSION SET TIME_ZONE = '+8:00';
```

- e. Display the CURRENT\_DATE, CURRENT\_TIMESTAMP, LOCALTIMESTAMP for this session.

**Note:** The output might be different, based on the date when the command is executed.

```
SELECT CURRENT_DATE, CURRENT_TIMESTAMP,
LOCALTIMESTAMP FROM DUAL;
```

## Practice 5: Solutions (continued)

**Note:** Observe in the preceding practice that `CURRENT_DATE`, `CURRENT_TIMESTAMP`, and `LOCALTIMESTAMP` are all sensitive to the session time zone.

3. Write a query to display `DBTIMEZONE` and `SESSIONTIMEZONE`.

```
SELECT DBTIMEZONE, SESSIONTIMEZONE
FROM DUAL;
```

4. Write a query to extract `YEAR` from the `HIRE_DATE` column of the `EMPLOYEES` table for those employees who work in department 80.

```
SELECT last_name, EXTRACT (YEAR FROM HIRE_DATE)
FROM employees
WHERE department_id = 80;
```

5. Alter the session to set `NLS_DATE_FORMAT` to `DD-MON-YYYY`.

```
ALTER SESSION SET NLS_DATE_FORMAT = 'DD-MON-YYYY';
```

6. Examine and run the `lab_05_06.sql` script to create the `SAMPLE_DATES` table and populate it.

- a. Select from the table and view the data.

```
SELECT * FROM sample_dates;
```

- b. Modify the data type of the `DATE_COL` column and change it to `TIMESTAMP`.  
Select from the table to view the data.

```
ALTER TABLE sample_dates MODIFY date_col TIMESTAMP;
SELECT * FROM sample_dates;
```

- c. Try to modify the data type of the `DATE_COL` column and change it to `TIMESTAMP WITH TIME ZONE`. What happens?

```
ALTER TABLE sample_dates MODIFY date_col
TIMESTAMP WITH TIME ZONE;
```

You are unable to change the data type of the `DATE_COL` column as the Oracle server does not permit you to convert from `TIMESTAMP` to `TIMESTAMP WITH TIMEZONE` by using the `ALTER` statement.

## Practice 5: Solutions (continued)

7. Create a query to retrieve last names from the EMPLOYEES table and calculate the review status. If the year hired was 1998, then display Needs Review for the review status; otherwise, display not this year! Name the review status column Review. Sort the results by the HIRE\_DATE column.

**Hint:** Use a CASE expression with the EXTRACT function to calculate the review status.

```
SELECT e.last_name
, (CASE extract(year from e.hire_date)
 WHEN 1998 THEN 'Needs Review'
 ELSE 'not this year!'
 END) AS "Review "
FROM employees e
ORDER BY e.hire_date;
```

8. Create a query to print the last names and the number of years of service for each employee. If the employee has been employed five or more years, then print 5 years of service. If the employee has been employed 10 or more years, then print 10 years of service. If the employee has been employed 15 or more years, then print 15 years of service. If none of these conditions match, then print maybe next year! Sort the results by the HIRE\_DATE column. Use the EMPLOYEES table.

**Hint:** Use CASE expressions and TO\_YMINTERVAL.

```
SELECT e.last_name, hire_date, sysdate,
 (CASE
 WHEN (sysdate -TO_YMINTERVAL('15-0'))>=
 hire_date THEN '15 years of service'
 WHEN (sysdate -TO_YMINTERVAL('10-0'))>=
hire_date
 THEN '10 years of service'
 WHEN (sysdate - TO_YMINTERVAL('5-0'))>=
hire_date
 THEN '5 years of service'
 ELSE 'maybe next year!'
 END) AS "Awards"
FROM employees e;
```



## Practice 6: Solutions

1. Write a query to display the last name, department number, and salary of any employee whose department number and salary both match the department number and salary of any employee who earns a commission.

```
SELECT last_name, department_id, salary
FROM employees
WHERE (salary, department_id) IN
 (SELECT salary, department_id
 FROM employees
 WHERE commission_pct IS NOT NULL);
```

2. Display the last name, department name, and salary of any employee whose salary and commission match the salary and commission of any employee located in location ID1700.

```
SELECT e.last_name, d.department_name, e.salary
FROM employees e, departments d
WHERE e.department_id = d.department_id
AND (salary, NVL(commission_pct,0)) IN
 (SELECT salary, NVL(commission_pct,0)
 FROM employees e, departments d
 WHERE e.department_id = d.department_id
 AND d.location_id = 1700);
```

3. Create a query to display the last name, hire date, and salary for all employees who have the same salary and commission as Kochhar.

**Note:** Do not display Kochhar in the result set.

```
SELECT last_name, hire_date, salary
FROM employees
WHERE (salary, NVL(commission_pct,0)) IN
 (SELECT salary, NVL(commission_pct,0)
 FROM employees
 WHERE last_name = 'Kochhar')
AND last_name != 'Kochhar';
```

## Practice 6: Solutions (continued)

4. Create a query to display the employees who earn a salary that is higher than the salary of all the sales managers (JOB\_ID = 'SA\_MAN'). Sort the results on salary from the highest to the lowest.

```
SELECT last_name, job_id, salary
FROM employees
WHERE salary > ALL
 (SELECT salary
 FROM employees
 WHERE job_id = 'SA_MAN')
ORDER BY salary DESC;
```

5. Display details such as the employee ID, last name, and department ID of those employees who live in cities the names of which begin with T.

```
SELECT employee_id, last_name, department_id
FROM employees
WHERE department_id IN (SELECT department_id
 FROM departments
 WHERE location_id IN
 (SELECT location_id
 FROM locations
 WHERE city LIKE 'T%'));
```

6. Write a query to find all employees who earn more than the average salary in their departments. Display last name, salary, department ID, and the average salary for the department. Sort by average salary. Use aliases for the columns retrieved by the query as shown in the sample output.

```
SELECT e.last_name ename, e.salary salary,
 e.department_id deptno, AVG(a.salary) dept_avg
FROM employees e, employees a
WHERE e.department_id = a.department_id
AND e.salary > (SELECT AVG(salary)
 FROM employees
 WHERE department_id = e.department_id)
GROUP BY e.last_name, e.salary, e.department_id
ORDER BY AVG(a.salary);
```

## Practice 6: Solutions (continued)

7. Find all employees who are not supervisors.
  - a. First, do this by using the NOT EXISTS operator.

```
SELECT outer.last_name
FROM employees outer
WHERE NOT EXISTS (SELECT 'X'
 FROM employees inner
 WHERE inner.manager_id =
 outer.employee_id);
```

- b. Can this be done by using the NOT IN operator? How, or why not?

```
SELECT outer.last_name
FROM employees outer
WHERE outer.employee_id
NOT IN (SELECT inner.manager_id
 FROM employees inner);
```

This alternative solution is not a good one. The subquery picks up a NULL value, so the entire query returns no rows. The reason is that all conditions that compare a NULL value result in NULL. Whenever NULL values are likely to be part of the value set, *do not* use NOT IN as a substitute for NOT EXISTS.

8. Write a query to display the last names of the employees who earn less than the average salary in their departments.

```
SELECT last_name
FROM employees outer
WHERE outer.salary < (SELECT AVG(inner.salary)
 FROM employees inner
 WHERE inner.department_id
 = outer.department_id);
```

## Practice 6: Solutions (continued)

9. Write a query to display the last names of employees who have one or more coworkers in their departments with later hire dates but higher salaries.

```
SELECT last_name
FROM employees outer
WHERE EXISTS (SELECT 'X'
 FROM employees inner
 WHERE inner.department_id =
 outer.department_id
 AND inner.hire_date > outer.hire_date
 AND inner.salary > outer.salary);
```

10. Write a query to display the employee ID, last names, and department names of all employees.

**Note:** Use a scalar subquery to retrieve the department name in the SELECT statement.

```
SELECT employee_id, last_name,
 (SELECT department_name
 FROM departments d
 WHERE e.department_id =
 d.department_id) department
FROM employees e
ORDER BY department;
```

11. Write a query to display the department names of those departments whose total salary cost is above one-eighth (1/8) of the total salary cost of the whole company. Use the WITH clause to write this query. Name the query SUMMARY.

```
WITH
summary AS (
 SELECT d.department_name, SUM(e.salary) AS dept_total
 FROM employees e, departments d
 WHERE e.department_id = d.department_id
 GROUP BY d.department_name)
SELECT department_name, dept_total
FROM summary
WHERE dept_total > (SELECT SUM(dept_total) * 1/8
 FROM summary)
ORDER BY dept_total DESC;
```

## Practice 7: Solutions

1. Write a query to search the EMPLOYEES table for all employees whose first names start with “Ki” or “Ko.”

```
SELECT first_name, last_name
FROM employees
WHERE REGEXP_LIKE (last_name, '^K(i|o).');
```

2. Create a query that removes the spaces in the STREET\_ADDRESS column of the LOCATIONS table in the display. Use “Street Address” as the column heading.

```
SELECT regexp_replace (street_address, ' ', '') AS "Street
Address"
FROM locations;
```

3. Create a query that displays “St” replaced by “Street” in the STREET\_ADDRESS column of the LOCATIONS table. Be careful that you do not affect any rows that already have “Street” in them. Display only those rows, which are affected.

```
SELECT regexp_replace (street_address, 'St$', 'Street')
FROM locations
WHERE regexp_like (street_address, 'St');
```

4. Create a contacts table and add a check constraint to the p\_number column to enforce the following format mask to ensure that phone numbers are entered into the database in the following standard format: (XXX) XXX-XXXX. The table should have the following columns:

- l\_name varchar2(30)
- p\_number varchar2 (30)

```
CREATE TABLE contacts
(
 l_name VARCHAR2(30),
 p_number VARCHAR2(30)
 CONSTRAINT p_number_format
 CHECK (REGEXP_LIKE (p_number, '^(\d{3}\) \d{3}-
\d{4}$'))
);
```

## Practice 7: Solutions (continued)

5. Run the `lab_07_05.sql` SQL script to insert the following seven phone numbers into the contacts table. Which numbers are added?  
**Only the first two INSERT statements use a format that conforms to the `c_contacts_pnf` constraint; the remaining statements generate CHECK constraint errors.**
6. Write a query to find the number of occurrences of the DNA pattern `ctc` in the string `gtctcgtctcgttctgtctgtcgttctg`. Use the alias `Count_DNA`. Ignore case sensitivity.

```
SELECT REGEXP_COUNT('gtctcgtctcgttctgtctgtcgttctg','ctc')
AS Count_DNA
FROM dual;
```

Oracle Internal & Oracle Academy  
Use Only

## Table Descriptions

B

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Oracle Internal & Oracle Academy  
Use Only

## Schema Description

### Overall Description

The Oracle database sample schemas portray a sample company that operates worldwide to fill orders for several different products. The company has three divisions:

- **Human Resources:** Tracks information about the employees and facilities
- **Order Entry:** Tracks product inventories and sales through various channels
- **Sales History:** Tracks business statistics to facilitate business decisions

Each of these divisions is represented by a schema. In this course, you have access to the objects in all the schemas. However, the emphasis of the examples, demonstrations, and practices is on the Human Resources (HR) schema.

All scripts necessary to create the sample schemas reside in the `$ORACLE_HOME/demo/schema/` folder.

### Human Resources (HR)

This is the schema that is used in this course. In the Human Resource (HR) records, each employee has an identification number, e-mail address, job identification code, salary, and manager. Some employees earn commissions in addition to their salary.

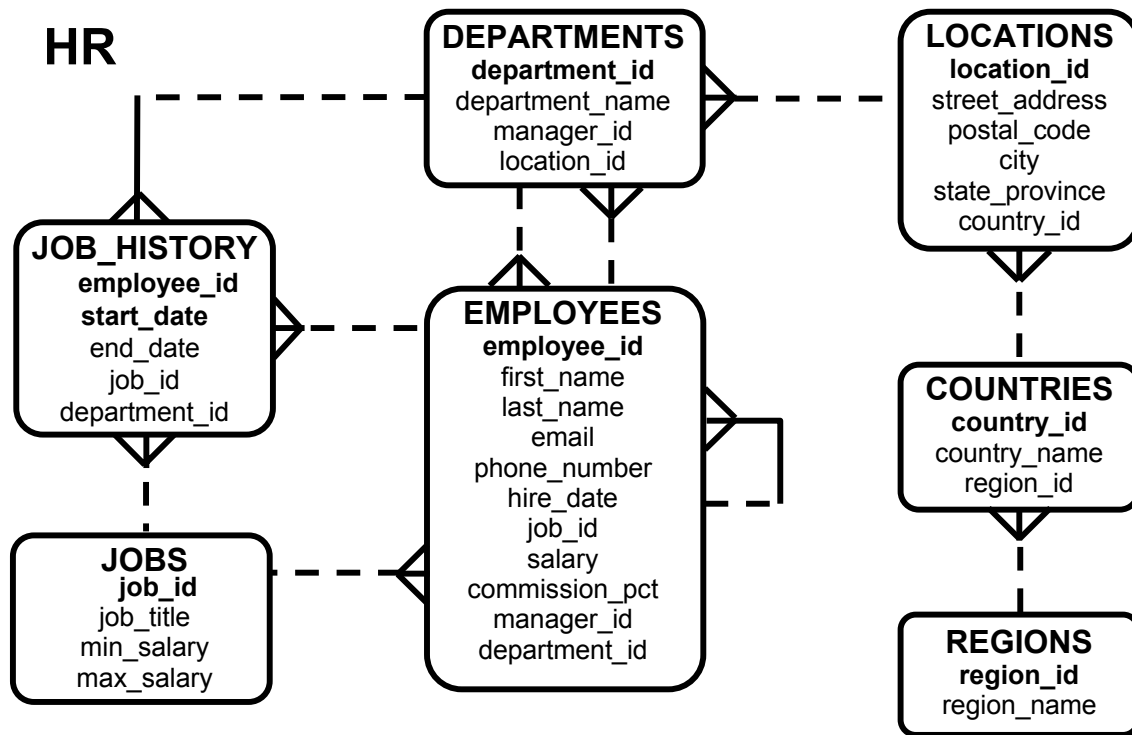
The company also tracks information about jobs within the organization. Each job has an identification code, job title, and a minimum and maximum salary range for the job. Some employees have been with the company for a long time and have held different positions within the company. When an employee resigns, the duration the employee was working for, the job identification number, and the department are recorded.

The sample company is regionally diverse, so it tracks the locations of its warehouses and departments. Each employee is assigned to a department, and each department is identified either by a unique department number or a short name. Each department is associated with one location, and each location has a full address that includes the street name, postal code, city, state or province, and the country code.

In places where the departments and warehouses are located, the company records details such as the country name, currency symbol, currency name, and the region where the country is located geographically.



## The HR Entity Relationship Diagram



## The Human Resources (HR) Table Descriptions

DESCRIBE countries

| Name         | Null     | Type         |
|--------------|----------|--------------|
| COUNTRY_ID   | NOT NULL | CHAR(2)      |
| COUNTRY_NAME |          | VARCHAR2(40) |
| REGION_ID    |          | NUMBER       |

SELECT \* FROM countries;

|   | COUNTRY_ID | COUNTRY_NAME             | REGION_ID |
|---|------------|--------------------------|-----------|
| 1 | CA         | Canada                   | 2         |
| 2 | DE         | Germany                  | 1         |
| 3 | UK         | United Kingdom           | 1         |
| 4 | US         | United States of America | 2         |

Oracle Internal & Oracle Academy  
Use Only

## The Human Resources (HR) Table Descriptions (continued)

DESCRIBE departments

| Name            | Null     | Type         |
|-----------------|----------|--------------|
| -----           | -----    | -----        |
| DEPARTMENT_ID   | NOT NULL | NUMBER(4)    |
| DEPARTMENT_NAME | NOT NULL | VARCHAR2(30) |
| MANAGER_ID      |          | NUMBER(6)    |
| LOCATION_ID     |          | NUMBER(4)    |

SELECT \* FROM departments;

|   |  DEPARTMENT_ID |  DEPARTMENT_NAME |  MANAGER_ID |  LOCATION_ID |
|---|-------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| 1 | 10                                                                                              | Administration                                                                                    | 200                                                                                          | 1700                                                                                            |
| 2 | 20                                                                                              | Marketing                                                                                         | 201                                                                                          | 1800                                                                                            |
| 3 | 50                                                                                              | Shipping                                                                                          | 124                                                                                          | 1500                                                                                            |
| 4 | 60                                                                                              | IT                                                                                                | 103                                                                                          | 1400                                                                                            |
| 5 | 80                                                                                              | Sales                                                                                             | 149                                                                                          | 2500                                                                                            |
| 6 | 90                                                                                              | Executive                                                                                         | 100                                                                                          | 1700                                                                                            |
| 7 | 110                                                                                             | Accounting                                                                                        | 205                                                                                          | 1700                                                                                            |
| 8 | 190                                                                                             | Contracting                                                                                       | (null)                                                                                       | 1700                                                                                            |

## The Human Resources (HR) Table Descriptions (continued)

DESCRIBE employees

| Name           | Null     | Type         |
|----------------|----------|--------------|
| -----          | -----    | -----        |
| EMPLOYEE_ID    | NOT NULL | NUMBER(6)    |
| FIRST_NAME     |          | VARCHAR2(20) |
| LAST_NAME      | NOT NULL | VARCHAR2(25) |
| EMAIL          | NOT NULL | VARCHAR2(25) |
| PHONE_NUMBER   |          | VARCHAR2(20) |
| HIRE_DATE      | NOT NULL | DATE         |
| JOB_ID         | NOT NULL | VARCHAR2(10) |
| SALARY         |          | NUMBER(8,2)  |
| COMMISSION_PCT |          | NUMBER(2,2)  |
| MANAGER_ID     |          | NUMBER(6)    |
| DEPARTMENT_ID  |          | NUMBER(4)    |

SELECT \* FROM employees;

|    | EMPLOYEE_ID | FIRST_N... | LAST_N... | EMAIL    | PHONE_NUMBER       | HIRE_DATE | JOB_ID    | SALARY | COMMI... | MANAGER_ID | DEPARTMENT_ID |
|----|-------------|------------|-----------|----------|--------------------|-----------|-----------|--------|----------|------------|---------------|
| 1  | 100         | Steven     | King      | SKING    | 515.123.4567       | 17-JUN-87 | AD_PRES   | 24000  | (null)   | (null)     | 90            |
| 2  | 101         | Neena      | Kochhar   | NKOCHHAR | 515.123.4568       | 21-SEP-89 | AD_VP     | 17000  | (null)   | 100        | 90            |
| 3  | 102         | Lex        | De Haan   | LDEHAAN  | 515.123.4569       | 13-JAN-93 | AD_VP     | 17000  | (null)   | 100        | 90            |
| 4  | 103         | Alexander  | Hunold    | AHUNOLD  | 590.423.4567       | 03-JAN-90 | IT_PROG   | 9000   | (null)   | 102        | 60            |
| 5  | 104         | Bruce      | Ernst     | BERNST   | 590.423.4568       | 21-MAY-91 | IT_PROG   | 6000   | (null)   | 103        | 60            |
| 6  | 107         | Diana      | Lorentz   | DLORENTZ | 590.423.5567       | 07-FEB-99 | IT_PROG   | 4200   | (null)   | 103        | 60            |
| 7  | 124         | Kevin      | Mourgos   | KMOURGOS | 650.123.5234       | 16-NOV-99 | ST_MAN    | 5800   | (null)   | 100        | 50            |
| 8  | 141         | Trenna     | Rajs      | TRAJS    | 650.121.8009       | 17-OCT-95 | ST_CLERK  | 3500   | (null)   | 124        | 50            |
| 9  | 142         | Curtis     | Davies    | CDAVIES  | 650.121.2994       | 29-JAN-97 | ST_CLERK  | 3100   | (null)   | 124        | 50            |
| 10 | 143         | Randall    | Matos     | RMATOS   | 650.121.2874       | 15-MAR-98 | ST_CLERK  | 2600   | (null)   | 124        | 50            |
| 11 | 144         | Peter      | Vargas    | PVARGAS  | 650.121.2004       | 09-JUL-98 | ST_CLERK  | 2500   | (null)   | 124        | 50            |
| 12 | 149         | Eleni      | Zlotkey   | EZLOTKEY | 011.44.1344.429018 | 29-JAN-00 | SA_MAN    | 10500  | 0.2      | 100        | 80            |
| 13 | 174         | Ellen      | Abel      | EABEL    | 011.44.1644.429267 | 11-MAY-96 | SA_REP    | 11000  | 0.3      | 149        | 80            |
| 14 | 176         | Jonathon   | Taylor    | JTAYLOR  | 011.44.1644.429265 | 24-MAR-98 | SA_REP    | 8600   | 0.2      | 149        | 80            |
| 15 | 178         | Kimberely  | Grant     | KGRANT   | 011.44.1644.429263 | 24-MAY-99 | SA_REP    | 7000   | 0.15     | 149        | (null)        |
| 16 | 200         | Jennifer   | Whalen    | JWHALEN  | 515.123.4444       | 17-SEP-87 | AD_ASST   | 4400   | (null)   | 101        | 10            |
| 17 | 201         | Michael    | Hartstein | MHARTSTE | 515.123.5555       | 17-FEB-96 | MK_MAN    | 13000  | (null)   | 100        | 20            |
| 18 | 202         | Pat        | Fay       | PFAY     | 603.123.6666       | 17-AUG-97 | MK_REP    | 6000   | (null)   | 201        | 20            |
| 19 | 205         | Shelley    | Higgins   | SHIGGINS | 515.123.8080       | 07-JUN-94 | AC_MGR    | 12000  | (null)   | 101        | 110           |
| 20 | 206         | William    | Gietz     | WGIEZT   | 515.123.8181       | 07-JUN-94 | AC_ACC... | 8300   | (null)   | 205        | 110           |

## The Human Resources (HR) Table Descriptions (continued)

DESCRIBE job\_history

| DESCRIBE job_history |          |              |
|----------------------|----------|--------------|
| Name                 | Null     | Type         |
| -----                |          |              |
| EMPLOYEE_ID          | NOT NULL | NUMBER(6)    |
| START_DATE           | NOT NULL | DATE         |
| END_DATE             | NOT NULL | DATE         |
| JOB_ID               | NOT NULL | VARCHAR2(10) |
| DEPARTMENT_ID        |          | NUMBER(4)    |

SELECT \* FROM job\_history

|    | EMPLOYEE_ID | START_DATE | END_DATE  | JOB_ID     | DEPARTMENT_ID |
|----|-------------|------------|-----------|------------|---------------|
| 1  | 102         | 13-JAN-93  | 24-JUL-98 | IT_PROG    | 60            |
| 2  | 101         | 21-SEP-89  | 27-OCT-93 | AC_ACCOUNT | 110           |
| 3  | 101         | 28-OCT-93  | 15-MAR-97 | AC_MGR     | 110           |
| 4  | 201         | 17-FEB-96  | 19-DEC-99 | MK_REP     | 20            |
| 5  | 114         | 24-MAR-98  | 31-DEC-99 | ST_CLERK   | 50            |
| 6  | 122         | 01-JAN-99  | 31-DEC-99 | ST_CLERK   | 50            |
| 7  | 200         | 17-SEP-87  | 17-JUN-93 | AD_ASST    | 90            |
| 8  | 176         | 24-MAR-98  | 31-DEC-98 | SA_REP     | 80            |
| 9  | 176         | 01-JAN-99  | 31-DEC-99 | SA_MAN     | 80            |
| 10 | 200         | 01-JUL-94  | 31-DEC-98 | AC_ACCOUNT | 90            |

## The Human Resources (HR) Table Descriptions (continued)

DESCRIBE jobs

| Name       | Null     | Type         |
|------------|----------|--------------|
| JOB_ID     | NOT NULL | VARCHAR2(10) |
| JOB_TITLE  | NOT NULL | VARCHAR2(35) |
| MIN_SALARY |          | NUMBER(6)    |
| MAX_SALARY |          | NUMBER(6)    |

SELECT \* FROM jobs

|    |  JOB_ID |  JOB_TITLE |  MIN_SALARY |  MAX_SALARY |
|----|------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| 1  | AD_PRES                                                                                  | President                                                                                   | 20000                                                                                        | 40000                                                                                          |
| 2  | AD_VP                                                                                    | Administration Vice President                                                               | 15000                                                                                        | 30000                                                                                          |
| 3  | AD_ASST                                                                                  | Administration Assistant                                                                    | 3000                                                                                         | 6000                                                                                           |
| 4  | AC_MGR                                                                                   | Accounting Manager                                                                          | 8200                                                                                         | 16000                                                                                          |
| 5  | AC_ACCOUNT                                                                               | Public Accountant                                                                           | 4200                                                                                         | 9000                                                                                           |
| 6  | SA_MAN                                                                                   | Sales Manager                                                                               | 10000                                                                                        | 20000                                                                                          |
| 7  | SA_REP                                                                                   | Sales Representative                                                                        | 6000                                                                                         | 12000                                                                                          |
| 8  | ST_MAN                                                                                   | Stock Manager                                                                               | 5500                                                                                         | 8500                                                                                           |
| 9  | ST_CLERK                                                                                 | Stock Clerk                                                                                 | 2000                                                                                         | 5000                                                                                           |
| 10 | IT_PROG                                                                                  | Programmer                                                                                  | 4000                                                                                         | 10000                                                                                          |
| 11 | MK_MAN                                                                                   | Marketing Manager                                                                           | 9000                                                                                         | 15000                                                                                          |
| 12 | MK_REP                                                                                   | Marketing Representative                                                                    | 4000                                                                                         | 9000                                                                                           |

## The Human Resources (HR) Table Descriptions (continued)

DESCRIBE locations

| Name           | Null     | Type         |
|----------------|----------|--------------|
| -----          | -----    | -----        |
| LOCATION_ID    | NOT NULL | NUMBER(4)    |
| STREET_ADDRESS |          | VARCHAR2(40) |
| POSTAL_CODE    |          | VARCHAR2(12) |
| CITY           | NOT NULL | VARCHAR2(30) |
| STATE_PROVINCE |          | VARCHAR2(25) |
| COUNTRY_ID     |          | CHAR(2)      |

SELECT \* FROM locations

|   |  LOCATION_ID |  STREET_ADDRESS |  POSTAL_CODE |  CITY |  STATE_PROVINCE | COUNTRY_ID |
|---|-----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|------------|
| 1 | 1400                                                                                          | 2014 Jabberwocky Rd                                                                              | 26192                                                                                         | Southlake                                                                              | Texas                                                                                              | US         |
| 2 | 1500                                                                                          | 2011 Interiors Blvd                                                                              | 99236                                                                                         | South San Francisco                                                                    | California                                                                                         | US         |
| 3 | 1700                                                                                          | 2004 Charade Rd                                                                                  | 98199                                                                                         | Seattle                                                                                | Washington                                                                                         | US         |
| 4 | 1800                                                                                          | 460 Bloor St. W.                                                                                 | ON M5S 1X8                                                                                    | Toronto                                                                                | Ontario                                                                                            | CA         |
| 5 | 2500                                                                                          | Magdalen Centre, The Oxford Science Park                                                         | OX9 9ZB                                                                                       | Oxford                                                                                 | Oxford                                                                                             | UK         |

Oracle Internal & Oracle Academy  
Use Only

## The Human Resources (HR) Table Descriptions (continued)

DESCRIBE regions

| Name        | Null     | Type         |
|-------------|----------|--------------|
| -----       | -----    | -----        |
| REGION_ID   | NOT NULL | NUMBER       |
| REGION_NAME |          | VARCHAR2(25) |

SELECT \* FROM regions

|   | REGION_ID | REGION_NAME            |
|---|-----------|------------------------|
| 1 | 1         | Europe                 |
| 2 | 2         | Americas               |
| 3 | 3         | Asia                   |
| 4 | 4         | Middle East and Africa |

Oracle Internal & Oracle Academy  
Use Only



# Using the SQL Developer GUI

ORACLE

Copyright © 2007, Oracle. All rights reserved.

Oracle Internal & Oracle Academy  
Use Only

## Objectives

After completing this appendix, you should be able to do the following:

- Perform data definition language (DDL) operations using the SQL Developer menu options
- Perform data manipulation language (DML) operations using the SQL Developer menu options

ORACLE

Copyright © 2007, Oracle. All rights reserved.

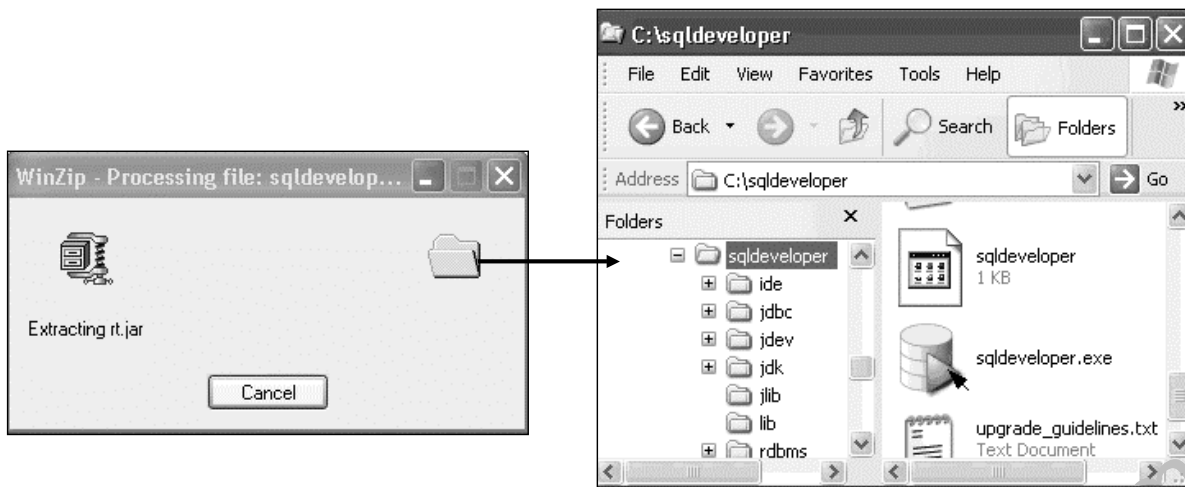
### Objectives

In this appendix, you are introduced to the graphical tool SQL Developer. You learn how to use the SQL Developer GUI interface for your database development tasks.

This appendix is an add-on to what was discussed in the lesson titled “Introduction.”

## Installing SQL Developer

Download the Oracle SQL Developer kit and unzip it into any directory on your machine.



Copyright © 2007, Oracle. All rights reserved.

### Installing SQL Developer

Oracle SQL Developer does not require an installer. To install SQL Developer, you need an unzip tool.

To install SQL Developer, perform the following steps:

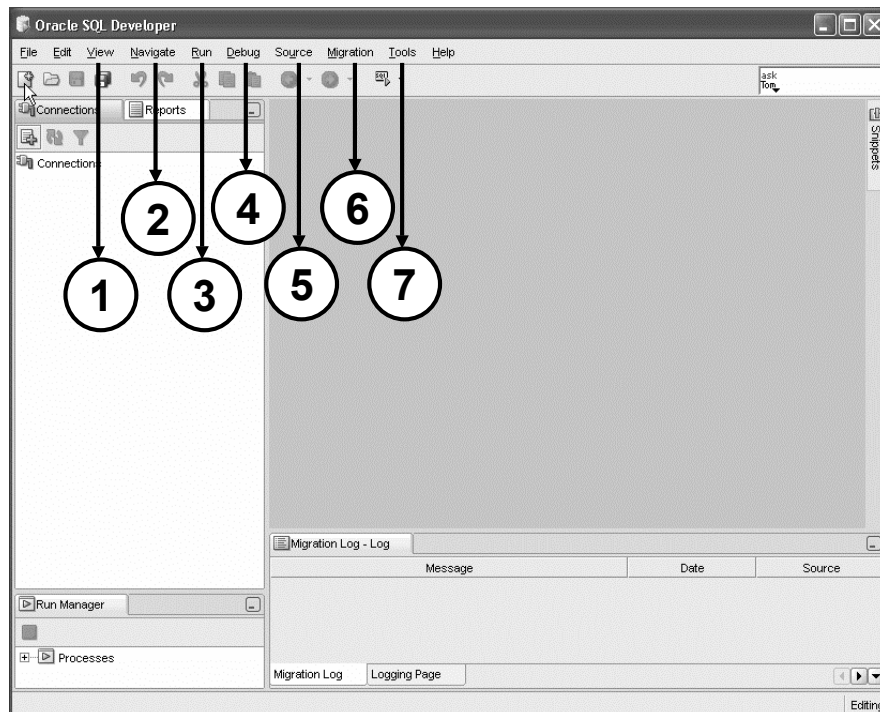
1. Create a folder as **<local drive>:\SQL Developer**.
2. Download the SQL Developer kit from the Oracle SQL Developer home page at:  
[http://www.oracle.com/technology/products/database/sql\\_developer/index.html](http://www.oracle.com/technology/products/database/sql_developer/index.html)
3. Unzip the downloaded SQL Developer kit into the folder created in step 1.

To start SQL Developer, go to **<local drive>:\SQL Developer**, and double-click **sqldeveloper.exe**.

**Note:** In addition to using the Oracle SQL Developer home page to download the software for free, you can use this page to do the following: View the Oracle SQL Developer online Help, presentations, white papers, technical information, lessons, tutorials, and so on.

**Note:** You learned how to create a database connection in the lesson titled “Introduction” of this course. This lesson assumes that you are already connected to the database using SQL Developer. You are now ready to perform database development tasks in the SQL Developer interface.

## Menus for SQL Developer



ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Menus for SQL Developer

SQL Developer has two main navigation tabs:

- **Connections Navigator:** By using this tab, you can browse database objects and users to which you have access.
- **Reporting Tab:** By using this tab, you can run predefined reports or create and add your own reports.

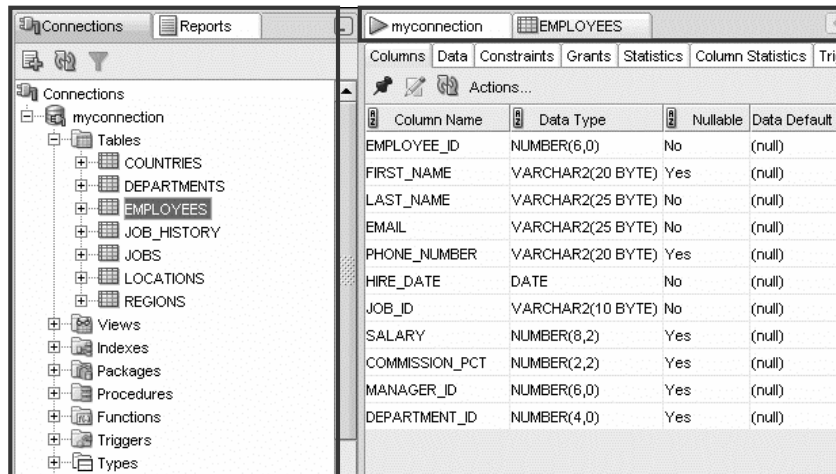
SQL Developer uses the left side for navigation to find and select objects, and the right side to display information about selected objects. You can customize many aspects of the appearance and behavior of SQL Developer by setting preferences.

The menus at the top contain standard entries, plus entries for features specific to the tool.

1. **View:** Contains options that affect what is displayed in the SQL Developer interface
2. **Navigate:** Contains options for navigating to panes and for the execution of subprograms
3. **Run:** Contains the Run File and Execution Profile options that are relevant when a function or procedure is selected
4. **Debug:** Contains options that are relevant when a function or procedure is selected
5. **Source:** Contains options for use when editing functions and procedures
6. **Migration:** Enables you to migrate from another database, such as Microsoft SQL Server and Microsoft Access, to an Oracle database.
7. **Tools:** Invokes SQL Developer tools such as SQL\*Plus, Preferences, and SQL Worksheet

## Browsing Database Objects

- Create a database connection object
- Use the Connections Navigator to:
  - Browse through many objects in a database schema
  - Review the definitions of objects at a glance



ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Browsing Database Objects

After you have created a database connection, you can use the Connections Navigator to browse through many objects in a database schema including Tables, Views, Indexes, Packages, Procedures, Triggers, Types, and so on.

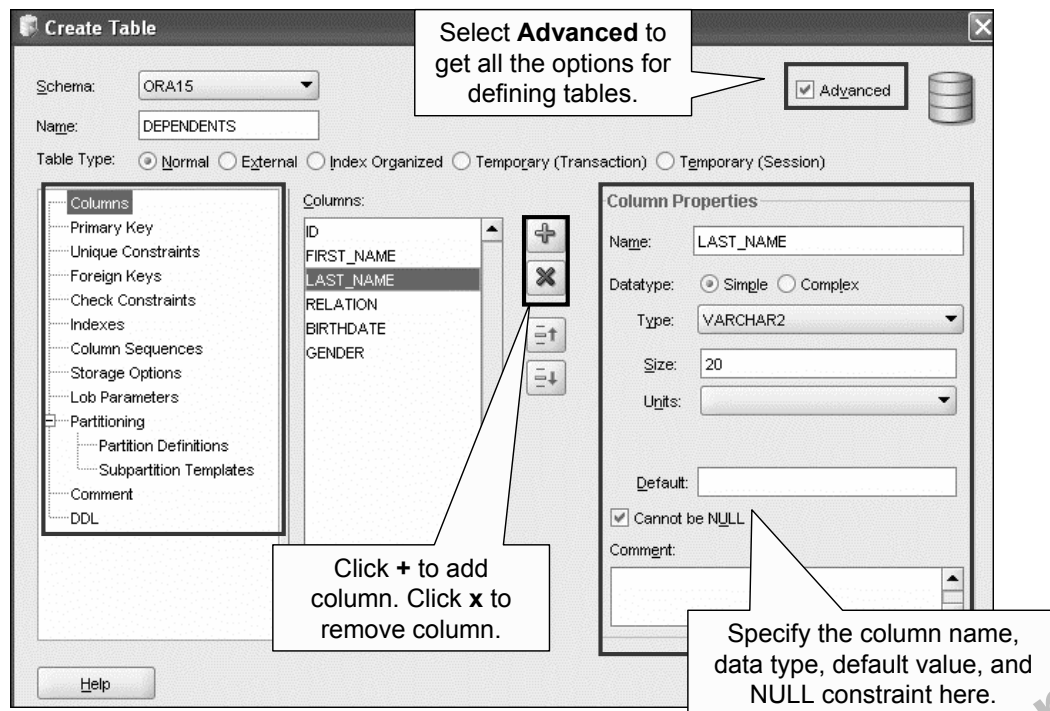
SQL Developer uses the left side for navigation to find and select objects, and the right side to display information about the selected objects. You can customize many aspects of the appearance of SQL Developer by setting preferences.

You can see the definition of the objects broken into tabs of information that is pulled out of the data dictionary. For example, if you select a table in the Navigator, the details about columns, constraints, grants, statistics, triggers, and so on are displayed in an easy-to-read tabbed page.

If you want to see the definition of the EMPLOYEES table as shown in the slide, perform the following steps:

1. Expand the Connections node in the Connections Navigator.
2. Expand **Tables**.
3. Click **EMPLOYEES**. By default, the Columns tab is selected. It shows the column description of the table. Using the Data tab, you can view the tables data and also enter new rows, update data, and commit these changes to the database.

## Creating a New Table: Example



Copyright © 2007, Oracle. All rights reserved.

### Creating a New Table: Example

In the Create Table dialog box, if you do not select the **Advanced** check box, you can create a table quickly by specifying columns and some frequently used features.

If you select the **Advanced** check box, the Create Table dialog box changes to one with multiple options, in which you can specify an extended set of features while creating the table.

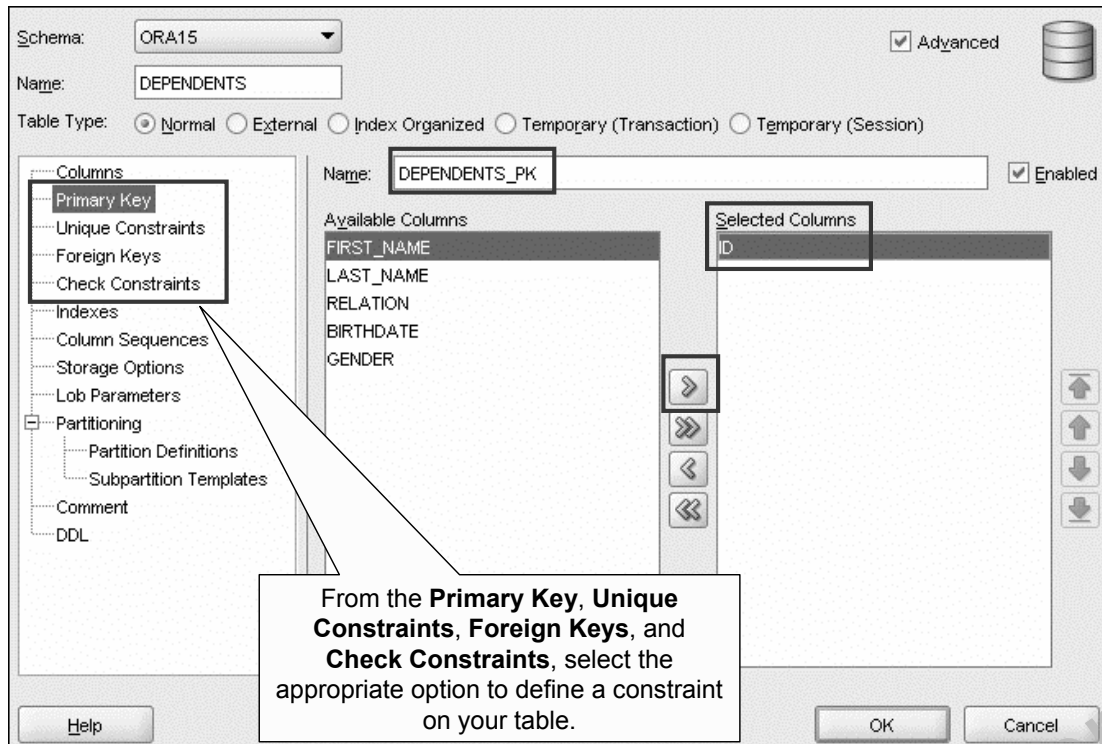
The example in the slide shows creating the DEPENDENTS table by selecting the **Advanced** check box.

To create a new table, perform the following steps:

1. In the Connections Navigator, right-click **Tables**.
2. Select **Create TABLE**.
3. In the Create Table dialog box, select **Advanced**.
4. Specify column information. In the column properties section, specify the column name, data types, default value, and select "Cannot be NULL" to define a NOT NULL column. Click + to add column and click x to remove column. Note, in the screenshot, the LAST\_NAME column is defined as varchar2 (20) and as a NOT NULL column by selecting the "Cannot be Null" option.
5. Click **OK**.

Although it is not required, you should also specify a primary key using the Primary Key tab in the dialog box. Sometimes, you may want to edit the table that you have created. To edit a table, right-click the table in the Connections Navigator and select **Edit**.

## Defining Constraints



Copyright © 2007, Oracle. All rights reserved.

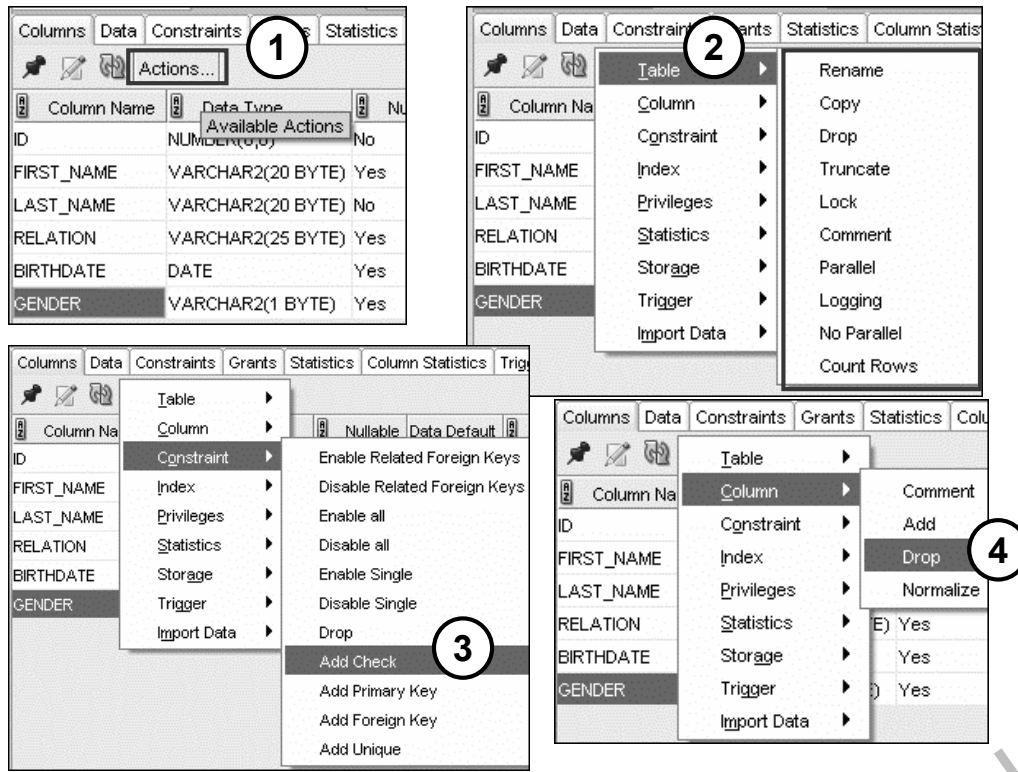
### Defining Constraints

From the same Create Table dialog box, you can define all types of constraints on the table. In the slide, the Primary Key option is selected. A default constraint name, `DEPENDENTS_PK`, is assigned. From the Available Columns list, you can select the appropriate column and click > to move it to the Selected Columns list. In the slide, ID is specified as the primary key. Similarly, click the UNIQUE Constraints option to define a unique constraint or click Foreign Keys to define a foreign key constraint. Click OK.

If you want to define constraints on an existing table, in the Connections Navigator, right-click the table, and select Edit. A similar dialog box as shown in the slide will appear. You can also click the Pencil icon to edit.



## Modifying a Table



ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Modifying a Table

When you create a table, it gets added under the Tables node in the Connections Navigator. You can view the table structure and even modify the column definition of the table. In the Connections Navigator, click the table you want to modify. On the right side, you get a set of tabs that gives you all the details about the table. The Columns tab shows the table structure, whereas the Data tab lets you view the tables data.

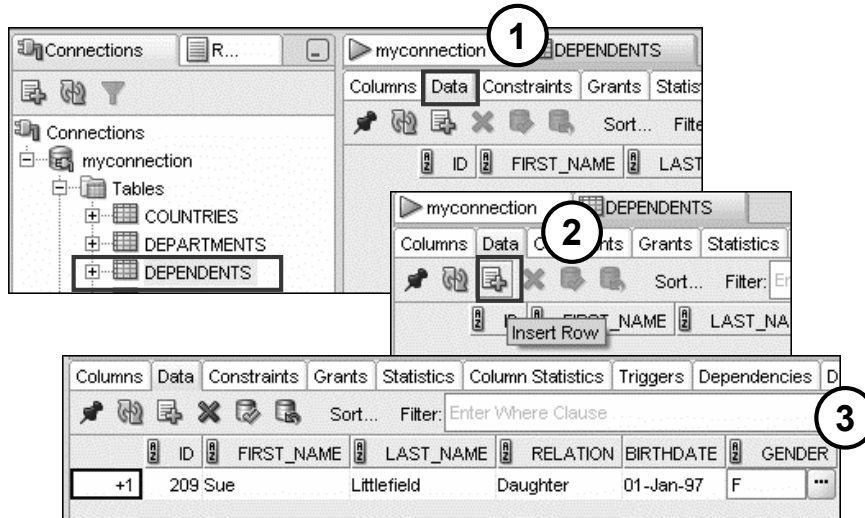
With the Columns tab selected, perform the following steps:

1. Click Actions. A submenu appears.
2. Select Table and a set of menu options appear. If you want to rename the table, select Rename. Alternatively, to delete all the rows from the table, select Truncate. Or, to drop the table, select Drop.
3. If you want to add a check constraint on the GENDER column such that it accepts only “M” and “F” as values, make sure that the GENDER column is selected and then select Constraint and click Add Check.
4. If you want to drop a column, select Column from the Actions submenu and click Drop.

**Note:** You also get the same menu options when you right-click the table in the Connections Navigator.



## Adding Data to a Table



ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Adding Data to a Table

To add rows in the table, perform the following steps:

1. In the Connections Navigator, select the table you want to add data into. Click the **Data** tab.
2. Click the **Insert Row** icon. Insert Row adds an empty row after the selected row, for you to enter new data.
3. Enter the field values for the record. To add another row, click the Insert Row icon again.

## Updating and Deleting Rows

| Columns                     | Data | Constraints | Grants      | Statistics | Column Statistics | Triggers | Dependencies | D |
|-----------------------------|------|-------------|-------------|------------|-------------------|----------|--------------|---|
| Sort... Filter: Enter Where |      |             |             |            |                   |          |              |   |
|                             | ID   | FIRST_NAME  | LAST_NAME   | RELATION   | BIRTHDATE         | GENDER   |              |   |
| +1                          | 209  | Sue         | Littlefield | Sister     | 01-Jan-97         | F        |              |   |

| Constraints                        | Grants | Statistics | Column Statistics | Triggers | Dependencies | Details | Partitions |
|------------------------------------|--------|------------|-------------------|----------|--------------|---------|------------|
| Sort... Filter: Enter Where Clause |        |            |                   |          |              |         |            |
|                                    | ID     | FIRST_NAME | LAST_NAME         | RELATION | BIRTHDATE    | GENDER  |            |
| +1                                 | 209    | Sue        | Littlefield       | Sister   | 01-Jan-97    | F       |            |

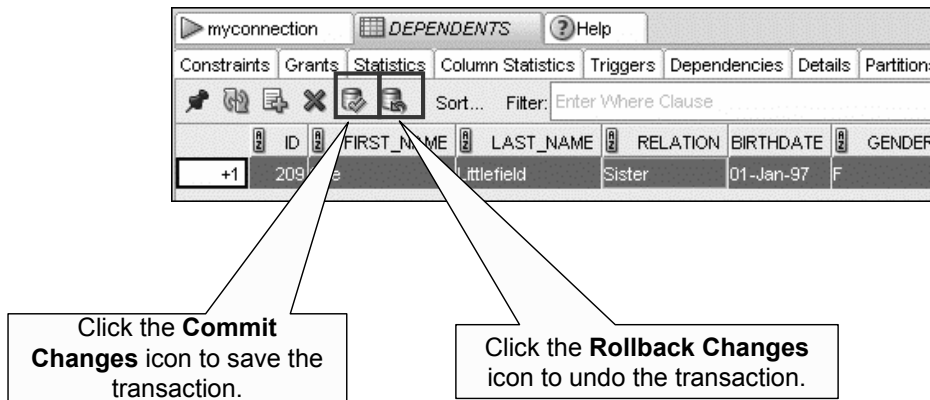
ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Updating and Deleting Rows

1. To update data in a table, make the changes directly in the grid of data values in the Data tab. When you enter a cell in the grid, you can directly edit the data for many data types. For all data types, you can click the ellipsis (...) button to edit the data.
2. Click Delete Selected Row(s) to mark the selected rows for deletion. The actual deletion does not occur until you commit the changes.

## Commit and Rollback



ORACLE

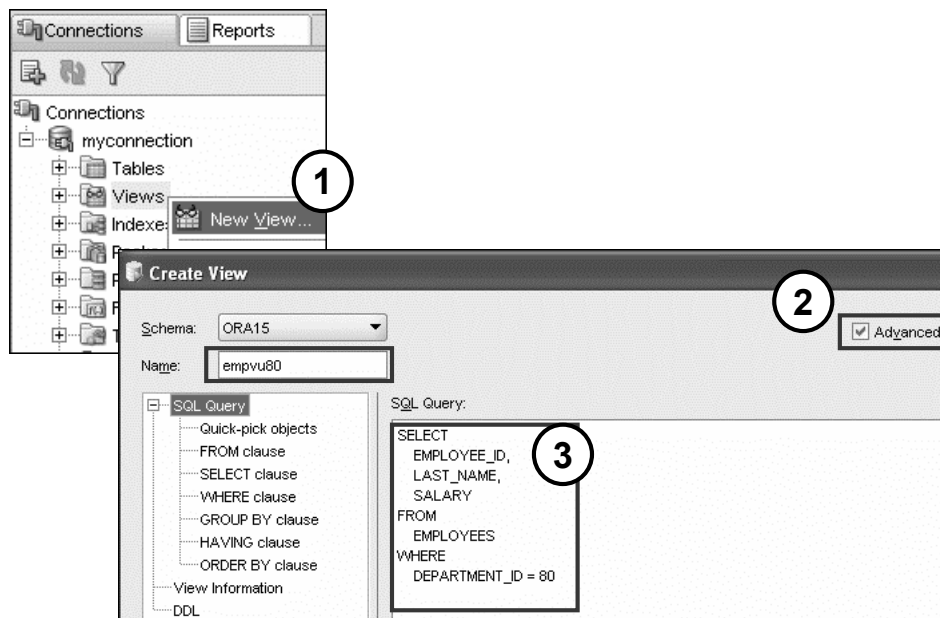
Copyright © 2007, Oracle. All rights reserved.

### Commit and Rollback

After you have inserted new rows or updated any rows, you can either save the changes or undo the changes.

1. To save the changes, click the **Commit Changes** icon. **Commit Changes** ends the current transaction and makes permanent all changes performed in the transaction.
2. To undo the changes, click the **Rollback Changes** icon. **Rollback Changes** undoes any work done in the current transaction.

## Creating a View



ORACLE

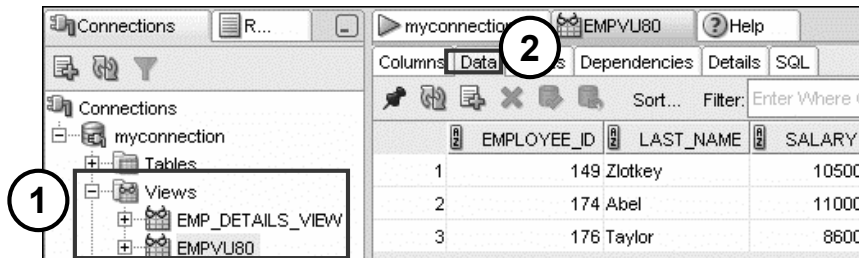
Copyright © 2007, Oracle. All rights reserved.

### Creating a View

Views are virtual tables (analogous to queries in some database products) that select data from one or more underlying tables. To create a view, perform the following steps:

1. In the Connections Navigator, right-click **Views** and select **New View**.
2. In the Create View dialog box, select the Advanced option. If this option is selected, the dialog box changes to include a pane that provides an extended set of features for creating the view.
3. In the SQL Query box, you can directly enter the SQL query. Alternatively, you can expand the **SQL Query** node and use its options individually to define the view step by step. Click **OK**. The view gets added in the Views node in the Connections Navigator.

## Retrieving Data From a View



ORACLE

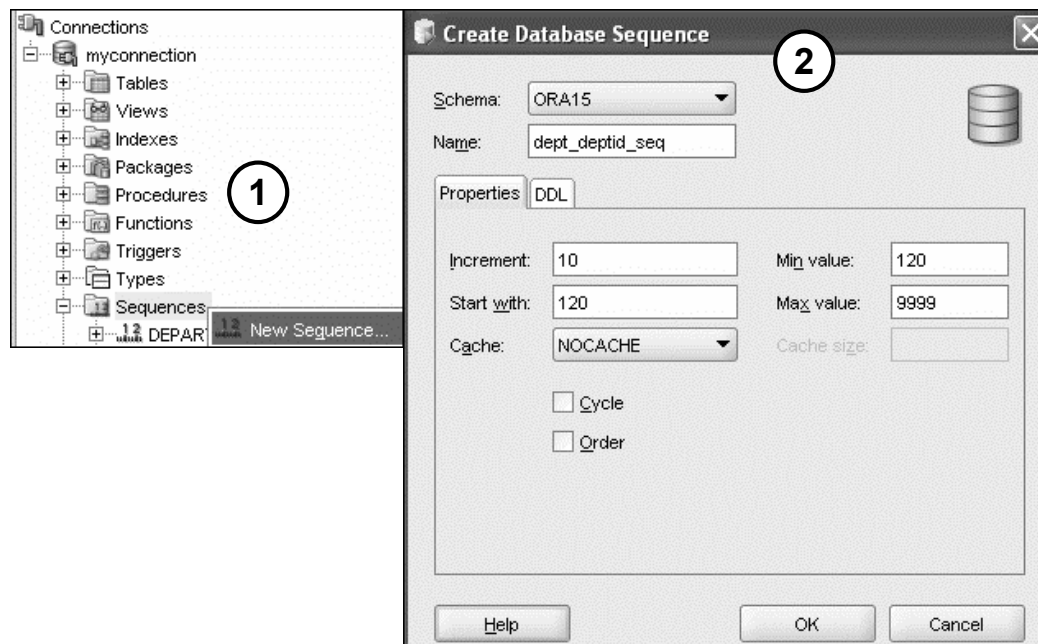
Copyright © 2007, Oracle. All rights reserved.

### Retrieving Data From a View

To retrieve data from a view, perform the following steps:

1. In the Connections Navigator, expand **Views**. Select the view you want to view the data of.
2. Click the **Data** tab to view its data.

## Creating a Sequence



ORACLE

Copyright © 2007, Oracle. All rights reserved.

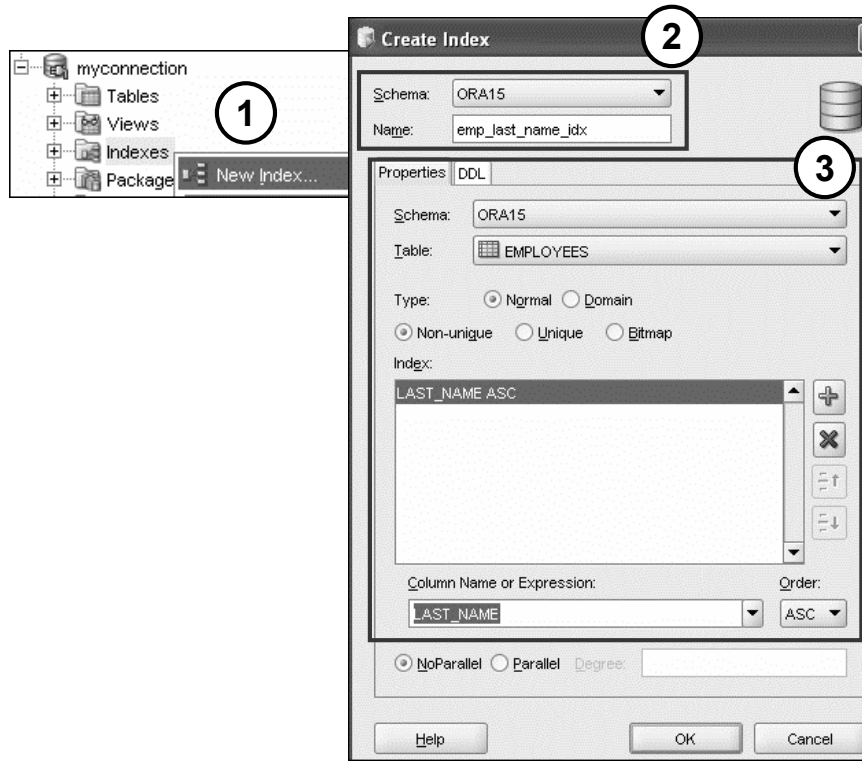
### Creating a Sequence

Sequences are used to generate unique integers. You can use sequences to automatically generate primary key values. To create a sequence, perform the following steps:

1. In the Connections Navigator, expand **Sequences**. Select **New Sequence**.
2. In the Create Database Sequence dialog box, specify the schema name and the sequence name. On the Properties tabbed page, specify the increment value, the minimum value, the “start with” value, the maximum value, and so on. You can review and make changes to the code for this sequence by clicking the **DDL** tab. Click OK.

**Note:** You can edit, drop, or alter a sequence by using the menu options when you right-click the sequence.

## Creating Indexes



Copyright © 2007, Oracle. All rights reserved.

### Creating Indexes

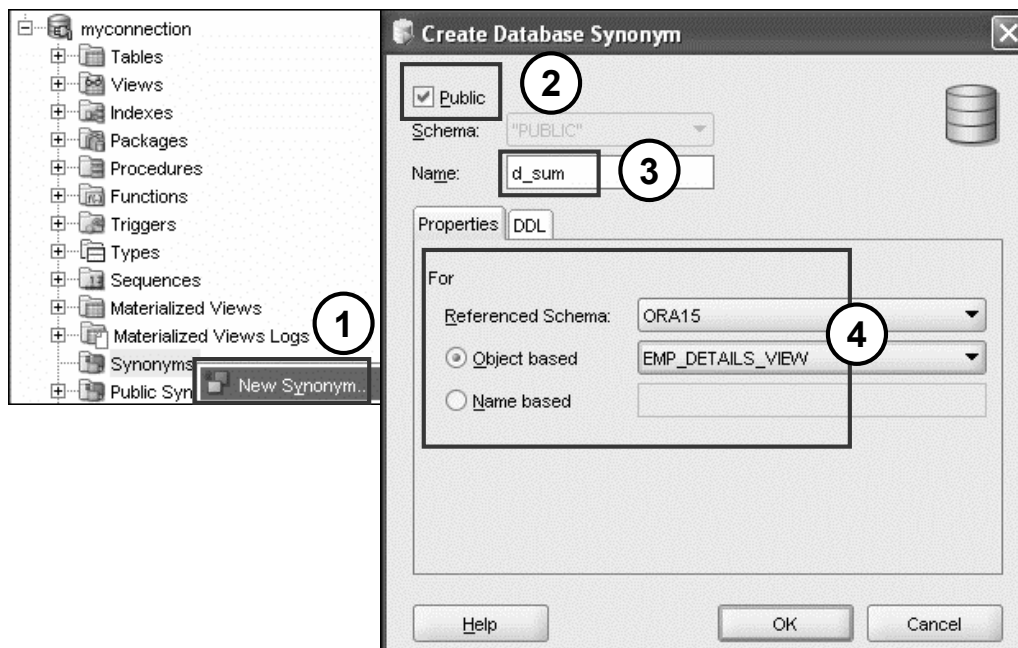
An index is a database object that contains an entry for each value that appears in the indexed columns of the table or cluster and provides direct, fast access to rows. Indexes are automatically created on primary key columns; however, you must create indexes on other columns to gain the benefits of indexing.

To create an index, perform the following steps:

1. Right-click **Indexes** in the Connections Navigator. Select **New Index**.
2. In the Create Index dialog box, select the schema that will own the index. Specify the name of the index.
3. On the Properties tabbed page, specify the schema that owns the table you want to index. Select the table that will be associated with the index. Select the type of index. Add a list of index expressions—that is, the table columns or column expressions in the index. To add an index expression, click the Add Column Expression (+) icon; this adds a column name here and in Column Expression, where you can edit it. You can also specify the order of the index.

**Note:** You right-click an index and then use the menu options to edit, drop, rebuild, and rename the index.

## Creating a Synonym



### Creating a Synonym

Synonyms provide alternative names for tables, views, sequences, or other synonyms. The Connections navigator has a Synonyms node for all synonyms (public and private) owned by the user associated with the specified connection, and a Public Synonyms node for all public synonyms on the database associated with the connection.

To create a synonym, perform the following steps:

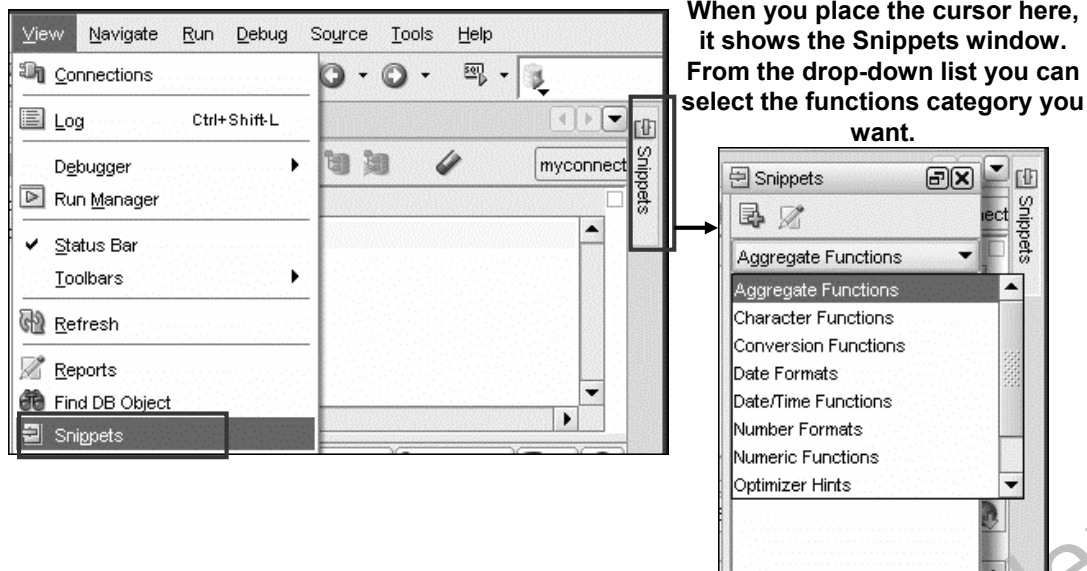
1. In the Connections Navigator, right-click Synonyms. Select New Synonym.
2. In the Create Database Synonym dialog box, select Public if you want the synonym to be accessible to all users. Private synonyms are accessible only within its schema.
3. Enter the name of the synonym.
4. On the Properties tabbed page, select the schema that contains the object for which this synonym is being defined. You can enter the name of the object directly (if “Name based” is selected) or you can select “Object based” to get a drop-down list of all the objects in the schema. Select the object from the list. Click OK.

**Note:** Public synonyms get added in a different node in the Connections Navigator. So, look for your public synonyms in the Public Synonyms node. To drop a synonym, simply right-click the synonym and select Drop from the menu.



## Using Snippets

Snippets are code fragments that may be just syntax or examples.



When you place the cursor here, it shows the Snippets window. From the drop-down list you can select the functions category you want.

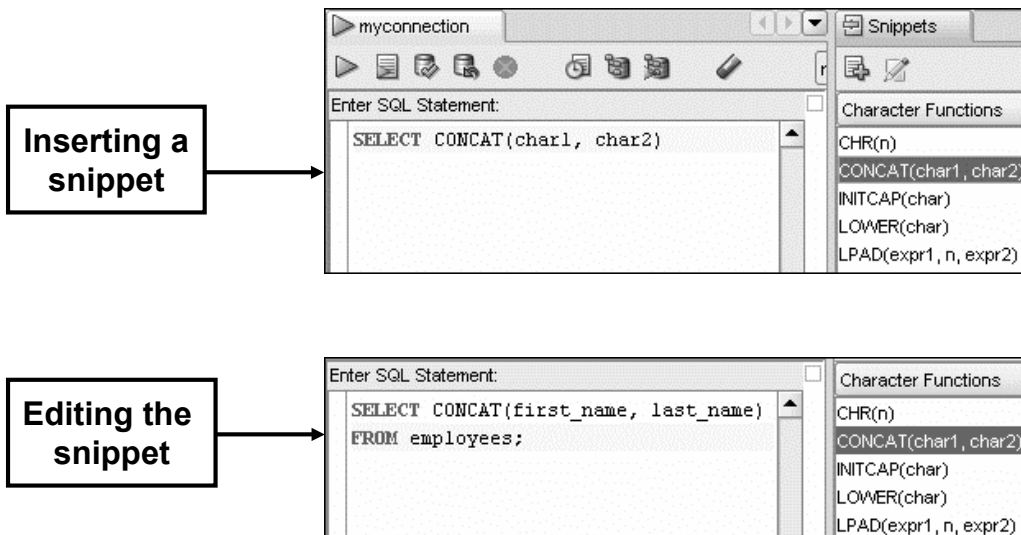
### Using Snippets

You may want to use certain code fragments when you are using SQL Worksheet or creating or editing a PL/SQL function or procedure. SQL Developer has the feature called Snippets. Snippets are code fragments, such as SQL functions, Optimizer hints, and miscellaneous PL/SQL programming techniques. You can drag snippets into the Editor window.

To display Snippets, select **View > Snippets**.

The Snippets window is displayed on the right side. You can use the drop-down list to select a group. A Snippets button is placed in the right window margin, so that you can display the Snippets window if it becomes hidden.

## Using Snippets: Example



ORACLE

Copyright © 2007, Oracle. All rights reserved.

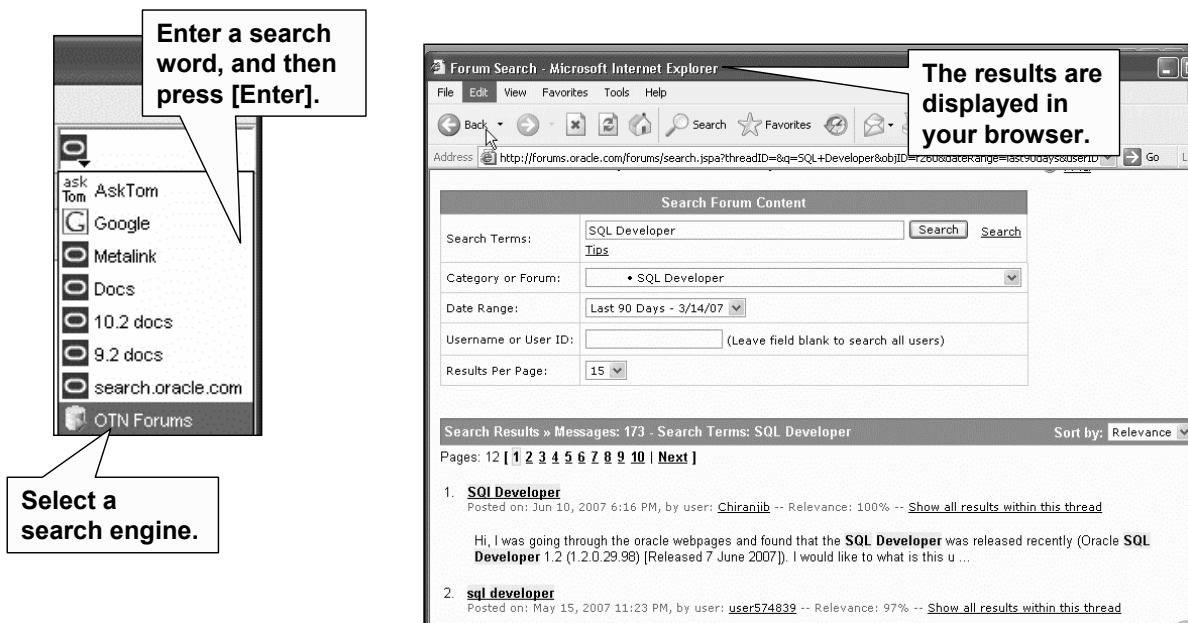
### Using Snippets: Example

To insert a Snippet into your code in a SQL Worksheet or in a PL/SQL function or procedure, drag the snippet from the Snippets window into the desired place in your code. Then you can edit the syntax so that the SQL function is valid in the current context. To see a brief description of a SQL function in a tool tip, place the cursor over the function name.

The example in the slide shows that `CONCAT(char1, char2)` is dragged from the Character Functions group in the Snippets window. Then the `CONCAT` function syntax is edited and the rest of the statement is added such as in the following:

```
SELECT CONCAT(first_name, last_name)
FROM employees;
```

# Browsing Through the Available Search Engines



Copyright © 2007, Oracle. All rights reserved.

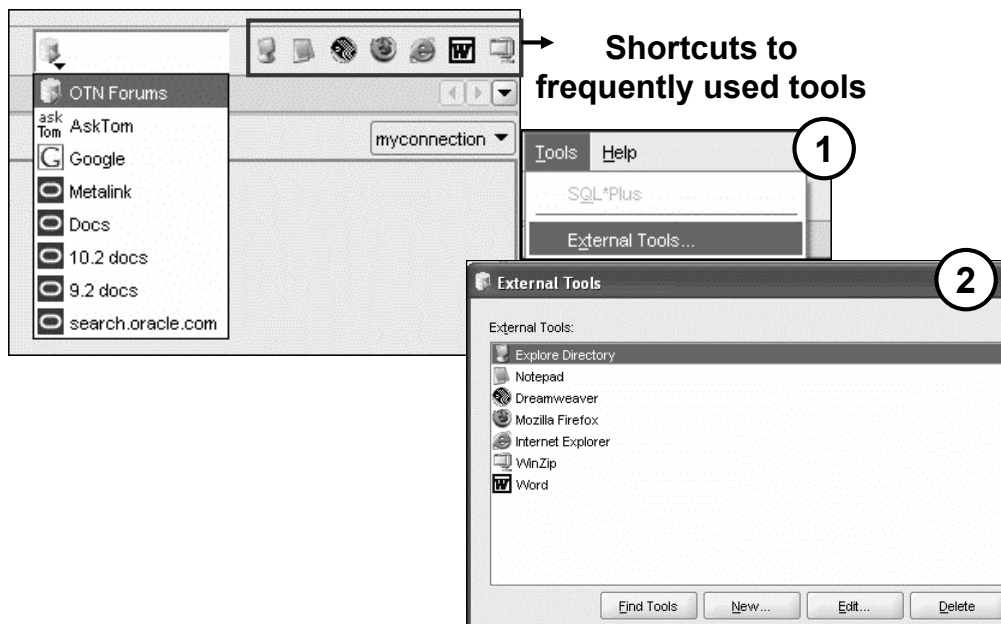
## Browsing Through the Available Search Engines

In SQL Developer 1.2, you have links to popular search engines and discussion forums. You can use the following search engines:

- Ask Tom
- Google
- MetaLink
- Docs
- 10.2 docs
- 9.2 docs
- Search.oracle.com
- OTN Forums

In the example in the slide, the OTN Forums are searched for the word “SQL Developer.” A resulting list displays in your browser window.

## Adding External Tools



ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Adding External Tools

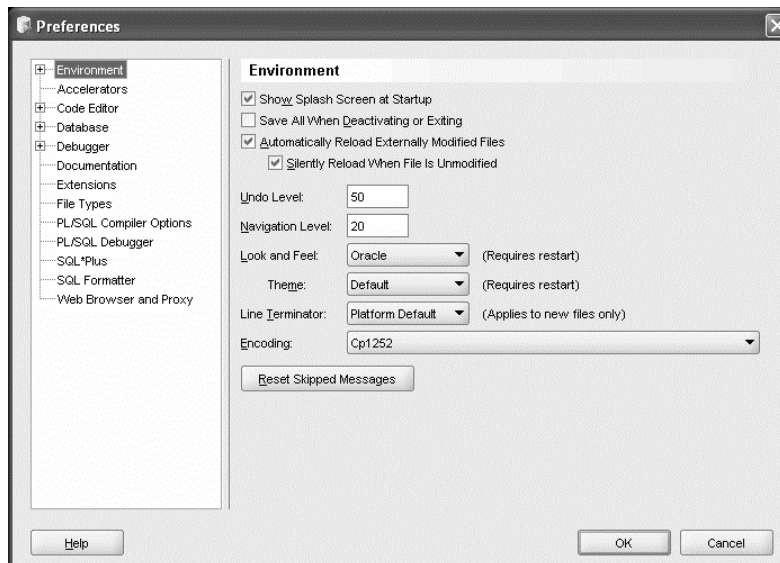
To enhance productivity of the SQL developers, SQL Developer 1.2 has the option to add external tools. You have shortcut icons to some of the frequently used tools such as Notepad, Microsoft Word, Dreamweaver, and so on.

You can add external tools to the existing list or even delete shortcuts to tools that you do not use frequently. To do so, perform the following steps:

1. From Tools menu, select **External Tools**.
2. In the External Tools dialog box, select **New** to add new tools. Select **Delete** to remove any tool from the list.

## Setting Preferences

- Customize the SQL Developer interface and environment.
- In the Tools menu, select Preferences.



ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Setting Preferences

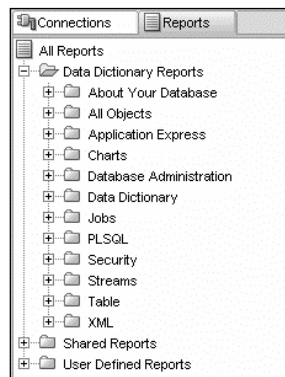
You can customize many aspects of the SQL Developer interface and environment by modifying SQL Developer preferences according to your preferences and needs. To modify SQL Developer preferences, select Tools and then Preferences.

The preferences are grouped in the following categories:

- Environment
- Accelerators (keyboard shortcuts)
- Code Editor
- Database
- Debugger
- Documentation
- Extensions
- File Types
- Migration
- PL/SQL Compilers
- PL/SQL Debugger, and so on

## Creating Reports

- SQL Developer provides you with a number of predefined reports about your database and objects.
- The Reports are organized into categories.
- You can create your own customized reports too.



ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Creating Reports

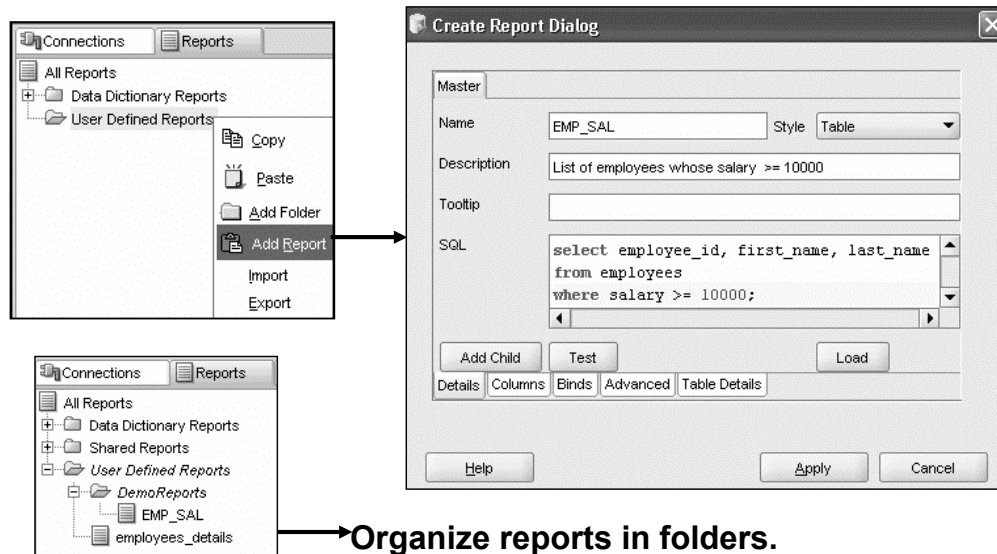
SQL Developer provides many reports about the database and its objects. These reports are grouped into the following categories:

- About Your Database reports
- Object reports
- Application Express reports
- Charts
- Database Administration reports
- Data Dictionary reports
- Jobs reports
- PL/SQL reports
- Security reports
- Streams reports
- Table reports
- XML reports

To display a report, click the Reports tabbed page and then select the report type. You can also create your own user-defined reports.

# Creating a User-Defined Report

Create and save user-defined reports for repeated use.



Organize reports in folders.

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Creating a User-Defined Report

User-defined reports are any reports that are created by SQL Developer users.

To create a user-defined report, perform the following steps:

- Right-click the **User Defined Reports** node under Reports, and select **Add Report**.
- In the **Create Report Dialog** box, specify the report name and the SQL query to retrieve information for the report. Then click **Apply**.

In the example in the slide, the report name is specified as EMP\_SAL. An optional description is provided indicating that the report contains details of employees whose salaries are greater than or equal to \$10,000. The complete SQL statement for retrieving the information to be displayed in the user-defined report is specified in the SQL box. You can also include an optional tool tip to be displayed when the cursor stays briefly over the report name in the Reports navigator display.

You can organize user-defined reports in folders, and you can create a hierarchy of folders and subfolders. To create a folder for user-defined reports, right-click the **User Defined Reports** node or any folder name under that node and select **Add Folder**. Information about user-defined reports, including any folders for these reports, is stored in a file named `UserReports.xml` under the directory for user-specific information.

## Summary

In this appendix, you should have learned how to use SQL Developer to do the following:

- Browse, create, and edit database objects using the SQL Developer GUI interface
- Insert, update, and delete rows from a table using the SQL Developer GUI interface

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Summary

SQL Developer is a free graphical tool to simplify database development tasks. Using SQL Developer, you can browse, create, and edit database objects. Using quick menu options and the graphical interface, you can perform the same database development tasks that you can using SQL statements.



# D

## Using SQL\*Plus

ORACLE<sup>®</sup>

Copyright © 2007, Oracle. All rights reserved.

Oracle Internal & Oracle Academy  
Use Only

## Objectives

After completing this appendix, you should be able to do the following:

- Log in to SQL\*Plus
- Edit SQL commands
- Format output using SQL\*Plus commands
- Interact with script files

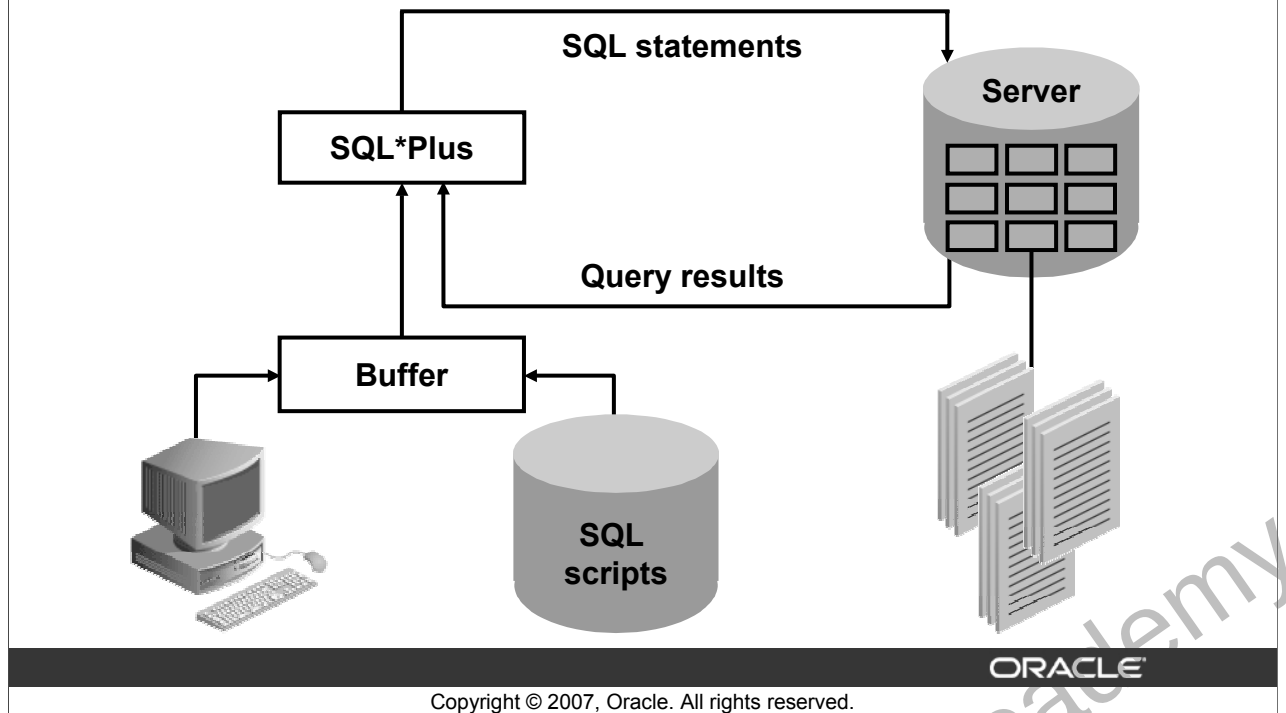
ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Objectives

You may want to create `SELECT` statements that can be used again and again. This appendix also covers the use of SQL\*Plus commands to execute SQL statements. You learn how to format output using SQL\*Plus commands, edit SQL commands, and save scripts in SQL\*Plus.

## SQL and SQL\*Plus Interaction



### SQL and SQL\*Plus

SQL is a command language used for communication with the Oracle server from any tool or application. Oracle SQL contains many extensions. When you enter a SQL statement, it is stored in a part of memory called the *SQL buffer* and remains there until you enter a new SQL statement. SQL\*Plus is an Oracle tool that recognizes and submits SQL statements to the Oracle9i Server for execution. It contains its own command language.

#### Features of SQL

- Can be used by a range of users, including those with little or no programming experience
- Is a nonprocedural language
- Reduces the amount of time required for creating and maintaining systems
- Is an English-like language

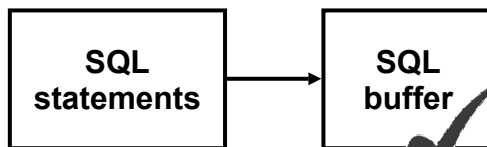
#### Features of SQL\*Plus

- Accepts ad hoc entry of statements
- Accepts SQL input from files
- Provides a line editor for modifying SQL statements
- Controls environmental settings
- Formats query results into basic reports
- Accesses local and remote databases

## SQL Statements Versus SQL\*Plus Commands

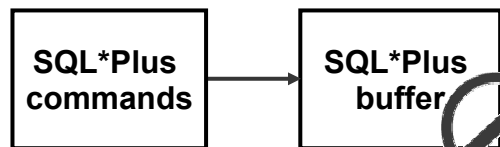
### SQL

- A language
- ANSI-standard
- Keywords cannot be abbreviated
- Statements manipulate data and table definitions in the database



### SQL\*Plus

- An environment
- Oracle-proprietary
- Keywords can be abbreviated
- Commands do not allow manipulation of values in the database



ORACLE

Copyright © 2007, Oracle. All rights reserved.

### SQL and SQL\*Plus (continued)

The following table compares SQL and SQL\*Plus:

| SQL                                                                   | SQL*Plus                                                                           |
|-----------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Is a language for communicating with the Oracle server to access data | Recognizes SQL statements and sends them to the server                             |
| Is based on American National Standards Institute (ANSI)–standard SQL | Is the Oracle-proprietary interface for executing SQL statements                   |
| Manipulates data and table definitions in the database                | Does not allow manipulation of values in the database                              |
| Is entered into the SQL buffer on one or more lines                   | Is entered one line at a time, not stored in the SQL buffer                        |
| Does not have a continuation character                                | Uses a dash (–) as a continuation character if the command is longer than one line |
| Cannot be abbreviated                                                 | Can be abbreviated                                                                 |
| Uses a termination character to execute commands immediately          | Does not require termination characters; executes commands immediately             |
| Uses functions to perform some formatting                             | Uses commands to format data                                                       |

## Overview of SQL\*Plus

- Log in to SQL\*Plus.
- Describe the table structure.
- Edit your SQL statement.
- Execute SQL from SQL\*Plus.
- Save SQL statements to files and append SQL statements to files.
- Execute saved files.
- Load commands from file to buffer to edit.

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### SQL\*Plus

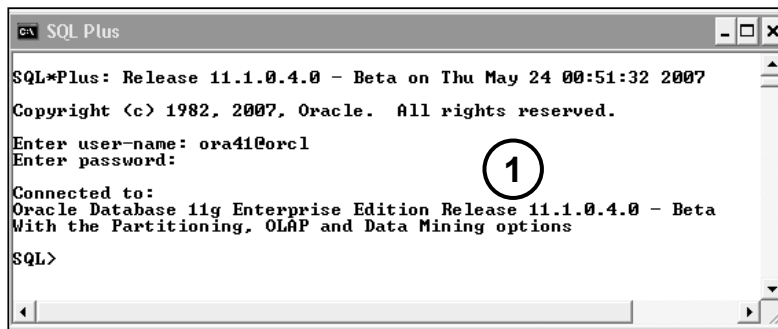
SQL\*Plus is an environment in which you can:

- Execute SQL statements to retrieve, modify, add, and remove data from the database
- Format, perform calculations on, store, and print query results in the form of reports
- Create script files to store SQL statements for repeated use in the future

SQL\*Plus commands can be divided into the following main categories:

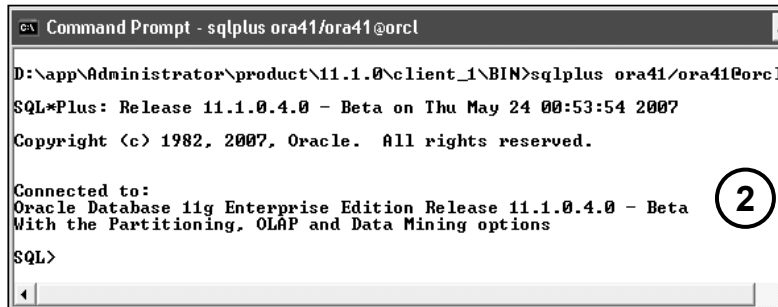
| Category          | Purpose                                                                                              |
|-------------------|------------------------------------------------------------------------------------------------------|
| Environment       | Affect the general behavior of SQL statements for the session                                        |
| Format            | Format query results                                                                                 |
| File manipulation | Save, load, and run script files                                                                     |
| Execution         | Send SQL statements from the SQL buffer to the Oracle server                                         |
| Edit              | Modify SQL statements in the buffer                                                                  |
| Interaction       | Create and pass variables to SQL statements, print variable values, and print messages to the screen |
| Miscellaneous     | Connect to the database, manipulate the SQL*Plus environment, and display column definitions         |

## Logging in to SQL\*Plus



```
SQL*Plus: Release 11.1.0.4.0 - Beta on Thu May 24 00:51:32 2007
Copyright (c) 1982, 2007, Oracle. All rights reserved.
Enter user-name: ora41@orcl
Enter password:
Connected to:
Oracle Database 11g Enterprise Edition Release 11.1.0.4.0 - Beta
With the Partitioning, OLAP and Data Mining options
SQL>
```

```
sqlplus [username[/password[@database]]]
```



```
D:\app\Administrator\product\11.1.0\client_1\BIN>sqlplus ora41/ora41@orcl
SQL*Plus: Release 11.1.0.4.0 - Beta on Thu May 24 00:53:54 2007
Copyright (c) 1982, 2007, Oracle. All rights reserved.
Connected to:
Oracle Database 11g Enterprise Edition Release 11.1.0.4.0 - Beta
With the Partitioning, OLAP and Data Mining options
SQL>
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Logging in to SQL\*Plus

How you invoke SQL\*Plus depends on which type of operating system or Windows environment you are running.

To log in from a Windows environment:

1. Select Start > Programs > Oracle > Application Development > SQL\*Plus.
2. Enter the username, password, and database name.

To log in from a command-line environment:

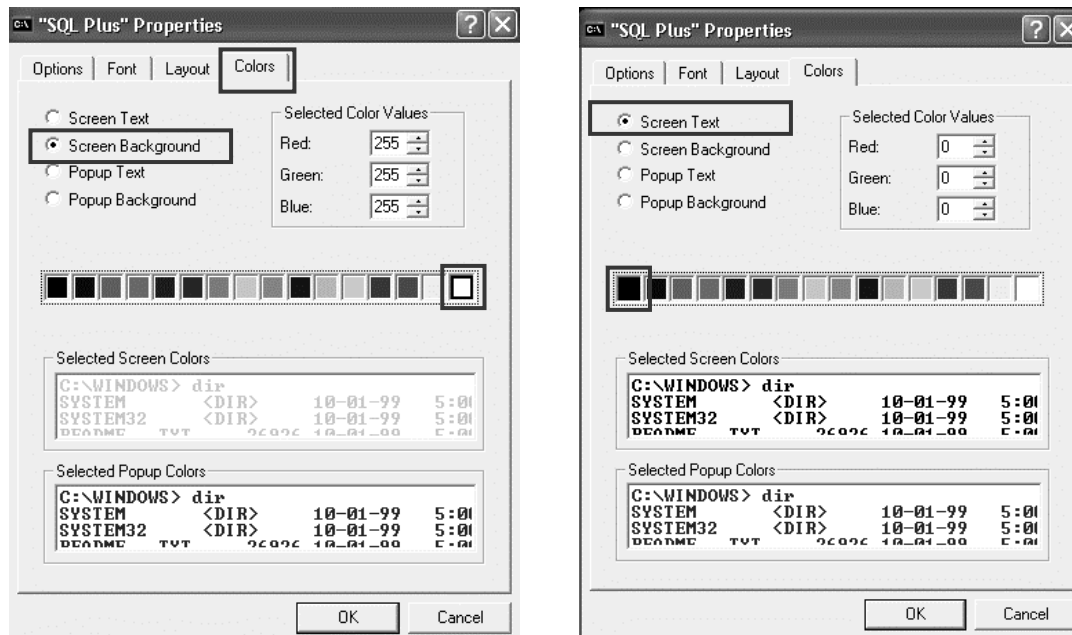
1. Log on to your machine.
2. Enter the sqlplus command shown in the slide.

In the syntax:

*username* Your database username  
*password* Your database password (Your password is visible if you enter it here.)  
*@database* The database connect string

**Note:** To ensure the integrity of your password, do not enter it at the operating system prompt. Instead, enter only your username. Enter your password at the password prompt.

## Changing the Settings of SQL\*Plus Environment



ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Changing Settings of SQL\*Plus Environment

You can optionally change the look of the SQL\*Plus environment by using the SQL\*Plus Properties dialog box.

In the SQL\*Plus window, right-click the title bar and in the shortcut menu that appears, select Properties. You can then use the Colors tab of the SQL\*Plus Properties dialog box to set Screen Background and Screen Text.

## Displaying Table Structure

Use the SQL\*Plus DESCRIBE command to display the structure of a table:

```
DESC[RIBE] tablename
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Displaying Table Structure

In SQL\*Plus, you can display the structure of a table using the DESCRIBE command. The result of the command is a display of column names and data types as well as an indication if a column must contain data.

In the syntax:

*tablename* The name of any existing table, view, or synonym that is accessible to the user

To describe the DEPARTMENTS table, use this command:

```
SQL> DESCRIBE DEPARTMENTS
Name Null? Type

DEPARTMENT_ID NOT NULL NUMBER(4)
DEPARTMENT_NAME NOT NULL VARCHAR2(30)
MANAGER_ID NUMBER(6)
LOCATION_ID NUMBER(4)
```



## Displaying Table Structure

```
DESCRIBE departments
```

| Name            | Null?    | Type         |
|-----------------|----------|--------------|
| DEPARTMENT_ID   | NOT NULL | NUMBER(4)    |
| DEPARTMENT_NAME | NOT NULL | VARCHAR2(30) |
| MANAGER_ID      |          | NUMBER(6)    |
| LOCATION_ID     |          | NUMBER(4)    |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Displaying Table Structure (continued)

The example in the slide displays the information about the structure of the DEPARTMENTS table. In the result:

Null? : Specifies whether a column must contain data (NOT NULL indicates that a column must contain data.)

Type : Displays the data type for a column

## SQL\*Plus Editing Commands

- A[PPEND] *text*
- C[HANGE] / *old* / *new*
- C[HANGE] / *text* /
- CL[EAR] BUFF[ER]
- DEL
- DEL *n*
- DEL *m n*

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### SQL\*Plus Editing Commands

SQL\*Plus commands are entered one line at a time and are not stored in the SQL buffer.

| Command                            | Description                                               |
|------------------------------------|-----------------------------------------------------------|
| A[PPEND] <i>text</i>               | Adds <i>text</i> to the end of the current line           |
| C[HANGE] / <i>old</i> / <i>new</i> | Changes <i>old</i> text to <i>new</i> in the current line |
| C[HANGE] / <i>text</i> /           | Deletes <i>text</i> from the current line                 |
| CL[EAR] BUFF[ER]                   | Deletes all lines from the SQL buffer                     |
| DEL                                | Deletes current line                                      |
| DEL <i>n</i>                       | Deletes line <i>n</i>                                     |
| DEL <i>m n</i>                     | Deletes lines <i>m</i> to <i>n</i> inclusive              |

#### Guidelines

- If you press [Enter] before completing a command, SQL\*Plus prompts you with a line number.
- You terminate the SQL buffer either by entering one of the terminator characters (semicolon or slash) or by pressing [Enter] twice. The SQL prompt then appears.

## SQL\*Plus Editing Commands

- I [NPUT]
- I [NPUT] *text*
- L [IST]
- L [IST] *n*
- L [IST] *m n*
- R [UN]
- *n*
- *n text*
- 0 *text*

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### SQL\*Plus Editing Commands (continued)

| Command              | Description                                               |
|----------------------|-----------------------------------------------------------|
| I [NPUT]             | Inserts an indefinite number of lines                     |
| I [NPUT] <i>text</i> | Inserts a line consisting of <i>text</i>                  |
| L [IST]              | Lists all lines in the SQL buffer                         |
| L [IST] <i>n</i>     | Lists one line (specified by <i>n</i> )                   |
| L [IST] <i>m n</i>   | Lists a range of lines ( <i>m</i> to <i>n</i> ) inclusive |
| R [UN]               | Displays and runs the current SQL statement in the buffer |
| <i>n</i>             | Specifies the line to make the current line               |
| <i>n text</i>        | Replaces line <i>n</i> with <i>text</i>                   |
| 0 <i>text</i>        | Inserts a line before line 1                              |

**Note:** You can enter only one SQL\*Plus command for each SQL prompt. SQL\*Plus commands are not stored in the buffer. To continue a SQL\*Plus command on the next line, end the first line with a hyphen (-).

## Using LIST, n, and APPEND

```
LIST
1 SELECT last_name
2* FROM employees
```

```
1
1* SELECT last_name
```

```
A , job_id
1* SELECT last_name, job_id
```

```
LIST
1 SELECT last_name, job_id
2* FROM employees
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Using LIST, n, and APPEND

- Use the L[IST] command to display the contents of the SQL buffer. The asterisk (\*) beside line 2 in the buffer indicates that line 2 is the current line. Any edits that you made apply to the current line.
- Change the number of the current line by entering the number (n) of the line that you want to edit. The new current line is displayed.
- Use the A[PPEND] command to add text to the current line. The newly edited line is displayed. Verify the new contents of the buffer by using the LIST command.

**Note:** Many SQL\*Plus commands, including LIST and APPEND, can be abbreviated to just their first letter. LIST can be abbreviated to L; APPEND can be abbreviated to A.

## Using the CHANGE Command

```
LIST
1* SELECT * from employees
```

```
c/employees/departments
1* SELECT * from departments
```

```
LIST
1* SELECT * from departments
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Using the CHANGE Command

- Use L[IST] to display the contents of the buffer.
- Use the C[HANGE] command to alter the contents of the current line in the SQL buffer. In this case, replace the employees table with the departments table. The new current line is displayed.
- Use the L[IST] command to verify the new contents of the buffer.

## SQL\*Plus File Commands

- `SAVE filename`
- `GET filename`
- `START filename`
- `@ filename`
- `EDIT filename`
- `SPOOL filename`
- `EXIT`

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### SQL\*Plus File Commands

SQL statements communicate with the Oracle server. SQL\*Plus commands control the environment, format query results, and manage files. You can use the commands described in the following table:

| Command                                                                          | Description                                                                                                                                                      |
|----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>SAV[E] filename [.ext]</code><br><code>[REP[LACE] APP[END]]</code>         | Saves current contents of SQL buffer to a file. Use APPEND to add to an existing file; use REPLACE to overwrite an existing file. The default extension is .sql. |
| <code>GET filename [.ext]</code>                                                 | Writes the contents of a previously saved file to the SQL buffer. The default extension for the file name is .sql.                                               |
| <code>STA[RT] filename [.ext]</code>                                             | Runs a previously saved command file                                                                                                                             |
| <code>@ filename</code>                                                          | Runs a previously saved command file (same as START)                                                                                                             |
| <code>ED[IT]</code>                                                              | Invokes the editor and saves the buffer contents to a file named <code>afiedt.buf</code>                                                                         |
| <code>ED[IT] [filename [.ext]]</code>                                            | Invokes the editor to edit the contents of a saved file                                                                                                          |
| <code>SPO[OL]</code><br><code>[filename [.ext]  </code><br><code>OFF OUT]</code> | Stores query results in a file. OFF closes the spool file. OUT closes the spool file and sends the file results to the printer.                                  |
| <code>EXIT</code>                                                                | Quits SQL*Plus                                                                                                                                                   |

## Using the SAVE, START, and EDIT Commands

```
LIST
1 SELECT last_name, manager_id, department_id
2* FROM employees
```

```
SAVE my_query
Created file my_query
```

```
START my_query

LAST_NAME MANAGER_ID DEPARTMENT_ID

King 100 90
Kochhar 100 90
...
107 rows selected.
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Using the SAVE, START, and EDIT Commands

#### SAVE

Use the SAVE command to store the current contents of the buffer in a file. In this way, you can store frequently used scripts for use in the future.

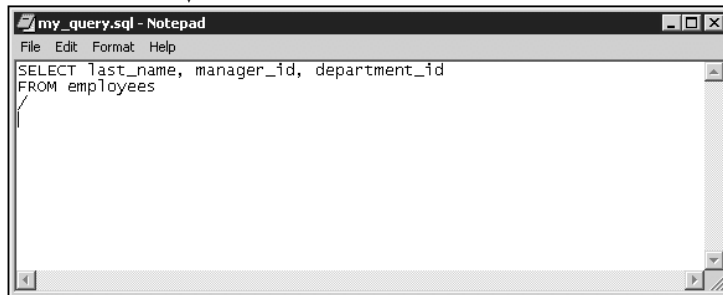
#### START

Use the START command to run a script in SQL\*Plus. You can also, alternatively, use the symbol @ to run a script.

```
@my_query
```

## Using the SAVE, START, and EDIT Commands

EDIT my\_query



ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Using the SAVE, START, and EDIT Commands (continued)

#### EDIT

Use the EDIT command to edit an existing script. This opens an editor with the script file in it. When you have made the changes, quit the editor to return to the SQL\*Plus command line.

**Note:** The "/" is a delimiter that signifies the end of the statement. When encountered in a file, SQL\*Plus runs the statement prior to this delimiter. The delimiter must be the first character of a new line immediately following the statement.



## SERVEROUTPUT Command

- Use the `SET SERVEROUT [PUT]` command to control whether to display the output of stored procedures or PL/SQL blocks in SQL\*Plus.
- The `DBMS_OUTPUT` line length limit is increased from 255 bytes to 32767 bytes.
- The default size is now unlimited.
- Resources are not pre-allocated when `SERVEROUTPUT` is set.
- Because there is no performance penalty, use `UNLIMITED` unless you want to conserve physical memory.

```
SET SERVEROUT [PUT] {ON | OFF} [SIZE {n | UNLIMITED}]
[FOR [MAT] {WRAPPED | WORD_WRAPPED} | TRUNCATED]
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### SERVEROUTPUT Command

Most of the PL/SQL programs perform input and output through SQL statements, to store data in database tables or query those tables. All other PL/SQL I/O is done through APIs that interact with other programs. For example, the `DBMS_OUTPUT` package has procedures such as `PUT_LINE`. To see the result outside of PL/SQL requires another program, such as SQL\*Plus, to read and display the data passed to `DBMS_OUTPUT`.

SQL\*Plus does not display `DBMS_OUTPUT` data unless you first issue the SQL\*Plus command `SET SERVEROUTPUT ON` as follows:

```
SET SERVEROUTPUT ON
```

#### Note

- `SIZE` sets the number of bytes of the output that can be buffered within the Oracle Database server. The default is `UNLIMITED`. `n` cannot be less than 2000 or greater than 1,000,000.
- For additional information on `SERVEROUTPUT`, see the *Oracle Database PL/SQL User's Guide and Reference 11g*

## Using the SQL\*Plus SPOOL Command

```
SPO[OL] [file_name[.ext] [CRE[ATE] | REP[LACE] |
APP[END]] | OFF | OUT]
```

| Option          | Description                                                                                      |
|-----------------|--------------------------------------------------------------------------------------------------|
| file_name[.ext] | Spools output to the specified file name.                                                        |
| CRE[ATE]        | Creates a new file with the name specified.                                                      |
| REP[LACE]       | Replaces the contents of an existing file. If the file does not exist, REPLACE creates the file. |
| APP[END]        | Adds the contents of the buffer to the end of the file you specify.                              |
| OFF             | Stops spooling.                                                                                  |
| OUT             | Stops spooling and sends the file to your computer's standard (default) printer.                 |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Using the SQL\*Plus SPOOL Command

The SPOOL command stores query results in a file, or optionally sends the file to a printer. The SPOOL command has been enhanced. You can now append to, or replace an existing file, where previously you could only use SPOOL to create (and replace) a file. REPLACE is the default.

To spool output generated by commands in a script without displaying the output on the screen, use SET TERMOUT OFF. SET TERMOUT OFF does not affect output from commands that run interactively.

You must use quotes around file names containing white space. To create a valid HTML file using SPOOL APPEND commands, you must use PROMPT or a similar command to create the HTML page header and footer. The SPOOL APPEND command does not parse HTML tags. SET SQLPLUSCOMPAT[IBILITY] to 9.2 or earlier to disable the CREATE, APPEND and SAVE parameters.

## Using the AUTOTRACE Command

- Displays a report after the successful execution of SQL DML statements such as `SELECT`, `INSERT`, `UPDATE` or `DELETE`.
- The report can now include execution statistics and the query execution path.

```
SET AUTOT[RACE] {ON | OFF | TRACE[ONLY]} [EXP[LAIN]]
[STAT[ISTICS]]
```

```
SET AUTOTRACE ON
-- The AUTOTRACE report includes both the optimizer
-- execution path and the SQL statement execution
-- statistics.
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Using the AUTOTRACE Command

`EXPLAIN` shows the query execution path by performing an `EXPLAIN PLAN`. `STATISTICS` displays SQL statement statistics. The formatting of your `AUTOTRACE` report may vary depending on the version of the server to which you are connected and the configuration of the server. The `DBMS_XPLAN` package provides an easy way to display the output of the `EXPLAIN PLAN` command in several, predefined formats.

#### Note

- For additional information about the package and subprograms, see the *Oracle Database PL/SQL Packages and Types Reference 11g* guide.
- For additional information about the `EXPLAIN PLAN`, see the *Oracle Database SQL Reference 11g*.
- For additional information about Execution Plans and the statistics, see the *Oracle Database Performance Tuning Guide 11g*.

## Summary

In this appendix, you should have learned how to use SQL\*Plus as an environment to do the following:

- Execute SQL statements
- Edit SQL statements
- Format output
- Interact with script files

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Summary

SQL\*Plus is an execution environment that you can use to send SQL commands to the database server and to edit and save SQL commands. You can execute commands from the SQL prompt or from a script file.

## Generating Reports by Grouping Related Data

ORACLE®

Copyright © 2007, Oracle. All rights reserved.

Oracle Internal & Oracle Academy  
Use Only

## Objectives

After completing this appendix, you should be able to do the following:

- Use the `ROLLUP` operation to produce subtotal values
- Use the `CUBE` operation to produce cross-tabulation values
- Use the `GROUPING` function to identify the row values created by `ROLLUP` or `CUBE`
- Use `GROUPING SETS` to produce a single result set

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Objectives

In this lesson, you learn how to Group data to obtain the following:

- Subtotal values by using the `ROLLUP` operator
- Cross-tabulation values by using the `CUBE` operator
- Use the `GROUPING` function to identify the level of aggregation in the result set produced by a `ROLLUP` or `CUBE` operator
- Use `GROUPING SETS` to produce a single result set that is equivalent to a `UNION ALL` approach

## Review of Group Functions

- Group functions operate on sets of rows to give one result per group.

```
SELECT [column,] group_function(column) . . .
FROM table
[WHERE condition]
[GROUP BY group_by_expression]
[ORDER BY column];
```

- Example:

```
SELECT AVG(salary), STDDEV(salary),
 COUNT(commission_pct), MAX(hire_date)
FROM employees
WHERE job_id LIKE 'SA%';
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Group Functions

You can use the GROUP BY clause to divide the rows in a table into groups. You can then use group functions to return summary information for each group. Group functions can appear in select lists and in ORDER BY and HAVING clauses. The Oracle server applies the group functions to each group of rows and returns a single result row for each group.

**Types of group functions:** Each of the group functions—AVG, SUM, MAX, MIN, COUNT, STDDEV, and VARIANCE—accepts one argument. The AVG, SUM, STDDEV, and VARIANCE functions operate only on numeric values. MAX and MIN can operate on numeric, character, or date data values. COUNT returns the number of non-null rows for the given expression. The example in the slide calculates the average salary, standard deviation on the salary, number of employees earning a commission, and the maximum hire date for those employees whose JOB\_ID begins with SA.

#### Guidelines for Using Group Functions

- The data types for the arguments can be CHAR, VARCHAR2, NUMBER, or DATE.
- All group functions except COUNT (\*) ignore null values. To substitute a value for null values, use the NVL function. COUNT returns either a number or zero.
- The Oracle server implicitly sorts the result set in ascending order of the grouping columns specified, when you use a GROUP BY clause. To override this default ordering, you can use DESC in an ORDER BY clause.

## Review of the GROUP BY Clause

- Syntax:

```
SELECT [column,] group_function(column) . . .
FROM table
[WHERE condition]
[GROUP BY group_by_expression]
[ORDER BY column];
```

- Example:

```
SELECT department_id, job_id, SUM(salary),
 COUNT(employee_id)
FROM employees
GROUP BY department_id, job_id;
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Review of the GROUP BY Clause

The example illustrated in the slide is evaluated by the Oracle server as follows:

- The SELECT clause specifies that the following columns be retrieved:
  - Department ID and job ID columns from the EMPLOYEES table
  - The sum of all the salaries and the number of employees in each group that you have specified in the GROUP BY clause
- The GROUP BY clause specifies how the rows should be grouped in the table. The total salary and the number of employees are calculated for each job ID within each department. The rows are grouped by department ID and then grouped by job within each department.



## Review of the HAVING Clause

- Use the HAVING clause to specify which groups are to be displayed.
- You further restrict the groups on the basis of a limiting condition.

```
SELECT [column,] group_function(column)...
FROM table
[WHERE condition]
[GROUP BY group_by_expression]
[HAVING having_expression]
[ORDER BY column];
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### HAVING Clause

Groups are formed and group functions are calculated before the HAVING clause is applied to the groups. The HAVING clause can precede the GROUP BY clause, but it is recommended that you place the GROUP BY clause first because it is more logical.

The Oracle server performs the following steps when you use the HAVING clause:

1. It groups rows.
2. It applies the group functions to the groups and displays the groups that match the criteria in the HAVING clause.

## GROUP BY with ROLLUP and CUBE Operators

- Use ROLLUP or CUBE with GROUP BY to produce superaggregate rows by cross-referencing columns.
- ROLLUP grouping produces a result set containing the regular grouped rows and the subtotal values.
- CUBE grouping produces a result set containing the rows from ROLLUP and cross-tabulation rows.

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### GROUP BY with the ROLLUP and CUBE Operators

You specify ROLLUP and CUBE operators in the GROUP BY clause of a query. ROLLUP grouping produces a result set containing the regular grouped rows and subtotal rows. The ROLLUP operator also calculates a grand total. The CUBE operation in the GROUP BY clause groups the selected rows based on the values of all possible combinations of expressions in the specification and returns a single row of summary information for each group. You can use the CUBE operator to produce cross-tabulation rows.

**Note:** When working with ROLLUP and CUBE, make sure that the columns following the GROUP BY clause have meaningful, real-life relationships with each other; otherwise, the operators return irrelevant information.

## ROLLUP Operator

- ROLLUP is an extension to the GROUP BY clause.
- Use the ROLLUP operation to produce cumulative aggregates, such as subtotals.

```
SELECT [column,] group_function(column). . .
FROM table
[WHERE condition]
[GROUP BY [ROLLUP] group_by_expression]
[HAVING having_expression];
[ORDER BY column];
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### ROLLUP Operator

The ROLLUP operator delivers aggregates and superaggregates for expressions within a GROUP BY statement. The ROLLUP operator can be used by report writers to extract statistics and summary information from result sets. The cumulative aggregates can be used in reports, charts, and graphs.

The ROLLUP operator creates groupings by moving in one direction, from right to left, along the list of columns specified in the GROUP BY clause. It then applies the aggregate function to these groupings.

#### Note

- To produce subtotals in  $n$  dimensions (that is,  $n$  columns in the GROUP BY clause) without a ROLLUP operator,  $n+1$  SELECT statements must be linked with UNION ALL. This makes the query execution inefficient because each of the SELECT statements causes table access. The ROLLUP operator gathers its results with just one table access. The ROLLUP operator is useful when there are many columns involved in producing the subtotals.
- Subtotals and totals are produced with ROLLUP. CUBE produces totals as well but effectively rolls up in each possible direction, producing cross-tabular data.

## ROLLUP Operator: Example

```
SELECT department_id, job_id, SUM(salary)
FROM employees
WHERE department_id < 60
GROUP BY ROLLUP(department_id, job_id);
```

|    | DEPARTMENT_ID | JOB_ID   | SUM(SALARY) |   |
|----|---------------|----------|-------------|---|
| 1  | 10            | AD_ASST  | 4400        | 1 |
| 2  | 10            |          | 4400        |   |
| 3  | 20            | MK_MAN   | 13000       |   |
| 4  | 20            | MK_REP   | 6000        |   |
| 5  | 20            |          | 19000       |   |
| 6  | 30            | PU_MAN   | 11000       |   |
| 7  | 30            | PU_CLERK | 13900       |   |
| 8  | 30            |          | 24900       |   |
| 9  | 40            | HR_REP   | 6500        |   |
| 10 | 40            |          | 6500        | 2 |
| 11 | 50            | ST_MAN   | 36400       |   |
| 12 | 50            | SH_CLERK | 64300       |   |
| 13 | 50            | ST_CLERK | 55700       |   |
| 14 | 50            |          | 156400      |   |
| 15 |               |          | 211200      | 3 |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Example of a ROLLUP Operator

In the example in the slide:

- Total salaries for every job ID within a department for those departments whose department ID is less than 60 are displayed by the GROUP BY clause
- The ROLLUP operator displays:
  - Total salary for each department whose department ID is less than 60
  - Total salary for all departments whose department ID is less than 60, irrespective of the job IDs

In this example, 1 indicates a group totaled by both DEPARTMENT\_ID and JOB\_ID, 2 indicates a group totaled only by DEPARTMENT\_ID, and 3 indicates the grand total.

The ROLLUP operator creates subtotals that roll up from the most detailed level to a grand total, following the grouping list specified in the GROUP BY clause. First, it calculates the standard aggregate values for the groups specified in the GROUP BY clause (in the example, the sum of salaries grouped on each job within a department). Then it creates progressively higher-level subtotals, moving from right to left through the list of grouping columns. (In the example, the sum of salaries for each department is calculated, followed by the sum of salaries for all departments.)

- Given  $n$  expressions in the ROLLUP operator of the GROUP BY clause, the operation results in  $n + 1$  (in this case,  $2 + 1 = 3$ ) groupings.
- Rows based on the values of the first  $n$  expressions are called rows or regular rows, and the others are called superaggregate rows.

## CUBE Operator

- CUBE is an extension to the GROUP BY clause.
- You can use the CUBE operator to produce cross-tabulation values with a single SELECT statement.

```
SELECT [column,] group_function(column)...
FROM table
[WHERE condition]
[GROUP BY [CUBE] group_by_expression]
[HAVING having_expression]
[ORDER BY column];
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### CUBE Operator

The CUBE operator is an additional switch in the GROUP BY clause in a SELECT statement. The CUBE operator can be applied to all aggregate functions, including AVG, SUM, MAX, MIN, and COUNT. It is used to produce result sets that are typically used for cross-tabular reports. ROLLUP produces only a fraction of possible subtotal combinations, whereas CUBE produces subtotals for all possible combinations of groupings specified in the GROUP BY clause, and a grand total.

The CUBE operator is used with an aggregate function to generate additional rows in a result set. Columns included in the GROUP BY clause are cross-referenced to produce a superset of groups. The aggregate function specified in the select list is applied to these groups to produce summary values for the additional superaggregate rows. The number of extra groups in the result set is determined by the number of columns included in the GROUP BY clause.

In fact, every possible combination of the columns or expressions in the GROUP BY clause is used to produce superaggregates. If you have  $n$  columns or expressions in the GROUP BY clause, there will be  $2^n$  possible superaggregate combinations. Mathematically, these combinations form an  $n$ -dimensional cube, which is how the operator got its name.

By using application or programming tools, these superaggregate values can then be fed into charts and graphs that convey results and relationships visually and effectively.

## CUBE Operator: Example

```
SELECT department_id, job_id, SUM(salary)
FROM employees
WHERE department_id < 60
GROUP BY CUBE (department_id, job_id) ;
```

|    | DEPARTMENT_ID | JOB_ID   | SUM(SALARY) |
|----|---------------|----------|-------------|
| 1  |               |          | 211200      |
| 2  |               | HR_REP   | 6500        |
| 3  |               | MK_MAN   | 13000       |
| 4  |               | MK_REP   | 6000        |
| 5  |               | PU_MAN   | 11000       |
| 6  |               | ST_MAN   | 36400       |
| 7  |               | AD_ASST  | 4400        |
| 8  |               | PU_CLERK | 13900       |
| 9  |               | SH_CLERK | 64300       |
| 10 |               | ST_CLERK | 55700       |
| 11 | 10            |          | 4400        |
| 12 | 10            | AD_ASST  | 4400        |
| 13 | 20            |          | 19000       |
| 14 | 20            | MK_MAN   | 13000       |
| 15 | 20            | MK_REP   | 6000        |
| 16 | 30            |          | 24900       |
| 17 | 30            | PU_MAN   | 11000       |

1

2

3

4

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Example of a CUBE Operator

The output of the `SELECT` statement in the example can be interpreted as follows:

- The total salary for every job within a department (for those departments whose department ID is less than 60)
- The total salary for each department whose department ID is less than 60
- The total salary for each job irrespective of the department
- The total salary for those departments whose department ID is less than 60, irrespective of the job titles

In this example, 1 indicates the grand total, 2 indicates the rows totaled by `JOB_ID` alone, 3 indicates some of the rows totaled by `DEPARTMENT_ID` and `JOB_ID`, and 4 indicates some of the rows totaled by `DEPARTMENT_ID` alone.

The `CUBE` operator has also performed the `ROLLUP` operation to display the subtotals for those departments whose department ID is less than 60 and the total salary for those departments whose department ID is less than 60, irrespective of the job titles. Further, the `CUBE` operator displays the total salary for every job irrespective of the department.

**Note:** Similar to the `ROLLUP` operator, producing subtotals in  $n$  dimensions (that is,  $n$  columns in the `GROUP BY` clause) without a `CUBE` operator requires that  $2^n$  `SELECT` statements be linked with `UNION ALL`. Thus, a report with three dimensions requires  $2^3 = 8$  `SELECT` statements to be linked with `UNION ALL`.

## GROUPING Function

The GROUPING function:

- Is used with either the CUBE or ROLLUP operator
- Is used to find the groups forming the subtotal in a row
- Is used to differentiate stored NULL values from NULL values created by ROLLUP or CUBE
- Returns 0 or 1

```
SELECT [column,] group_function(column) .. ,
 GROUPING(expr)
FROM table
[WHERE condition]
[GROUP BY [ROLLUP] [CUBE] group_by_expression]
[HAVING having_expression]
[ORDER BY column];
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### GROUPING Function

The GROUPING function can be used with either the CUBE or ROLLUP operator to help you understand how a summary value has been obtained.

The GROUPING function uses a single column as its argument. The *expr* in the GROUPING function must match one of the expressions in the GROUP BY clause. The function returns a value of 0 or 1.

The values returned by the GROUPING function are useful to:

- Determine the level of aggregation of a given subtotal (that is, the group or groups on which the subtotal is based)
- Identify whether a NULL value in the expression column of a row of the result set indicates:
  - A NULL value from the base table (stored NULL value)
  - A NULL value created by ROLLUP or CUBE (as a result of a group function on that expression)

A value of 0 returned by the GROUPING function based on an expression indicates one of the following:

- The expression has been used to calculate the aggregate value.
- The NULL value in the expression column is a stored NULL value.

A value of 1 returned by the GROUPING function based on an expression indicates one of the following:

- The expression has not been used to calculate the aggregate value.
- The NULL value in the expression column is created by ROLLUP or CUBE as a result of grouping.

## GROUPING Function: Example

```
SELECT department_id DEPTID, job_id JOB,
 SUM(salary),
 GROUPING(department_id) GRP_DEPT,
 GROUPING(job_id) GRP_JOB
FROM employees
WHERE department_id < 50
GROUP BY ROLLUP(department_id, job_id);
```

|    | DEPTID | JOB      | SUM(SALARY) | GRP_DEPT | GRP_JOB |
|----|--------|----------|-------------|----------|---------|
| 1  | 10     | AD_ASST  | 4400        | 0        | 0       |
| 2  | 10     |          | 4400        | 0        | 1       |
| 3  | 20     | MK_MAN   | 13000       | 0        | 0       |
| 4  | 20     | MK_REP   | 6000        | 0        | 0       |
| 5  | 20     |          | 19000       | 0        | 1       |
| 6  | 30     | PU_MAN   | 11000       | 0        | 0       |
| 7  | 30     | PU_CLERK | 13900       | 0        | 0       |
| 8  | 30     |          | 24900       | 0        | 1       |
| 9  | 40     | HR_REP   | 6500        | 0        | 0       |
| 10 | 40     |          | 6500        | 0        | 1       |
| 11 |        |          | 54800       | 1        | 1       |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Example of a GROUPING Function

In the example in the slide, consider the summary value 4400 in the first row (labeled 1). This summary value is the total salary for the job ID of AD\_ASST within department 10. To calculate this summary value, both the DEPARTMENT\_ID and JOB\_ID columns have been taken into account. Thus, a value of 0 is returned for both the GROUPING(department\_id) and GROUPING(job\_id) expressions.

Consider the summary value 4400 in the second row (labeled 2). This value is the total salary for department 10 and has been calculated by taking into account the DEPARTMENT\_ID column; thus, a value of 0 has been returned by GROUPING(department\_id). Because the JOB\_ID column has not been taken into account to calculate this value, a value of 1 has been returned for GROUPING(job\_id). You can observe similar output in the fifth row.

In the last row, consider the summary value 54800 (labeled 3). This is the total salary for those departments whose department ID is less than 50 and all job titles. To calculate this summary value, neither of the DEPARTMENT\_ID and JOB\_ID columns have been taken into account. Thus, a value of 1 is returned for both the GROUPING(department\_id) and GROUPING(job\_id) expressions.



## GROUPING SETS

- The GROUPING SETS syntax is used to define multiple groupings in the same query.
- All groupings specified in the GROUPING SETS clause are computed and the results of individual groupings are combined with a UNION ALL operation.
- Grouping set efficiency:
  - Only one pass over the base table is required.
  - There is no need to write complex UNION statements.
  - The more elements GROUPING SETS has, the greater is the performance benefit.

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### GROUPING SETS

GROUPING SETS is a further extension of the GROUP BY clause that you can use to specify multiple groupings of data. Doing so facilitates efficient aggregation and, therefore, facilitates analysis of data across multiple dimensions.

A single SELECT statement can now be written using GROUPING SETS to specify various groupings (which can also include ROLLUP or CUBE operators), rather than multiple SELECT statements combined by UNION ALL operators. For example:

```
SELECT department_id, job_id, manager_id, AVG(salary)
FROM employees
GROUP BY
GROUPING SETS
((department_id, job_id, manager_id),
 (department_id, manager_id), (job_id, manager_id));
```

This statement calculates aggregates over three groupings:

(department\_id, job\_id, manager\_id), (department\_id, manager\_id) and (job\_id, manager\_id)

Without this feature, multiple queries combined together with UNION ALL are required to obtain the output of the preceding SELECT statement. A multiquery approach is inefficient because it requires multiple scans of the same data.

## GROUPING SETS (continued)

Compare the previous example with the following alternative:

```
SELECT department_id, job_id, manager_id, AVG(salary)
FROM employees
GROUP BY CUBE(department_id, job_id, manager_id);
```

This statement computes all the 8 (2 \* 2 \* 2) groupings, though only the (department\_id, job\_id, manager\_id), (department\_id, manager\_id), and (job\_id, manager\_id) groups are of interest to you.

Another alternative is the following statement:

```
SELECT department_id, job_id, manager_id, AVG(salary)
FROM employees
GROUP BY department_id, job_id, manager_id
UNION ALL
SELECT department_id, NULL, manager_id, AVG(salary)
FROM employees
GROUP BY department_id, manager_id
UNION ALL
SELECT NULL, job_id, manager_id, AVG(salary)
FROM employees
GROUP BY job_id, manager_id;
```

This statement requires three scans of the base table, which makes it inefficient.

CUBE and ROLLUP can be thought of as grouping sets with very specific semantics and results.

The following equivalencies show this fact:

|                                     |                                                                            |
|-------------------------------------|----------------------------------------------------------------------------|
| CUBE(a, b, c)<br>is equivalent to   | GROUPING SETS<br>((a, b, c), (a, b), (a, c), (b, c),<br>(a), (b), (c), ()) |
| ROLLUP(a, b, c)<br>is equivalent to | GROUPING SETS ((a, b, c), (a, b), (a), ())                                 |

## GROUPING SETS: Example

```
SELECT department_id, job_id,
 manager_id, AVG(salary)
FROM employees
GROUP BY GROUPING SETS
 ((department_id, job_id), (job_id, manager_id));
```

| DEPARTMENT_ID | JOB_ID   | MANAGER_ID | AVG(SALARY) |
|---------------|----------|------------|-------------|
|               | SH_CLERK | 122        | 3200        |
|               | AC_MGR   | 101        | 12000       |
|               | ST_MAN   | 100        | 7280        |
| ...           | ST_CLERK | 121        | 2675        |

①

| DEPARTMENT_ID | JOB_ID      | MANAGER_ID | AVG(SALARY) |
|---------------|-------------|------------|-------------|
| 110           | AC_MGR      |            | 12000       |
| 90            | AD_PRES     |            | 24000       |
| 60            | IT_PROG     |            | 5760        |
| 100           | FI_MGR      |            | 12000       |
| 30            | PU_CLERK    |            | 2780        |
| ...           | 50 SH_CLERK |            | 3215        |

②

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### GROUPING SETS: Example

The query in the slide calculates aggregates over two groupings. The table is divided into the following groups:

- Department ID, Job ID
- Job ID, Manager ID

The average salaries for each of these groups are calculated. The result set displays the average salary for each of the two groups.

In the output, the group marked as 1 can be interpreted as the following:

- The average salary of all employees with the SH\_CLERK job ID under manager 122 is 3,200.
- The average salary of all employees with the AC\_MGR job ID under manager 101 is 12,000, and so on.

The group marked as 2 in the output is interpreted as the following:

- The average salary of all employees with the AC\_MGR job ID in department 110 is 12,000.
- The average salary of all employees with the AD\_PRES job ID in department 90 is 24,000, and so on.

### GROUPING SETS: Example (continued)

The example in the slide can also be written as:

```
SELECT department_id, job_id, NULL as manager_id,
 AVG(salary) as AVGSAL
FROM employees
GROUP BY department_id, job_id
UNION ALL
SELECT NULL, job_id, manager_id, avg(salary) as AVGSAL
FROM employees
GROUP BY job_id, manager_id;
```

In the absence of an optimizer that looks across query blocks to generate the execution plan, the preceding query would need two scans of the base table, EMPLOYEES. This could be very inefficient. Therefore, the usage of the GROUPING SETS statement is recommended.

Oracle Internal & Oracle Academy  
Use Only

## Composite Columns

- A composite column is a collection of columns that are treated as a unit.  
`ROLLUP (a, (b, c), d)`
- Use parentheses within the `GROUP BY` clause to group columns, so that they are treated as a unit while computing `ROLLUP` or `CUBE` operations.
- When used with `ROLLUP` or `CUBE`, composite columns would require skipping aggregation across certain levels.

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Composite Columns

A composite column is a collection of columns that are treated as a unit during the computation of groupings. You specify the columns in parentheses as in the following statement: `ROLLUP (a, (b, c), d)`

Here, `(b, c)` forms a composite column and is treated as a unit. In general, composite columns are useful in `ROLLUP`, `CUBE`, and `GROUPING SETS`. For example, in `CUBE` or `ROLLUP`, composite columns would require skipping aggregation across certain levels.

That is, `GROUP BY ROLLUP(a, (b, c))` is equivalent to:

```
GROUP BY a, b, c UNION ALL
GROUP BY a UNION ALL
GROUP BY ()
```

Here, `(b, c)` is treated as a unit and `ROLLUP` is not applied across `(b, c)`. It is as though you have an alias—for example, `z` as an alias for `(b, c)`, and the `GROUP BY` expression reduces to: `GROUP BY ROLLUP(a, z)`.

**Note:** `GROUP BY ( )` is typically a `SELECT` statement with `NULL` values for the columns `a` and `b` and only the aggregate function. It is generally used for generating grand totals.

```
SELECT NULL, NULL, aggregate_col
FROM <table_name>
GROUP BY ();
```

## Composite Columns (continued)

Compare this with the normal ROLLUP as in:

```
GROUP BY ROLLUP (a, b, c)
```

This would be:

```
GROUP BY a, b, c UNION ALL
GROUP BY a, b UNION ALL
GROUP BY a UNION ALL
GROUP BY ()
```

Similarly:

```
GROUP BY CUBE ((a, b), c)
```

This would be equivalent to:

```
GROUP BY a, b, c UNION ALL
GROUP BY a, b UNION ALL
GROUP BY c UNION ALL
GROUP BY ()
```

The following table shows the GROUPING SETS specification and the equivalent GROUP BY specification.

| GROUPING SETS Statements                                                                               | Equivalent GROUP BY Statements                                |
|--------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| GROUP BY GROUPING SETS (a, b, c)                                                                       | GROUP BY a UNION ALL<br>GROUP BY b UNION ALL<br>GROUP BY c    |
| GROUP BY GROUPING SETS (a, b, (b, c))<br>(The GROUPING SETS expression has a composite column.)        | GROUP BY a UNION ALL<br>GROUP BY b UNION ALL<br>GROUP BY b, c |
| GROUP BY GROUPING SETS ((a, b, c))                                                                     | GROUP BY a, b, c                                              |
| GROUP BY GROUPING SETS (a, (b), ())                                                                    | GROUP BY a UNION ALL<br>GROUP BY b UNION ALL<br>GROUP BY ()   |
| GROUP BY GROUPING SETS<br>(a, ROLLUP (b, c))<br>(The GROUPING SETS expression has a composite column.) | GROUP BY a UNION ALL<br>GROUP BY ROLLUP (b, c)                |

## Composite Columns: Example

```
SELECT department_id, job_id, manager_id,
 SUM(salary)
FROM employees
GROUP BY ROLLUP(department_id, (job_id, manager_id));
```

| 1   | DEPARTMENT_ID | JOB_ID     | MANAGER_ID | SUM(SALARY) |
|-----|---------------|------------|------------|-------------|
|     |               | SA_REP     | 149        | 7000        |
|     |               |            |            | 7000        |
|     | 10            | AD_ASST    | 101        | 4400        |
|     | 10            |            |            | 4400        |
|     | 20            | MK_MAN     | 100        | 13000       |
|     | 20            | MK_REP     | 201        | 6000        |
|     | 20            |            |            | 19000       |
| ... |               |            |            |             |
|     | 100           | FI_MGR     | 101        | 12000       |
|     | 100           | FI_ACCOUNT | 108        | 39600       |
|     | 100           |            |            | 51600       |
|     | 110           | AC_MGR     | 101        | 12000       |
|     | 110           | AC_ACCOUNT | 205        | 8300        |
|     | 110           |            |            | 20300       |
|     |               |            |            | 691400      |

Annotations: 1 points to DEPARTMENT\_ID, 2 points to the first summary row for department 20, 3 points to the first summary row for department 100, 4 points to the grand total row.

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Composite Columns: Example

Consider the example:

```
SELECT department_id, job_id, manager_id, SUM(salary)
FROM employees
GROUP BY ROLLUP(department_id, job_id, manager_id);
```

This query results in the Oracle server computing the following groupings:

- (job\_id, manager\_id)
- (department\_id, job\_id, manager\_id)
- (department\_id)
- Grand total

If you are interested only in specific groups, you cannot limit the calculation to those groupings without using composite columns. With composite columns, this is possible by treating JOB\_ID and MANAGER\_ID columns as a single unit while rolling up. Columns enclosed in parentheses are treated as a unit while computing ROLLUP and CUBE. This is illustrated in the example in the slide. By enclosing the JOB\_ID and MANAGER\_ID columns in parentheses, you indicate to the Oracle server to treat JOB\_ID and MANAGER\_ID as a single unit—that is, a composite column.

## Composite Columns: Example (continued)

The example in the slide computes the following groupings:

- (department\_id, job\_id, manager\_id)
- (department\_id)
- ( )

The example in the slide displays the following:

- Total salary for every job and manager (labeled 1)
- Total salary for every department, job, and manager (labeled 2)
- Total salary for every department (labeled 3)
- Grand total (labeled 4)

The example in the slide can also be written as:

```
SELECT department_id, job_id, manager_id, SUM(salary)
FROM employees
GROUP BY department_id, job_id, manager_id
UNION ALL
SELECT department_id, TO_CHAR(NULL), TO_NUMBER(NULL),
 SUM(salary)
FROM employees
GROUP BY department_id
UNION ALL
SELECT TO_NUMBER(NULL), TO_CHAR(NULL), TO_NUMBER(NULL),
 SUM(salary)
FROM employees
GROUP BY ();
```

In the absence of an optimizer that looks across query blocks to generate the execution plan, the preceding query would need three scans of the base table, EMPLOYEES. This could be very inefficient. Therefore, the use of composite columns is recommended.



## Concatenated Groupings

- Concatenated groupings offer a concise way to generate useful combinations of groupings.
- To specify concatenated grouping sets, you separate multiple grouping sets, ROLLUP, and CUBE operations with commas so that the Oracle server combines them into a single GROUP BY clause.
- The result is a cross-product of groupings from each GROUPING SET.

```
GROUP BY GROUPING SETS(a, b), GROUPING SETS(c, d)
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Concatenated Columns

Concatenated groupings offer a concise way to generate useful combinations of groupings. The concatenated groupings are specified by listing multiple grouping sets, CUBEs, and ROLLUPs, and separating them with commas. The following is an example of concatenated grouping sets:

```
GROUP BY GROUPING SETS(a, b), GROUPING SETS(c, d)
```

This SQL example defines the following groupings:

```
(a, c), (a, d), (b, c), (b, d)
```

Concatenation of grouping sets is very helpful for these reasons:

- **Ease of query development:** You need not manually enumerate all groupings.
- **Use by applications:** SQL generated by online analytical processing (OLAP) applications often involves concatenation of grouping sets, with each GROUPING SET defining groupings needed for a dimension.

## Concatenated Groupings: Example

```
SELECT department_id, job_id, manager_id,
 SUM(salary)
FROM employees
GROUP BY department_id,
 ROLLUP(job_id),
 CUBE(manager_id);
```

|   | DEPARTMENT_ID | JOB_ID | MANAGER_ID | SUM(SALARY) |
|---|---------------|--------|------------|-------------|
|   | SA_REP        |        | 149        | 7000        |
|   | 10 AD_ASST    |        | 101        | 4400        |
|   | 20 MK_MAN     |        | 100        | 13000       |
|   | 20 MK_REP     |        | 201        | 6000        |
|   | ...           |        |            |             |
| 1 | 90 AD_VP      |        |            | 34000       |
|   | 90 AD PRES    |        |            | 24000       |
|   | ...           |        |            |             |
|   | SA_REP        |        |            | 7000        |
|   | 10 AD_ASST    |        |            | 4400        |
|   | ...           |        |            |             |
| 2 | 110           |        | 101        | 12000       |
|   | 110           |        | 205        | 8300        |
|   | 110           |        |            | 20300       |
|   | ...           |        |            |             |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Concatenated Groupings: Example

The example in the slide results in the following groupings:

- (department\_id, job\_id,) (1)
- (department\_id, manager\_id) (2)
- (department\_id) (3)

The total salary for each of these groups is calculated.

The following is another example of a concatenated grouping.

```
SELECT department_id, job_id, manager_id, SUM(salary) totalsal
FROM employees
WHERE department_id < 60
GROUP BY GROUPING SETS (department_id),
 GROUPING SETS (job_id, manager_id);
```

## Summary

In this appendix, you should have learned how to use the:

- ROLLUP operation to produce subtotal values
- CUBE operation to produce cross-tabulation values
- GROUPING function to identify the row values created by ROLLUP or CUBE
- GROUPING SETS syntax to define multiple groupings in the same query
- GROUP BY clause to combine expressions in various ways:
  - Composite columns
  - Concatenated grouping sets

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Summary

- ROLLUP and CUBE are extensions of the GROUP BY clause.
- ROLLUP is used to display subtotal and grand total values.
- CUBE is used to display cross-tabulation values.
- The GROUPING function enables you to determine whether a row is an aggregate produced by a CUBE or ROLLUP operator.
- With the GROUPING SETS syntax, you can define multiple groupings in the same query. GROUP BY computes all the groupings specified and combines them with UNION ALL.
- Within the GROUP BY clause, you can combine expressions in various ways:
  - To specify composite columns, you group columns within parentheses so that the Oracle server treats them as a unit while computing ROLLUP or CUBE operations.
  - To specify concatenated grouping sets, you separate multiple grouping sets, ROLLUP, and CUBE operations with commas so that the Oracle server combines them into a single GROUP BY clause. The result is a cross-product of groupings from each grouping set.



# **Hierarchical Retrieval**

ORACLE<sup>®</sup>

Copyright © 2007, Oracle. All rights reserved.

Oracle Internal & Oracle Academy  
Use Only

## Objectives

After completing this lesson, you should be able to do the following:

- Interpret the concept of a hierarchical query
- Create a tree-structured report
- Format hierarchical data
- Exclude branches from the tree structure

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Objectives

In this lesson, you learn how to use hierarchical queries to create tree-structured reports.

## Sample Data from the EMPLOYEES Table

|   | EMPLOYEE_ID | LAST_NAME | JOB_ID  | MANAGER_ID |
|---|-------------|-----------|---------|------------|
| 1 | 100         | King      | AD_PRES | (null)     |
| 2 | 101         | Kochhar   | AD_VP   | 100        |
| 3 | 102         | De Haan   | AD_VP   | 100        |
| 4 | 103         | Hunold    | IT_PROG | 102        |
| 5 | 104         | Ernst     | IT_PROG | 103        |
| 6 | 107         | Lorentz   | IT_PROG | 103        |

...

|    |     |           |            |     |
|----|-----|-----------|------------|-----|
| 16 | 200 | Whalen    | AD_ASST    | 101 |
| 17 | 201 | Hartstein | MK_MAN     | 100 |
| 18 | 202 | Fay       | MK_REP     | 201 |
| 19 | 205 | Higgins   | AC_MGR     | 101 |
| 20 | 206 | Gietz     | AC_ACCOUNT | 205 |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Sample Data from the EMPLOYEES Table

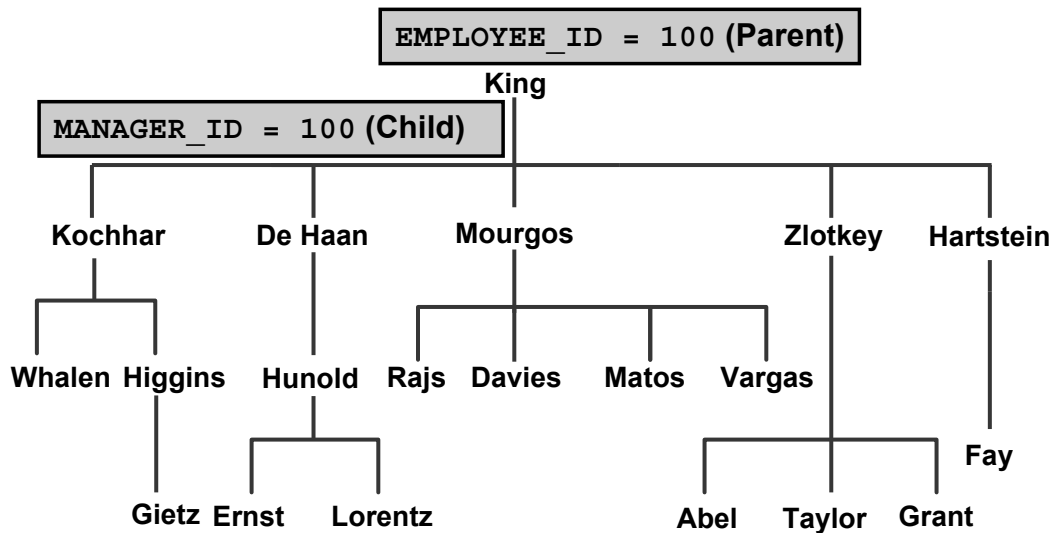
Using hierarchical queries, you can retrieve data based on a natural hierarchical relationship between rows in a table. A relational database does not store records in a hierarchical way. However, where a hierarchical relationship exists between the rows of a single table, a process called *tree walking* enables the hierarchy to be constructed. A hierarchical query is a method of reporting, with the branches of a tree in a specific order.

Imagine a family tree with the eldest members of the family found close to the base or trunk of the tree and the youngest members representing branches of the tree. Branches can have their own branches, and so on.

A hierarchical query is possible when a relationship exists between rows in a table. For example, in the slide, you see that Kochhar, De Haan, and Hartstein report to MANAGER\_ID 100, which is King's EMPLOYEE\_ID.

**Note:** Hierarchical trees are used in various fields such as human genealogy (family trees), livestock (breeding purposes), corporate management (management hierarchies), manufacturing (product assembly), evolutionary research (species development), and scientific research.

## Natural Tree Structure



ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Natural Tree Structure

The EMPLOYEES table has a tree structure representing the management reporting line. The hierarchy can be created by looking at the relationship between equivalent values in the EMPLOYEE\_ID and MANAGER\_ID columns. This relationship can be exploited by joining the table to itself. The MANAGER\_ID column contains the employee number of the employee's manager.

The parent-child relationship of a tree structure enables you to control:

- The direction in which the hierarchy is walked
- The starting point inside the hierarchy

**Note:** The slide displays an inverted tree structure of the management hierarchy of the employees in the EMPLOYEES table.



# Hierarchical Queries

```
SELECT [LEVEL], column, expr...
FROM table
[WHERE condition(s)]
[START WITH condition(s)]
[CONNECT BY PRIOR condition(s)] ;
```

*condition:*

```
expr comparison_operator expr
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Keywords and Clauses

Hierarchical queries can be identified by the presence of the CONNECT BY and START WITH clauses.

In the syntax:

|                   |                                                                                                                                              |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| SELECT            | Is the standard SELECT clause                                                                                                                |
| LEVEL             | For each row returned by a hierarchical query, the LEVEL pseudocolumn returns 1 for a root row, 2 for a child of a root, and so on.          |
| FROM <i>table</i> | Specifies the table, view, or snapshot containing the columns. You can select from only one table.                                           |
| WHERE             | Restricts the rows returned by the query without affecting other rows of the hierarchy                                                       |
| <i>condition</i>  | Is a comparison with expressions                                                                                                             |
| START WITH        | Specifies the root rows of the hierarchy (where to start). This clause is required for a true hierarchical query.                            |
| CONNECT BY        | Specifies the columns in which the relationship between parent and child PRIOR rows exist. This clause is required for a hierarchical query. |

## Walking the Tree

### Starting Point

- Specifies the condition that must be met
- Accepts any valid condition

```
START WITH column1 = value
```

Using the EMPLOYEES table, start with the employee whose last name is Kochhar.

```
...START WITH last_name = 'Kochhar'
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Walking the Tree

The row or rows to be used as the root of the tree are determined by the START WITH clause. The START WITH clause can contain any valid condition.

### Examples

Using the EMPLOYEES table, start with King, the president of the company.

```
... START WITH manager_id IS NULL
```

Using the EMPLOYEES table, start with employee Kochhar. A START WITH condition can contain a subquery.

```
... START WITH employee_id = (SELECT employee_id
 FROM employees
 WHERE last_name = 'Kochhar')
```

If the START WITH clause is omitted, the tree walk is started with all the rows in the table as root rows.

**Note:** The CONNECT BY and START WITH clauses are not ANSI SQL standard.

## Walking the Tree

```
CONNECT BY PRIOR column1 = column2
```

Walk from the top down, using the EMPLOYEES table.

```
... CONNECT BY PRIOR employee_id = manager_id
```

### Direction

Top down      →      Column1 = Parent Key  
                             Column2 = Child Key

Bottom up      →      Column1 = Child Key  
                             Column2 = Parent Key

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Walking the Tree (continued)

The direction of the query, whether it is from parent to child or from child to parent, is determined by the CONNECT BY PRIOR column placement. The PRIOR operator refers to the parent row. To find the child rows of a parent row, the Oracle server evaluates the PRIOR expression for the parent row and the other expressions for each row in the table. Rows for which the condition is true are the child rows of the parent. The Oracle server always selects child rows by evaluating the CONNECT BY condition with respect to a current parent row.

#### Examples

Walk from the top down using the EMPLOYEES table. Define a hierarchical relationship in which the EMPLOYEE\_ID value of the parent row is equal to the MANAGER\_ID value of the child row:

```
... CONNECT BY PRIOR employee_id = manager_id
```

Walk from the bottom up using the EMPLOYEES table:

```
... CONNECT BY PRIOR manager_id = employee_id
```

The PRIOR operator does not necessarily need to be coded immediately following CONNECT BY. Thus, the following CONNECT BY PRIOR clause gives the same result as the one in the preceding example:

```
... CONNECT BY employee_id = PRIOR manager_id
```

**Note:** The CONNECT BY clause cannot contain a subquery.

## Walking the Tree: From the Bottom Up

```
SELECT employee_id, last_name, job_id, manager_id
FROM employees
START WITH employee_id = 101
CONNECT BY PRIOR manager_id = employee_id ;
```

|   | EMPLOYEE_ID | LAST_NAME | JOB_ID  | MANAGER_ID |
|---|-------------|-----------|---------|------------|
| 1 | 101         | Kochhar   | AD_VP   | 100        |
| 2 | 100         | King      | AD_PRES | (null)     |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Walking the Tree: From the Bottom Up

The example in the slide displays a list of managers starting with the employee whose employee ID is 101.

#### Example

In the following example, EMPLOYEE\_ID values are evaluated for the parent row and MANAGER\_ID, and SALARY values are evaluated for the child rows. The PRIOR operator applies only to the EMPLOYEE\_ID value.

```
... CONNECT BY PRIOR employee_id = manager_id
 AND salary > 15000;
```

To qualify as a child row, a row must have a MANAGER\_ID value equal to the EMPLOYEE\_ID value of the parent row and must have a SALARY value greater than \$15,000.

## Walking the Tree: From the Top Down

```
SELECT last_name || ' reports to ' ||
PRIOR last_name "Walk Top Down"
FROM employees
START WITH last_name = 'King'
CONNECT BY PRIOR employee_id = manager_id ;
```

|   | Walk Top Down              |
|---|----------------------------|
| 1 | King reports to            |
| 2 | Kochhar reports to King    |
| 3 | Whalen reports to Kochhar  |
| 4 | Higgins reports to Kochhar |

|     |                           |
|-----|---------------------------|
| ... |                           |
| 18  | Grant reports to Zlotkey  |
| 19  | Hartstein reports to King |
| 20  | Fay reports to Hartstein  |

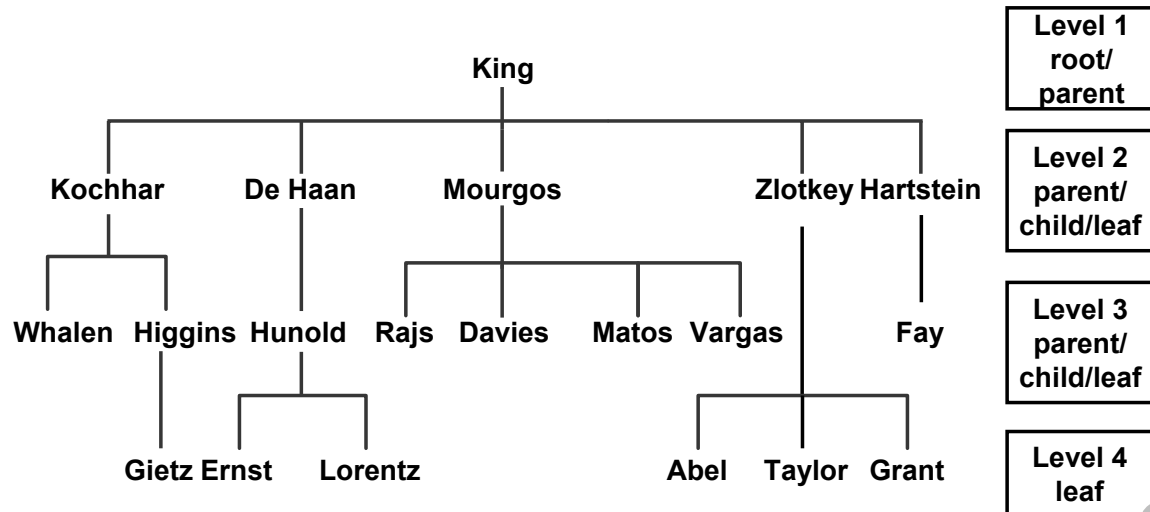
ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Walking the Tree: From the Top Down

Walking from the top down, display the names of the employees and their manager. Use employee King as the starting point. Print only one column.

## Ranking Rows with the LEVEL Pseudocolumn



ORACLE

Copyright © 2007, Oracle. All rights reserved.

## Ranking Rows with the LEVEL Pseudocolumn

You can explicitly show the rank or level of a row in the hierarchy by using the LEVEL pseudocolumn. This will make your report more readable. The forks where one or more branches split away from a larger branch are called nodes, and the very end of a branch is called a leaf or leaf node. The diagram in the slide shows the nodes of the inverted tree with their LEVEL values. For example, employee Higgins is a parent and a child, whereas employee Davies is a child and a leaf.

### LEVEL Pseudocolumn

| Value | Level                         |
|-------|-------------------------------|
| 1     | A root node                   |
| 2     | A child of a root node        |
| 3     | A child of a child, and so on |

In the slide, King is the root or parent (LEVEL = 1). Kochhar, De Haan, Mourgos, Zlotkey, Hartstein, Higgins, and Hunold are children and also parents (LEVEL = 2). Whalen, Rajs, Davies, Matos, Vargas, Gietz, Ernst, Lorentz, Abel, Taylor, Grant, and Fay are children and leaves (LEVEL = 3 and LEVEL = 4).

**Note:** A *root node* is the highest node within an inverted tree. A *child node* is any nonroot node. A parent node is any node that has children. A leaf node is any node without children. The number of levels returned by a hierarchical query may be limited by available user memory.

## Formatting Hierarchical Reports Using LEVEL and LPAD

Create a report displaying company management levels, beginning with the highest level and indenting each of the following levels.

```
COLUMN org_chart FORMAT A12
SELECT LPAD(last_name, LENGTH(last_name)+(LEVEL*2)-2, '_')
 AS org_chart
FROM employees
START WITH first_name='Steven' AND last_name='King'
CONNECT BY PRIOR employee_id=manager_id
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Formatting Hierarchical Reports Using LEVEL and LPAD

The nodes in a tree are assigned level numbers from the root. Use the LPAD function in conjunction with the LEVEL pseudocolumn to display a hierarchical report as an indented tree.

In the example in the slide:

- `LPAD(char1, n [, char2])` returns `char1`, left-padded to length `n` with the sequence of characters in `char2`. The argument `n` is the total length of the return value as it is displayed on your terminal screen.
- `LPAD(last_name, LENGTH(last_name)+(LEVEL*2)-2, '_')` defines the display format
- `char1` is the `LAST_NAME`, `n` the total length of the return value, is length of the `LAST_NAME + (LEVEL*2) - 2`, and `char2` is `'_'`.

That is, this tells SQL to take the `LAST_NAME` and left-pad it with the `'_'` character until the length of the resultant string is equal to the value determined by `LENGTH(last_name) + (LEVEL*2) - 2`.

For King, `LEVEL = 1`. Therefore,  $(2 * 1) - 2 = 2 - 2 = 0$ . So King does not get padded with any `'_'` character and is displayed in column 1.

For Kochhar, `LEVEL = 2`. Therefore,  $(2 * 2) - 2 = 4 - 2 = 2$ . So Kochhar gets padded with 2 `'_'` characters and is displayed indented.

The rest of the records in the `EMPLOYEES` table are displayed similarly.

## Formatting Hierarchical Reports Using LEVEL and LPAD (continued)

|  ORG_CHART |  |
|---------------------------------------------------------------------------------------------|--|
| 1 King                                                                                      |  |
| 2 __Kochhar                                                                                 |  |
| 3 ___Whalen                                                                                 |  |
| 4 ___Higgins                                                                                |  |
| 5 ____Gietz                                                                                 |  |
| 6 __De Haan                                                                                 |  |
| 7 ___Hunold                                                                                 |  |
| 8 ____Ernst                                                                                 |  |
| 9 ____Lorentz                                                                               |  |
| 10 __Mourgos                                                                                |  |
| 11 ___Rajs                                                                                  |  |
| 12 ___Davies                                                                                |  |
| 13 ___Matos                                                                                 |  |
| 14 ___Vargas                                                                                |  |
| 15 __Zlotkey                                                                                |  |
| 16 ___Abel                                                                                  |  |
| 17 ___Taylor                                                                                |  |
| 18 ___Grant                                                                                 |  |
| 19 __Hartstein                                                                              |  |
| 20 ___Fay                                                                                   |  |

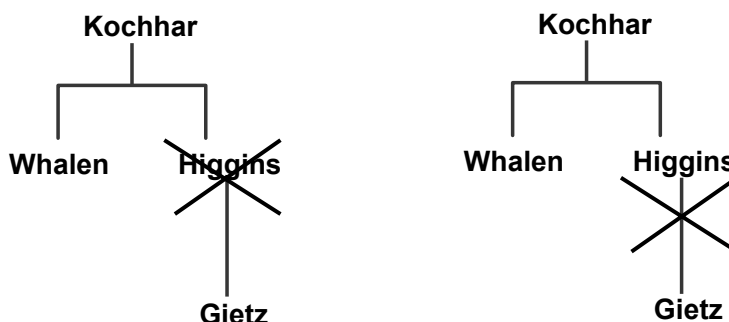


## Pruning Branches

Use the WHERE clause  
to eliminate a node.

Use the CONNECT BY clause  
to eliminate a branch.

```
WHERE last_name != 'Higgins'
CONNECT BY PRIOR
employee_id = manager_id
AND last_name != 'Higgins'
```



ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Pruning Branches

You can use the WHERE and CONNECT BY clauses to prune the tree (that is, to control which nodes or rows are displayed). The predicate you use acts as a Boolean condition.

#### Examples

Starting at the root, walk from the top down, and eliminate employee Higgins in the result, but process the child rows.

```
SELECT department_id, employee_id, last_name, job_id, salary
FROM employees
WHERE last_name != 'Higgins'
START WITH manager_id IS NULL
CONNECT BY PRIOR employee_id = manager_id;
```

Starting at the root, walk from the top down, and eliminate employee Higgins and all child rows.

```
SELECT department_id, employee_id, last_name, job_id, salary
FROM employees
START WITH manager_id IS NULL
CONNECT BY PRIOR employee_id = manager_id
AND last_name != 'Higgins';
```

## Summary

In this lesson, you should have learned that:

- You can use hierarchical queries to view a hierarchical relationship between rows in a table
- You specify the direction and starting point of the query
- You can eliminate nodes or branches by pruning

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Summary

You can use hierarchical queries to retrieve data based on a natural hierarchical relationship between rows in a table. The `LEVEL` pseudocolumn counts how far down a hierarchical tree you have traveled. You can specify the direction of the query using the `CONNECT BY PRIOR` clause. You can specify the starting point using the `START WITH` clause. You can use the `WHERE` and `CONNECT BY` clauses to prune the tree branches.

# G

## Writing Advanced Scripts

ORACLE®

Copyright © 2007, Oracle. All rights reserved.

Oracle Internal & Oracle Academy  
Use Only

## Objectives

After completing this appendix, you should be able to do the following:

- Describe the type of problems that are solved by using SQL to generate SQL
- Write a script that generates a script of `DROP TABLE` statements
- Write a script that generates a script of `INSERT INTO` statements

ORACLE<sup>®</sup>

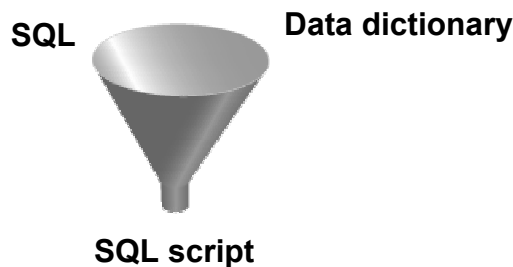
Copyright © 2007, Oracle. All rights reserved.

## Objectives

In this appendix, you learn how to write a SQL script to generate a SQL script.

## Using SQL to Generate SQL

- SQL can be used to generate scripts in SQL.
- The data dictionary:
  - Is a collection of tables and views that contain database information
  - Is created and maintained by the Oracle server



ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Using SQL to Generate SQL

SQL can be a powerful tool to generate other SQL statements. In most cases, this involves writing a script file. You can use SQL from SQL to:

- Avoid repetitive coding
- Access information from the data dictionary
- Drop or re-create database objects
- Generate dynamic predicates that contain run-time parameters

The examples used in this lesson involve selecting information from the data dictionary. The data dictionary is a collection of tables and views that contain information about the database. This collection is created and maintained by the Oracle server. All data dictionary tables are owned by the SYS user. Information stored in the data dictionary includes names of Oracle server users, privileges granted to users, database object names, table constraints, and audit information. There are four categories of data dictionary views. Each category has a distinct prefix that reflects its intended use.

| Prefix | Description                                                                                                            |
|--------|------------------------------------------------------------------------------------------------------------------------|
| USER_  | Contains details of objects owned by the user                                                                          |
| ALL_   | Contains details of objects to which the user has been granted access rights, in addition to objects owned by the user |
| DBA_   | Contains details of users with DBA privileges to access any object in the database                                     |
| V\$_   | Stores information about database server performance and locking; available only to the DBA                            |

## Creating a Basic Script

```
SELECT 'CREATE TABLE ' || table_name ||
 ' _test ' || 'AS SELECT * FROM '
 || table_name || ' WHERE 1=2;'
 AS "Create Table Script"
FROM user tables;
```

|   | Create Table Script                                                   |
|---|-----------------------------------------------------------------------|
| 1 | CREATE TABLE REGIONS_test AS SELECT * FROM REGIONS WHERE 1=2;         |
| 2 | CREATE TABLE LOCATIONS_test AS SELECT * FROM LOCATIONS WHERE 1=2;     |
| 3 | CREATE TABLE DEPARTMENTS_test AS SELECT * FROM DEPARTMENTS WHERE 1=2; |
| 4 | CREATE TABLE JOBS_test AS SELECT * FROM JOBS WHERE 1=2;               |
| 5 | CREATE TABLE EMPLOYEES_test AS SELECT * FROM EMPLOYEES WHERE 1=2;     |
| 6 | CREATE TABLE JOB_HISTORY_test AS SELECT * FROM JOB_HISTORY WHERE 1=2; |
| 7 | CREATE TABLE JOB_GRADES_test AS SELECT * FROM JOB_GRADES WHERE 1=2;   |
| 8 | CREATE TABLE COUNTRIES_test AS SELECT * FROM COUNTRIES WHERE 1=2;     |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### A Basic Script

The example in the slide produces a report with CREATE TABLE statements from every table you own. Each CREATE TABLE statement produced in the report includes the syntax to create a table using the table name with a suffix of \_test and having only the structure of the corresponding existing table. The old table name is obtained from the TABLE\_NAME column of the data dictionary view USER\_TABLES.

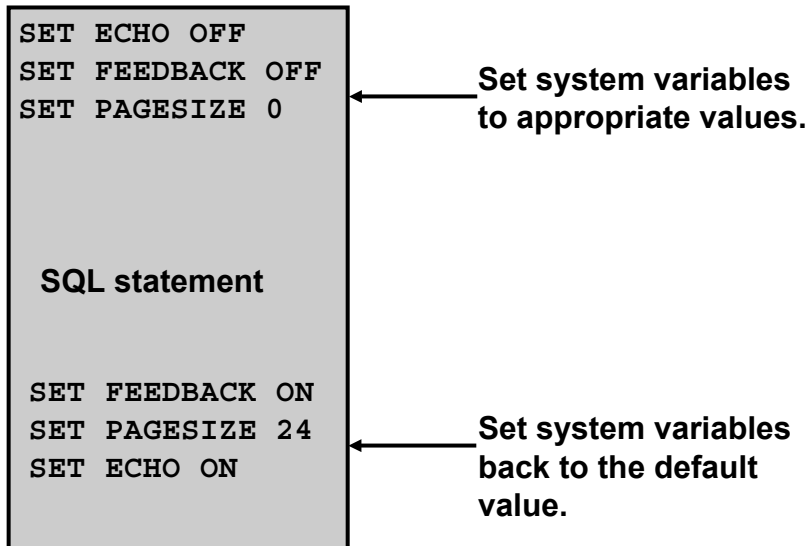
The next step is to enhance the report to automate the process.

**Note:** You can query the data dictionary tables to view various database objects that you own.

The data dictionary views frequently used include:

- USER\_TABLES: Displays description of the user's own tables
- USER\_OBJECTS: Displays all the objects owned by the user
- USER\_TAB\_PRIVS\_MADE: Displays all grants on objects owned by the user
- USER\_COL\_PRIVS\_MADE: Displays all grants on columns of objects owned by the user

## Controlling the Environment



ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Controlling the Environment

To execute the SQL statements that are generated, you must capture them in a file that can then be run. You must also plan to clean up the output that is generated and make sure that you suppress elements such as headings, feedback messages, top titles, and so on. In SQL Developer, you can save these statements to a script. To save the contents of the Enter SQL Statement box, click the Save icon or use the **File > Save** menu item. Alternatively, you can right-click in the Enter SQL Statement box and select the Save File option from the drop-down menu.

**Note:** Some of the SQL\*Plus statements are not supported by SQL Worksheet. For the complete list of SQL\*Plus statements that are supported, and not supported by SQL Worksheet, refer to the topic titled *SQL\*Plus Statements Supported and Not Supported in SQL Worksheet* in the SQL Developer online Help.

## The Complete Picture

```
SET ECHO OFF
SET FEEDBACK OFF
SET PAGESIZE 0

SELECT 'DROP TABLE ' || object_name || ';'
FROM user_objects
WHERE object_type = 'TABLE'
/

SET FEEDBACK ON
SET PAGESIZE 24
SET ECHO ON
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### The Complete Picture

The output of the command in the slide is saved into a file called `dropem.sql` in SQL Developer. To save the output into a file in SQL Developer, you use the Save File option under the Script Output pane. The `dropem.sql` file contains the following data. This file can now be started from SQL Developer by locating the script file, loading it, and executing it.

|   | 'DROPTABLE'  OBJECT_NAME  ';' |
|---|-------------------------------|
| 1 | DROP TABLE REGIONS;           |
| 2 | DROP TABLE COUNTRIES;         |
| 3 | DROP TABLE LOCATIONS;         |
| 4 | DROP TABLE DEPARTMENTS;       |
| 5 | DROP TABLE JOBS;              |
| 6 | DROP TABLE EMPLOYEES;         |
| 7 | DROP TABLE JOB_HISTORY;       |
| 8 | DROP TABLE JOB_GRADES;        |



## Dumping the Contents of a Table to a File

```
SET HEADING OFF ECHO OFF FEEDBACK OFF
SET PAGESIZE 0

SELECT
 'INSERT INTO departments_test VALUES
 (' || department_id || ', ' || department_name ||
 ', ' || location_id || ');'
 AS "Insert Statements Script"
FROM departments
/

SET PAGESIZE 24
SET HEADING ON ECHO ON FEEDBACK ON
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Dumping Table Contents to a File

Sometimes, it is useful to have the values for the rows of a table in a text file in the format of an INSERT INTO VALUES statement. This script can be run to populate the table in case the table has been dropped accidentally.

The example in the slide produces INSERT statements for the DEPARTMENTS\_TEST table, captured in the data.sql file using the Save File option in SQL Developer.

The contents of the data.sql script file are as follows:

```
INSERT INTO departments_test VALUES
(10, 'Administration', 1700);
INSERT INTO departments_test VALUES
(20, 'Marketing', 1800);
INSERT INTO departments_test VALUES
(50, 'Shipping', 1500);
INSERT INTO departments_test VALUES
(60, 'IT', 1400);
...
```

## Dumping the Contents of a Table to a File

| Source                                   | Result                        |
|------------------------------------------|-------------------------------|
| <code>'''X'''</code>                     | <code>'X'</code>              |
| <code>'''</code>                         | <code>'</code>                |
| <code>'''   department_name   '''</code> | <code>'Administration'</code> |
| <code>','</code>                         | <code>','</code>              |
| <code>')) ;</code>                       | <code>')) ;</code>            |

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Dumping Table Contents to a File (continued)

You may have noticed the large number of single quotation marks in the previous slide. A set of four single quotation marks produces one single quotation mark in the final statement. Also remember that character and date values must be surrounded by quotation marks.

Within a string, to display one single quotation mark, you need to prefix it with another single quotation mark. For example, in the fifth example in the slide, the surrounding quotation marks are for the entire string. The second quotation mark acts as a prefix to display the third quotation mark. Thus, the result is one single quotation mark followed by the parenthesis, followed by the semicolon.

## Generating a Dynamic Predicate

```
COLUMN my_col NEW_VALUE dyn_where_clause

SELECT DECODE('&deptno', null,
DECODE ('&&hiredate', null, ' ',
'WHERE hire_date=TO_DATE('' || '&&hiredate'', 'DD-MON-YYYY'')),
DECODE ('&&hiredate', null,
'WHERE department_id = ' || '&deptno',
'WHERE department_id = ' || '&&deptno' ||
' AND hire_date = TO_DATE('' || '&&hiredate'', 'DD-MON-YYYY''))
AS my_col FROM dual;
```

```
SELECT last_name FROM employees &dyn_where_clause;
```

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Generating a Dynamic Predicate

The example in the slide generates a `SELECT` statement that retrieves data of all employees in a department who were hired on a specific day. The script generates the `WHERE` clause dynamically.

**Note:** After the user variable is in place, you must use the `UNDEFINE` command to delete it.

The first `SELECT` statement prompts you to enter the department number. If you do not enter any department number, the department number is treated as null by the `DECODE` function, and the user is then prompted for the hire date. If you do not enter any hire date, the hire date is treated as null by the `DECODE` function and the dynamic `WHERE` clause that is generated is also a null, which causes the second `SELECT` statement to retrieve all rows from the `EMPLOYEES` table.

**Note:** The `NEW_V[ALUE]` variable specifies a variable to hold a column value. You can reference the variable in `TTITLE` commands. Use `NEW_VALUE` to display column values or the date in the top title. You must include the column in a `BREAK` command with the `SKIP PAGE` action. The variable name cannot contain a pound sign (`#`). `NEW_VALUE` is useful for master/detail reports in which there is a new master record for each page.

## Generating a Dynamic Predicate (continued)

**Note:** Here, the hire date must be entered in the DD-MON-YYYY format.

The SELECT statement in the slide can be interpreted as follows:

```
IF (<<deptno>> is not entered) THEN
 IF (<<hiredate>> is not entered) THEN
 return empty string
 ELSE
 return the string 'WHERE hire_date =
TO_DATE('<<hiredate>>', 'DD-MON-YYYY')'
ELSE
 IF (<<hiredate>> is not entered) THEN
 return the string 'WHERE department_id =
<<deptno>> entered'
 ELSE
 return the string 'WHERE department_id =
<<deptno>> entered
AND hire_date =
TO_DATE(' <<hiredate>>', 'DD-MON-YYYY')'
END IF
```

The returned string becomes the value of the DYN\_WHERE\_CLAUSE variable, which will be used in the second SELECT statement.

**Note:** Use SQL\*Plus for these examples.

When the first example in the slide is executed, the user is prompted for the values for DEPTNO and HIREDATE:

Enter value for deptno: 10

Enter value for hiredate: 17-SEP-1987

The following value for MY\_COL is generated:

```
MY_COL

WHERE department_id = 10 AND hire_date = TO_DATE('27-SEP-1987','DD-MON-YYYY')
```

When the second example in the slide is executed, the following output is generated:

```
LAST_NAME

Whalen
```

## Summary

In this appendix, you should have learned that:

- You can write a SQL script to generate another SQL script
- Script files often use the data dictionary
- You can capture the output in a file

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Summary

SQL can be used to generate SQL scripts. These scripts can be used to avoid repetitive coding, drop or re-create objects, get help from the data dictionary, and generate dynamic predicates that contain run-time parameters.



# Oracle Database Architectural Components



ORACLE<sup>®</sup>

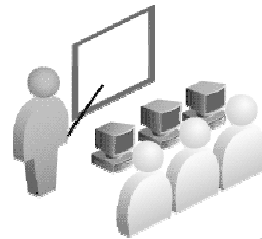
Copyright © 2007, Oracle. All rights reserved.

Oracle Internal & Oracle Academy  
Use Only

## Objectives

After completing this lesson, you should be able to do the following:

- List the major database architectural components
- Describe the background processes
- Explain the memory structures
- Correlate the logical and physical storage structures



ORACLE®

Copyright © 2007, Oracle. All rights reserved.

## Objectives

This lesson provides an overview of the Oracle Database architecture. You learn about the physical and logical structures and various components of the Oracle Database and their functions.



## Oracle Database Architecture: Overview

The Oracle Relational Database Management System (RDBMS) is a database management system that provides an open, comprehensive, integrated approach to information management.



ORACLE

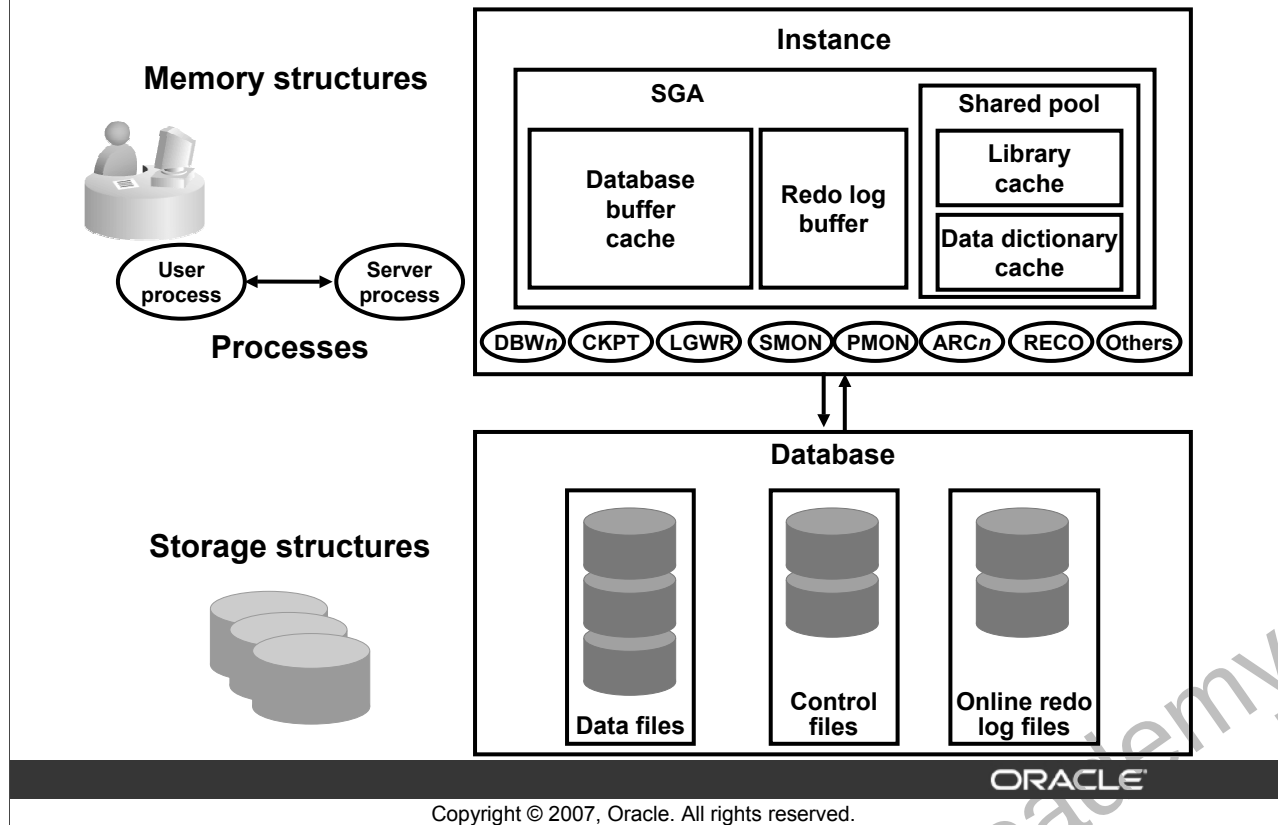
Copyright © 2007, Oracle. All rights reserved.

### Oracle Database Architecture: Overview

A database is a collection of data treated as a unit. The purpose of a database is to store and retrieve related information.

An Oracle Database reliably manages a large amount of data in a multiuser environment so that many users can concurrently access the same data. This is accomplished while delivering high performance. At the same time, it prevents unauthorized access and provides efficient solutions for failure recovery.

# Oracle Database Server Structures



## Oracle Database Server Structures

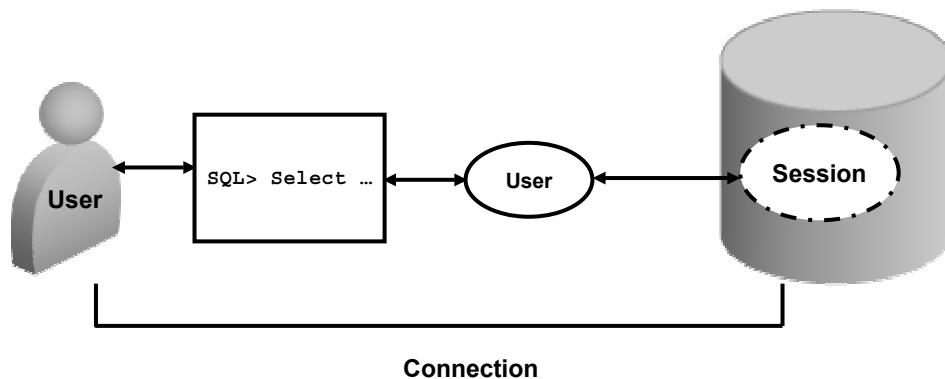
The Oracle Database consists of two main components—the instance and the database.

- The instance consists of the System Global Area (SGA), which is a collection of memory structures, and the background processes that perform tasks within the database. Every time an instance is started, the SGA is allocated and the background processes are started.
- The database consists of both physical structures and logical structures. Because the physical and logical structures are separate, the physical storage of data can be managed without affecting access to logical storage structures. The physical storage structures include:
  - The control files where the database configuration is stored
  - The redo log files that have information required for database recovery
  - The data files where all data is stored

An Oracle instance uses memory structures and processes to manage and access the database storage structures. All memory structures exist in the main memory of the computers that constitute the database server. Processes are jobs that work in the memory of these computers. A process is defined as a “thread of control” or a mechanism in an operating system that can run a series of steps.

## Connecting to the Database

- Connection: Communication pathway between a user process and a database instance
- Session: A specific connection of a user to a database instance through a user process



ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Connecting to the Database

To access information in the database, the user needs to connect to the database using a tool (such as SQL\*Plus). After the user establishes connection, a session is created for the user. Connection and session are closely related to user process but are very different in meaning.

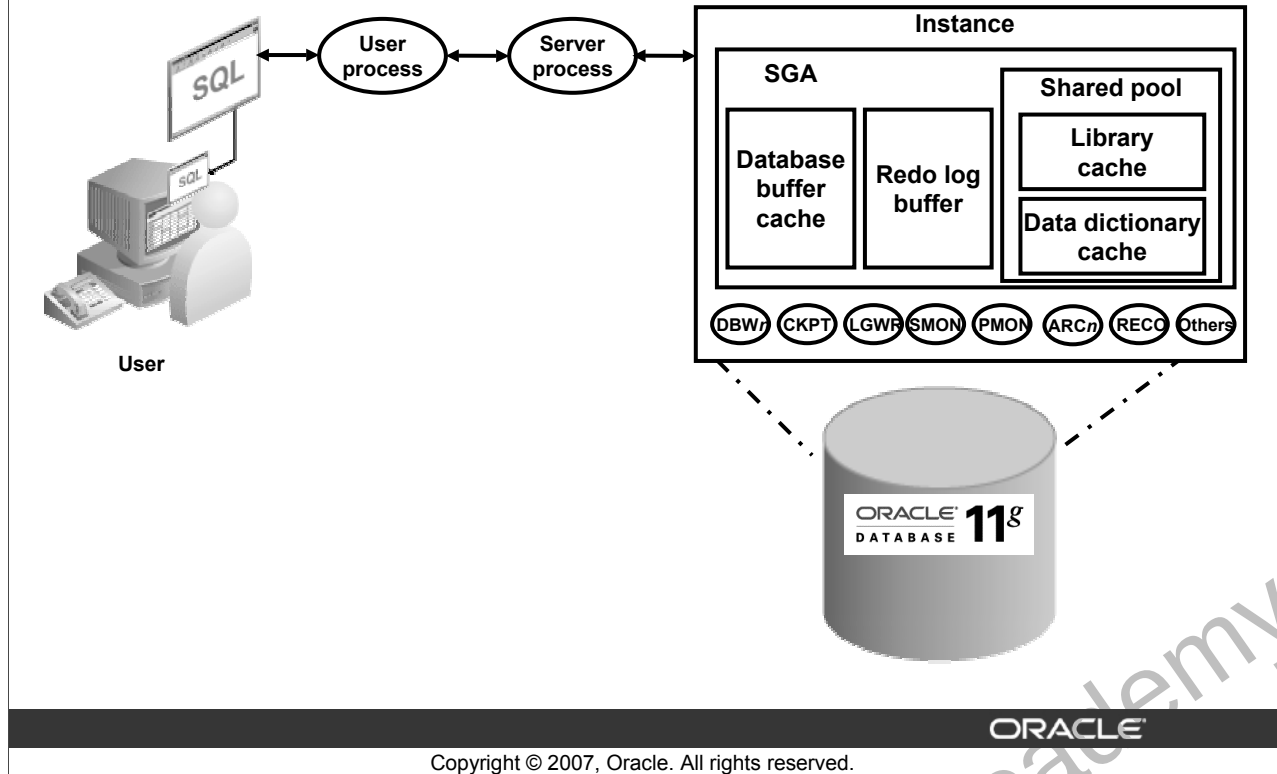
A connection is a communication pathway between a user process and an Oracle Database instance. A communication pathway is established using available interprocess communication mechanisms or network software (when different computers run the database application and Oracle Database, and communicate through a network).

A session represents the state of a current user login to the database instance. For example, when a user starts SQL\*Plus, the user must provide a valid username and password, and then a session is established for that user. A session lasts from the time the user connects until the time the user disconnects or exits the database application.

In the case of a dedicated connection, the session is serviced by a permanent dedicated process. In the case of a shared connection, the session is serviced by an available server process selected from a pool, either by the middle tier or by Oracle shared server architecture.

Multiple sessions can be created and exist concurrently for a single Oracle Database user using the same username, but through different applications, or multiple invocations of the same application.

## Interacting with an Oracle Database



### Interacting with an Oracle Database

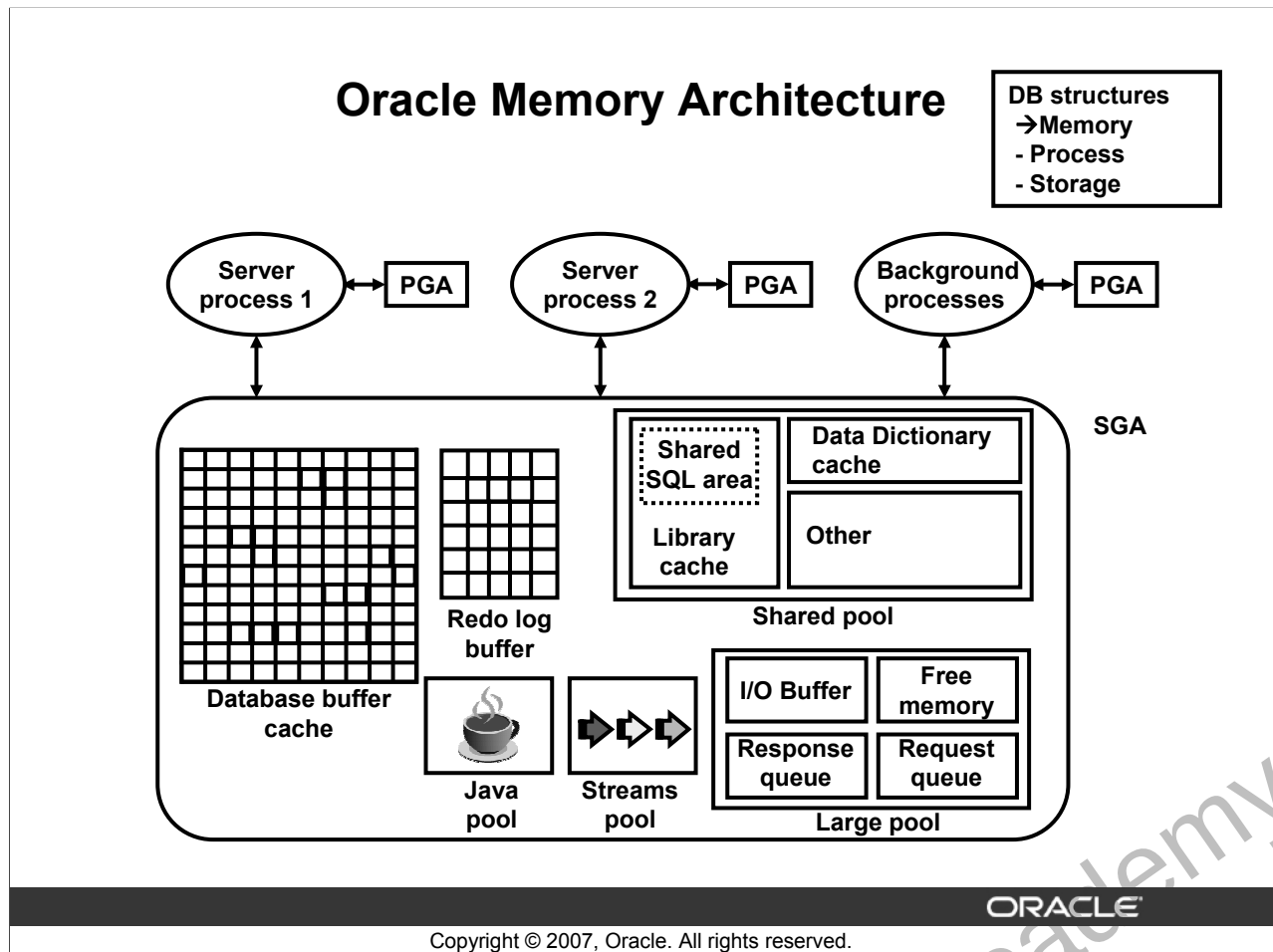
The following example describes Oracle Database operations at the most basic level. It illustrates an Oracle Database configuration where the user and associated server process are on separate computers, connected through a network.

1. An instance has started on a node where Oracle Database is installed, often called the host or database server.
2. A user starts an application spawning a user process. The application attempts to establish a connection to the server. (The connection may be local, client server, or a three-tier connection from a middle tier.)
3. The server runs a listener that has the appropriate Oracle Net Services handler. The server detects the connection request from the application and creates a dedicated server process on behalf of the user process.
4. The user runs a DML-type SQL statement and commits the transaction. For example, the user changes the address of a customer in a table and commits the change.
5. The server process receives the statement and checks the shared pool (an SGA component) for any shared SQL area that contains a similar SQL statement. If a shared SQL area is found, then the server process checks the user's access privileges to the requested data, and the existing shared SQL area is used to process the statement. If not, then a new shared SQL area is allocated for the statement, so it can be parsed and processed.

## Interacting with an Oracle Database (continued)

6. The server process retrieves any necessary data values, either from the actual data file (in which the table is stored) or those cached in the SGA.
7. The server process modifies data in the SGA. Because the transaction is committed, the log writer process (LGWR) immediately records the transaction in the redo log file. The database writer process (DBW $n$ ) writes modified blocks permanently to disk when doing so is efficient.
8. If the transaction is successful, then the server process sends a message across the network to the application. If it is not successful, then an error message is transmitted.
9. Throughout this entire procedure, the other background processes run, watching for conditions that require intervention. In addition, the database server manages other users' transactions and prevents contention between transactions that request the same data.

Oracle Internal & Oracle Academy  
Use Only



### Oracle Memory Structures

Oracle Database creates and uses memory structures for various purposes. For example, memory stores program code being run, data shared among users, and private data areas for each connected user.

Two basic memory structures are associated with an instance:

- The System Global Area (SGA) is a group of shared memory structures, known as SGA components, that contain data and control information for one Oracle Database instance. The SGA is shared by all server and background processes. Examples of data stored in the SGA include cached data blocks and shared SQL areas.
- The Program Global Areas (PGA) are memory regions that contain data and control information for a server or background process. A PGA is nonshared memory created by Oracle Database when a server or background process is started. Access to the PGA is exclusive to the server process. Each server process and background process has its own PGA.

## Oracle Memory Structures (continued)

The SGA is the memory area that contains data and control information for the instance. The SGA includes the following data structures:

- **Database buffer cache:** Caches blocks of data retrieved from the database
- **Redo Log buffer:** Caches redo information (used for instance recovery) until it can be written to the physical redo log files stored on the disk
- **Shared pool:** Caches various constructs that can be shared among users
- **Large pool:** Is an optional area that provides large memory allocations for certain large processes, such as Oracle backup and recovery operations, and I/O server processes
- **Java pool:** Is used for all session-specific Java code and data within the Java Virtual Machine (JVM)
- **Streams pool:** Is used by Oracle Streams to store information required by capture and apply

When you start the instance by using Enterprise Manager or SQL\*Plus, the amount of memory allocated for the SGA is displayed.

With the dynamic SGA infrastructure, the size of the database buffer cache, the shared pool, the large pool, the Java pool, and the Streams pool changes without shutting down the instance.

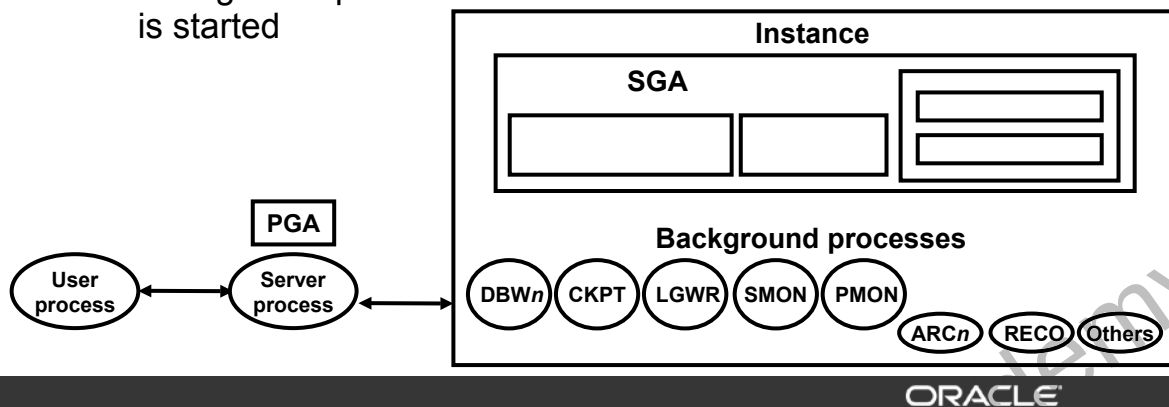
The Oracle database uses initialization parameters to create and configure memory structures. For example, the `SGA_TARGET` parameter specifies the total size of the SGA components. If you set `SGA_TARGET` to 0, Automatic Shared Memory Management is disabled.

Oracle Internal & Oracle Academy  
Use Only

## Process Architecture

DB structures  
- Memory  
→ Process  
- Storage

- User process:
  - Is started when a database user or a batch process connects to the Oracle Database
- Database processes:
  - Server process: Connects to the Oracle instance and is started when a user establishes a session
  - Background processes: Are started when an Oracle instance is started



Copyright © 2007, Oracle. All rights reserved.

## Process Architecture

The processes in an Oracle Database server can be categorized into two major groups:

- User processes that run the application or Oracle tool code
- Oracle Database processes that run the Oracle database server code. These include server processes and background processes.

When a user runs an application program or an Oracle tool such as SQL\*Plus, Oracle Database creates a **user process** to run the user's application. The Oracle Database also creates a **server process** to execute the commands issued by the user process. In addition, the Oracle server also has a set of **background processes** for an instance that interact with each other and with the operating system to manage the memory structures and asynchronously perform I/O to write data to disk, and perform other required tasks.

The process structure varies for different Oracle Database configurations, depending on the operating system and the choice of Oracle Database options. The code for connected users can be configured as a dedicated server or a shared server.

- With dedicated server, for each user, the database application is run by a user process, which is served by a dedicated server process that executes Oracle database server code.
- A shared server eliminates the need for a dedicated server process for each connection. A dispatcher directs multiple incoming network session requests to a pool of shared server processes. A shared server process serves any client request.



## Process Architecture (continued)

### Server Processes

Oracle Database creates server processes to handle the requests of user processes connected to the instance. In some situations when the application and Oracle Database operate on the same computer, it is possible to combine the user process and the corresponding server process into a single process to reduce system overhead. However, when the application and Oracle Database operate on different computers, a user process always communicates with Oracle Database through a separate server process.

Server processes created on behalf of each user's application can perform one or more of the following:

- Parse and run SQL statements issued through the application.
- Read necessary data blocks from data files on disk into the shared database buffers of the SGA, if the blocks are not already present in the SGA.
- Return results in such a way that the application can process the information.

### Background Processes

To maximize performance and accommodate many users, a multiprocess Oracle Database system uses some additional Oracle Database processes called background processes. An Oracle Database instance can have many background processes.

The following background processes are required for a successful startup of the database instance:

- Database Writer Process (DBW $n$ )
- Log Writer Process (LGWR)
- Checkpoint Process (CKPT)
- System Monitor Process (SMON)
- Process Monitor Process (PMON)

The following background processes are a few examples of optional background processes that can be started if required:

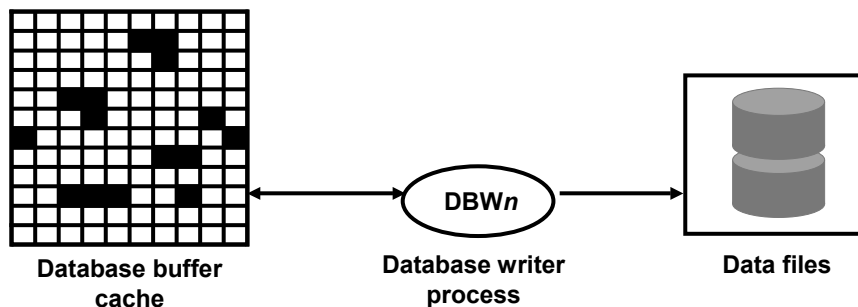
- Recoverer Process (RECO)
- Job Queue Processes
- Archiver Processes (ARC $n$ )
- Queue Monitor Processes (QM $Nn$ )

Other background processes may be found in more advanced configurations such as RAC. See the V\$BGPROCESS view for more information about the background processes.

On many operating systems, background processes are created automatically when an instance is started.

## Database Writer Process

- Writes modified (dirty) buffers in the database buffer cache to disk:
  - Asynchronously while performing other processing
  - Periodically to advance the checkpoint



ORACLE®

Copyright © 2007, Oracle. All rights reserved.

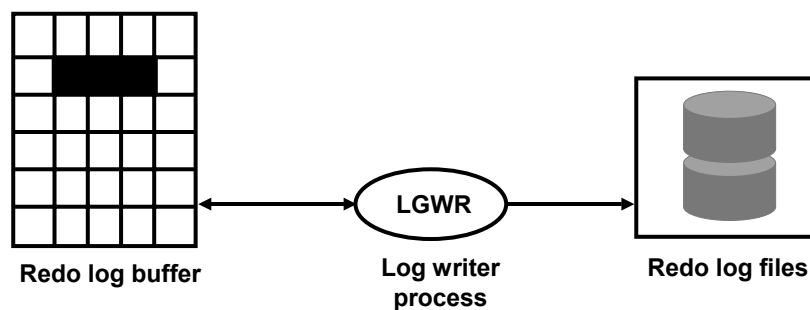
### Database Writer Process (DBW<sub>n</sub>)

The database writer process (DBW<sub>n</sub>) writes the contents of buffers to data files. The DBW<sub>n</sub> processes are responsible for writing modified (dirty) buffers in the database buffer cache to disk. Although one database writer process (DBW0) is adequate for most systems, you can configure additional processes (DBW1 through DBW9 and DBWa through DBWj) to improve write performance if your system modifies data heavily. These additional DBW<sub>n</sub> processes are not useful on uniprocessor systems.

When a buffer in the database buffer cache is modified, it is marked dirty and is added to the LRUW list of dirty buffers that is kept in SCN order, thereby matching the order of Redo corresponding to these changed buffers that is written to the Redo logs. When the number of available buffers in the buffer cache falls below an internal threshold such that server processes find it difficult to obtain available buffers, DBW<sub>n</sub> writes dirty buffers to the data files in the order that they were modified by following the order of the LRUW list.

## Log Writer Process

- Writes the redo log buffer to a redo log file on disk
- LGWR writes:
  - A process commits a transaction
  - When the redo log buffer is one-third full
  - Before a DBWn process writes modified buffers to disk



ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Log Writer Process (LGWR)

The log writer process (LGWR) is responsible for redo log buffer management by writing the redo log buffer entries to a redo log file on disk. LGWR writes all redo entries that have been copied into the buffer since the last time it wrote.

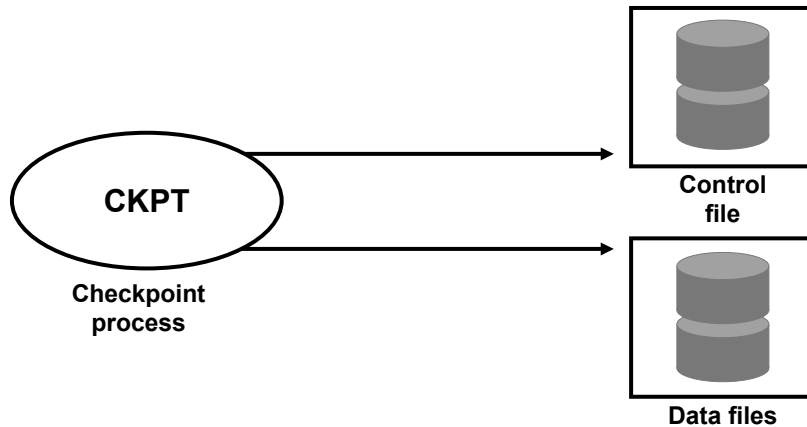
The redo log buffer is a circular buffer. When LGWR writes redo entries from the redo log buffer to a redo log file, server processes can then copy new entries over the entries in the redo log buffer that have been written to disk. LGWR normally writes fast enough to ensure that space is always available in the buffer for new entries, even when access to the redo log is heavy.

LGWR writes one contiguous portion of the buffer to disk. LGWR writes:

- When a user process commits a transaction
- When the redo log buffer is one-third full
- Before a DBWn process writes modified buffers to disk, if necessary

## Checkpoint Process

- Records checkpoint information in:
  - The control file
  - Each datafile header



ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Checkpoint Process (CKPT)

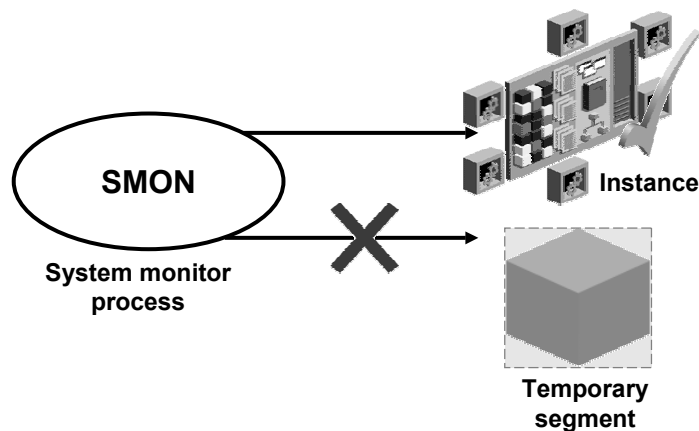
A checkpoint is a data structure that defines a system change number (SCN) in the redo thread of a database. Checkpoints are recorded in the control file and each data file header, and are a crucial element of recovery.

When a checkpoint occurs, Oracle Database must update the headers of all data files to record the details of the checkpoint. This is done by the CKPT process. The CKPT process does not write blocks to disk; DBWR always performs that work. The SCNs recorded in the file headers guarantee that all the changes made to database blocks before that SCN have been written to disk.

The statistic DBWR checkpoints displayed by the `SYSTEM_STATISTICS` monitor in Oracle Enterprise Manager indicates the number of checkpoint requests completed.

## System Monitor Process

- Performs recovery at instance startup
- Cleans up unused temporary segments



ORACLE®

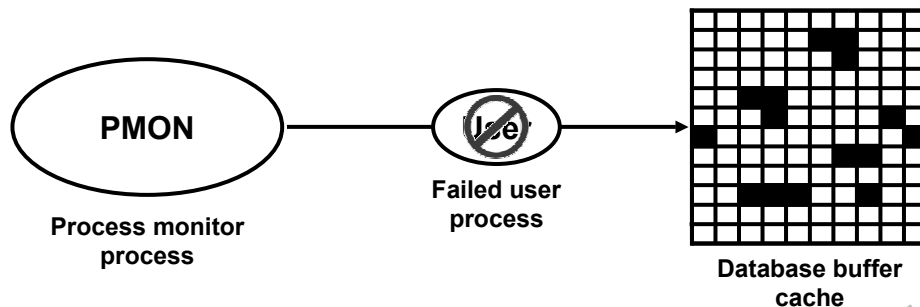
Copyright © 2007, Oracle. All rights reserved.

### System Monitor Process (SMON)

The system monitor process (SMON) performs recovery, if necessary, at instance startup. SMON is also responsible for cleaning up temporary segments that are no longer in use. If any terminated transactions were skipped during instance recovery because of file-read or offline errors, SMON recovers them when the tablespace or file is brought back online. SMON checks regularly to see whether it is needed. Other processes can call SMON if they detect a need for it.

## Process Monitor Process

- Performs process recovery when a user process fails
  - Cleans up the database buffer cache
  - Frees resources used by the user process
- Monitors sessions for idle session timeout
- Dynamically registers database services with listeners



Copyright © 2007, Oracle. All rights reserved.

### Process Monitor Process (PMON)

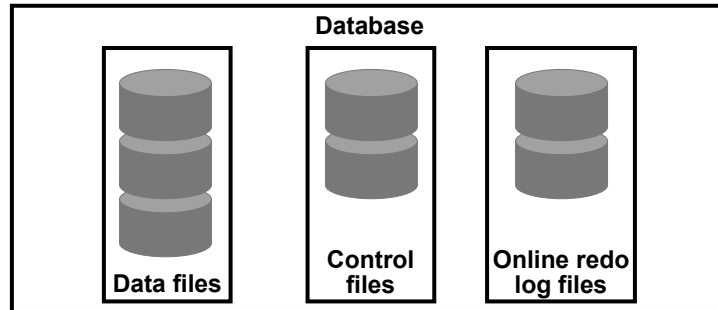
The process monitor (PMON) performs process recovery when a user process fails. PMON is responsible for cleaning up the database buffer cache and freeing resources that the user process was using. For example, it resets the status of the active transaction table, releases locks, and removes the process ID from the list of active processes.

PMON periodically checks the status of dispatcher and server processes, and restarts any that have stopped running (but not any that Oracle Database has terminated intentionally). PMON also registers information about the instance and dispatcher processes with the network listener.

Like SMON, PMON checks regularly to see whether it is needed and can be called if another process detects the need for it.

# Oracle Database Storage Architecture

DB structures  
- Memory  
- Process  
→ Storage



Copyright © 2007, Oracle. All rights reserved.

## Oracle Database Storage Architecture

The files that constitute an Oracle database are organized into the following:

- **Control files:** Contain data about the database itself (that is, physical database structure information). These files are critical to the database. Without them, you cannot open data files to access the data within the database.
- **Data files:** Contain the user or application data of the database, as well as metadata and the data dictionary.
- **Online redo log files:** Allow for instance recovery of the database. If the database server crashes and does not lose any data files, then the instance can recover the database with the information in these files.

The following additional files are important to the successful running of the database:

- **Backup files:** Are used for database recovery. You typically restore a backup file when a media failure or user error has damaged or deleted the original file.
- **Archived log files:** Contain an ongoing history of the data changes (redo) that are generated by the instance. Using these files and a backup of the database, you can recover a lost data file. That is, archive logs enable the recovery of restored data files.
- **Parameter file:** Is used to define how the instance is configured when it starts up.
- **Password file:** Allows sysdba/sysoper/sysasm to connect remotely to the database and perform administrative tasks.

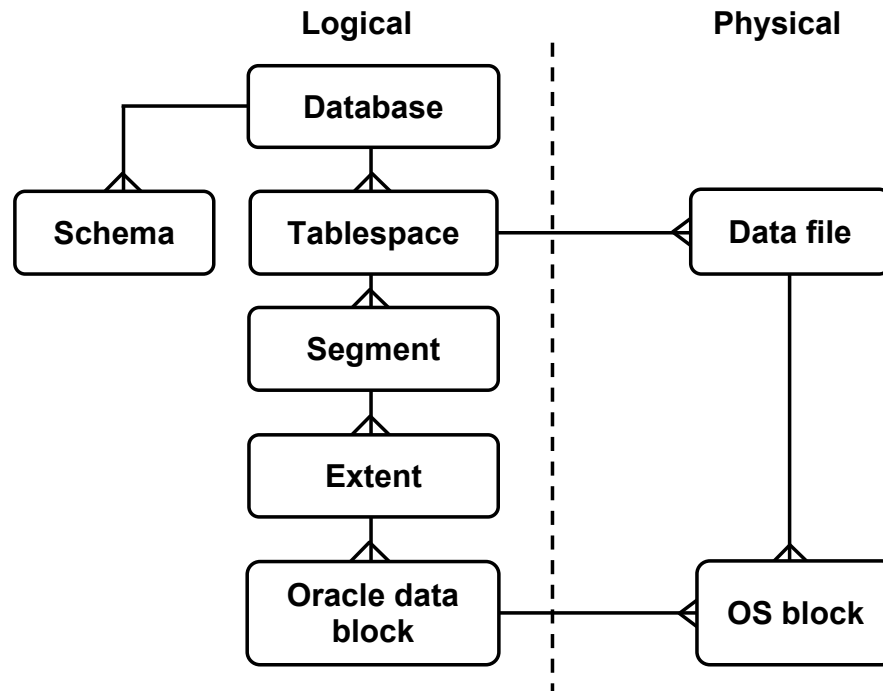
## Oracle Database Storage Architecture (continued)

- **Network files:** Are used for starting the database listener and store information required for user connections
- **Trace files:** Each server and background process can write to an associated trace file. When an internal error is detected by a process, the process dumps information about the error to its trace file. Some of the information written to a trace file is intended for the database administrator, whereas other information is for Oracle Support Services.
- **Alert log files:** These are special trace entries. The alert log of a database is a chronological log of messages and errors. Each instance has one alert log file. Oracle recommends that you review this alert log periodically.

Oracle Internal & Oracle Academy  
Use Only



## Logical and Physical Database Structures



Copyright © 2007, Oracle. All rights reserved.

### Logical and Physical Database Structures

An Oracle Database has logical and physical storage structures.

#### Tablespaces

A database is divided into logical storage units called tablespaces, which group related logical structures together. For example, tablespaces commonly group all of an application's objects to simplify some administrative operations. You may have a tablespace for application data and an additional one for application indexes.

#### Databases, Tablespaces, and Data Files

The relationship among databases, tablespaces, and data files is illustrated in the slide. Each database is logically divided into one or more tablespaces. One or more data files are explicitly created for each tablespace to physically store the data of all logical structures in a tablespace. If it is a TEMPORARY tablespace, instead of a data file, then the tablespace has a temporary file.

## Logical and Physical Database Structures (continued)

### Schemas

A schema is a collection of database objects that are owned by a database user. Schema objects are the logical structures that directly refer to the database's data. Schema objects include such structures as tables, views, sequences, stored procedures, synonyms, indexes, clusters, and database links. In general, schema objects include everything that your application creates in the database.

### Data Blocks

At the finest level of granularity, an Oracle database's data is stored in data blocks. One data block corresponds to a specific number of bytes of physical database space on the disk. A data block size is specified for each tablespace when it is created. A database uses and allocates free database space in Oracle data blocks.

### Extents

The next level of logical database space is called an extent. An extent is a specific number of contiguous data blocks (obtained in a single allocation) that are used to store a specific type of information.

### Segments

The level of logical database storage above an extent is called a segment. A segment is a set of extents allocated for a certain logical structure. For example, the different types of segments include:

- **Data segments:** Each nonclustered, non-indexed-organized table has a data segment with the exception of external tables, global temporary tables, and partitioned tables, where each table has one or more segments. All of the table's data is stored in the extents of its data segment. For a partitioned table, each partition has a data segment. Each cluster has a data segment. The data of every table in the cluster is stored in the cluster's data segment.
- **Index segments:** Each index has an index segment that stores all of its data. For a partitioned index, each partition has an index segment.
- **Undo segments:** One UNDO tablespace is created per database instance that contains numerous undo segments to temporarily store *undo* information. The information in an undo segment is used to generate read-consistent database information and, during database recovery, to roll back uncommitted transactions for users.
- **Temporary segments:** Temporary segments are created by the Oracle Database when a SQL statement needs a temporary work area to complete execution. When the statement finishes execution, the temporary segment's extents are returned to the instance for future use. Specify a default temporary tablespace for every user or a default temporary tablespace, which is used databasewide.

The Oracle Database dynamically allocates space. When the existing extents of a segment are full, additional extents are added. Because extents are allocated as needed, the extents of a segment may or may not be contiguous on the disk.

## Processing a SQL Statement

- Connect to an instance using:
  - The user process
  - The server process
- The Oracle server components that are used depend on the type of SQL statement:
  - Queries return rows.
  - Data manipulation language (DML) statements log changes.
  - Commit ensures transaction recovery.
- Some Oracle server components do not participate in SQL statement processing.

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Processing a SQL Statement

Not all the components of an Oracle instance are used to process SQL statements. The user and server processes are used to connect a user to an Oracle instance. These processes are not part of the Oracle instance, but are required to process a SQL statement.

Some of the background processes, SGA structures, and database files are used to process SQL statements. Depending on the type of SQL statement, different components are used:

- Queries require additional processing to return rows to the user.
- Data manipulation language (DML) statements require additional processing to log the changes made to the data.
- Commit processing ensures that the modified data in a transaction can be recovered.

Some required background processes do not directly participate in processing a SQL statement, but are used to improve performance and to recover the database. For example, the optional Archiver background process, ARC*n*, is used to ensure that a production database can be recovered.

## Processing a Query

- Parse:
  - Search for identical statement.
  - Check syntax, object names, and privileges.
  - Lock objects used during parse.
  - Create and store execution plan.
- Execute: Identify rows selected.
- Fetch: Return rows to user process.

ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Processing a Query

Queries are different from other types of SQL statements because, if successful, they return data as results. Other statements simply return success or failure, whereas a query can return one row or thousands of rows.

There are three main stages in the processing of a query:

- Parse
- Execute
- Fetch

During the *parse* stage, the SQL statement is passed from the user process to the server process, and a parsed representation of the SQL statement is loaded into a shared SQL area.

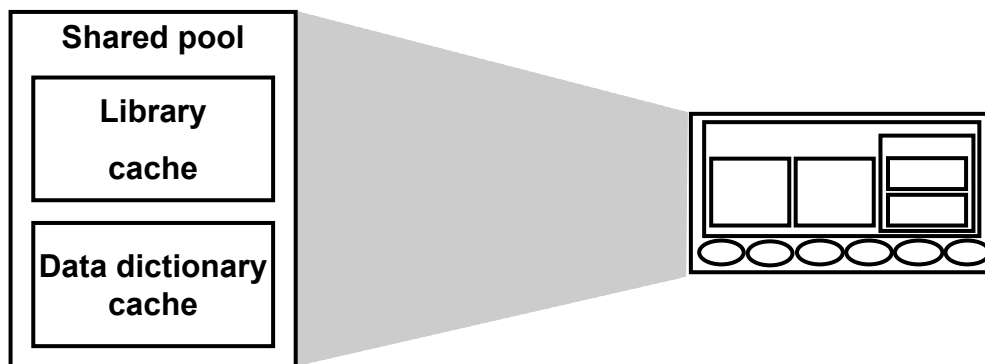
During parse, the server process performs the following functions:

- Searches for an existing copy of the SQL statement in the shared pool
- Validates the SQL statement by checking its syntax
- Performs data dictionary lookups to validate table and column definitions

The execute fetch executes the statement using the best optimizer approach and the fetch retrieves the rows back to the user.

## Shared Pool

- The library cache contains the SQL statement text, parsed code, and execution plan.
- The data dictionary cache contains table, column, and other object definitions and privileges.
- The shared pool is sized by `SHARED_POOL_SIZE`.



ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Shared Pool

During the parse stage, the server process uses the area in the SGA known as the shared pool to compile the SQL statement. The shared pool has two primary components:

- Library cache
- Data dictionary cache

#### Library Cache

The library cache stores information about the most recently used SQL statements in a memory structure called a shared SQL area. The shared SQL area contains:

- The text of the SQL statement
- The parse tree, which is a compiled version of the statement
- The execution plan, with steps to be taken when executing the statement

The optimizer is the function in the Oracle server that determines the optimal execution plan.

If a SQL statement is reexecuted and a shared SQL area already contains the execution plan for the statement, then the server process does not need to parse the statement. The library cache improves the performance of applications that reuse SQL statements by reducing parse time and memory requirements. If the SQL statement is not reused, it is eventually aged out of the library cache.

## **Shared Pool (continued)**

### **Data Dictionary Cache**

The data dictionary cache, also known as the dictionary cache or row cache, is a collection of the most recently used definitions in the database. It includes information about database files, tables, indexes, columns, users, privileges, and other database objects.

During the parse phase, the server process looks for the information in the dictionary cache to resolve the object names specified in the SQL statement and to validate the access privileges. If necessary, the server process initiates the loading of this information from the data files.

### **Sizing the Shared Pool**

The size of the shared pool is specified by the `SHARED_POOL_SIZE` initialization parameter.

Oracle Internal & Oracle Academy  
Use Only

## Database Buffer Cache

- The database buffer cache stores the most recently used blocks.
- The size of a buffer is based on `DB_BLOCK_SIZE`.
- The number of buffers is defined by `DB_BLOCK_BUFFERS`.



ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Database Buffer Cache

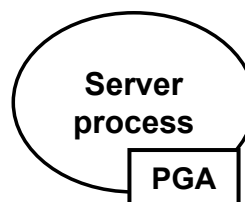
When a query is processed, the server process looks in the database buffer cache for any blocks it needs. If the block is not found in the database buffer cache, the server process reads the block from the data file and places a copy in the buffer cache. Because subsequent requests for the same block may find the block in memory, the requests may not require physical reads. The Oracle server uses a least recently used algorithm to age out buffers that have not been accessed recently to make room for new blocks in the buffer cache.

#### Sizing the Database Buffer Cache

The size of each buffer in the buffer cache is equal to the size of an Oracle block, and it is specified by the `DB_BLOCK_SIZE` parameter. The number of buffers is equal to the value of the `DB_BLOCK_BUFFERS` parameter.

## Program Global Area (PGA)

- Is not shared
- Is writable only by the server process
- Contains:
  - Sort area
  - Session information
  - Cursor state
  - Stack space



ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Program Global Area (PGA)

A Program Global Area (PGA) is a memory region that contains data and control information for a server process. It is a nonshared memory created by Oracle when a server process is started. Access to it is exclusive to that server process, and is read and written only by the Oracle server code acting on behalf of it. The PGA memory allocated by each server process attached to an Oracle instance is referred to as the aggregated PGA memory allocated by the instance.

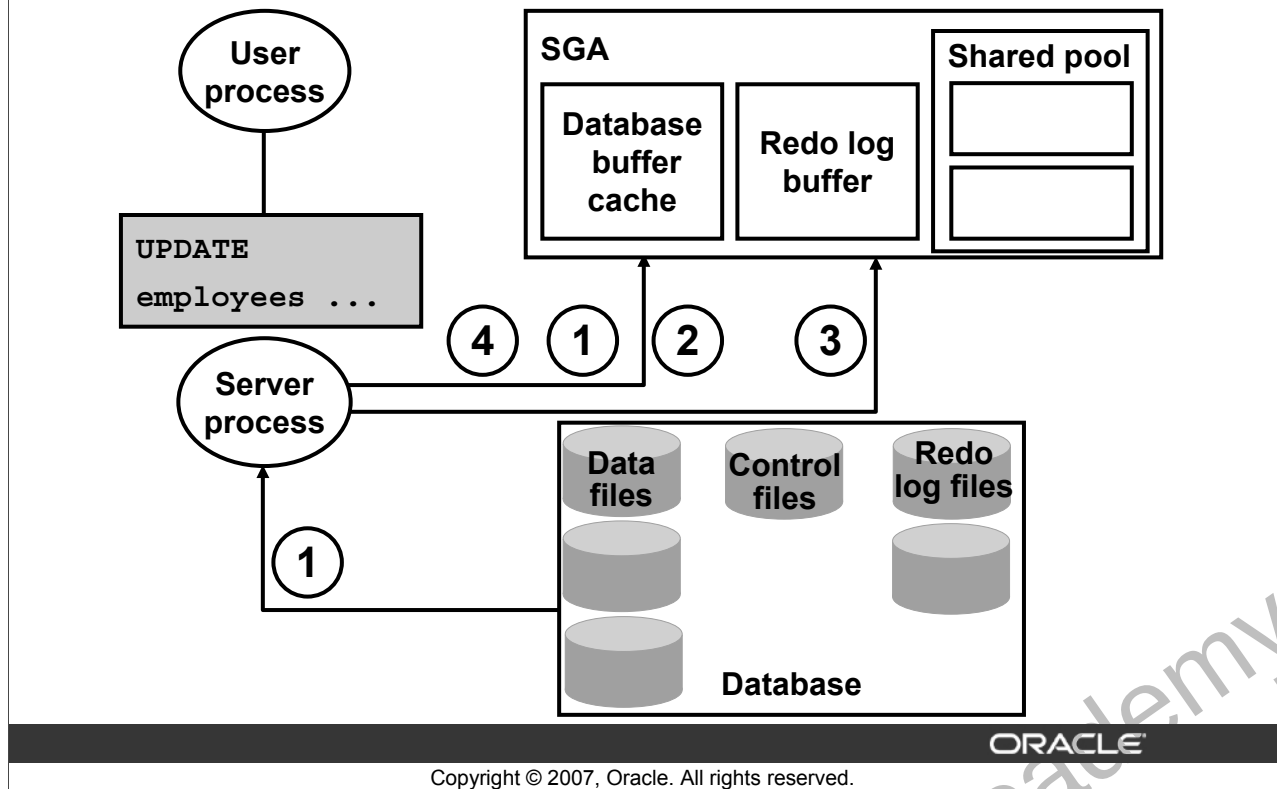
In a dedicated server configuration, the PGA of the server includes the following components:

- **Sort area:** Is used for any sorts that may be required to process the SQL statement
- **Session information:** Includes user privileges and performance statistics for the session
- **Cursor state:** Indicates the stage in the processing of the SQL statements that are currently used by the session
- **Stack space:** Contains other session variables

The PGA is allocated when a process is created, and deallocated when the process is terminated.



## Processing a DML Statement



### Processing a DML Statement

A data manipulation language (DML) statement requires only two phases of processing:

- Parse is the same as the parse phase used for processing a query.
- Execute requires additional processing to make data changes.

#### DML Execute Phase

To execute a DML statement:

- If the data and rollback blocks are not already in the buffer cache, the server process reads them from the data files into the buffer cache
- The server process places locks on the rows that are to be modified
- In the redo log buffer, the server process records the changes to be made to the rollback and data blocks
- The rollback block changes record the values of the data before it is modified. The rollback block is used to store the “before image” of the data, so that the DML statements can be rolled back if necessary.
- The data block changes record the new values of the data

## **Processing a DML Statement (continued)**

### **DML Execute Phase (continued)**

The server process records the “before image” to the rollback block and updates the data block. Both of these changes are done in the database buffer cache. Any changed blocks in the buffer cache are marked as dirty buffers (that is, buffers that are not the same as the corresponding blocks on the disk).

The processing of a DELETE or INSERT command uses similar steps. The “before image” for a DELETE contains the column values in the deleted row, and the “before image” of an INSERT contains the row location information.

Because the changes made to the blocks are only recorded in memory structures and are not written immediately to disk, a computer failure that causes the loss of the SGA can also lose these changes.

Oracle Internal & Oracle Academy  
Use Only

## Redo Log Buffer

- Has its size defined by `LOG_BUFFER`
- Records changes made through the instance
- Is used sequentially
- Is a circular buffer



ORACLE

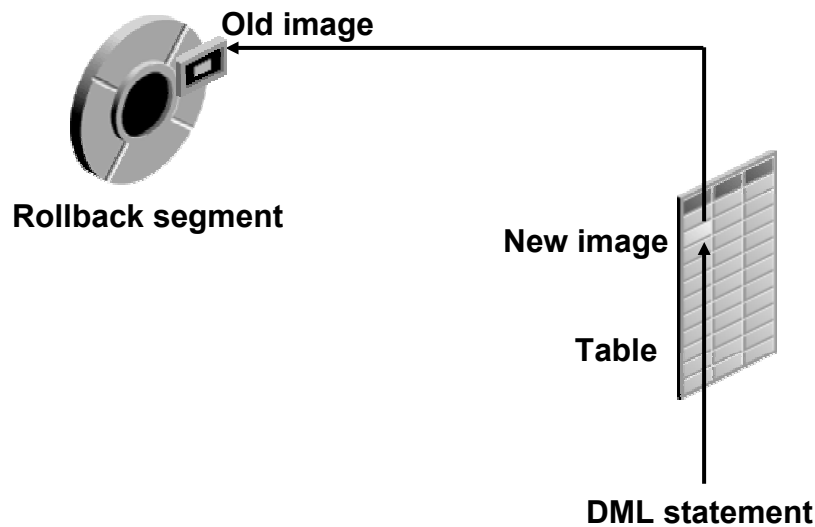
Copyright © 2007, Oracle. All rights reserved.

### Redo Log Buffer

The server process records most of the changes made to data file blocks in the redo log buffer, which is a part of the SGA. The redo log buffer has the following characteristics:

- Its size in bytes is defined by the `LOG_BUFFER` parameter.
- It records the block that is changed, the location of the change, and the new value in a redo entry. A redo entry makes no distinction between the types of block that is changed; it only records which bytes are changed in the block.
- The redo log buffer is used sequentially, and changes made by one transaction may be interleaved with changes made by other transactions.
- It is a circular buffer that is reused after it is filled, but only after all the old redo entries are recorded in the redo log files.

## Rollback Segment



ORACLE

Copyright © 2007, Oracle. All rights reserved.

### Rollback Segment

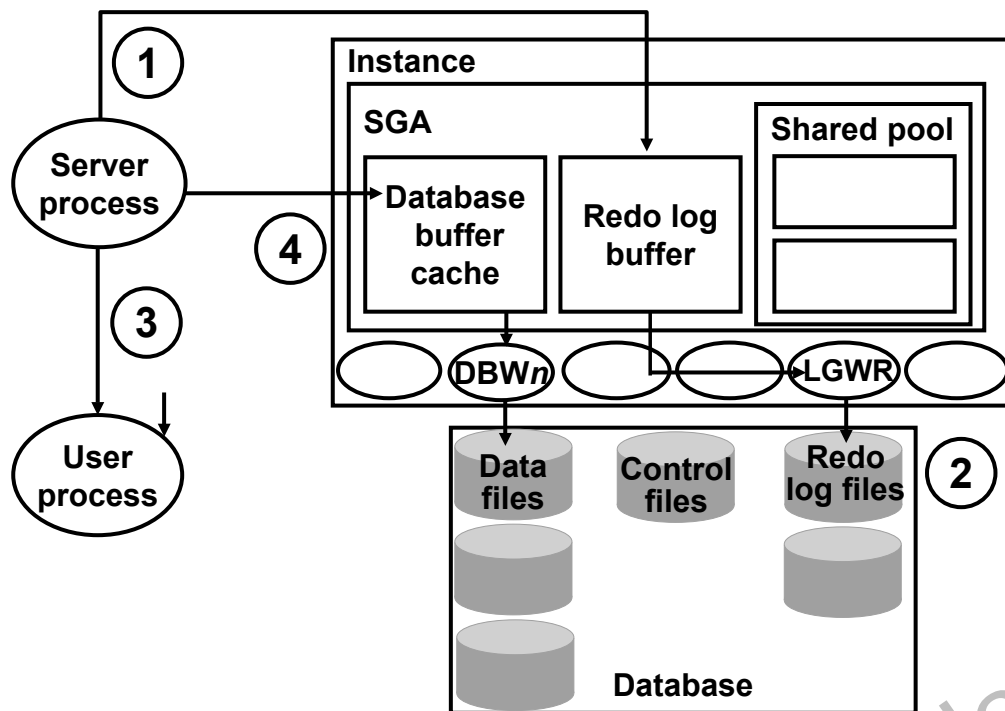
Before making a change, the server process saves the old data value in a rollback segment. This “before image” is used to:

- Undo the changes if the transaction is rolled back
- Provide read consistency by ensuring that other transactions do not see uncommitted changes made by the DML statement
- Recover the database to a consistent state in case of failures

Rollback segments, such as tables and indexes, exist in data files, and rollback blocks are brought into the database buffer cache as required. Rollback segments are created by the DBA.

Changes to rollback segments are recorded in the redo log buffer.

## COMMIT Processing



Copyright © 2007, Oracle. All rights reserved.

## COMMIT Processing

The Oracle server uses a fast COMMIT mechanism that guarantees that the committed changes can be recovered in case of instance failure.

### System Change Number

Whenever a transaction commits, the Oracle server assigns a commit system change number (SCN) to the transaction. The SCN is monotonically incremented and is unique within the database. It is used by the Oracle server as an internal time stamp to synchronize data and to provide read consistency when data is retrieved from the data files. Using the SCN enables the Oracle server to perform consistency checks without depending on the date and time of the operating system.

### Steps in Processing COMMITs

When a COMMIT is issued, the following steps are performed:

1. The server process places a commit record, along with the SCN, in the redo log buffer.
2. LGWR performs a contiguous write of all the redo log buffer entries up to and including the commit record to the redo log files. After this point, the Oracle server can guarantee that the changes will not be lost even if there is an instance failure.

## **COMMIT Processing (continued)**

### **Steps in Processing COMMITs (continued)**

3. The user is informed that the COMMIT is complete.
4. The server process records information to indicate that the transaction is complete and that resource locks can be released.

Flushing of the dirty buffers to the data file is performed independently by DBW0 and can occur either before or after the commit.

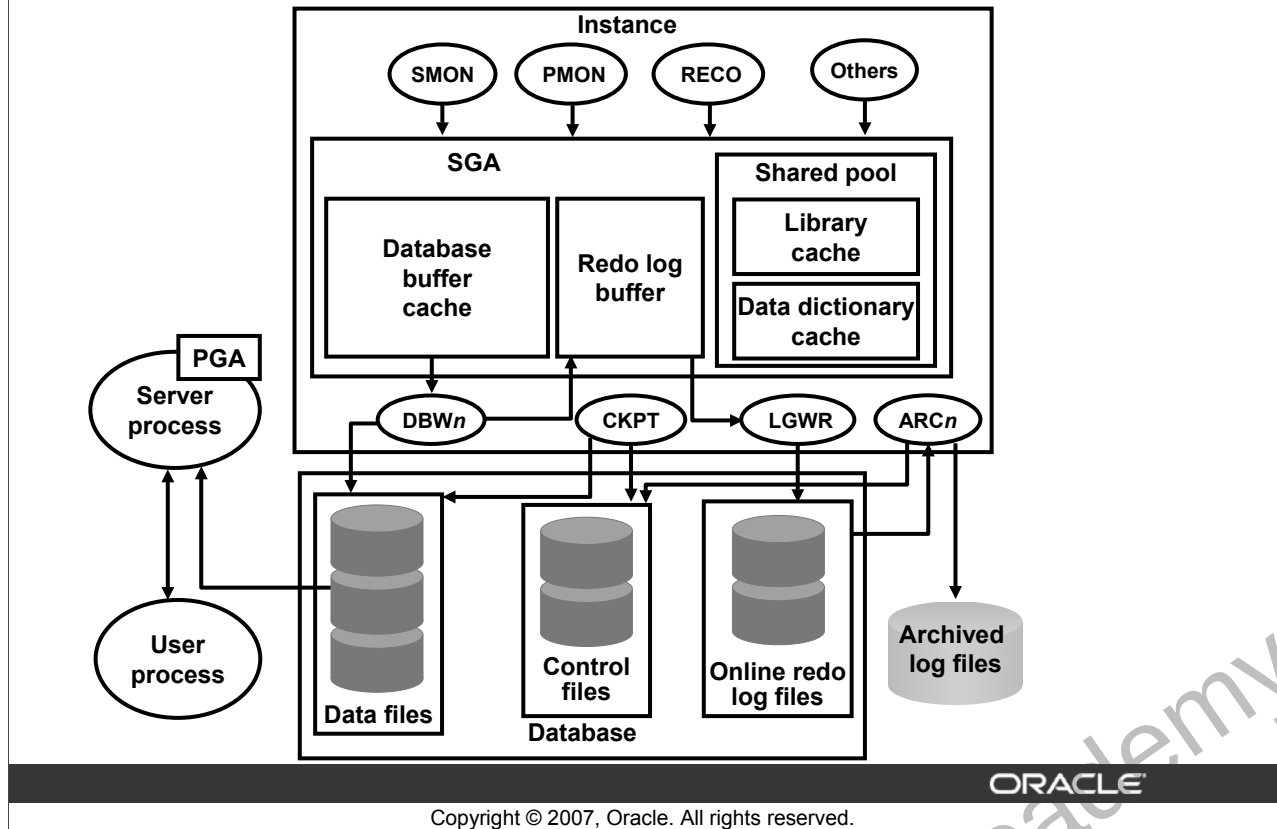
### **Advantages of the Fast COMMIT**

The fast COMMIT mechanism ensures data recovery by writing changes to the redo log buffer instead of the data files. It has the following advantages:

- Sequential writes to the log files are faster than writing to different blocks in the data file.
- Only the minimal information that is necessary to record changes is written to the log files; writing to the data files would require whole blocks of data to be written.
- If multiple transactions request to commit at the same time, the instance piggybacks redo log records into a single write.
- Unless the redo log buffer is particularly full, only one synchronous write is required per transaction. If piggybacking occurs, there can be less than one synchronous write per transaction.
- Because the redo log buffer may be flushed before the COMMIT, the size of the transaction does not affect the amount of time needed for an actual COMMIT operation.

**Note:** Rolling back a transaction does not trigger LGWR to write to disk. The Oracle server always rolls back uncommitted changes when recovering from failures. If there is a failure after a rollback, before the rollback entries are recorded on disk, the absence of a commit record is sufficient to ensure that the changes made by the transaction are rolled back.

## Summary of the Oracle Database Architecture



## Summary of the Oracle Database Architecture

An Oracle Database comprises an instance and its associated database:

- An instance comprises the System Global Area (SGA) and the background processes
  - **SGA:** Database buffer cache, redo log buffer, shared pool, and so on
  - **Background processes:** SMON, PMON, DBWn, CKPT, LGWR, and so on
- A database comprises storage structures:
  - **Logical:** Tablespaces, schemas, segments, extents, and Oracle block
  - **Physical:** Data files, control files, redo log files

When a user accesses the Oracle Database through an application, a server process communicates with the instance on behalf of the user process.





---

# Index

---

Oracle Internal & Oracle Academy  
Use Only

## A

ALTER 1-13, 1-15, 1-23, 2-3, 2-4, 2-5, 2-6, 2-7, 2-8, 2-9,  
2-10, 2-11, 2-12, 2-13, 2-14, 2-15, 2-18, 2-19, 2-20, 2-22, 2-23,  
2-24, 2-25, 2-28, 2-32, 2-36, 2-47, 5-5, 5-7, 5-9, 5-12, 6-24,  
7-24

ALTER SESSION 2-15, 5-5, 5-7, 5-9

ALTER TABLE 2-3, 2-4, 2-5, 2-6, 2-7, 2-8, 2-9, 2-10, 2-11,  
2-12, 2-13, 2-14, 2-15, 2-18, 2-19, 2-20, 2-22, 2-23, 2-24, 2-25,  
2-28, 2-32, 2-36, 2-47, 5-12, 6-24, 7-24

ALTER USER 1-13, 1-23

## C

Cascading Constraints 2-22, 2-23

Check Constraints 7-24, C-7

Conditional INSERT 4-2, 4-3, 4-11, 4-15, 4-18, 4-19, 4-20, 4-23,  
4-24, 4-25, 4-26, 4-27, 4-31, 4-37, 4-42

CONNECT BY 6-11, F-5, F-6, F-7, F-8, F-9, F-11, F-13, F-14

CONNECT BY PRIOR F-5, F-7, F-8, F-9, F-11, F-13, F-14

Constraint 2-12, 2-13, 2-18, 2-19, 2-20, 2-21, 3-3, 3-10, 3-14,  
3-15, 3-17, 3-25, C-8

Conversion functions i-24

Correlated Subquery 6-15

Create Database C-14, C-16

Create Index C-15

Creating a Database Connection i-13

CUBE E-2, E-6, E-7, E-9, E-10, E-11, E-13, E-14, E-17, E-18,  
E-19, E-21, E-22, E-23

CURRENT\_DATE 5-2, 5-3, 5-6, 5-7, 5-8, 5-13, 5-14, 5-21, 5-31,  
5-32, 5-33

CURRENT\_TIMESTAMP 5-2, 5-3, 5-6, 5-7, 5-8, 5-13, 5-14, 5-21,  
5-31, 5-32, 5-33

## D

Database Architecture H-3, H-33

Datetime 5-11

Daylight Saving 5-29, 5-30

DBTIMEZONE 5-2, 5-9, 5-23, 5-31, 5-33

## D

DEFAULT DIRECTORY 2-40, 2-41, 2-42, 2-45

Deferring Constraints 2-15, 2-17

Disabling Constraints 2-19

DROP TABLE 2-31, 2-35, G-2

Dropping a column 2-8

## E

Enabling Constraints 2-20

External Table 2-38, 2-39, 2-40, 2-41, 2-42, 2-43, 2-45

EXTRACT 5-2, 5-3, 5-14, 5-21, 5-22, 5-31, 5-32, 5-34

## F

FLASHBACK TABLE 2-31, 2-33, 2-34, 2-35

Flashback Version Query 4-38, 4-39, 4-40

FROM\_TZ 5-2, 5-3, 5-14, 5-21, 5-25, 5-31

## G

Greenwich Mean Time 5-4, 5-7

GROUP BY 6-11, 6-26, 6-31, 6-32, E-3, E-4, E-5, E-6, E-7,  
E-8, E-9, E-10, E-11, E-12, E-13, E-14, E-15, E-16, E-17, E-18,  
E-19, E-20, E-21, E-22, E-23

GROUP BY ROLLUP E-12, E-17, E-18, E-19

GROUPING E-2, E-11, E-12, E-13, E-14, E-15, E-16, E-17, E-18,  
E-21, E-22, E-23

GROUPING SET E-21

## H

HAVING i-26, i-35, 6-26, E-3, E-5, E-7, E-9, E-11

Hierarchical Queries F-5

## I

INSERT i-27, 1-15, 1-22, 2-16, 2-17, 2-43, 3-5, 4-2, 4-3, 4-4,  
4-7, 4-8, 4-9, 4-10, 4-11, 4-12, 4-13, 4-14, 4-15, 4-16, 4-17,  
4-18, 4-19, 4-20, 4-21, 4-22, 4-23, 4-24, 4-25, 4-26, 4-27, 4-28,  
4-29, 4-30, 4-31, 4-32, 4-33, 4-34, 4-36, 4-37, 4-42, 4-47, 5-13,  
5-18, 5-20, 7-24, D-19, G-2, G-7, H-28

INTERVAL DAY TO SECOND 5-15, 5-17, 5-19, 5-20, 5-28

INTERVAL YEAR TO MONTH 5-15, 5-17, 5-18, 5-19, 5-27

## **L**

LEVEL F-5, F-10, F-11, F-12, F-14

LOCALTIMESTAMP 5-2, 5-3, 5-6, 5-7, 5-8, 5-14, 5-21, 5-31,  
5-32, 5-33

LPAD F-11, F-12

## **M**

MERGE i-27, 4-2, 4-32, 4-33, 4-34, 4-36, 4-42, 4-43

Multiple-Column Subqueries 6-4

## **N**

Nonpairwise Comparisons 6-5

NOT IN 6-22, 6-36

## **O**

ON DELETE CASCADE 2-14

ORACLE\_LOADER 2-37, 2-40, 2-42, 2-43, 2-45, 2-50, 2-51

ORDER BY i-22, 2-29, 3-9, 6-12, 6-13, 6-31, E-3, E-4, E-5,  
E-7, E-9, E-11

ORGANIZATION EXTERNAL 2-37, 2-40, 2-41, 2-42, 2-45

## **P**

Pseudocolumn F-10

## **R**

Ranking Rows F-10

REGEXP\_COUNT 7-3, 7-6, 7-7, 7-11, 7-12, 7-17, 7-22, 7-23

REGEXP\_INSTR 7-3, 7-6, 7-7, 7-11, 7-12, 7-15, 7-17, 7-18, 7-19,  
7-20, 7-22

REGEXP\_LIKE 7-3, 7-6, 7-7, 7-11, 7-12, 7-13, 7-17, 7-22, 7-24

REGEXP\_REPLACE 7-3, 7-6, 7-7, 7-11, 7-12, 7-14, 7-17, 7-22

REGEXP\_SUBSTR 7-3, 7-6, 7-7, 7-11, 7-12, 7-16, 7-17, 7-18,  
7-19, 7-21, 7-22

Regular Expression Support i-6, 7-1, 7-19

Replacing Patterns 7-14

ROLLUP E-2, E-6, E-7, E-8, E-9, E-10, E-11, E-12, E-13, E-14,  
E-17, E-18, E-19, E-21, E-22, E-23

Row Versions 4-48

## **S**

Scalar Subquery 6-11

## **S**

SESSIONTIMEZONE 5-2, 5-7, 5-9, 5-23, 5-31, 5-33

SET TIME\_ZONE 5-5, 5-7, 5-9

SET UNUSED 2-9

SQL Developer i-2, i-3, i-7, i-9, i-10, i-11, i-12, i-13,  
i-14, i-15, i-16, i-19, i-20, i-28, i-29, i-32, i-33, 1-13, 1-25,  
3-30, 7-24, C-1, C-2, C-3, C-4, C-5, C-17, C-19, C-20, C-21,  
C-22, C-23, C-24, G-5, G-6, G-7

SQL Worksheet i-14, i-15, i-16, i-17, i-18, i-32, i-34, 2-48,  
3-31, C-4, C-17, C-18, G-5

START WITH F-5, F-6, F-11, F-14

Subquery 4-5, 4-6, 4-7, 4-8, 4-16, 6-4, 6-6, 6-7, 6-8,  
6-9, 6-11, 6-15

## **T**

Time Zone 5-7, 5-8

TIMESTAMP 2-34, 4-39, 5-2, 5-3, 5-6, 5-7, 5-8, 5-10, 5-11,  
5-12, 5-13, 5-14, 5-21, 5-25, 5-26, 5-31, 5-32, 5-33, 5-34

TIMESTAMP WITH TIME ZONE 5-6, 5-8, 5-10, 5-12, 5-13, 5-25, 5-26,  
5-34

TO\_DSINTERVAL 5-2, 5-3, 5-14, 5-21, 5-28, 5-31

TO\_YMINTERVAL 5-2, 5-3, 5-14, 5-21, 5-27, 5-31, 5-36

Tracking Changes 4-38

TZ\_OFFSET 5-2, 5-3, 5-14, 5-21, 5-23, 5-24, 5-31, 5-33

## **U**

Unconditional INSERT ALL 4-21, 4-22

UNION E-7, E-10, E-13, E-14, E-16, E-17, E-18, E-20, E-23

UNION ALL E-7, E-10, E-13, E-14, E-16, E-17, E-18, E-20, E-23

Using Snippets C-17, C-18

## **V**

VERSIONS BETWEEN 4-39, 4-41

## **W**

Walking the Tree F-6, F-7, F-8, F-9

WITH Clause 6-29, 6-30, 6-31